



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΑΚΑΔ. ΕΤΟΣ 2024-2025

ΑΘΗΝΑ 4 Οκτωβρίου 2024

1η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"

Ανάπτυξη κώδικα για το μικροελεγκτή ATmega328 και προσομοίωση της εκτέλεσης του στο αναπτυξιακό περιβάλλον MPLAB X

Εξέταση – Επίδειξη: Παρασκευή 11/10/2024.

Προθεσμία για παράδοση Έκθεσης: Κυριακή 13/10/2024 (23:59)

Ζήτημα 1.1

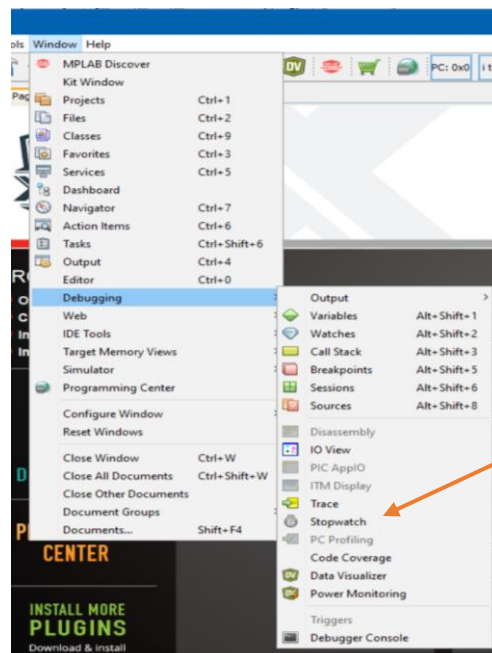
Να υλοποιηθεί κώδικας assembly, για τον μικροελεγκτή ATmega328, στη μορφή που εμφανίζεται παρακάτω, ο οποίος να παράγει ρυθμιζόμενες χρονικές καθυστερήσεις.

```
...  
rcall wait_x_msec  
...  
  
wait_x_msec:  
....  
ret
```

- Η εκτέλεση της εντολής `rcall wait_x_msec` να διαρκεί χρόνο x msec, όπου x είναι ένας αριθμός από 1 έως 65535, ο οποίος είναι αποθηκευμένος στο ζεύγος των καταχωρητών `r24`(Low byte) και `r25`(High byte).
- Το τμήμα αυτό του κώδικα να ενσωματωθεί σε ένα ολοκληρωμένο πρόγραμμα και να γίνει προσομοίωση σωστής λειτουργίας, στον αναπτυξιακό περιβάλλον Microchip MPLAB X.
- Στο τμήμα του κώδικα που γίνεται μέτρηση του χρόνου να ενσωματωθεί η εντολή `"sbiw"` η οποία χειρίζεται ένα ζεύγος καταχωρητών και όχι έναν απλό καταχωρητή.

Ο χρόνος εκτέλεσης της προσομοίωσης, εξαρτάται από τον υπολογιστή που εκτελεί την προσομοίωση και είναι διαφορετικός από το χρόνο εκτέλεσης του κώδικα από το φυσικό μικροελεγκτή.

Οπότε για να μετρηθούν οι χρόνοι εκτέλεσης τμημάτων του κώδικα, θα χρησιμοποιηθεί το εργαλείο Stopwatch, του MPLAB X, το οποίο ενεργοποιείται όπως φαίνεται στο σχήμα δίπλα.



Ζήτημα 1.2

Να υλοποιηθεί κώδικας assembly, για τον μικροελεγκτή ATmega328, για τον υπολογισμό των λογικών συναρτήσεων:

$$F0 = (A \cdot B' + B' \cdot D)'$$

$$F1 = (A+C') \cdot (B+D')$$

Ο υπολογισμός των συναρτήσεων να εισαχθεί σε ένα loop, το οποίο θα εκτελεστεί 6 φορές. Σε κάθε κύκλο η μεταβλητή A θα αυξάνεται κατά 0x01, η μεταβλητή B θα αυξάνεται κατά 0x02, η μεταβλητή C θα αυξάνεται κατά 0x03 και η μεταβλητή D θα αυξάνεται κατά 0x04.

Οι μεταβλητές A, B, C, D είναι μεγέθους ενός byte και έχουν αρχικές τιμές A=0x51, B=0x41, C=0x21 και D=0x01.

Να γίνει προσομοίωση στο MPLAB X και να συμπληρωθεί ο παρακάτω πίνακας:

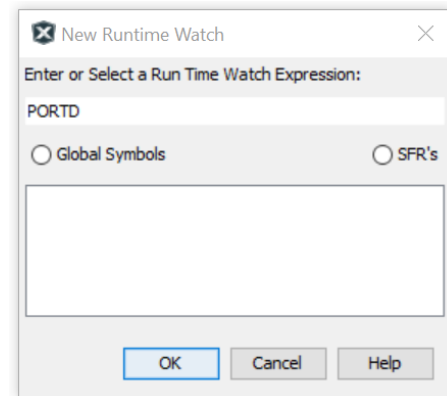
A	B	C	D	F0	F1
0x51	0x41	0x21	0x01		

Ζήτημα 1.3

Να υλοποιηθεί κώδικας assembly, για τον μικροελεγκτή ATmega328, ο οποίος να ελέγχει ένα αυτοματισμό βαγονέτου που κινείται συνεχώς, αρχικά από δεξιά προς τα αριστερά και στη συνέχεια αντίστροφα.

Το βαγονέτο να προσομοιώνεται με ένα bit της θύρας εξόδου PORTD που κινείται συνεχώς από το LSb προς το MSb και αντίστροφα.

Για να είναι εφικτή η παρακολούθηση της αλλαγής θέσης του bit της PORTD, κατά τη διάρκεια της εκτέλεσης της προσομοίωσης, είναι απαραίτητο να εισαχθεί ένα Run Time Watch, στο MPLAB X, για την PORTD, όπως φαίνεται στην εικόνα δίπλα.



- Η κίνησή του βαγονέτου κατά μία θέση, θα γίνεται κάθε 1 sec περίπου.
- Η κατεύθυνση της κίνησης να αποθηκεύεται στο T flag του SREG.
- Το βαγονέτο, κάθε φορά που αλλάζει κατεύθυνση, θα κάνει μία πρόσθετη στάση 1 sec περίπου δηλ. θα παραμένει στα άκρα 2 sec περίπου.
- Για τη δημιουργία των χρονικών καθυστερήσεων να χρησιμοποιηθεί ο κώδικας που υλοποιήθηκε στο ζήτημα 1.1
- Ο χρόνος εκτέλεσης της προσομοίωσης, εξαρτάται από τον υπολογιστή που εκτελεί την προσομοίωση. Οπότε οι διάφοροι χρόνοι θα επιτευχθούν κατόπιν δοκιμών.



ΑΘΗΝΑ, 11 Οκτωβρίου 2024

2^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Χρήση εξωτερικών διακοπών στον Μικροελεγκτή AVR

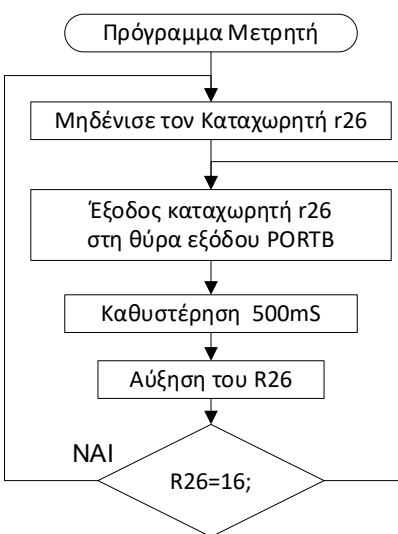
Εξέταση – Επίδειξη: Παρασκευή 18/10/2023
Προθεσμία για παράδοση Έκθεσης: Τρίτη 22/10/2023 (23:59)

Εισαγωγή

Μια χρήσιμη δυνατότητα των Μικροελεγκτών είναι η άμεση ανταπόκρισή τους σε εξωτερικές συνθήκες. Η ανταπόκριση αυτή επιτυγχάνεται με την εκμετάλλευση του συστήματος διακοπών τους. Κάθε μικροελεγκτής είναι εφοδιασμένος με μια ή περισσότερες εισόδους διακοπών. Η ενεργοποίηση μιας εισόδου διακοπής υποχρεώνει το μικροελεγκτή να σταματά άμεσα την τρέχουσα εργασία και να εκτελεί τον κώδικα που υπάρχει σε μια προκαθορισμένη διεύθυνση, που ονομάζεται διάνυσμα διακοπής. Στο σημείο αυτό συνδέεται συνήθως μια ρουτίνα εξυπηρέτησης διακοπής (διαφορετική για κάθε εφαρμογή). Μετά το τέλος της ρουτίνας εξυπηρέτησης διακοπής, ο Μικροελεγκτής συνεχίζει την εργασία που διέκοψε, επιστρέφοντας στο σημείο ακριβώς που είχε διακοπεί. Στην συνέχεια δίνεται ένα παράδειγμα προγράμματος μετρητή που διακόπτεται από την εξωτερική διακοπή INT0. Η διακοπή αυτή είναι συνδεδεμένη στο 3^ο bit της PORTD (PD2).

Παράδειγμα 2.1 Ένα πρόγραμμα μέτρησης που διακόπτεται από την INT0

Αρχικά εισάγουμε ένα πρόγραμμα που μετράει δυαδικά από το 0 έως το 15 σε συνεχόμενη λειτουργία με καθυστέρηση 500 mS σε κάθε μέτρηση. Το αποτέλεσμα της μέτρησης εμφανίζεται στα Leds της θύρας PB3-PB0. Το διάγραμμα ροής του φαίνεται στο σχήμα 2.1α.



Σχήμα 2.1α Πρόγραμμα μετρητή.

Το πρόγραμμα φαίνεται στο σχήμα 2.1β παρακάτω:

Σχήμα 2.1β Πρόγραμμα μετρητή

```
.include "m328PBdef.inc" ;ATmega328P microcontroller definitions

.equ FOSC_MHZ=16          ; Microcontroller operating frequency in MHz

.equ DEL_mS=500           ; Delay in mS (valid number from 1 to 4095)

.equ DEL_NU=FOSC_MHZ*DEL_mS ; delay_mS routine: (1000*DEL_NU+6) cycles

;Init Stack Pointer
ldi r24, LOW(RAMEND)
out SPL, r24
ldi r24, HIGH(RAMEND)
out SPH, r24

;Init PORTB as output
ser r26
out DDRB , r26

loop1:
clr r26
loop2:
out PORTB, r26

ldi r24, low(DEL_NU)      ;
ldi r25, high(DEL_NU)     ; Set delay (number of cycles)
rcall delay_mS            ;

inc r26

cpi r26, 16               ; compare r26 with 16
breq loop1
rjmp loop2

; delay of 1000*F1+6 cycles (almost equal to 1000*F1 cycles)
delay_mS:

; total delay of next 4 insruction group = 1+(249*4-1) = 996 cycles
ldi r23, 249              ; (1 cycle)
loop_inn:
dec r23                   ; 1 cycle
nop                        ; 1 cycle
brne loop_inn             ; 1 or 2 cycles

sbiw r24 ,1               ; 2 cycles
brne delay_mS             ; 1 or 2 cycles

ret                        ; 4 cycles
```

Εξωτερικές διακοπές στον ATmega328PB

Για τη λειτουργία του συστήματος διακοπών κάθε Μικροελεγκτή είναι απαραίτητη αρχικά η ενεργοποίηση σημαιών και επιλογών, που καθορίζουν τον ακριβή τρόπο λειτουργίας.

Στον Μικροελεγκτή AVR ATmega328PB, η επιλογή του επιπέδου ενεργοποίησης των εξωτερικών διακοπών γίνεται δια μέσω του καταχωρητή EICRA (offset 0x69), γράφοντας κατάλληλες τιμές στα τέσσερα λιγότερα σημαντικά ψηφία, σύμφωνα με τους παρακάτω πίνακες:

EICRA:

				ISC11	ISC10	ISC01	ISC00
--	--	--	--	-------	-------	-------	-------

ISC11	ISC10	Περιγραφή
0	0	Διακοπή στη χαμηλή στάθμη του INT1
0	1	Διακοπή σε κάθε αλλαγή στάθμης του INT1
1	0	Διακοπή στην κατερχόμενη ακμή του INT1
1	1	Διακοπή στην ανερχόμενη ακμή του INT1

ISC01	ISC00	Περιγραφή
0	0	Διακοπή στη χαμηλή στάθμη του INT0
0	1	Διακοπή σε κάθε αλλαγή στάθμης του INT0
1	0	Διακοπή στην κατερχόμενη ακμή του INT0
1	1	Διακοπή στην ανερχόμενη ακμή του INT0

Η επίτρηση των εξωτερικών διακοπών ενεργοποιείται γράφοντας στον καταχωρητή EIMSK (offset 0x3D) την τιμή 1 στο ψηφίο που αντιστοιχεί στην είσοδο διακοπής που επιθυμούμε να επιτρέψουμε, σύμφωνα με το παρακάτω σχήμα:

EIMSK:

						INT1	INT0
--	--	--	--	--	--	------	------

Ο καταχωρητής EIFR(offset 0x3C) περιέχει τις σημαίες των διακοπών INT0 και INT1. Κάθε μια από αυτές τίθεται όταν συμβεί η αντίστοιχη διακοπή και μηδενίζεται μετά την εξυπηρέτηση της διακοπής.

EIFR:

						INTF1	INTF0
--	--	--	--	--	--	-------	-------

Επίσης, απαραίτητη είναι και η εκτέλεση της εντολής sei, που επιτρέπει γενικώς την πρόκληση διακοπών.

Στον Μικροελεγκτή AVR ATmega328P, ο ακροδέκτης PD2, εκτός από ακροδέκτης εισόδου/εξόδου γενικής χρήσης, λειτουργεί εναλλακτικά και ως εξωτερική είσοδος της διακοπής INT0. Ομοίως ο ακροδέκτης PD3 λειτουργεί εναλλακτικά και ως εξωτερική είσοδος της διακοπής INT1.

Η ρουτίνα εξυπηρέτησης της διακοπής πρέπει να δηλωθεί στην κατάλληλη διεύθυνση του προγράμματος. Το διάνυσμα της διακοπής INT0 είναι 0x002 και το διάνυσμα της διακοπής INT1 είναι 0x004.

Για παράδειγμα, το παρακάτω τμήμα κώδικα (Σχήμα 2.2) ενεργοποιεί διακοπές στην είσοδο INT0 κατά την ανερχόμενη ακμή. Πρέπει να δοθεί προσοχή στο γεγονός ότι ο I/O καταχωρητής EICRA έχει η διεύθυνση μεγαλύτερη από 0x60, οπότε χρησιμοποιείται η εντολή STS και όχι η εντολή OUT.

Στη ρουτίνα εξυπηρέτησης της εξωτερική διακοπής INT0, ανάβουν τα led της θύρας PORTB για 500mS και στην συνέχεια συνεχίζεται η μέτρηση από το σημείο που είχε μείνει.

Η ενεργοποίηση της διακοπής INT0 δίνεται στο Σχήμα 2.2 και η ρουτίνα εξυπηρέτησης διακοπής στο Σχήμα 2.3.

Σχήμα 2.2 Εντολές που ενεργοποιούν τη διακοπή INT0

```
.org 0x0
rjmp reset
.org 0x2
rjmp ISR0

reset:
; Interrupt on rising edge of INT0 pin
ldi r24, (1 << ISC01) | ( 1 << ISC00)
sts EICRA, r24

;Enable the INT0 interrupt(PD2)
ldi r24, (1 << INT0)
out EIMSK, r24

sei ;Sets the Global Interrupt Flag
```

Σχήμα 2.3 Ρουτίνα εξυπηρέτησης διακοπής

```
ISR0:
push r25 ;
push r24 ;
in r24 , SREG ; Save r24, r25, SREG
push r24 ;

ser r24
out PORTB , r24

ldi r24, low(16*500) ;
ldi r25, high(16*500) ; Set delay (number of cycles)
rcall delay_mS

pop r24 ;
out SREG , r24 ; Restore r24, r25, SREG
pop r24 ;
pop r25 ;

reti
```

Το Φαινόμενο της Αναπήδησης (ή Σπινθηρισμού)

Εάν ο χρόνος που διαρκεί η εκτέλεση μιας ρουτίνας εξυπηρέτησης διακοπής είναι πολύ μικρός, ενδέχεται να εμφανιστεί το φαινόμενο της αναπήδησης, λόγω σπινθηρισμού στον πιεστικό διακόπτη που προκαλεί την εξωτερική διακοπή. Συγκεκριμένα, ένας πιεστικός διακόπτης μπορεί να δώσει περισσότερα του ενός σήματα διακοπής τόσο κατά την πίεση του όσο και κατά την απελευθέρωση του, οπότε μετά την ολοκλήρωση της ρουτίνας εξυπηρέτησης της διακοπής, μπορεί να καταφθάσουν και άλλα σήματα διακοπής που οφείλονται στο ίδιο πάτημα ή απελευθέρωση του πιεστικού διακόπτη. Μια λύση για το πρόβλημα αυτό για τον μικροελεγκτή ATmega328PB είναι ο έλεγχος του καταχωρητή EIFR μέσα στη ρουτίνα εξυπηρέτησης διακοπής.

Τα ψηφία INTF1 και INTF0 του καταχωρητή EIFR τίθενται αυτόματα από το μικροελεγκτή όταν υπάρχει αίτηση εξωτερικής διακοπής INT1 και INT0 αντίστοιχα, και μηδενίζονται όταν εξυπηρετηθεί η αίτηση. Τα ψηφία INTF1 και INTF0 μπορούν να μηδενιστούν και από τον κώδικα, με εγγραφή λογικού 1 στο αντίστοιχο ψηφίο.

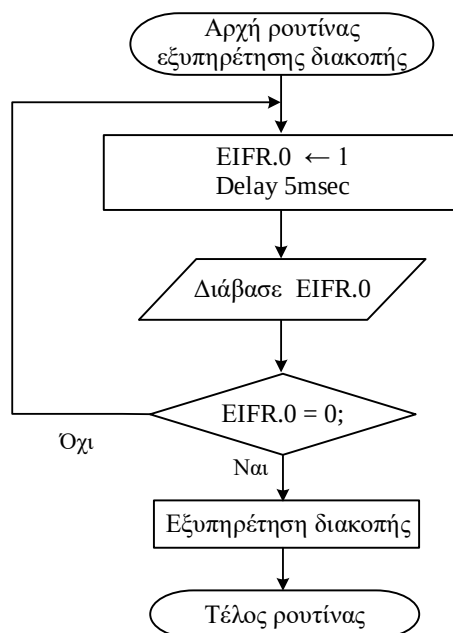
Π.χ. το INT0 μπορεί να μηδενιστεί με τις παρακάτω εντολές:

```
ldi r24, (1 << INTF0)
out EIFR, r24
```

Ένας απλός αλγόριθμος αποφυγής της αναπήδησης είναι ο εξής:

Στην αρχή της ρουτίνας εξυπηρέτησης διακοπής μηδενίζεται το ψηφίο INTF1 ή INTF0. Μετά από ένα μικρό χρονικό διάστημα (π.χ. 5 msec) το ψηφίο ελέγχεται ξανά και αν είναι λογικό 1 τότε κάποιος σπινθηρισμός έχει δώσει νέο σήμα διακοπής για το ίδιο πάτημα και το ψηφίο μηδενίζεται ξανά. Η διαδικασία αυτή επαναλαμβάνεται μέχρις ότου το ψηφίο σταθεροποιηθεί σε λογικό 0, οπότε έχουν σταματήσει οι αναπηδήσεις, και ολοκληρώνεται η ρουτίνα εξυπηρέτησης.

Τα παραπάνω, για την περίπτωση της εξωτερικής εισόδου INT0 φαίνονται και στο επόμενο λογικό διάγραμμα.



Σχήμα 2.4 Λογικό διάγραμμα αποφυγής σπινθηρισμού (debouncing) πλήκτρου διακοπής INT0.

Παράδειγμα 2.2 Ένα πρόγραμμα με εξωτερικά Interrupts σε γλώσσα C

Το πρόγραμμα που φαίνεται στο παρακάτω σχήμα (Σχήμα 2.5) αποτελείται από τον κώδικα της κυρίας ρουτίνας, ο οποίος αναβοσβήνει συνεχώς τα led της PORTB, με καθυστέρηση 500mS. Επίσης ενεργοποιεί τις εξωτερικές διακοπές INT0 και INT1 κατά την ανερχόμενη ακμή τάσης στα αντίστοιχα pins. Οι ρουτίνες εξυπηρέτησης των διακοπών ανάβουν τα led της PORTC για 2 Sec.

Σχήμα 2.5 Κώδικας που αναβοσβήνει τα led της PORTB και διακόπτεται από τα INT0 και INT1.

```
#define F_CPU 16000000UL
#include<avr/io.h>
#include<avr/interrupt.h>
#include<util/delay.h>

ISR (INT0_vect)           //External INT0 ISR
{
    PORTC = 0xFF;
    _delay_ms(2000);
    PORTC = 0x00;
}

ISR (INT1_vect)           //External INT1 ISR
{
    PORTC = 0xFF;
    _delay_ms(2000);
    PORTC = 0x00;
}

int main()
{
    //Interrupt on rising edge of INT0 and INT1 pin
    EICRA=(1 << ISC01) | ( 1 << ISC00) | (1 << ISC11) | ( 1 << ISC10);
    //Enable the INT0 interrupt(PD2), INT1 interrupt(PD3)
    EIMSK=(1 << INT0) | (1 << INT1);
    sei();           // Enable global interrupts

    DDRB=0xFF;       //Set PORTB as output
    DDRC=0xFF;       //Set PORTC as output
    PORTC=0x00;

    while(1)
    {
        PORTB = 0x00;
        _delay_ms(500);
        PORTB = 0xFF;
        _delay_ms(500);
    }
}
```

Τα ζητούμενα της 2^{ης} εργαστηριακής άσκησης

Ζήτημα 2.1

A) Στον κώδικα του μετρητή του σχήματος 2.1, να προστεθεί ρουτίνα εξυπηρέτησης της εξωτερικής διακοπής **INT1** (PD3), σε γλώσσα Assembly, η οποία να απαριθμεί το πλήθος των διακοπών INT1 από 0 έως 63. Όταν φτάσει 63 να αρχίζει ξανά από το μηδέν. Όσο είναι πατημένο το μπουτόν **PD5** (λογικό 0) η μέτρηση των διακοπών παγώνει. Όταν το μπουτόν **PD5** αφεθεί ξανά, η μέτρηση συνεχίζεται από το σημείο που είχε μείνει. Το πλήθος των εξωτερικών διακοπών να απεικονίζεται, σε δυαδική μορφή, στα leds **PC5-PC0**.

B) Επειδή υπάρχει η πιθανότητα να εμφανιστεί το φαινόμενο του σπινθηρισμού, με το πάτημα του μπουτόν **PD3** του προηγούμενου παραδείγματος, να προστεθούν οι κατάλληλες εντολές στη ρουτίνα εξυπηρέτησης της εξωτερικής διακοπής **INT1**, ώστε να αποφευχθεί το φαινόμενο αυτό.

Ζήτημα 2.2

A) Να υλοποιηθεί κώδικας Assembly για το μικροελεγκτή ATmega328PB, αντίστοιχο με τον κώδικα του μετρητή του σχήματος 2.1, με τη διαφορά ότι η μέτρηση θα είναι 0 έως 31, η απεικόνιση θα γίνεται στα leds **PC4-PC0** και ο χρόνος καθυστέρησης μεταξύ των εναλλαγών θα κυμαίνεται στα 2000 mS.

B) Η ρουτίνα εξυπηρέτησης της εξωτερικής διακοπής **INT0** (PD2) να τροποποιηθεί έτσι ώστε όταν ενεργοποιείται να ανάβει τόσα LEDs της θύρας **PORTC** αρχίζοντας από τα **LSB** όσο το πλήθος των 4 μπουτόν PB3 - PB0 που είναι πατημένα.

Ζήτημα 2.3

Να υλοποιηθεί αυτοματισμός που να ελέγχει το άναμμα και το σβήσιμο ενός φωτιστικού σώματος. Όταν πατάμε το push button **PD3** (δηλαδή με την ενεργοποίηση της **INT1**) να ανάβει το led **PB0** της θύρας **PORTB** (που υποθέτουμε ότι αντιπροσωπεύει το φωτιστικό σώμα). Το led θα σβήνει μετά από 5 sec, εκτός και αν ενδιάμεσα υπάρξει νέο πάτημα του PD3, οπότε και ο χρόνος των 5 sec θα ανανεώνεται. Κάθε φορά που γίνεται ανανέωση να ανάβουν τα led της θύρας **PORTB** (PB5-PB0) για 0.5 sec, μετά να σβήνουν εκτός από το led **PB0** που παραμένει συνολικά για 5 sec εκτός και αν ανανεωθεί. Ο κώδικας να δοθεί σε **Assembly** και σε **C**.



Ε.Μ.Π. - ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΑΚΑΔ. ΕΤΟΣ 2024-2025

ΑΘΗΝΑ, 18 Οκτωβρίου 2024

3^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Χρήση των Timers και του ADC στον AVR

Εξέταση – Επίδειξη: Παρασκευή 25/10/2024
Προθεσμία για παράδοση Έκθεσης: Τρίτη 29/10/2024 (23:59)

Χρονιστές(Timers)

Ένα πλεονέκτημα των σύγχρονων μικροελεγκτών είναι η ενσωμάτωση στην ίδια ψηφίδα χρήσιμων περιφερειακών συσκευών, όπως οι χρονιστές, με σύνδεση στο σύστημα διακοπών του Μικροελεγκτή. Οι χρονιστές είναι καταχωρητές που μεταβάλλονται με την πάροδο του χρόνου και μπορούν να προγραμματιστούν να προκαλέσουν διακοπή στον μικροελεγκτή σε συγκεκριμένα χρονικά διαστήματα.

Ο μικροελεγκτής ATmega328PB διαθέτει 8-ψήφιους και 16-ψήφιους χρονιστές. Στους χρονιστές αυτούς μπορεί να τοποθετηθεί μια αρχική τιμή η οποία αυξάνεται με επιλεγμένη συχνότητα. Όταν ένας χρονιστής υπερχειλίζει, μπορεί να δημιουργήσει κατάλληλο σήμα διακοπής.

Timer/Counter1(TCNT1)

Ο Timer/Counter1(TCNT1) είναι 16 bit. Το TCNT1L με διεύθυνση 0x84 είναι το Low Byte και το TCNT1H με διεύθυνση 0x85 είναι το Hi Byte.

Επιλογή συχνότητας λειτουργίας του Timer/Counter1(TCNT1)

Για την επιλογή της συχνότητας λειτουργίας του Timer/Counter1(TCNT1) χρησιμοποιούνται τα flags CS10, CS11 και CS12 του καταχωρητή TCCR1B σύμφωνα με τον Πίνακα 3.1 παρακάτω:

TCCR1B (Διεύθυνση: 0x81)

Bit	7	6	5	4	3	2	1	0
	ICNC1	ICES1		WGM13	WGM12	CS12	CS11	CS10
Access	R/W	R/W		R/W	R/W	R/W	R/W	R/W
Reset	0	0		0	0	0	0	0

CS12	CS11	CS10	Περιγραφή σήματος εισόδου χρονιστή
0	0	0	Κανένα. Χρονιστής σταματημένος
0	0	1	CLK
0	1	0	CLK/8
0	1	1	CLK/64
1	0	0	CLK/256
1	0	1	CLK/1024
1	1	0	Κατερχόμενη ακμή εξωτερικού σήματος ακροδέκτη T1
1	1	1	Ανερχόμενη ακμή εξωτερικού σήματος ακροδέκτη T1

Πίνακας 3.1: Επιλογή συχνότητας λειτουργίας του Timer/Counter1(TCNT1)

Για παράδειγμα, με τις παρακάτω εντολές επιλέγεται συχνότητα λειτουργίας του Timer/Counter1(TCNT1) ίση με τη 1/1024 της συχνότητας ρολογιού του μικροελεγκτή (δηλαδή, κάθε 1024 κύκλους ρολογιού αυξάνεται κατά 1 ο TCNT1).

```
ldi r24, (1<<CS12) | (0<<CS11) | (1<<CS10) ; CK/1024
sts TCCR1B, r24
```

Στην εκπαιδευτική κάρτα ntuAboard_G1 με συχνότητα ρολογιού 16MHz, η επιλογή αυτή ισοδυναμεί με συχνότητα αύξησης του TCNT1 ίση με 16MHz/1024=15625Hz.

Αν με αυτές τις επιλογές θέλουμε ο TCNT1 να δημιουργήσει σήμα διακοπής υπερχειλίσσης μετά από 3sec, πρέπει να ρυθμιστεί για μέτρημα $3 \times 15625 = 46875$ κύκλων. Επειδή η υπερχειλίση γίνεται μετά από 65536 κύκλους (16 ψηφία), θα πρέπει η αρχική τιμή που θα του δοθεί πριν αρχίσει τη μέτρηση προς τα πάνω να είναι $65536 - 46875 = 18660$. Αυτό γίνεται με τον παρακάτω κώδικα:

```
ldi r24, HIGH(18660) ; αρχικοποίηση του TCNT1
out TCNT1H, r24 ; για υπερχειλίση μετά από 3 sec
ldi r24, LOW(18660)
out TCNT1L, r24
```

Ανάγνωση/Εγγραφή του Timer/Counter1(TCNT1)

Για μέγιστη ασφάλεια, τα δύο τμήματα του μετρητή TCNT1, TCNT1H και TCNT1L πρέπει να διαβάζονται αδιαίρετα, χωρίς να μεσολαβήσει για παράδειγμα κάποια άλλη διακοπή. Για το λόγο αυτό ο μικροελεγκτής κάνει μια ειδική διαδικασία. Όταν μια τιμή γράφεται στον TCNT1H, αυτή τοποθετείται στον προσωρινό καταχωρητή TEMP. Στη συνέχεια, όταν γραφεί τιμή στον TCNT1L η τιμή που υπάρχει στον TEMP συνδυάζεται με αυτή και τα 16 ψηφία γράφονται ταυτόχρονα σε όλο το μήκος του TCNT1. Κατά την ανάγνωση, όταν διαβάζεται μια τιμή από τον TCNT1L αυτή τοποθετείται στον επιλεγμένο καταχωρητή του μικροελεγκτή και ταυτόχρονα η τιμή του TCNT1H μεταφέρεται στον καταχωρητή TEMP. Όταν στην συνέχεια διαβαστεί και ο TCNT1H, μεταφέρεται στον επιλεγμένο καταχωρητή η τιμή που έχει τοποθετηθεί στον TEMP.

Με βάση αυτή τη διαδικασία κατά την εγγραφή πρέπει πάντα να γράφεται πρώτα η τιμή στον TCNT1H και μετά στον TCNT1L ενώ κατά την ανάγνωση πρέπει πάντα να διαβάζεται πρώτα ο TCNT1L και μετά ο TCNT1H. Σε πολύπλοκες εφαρμογές πριν την πρόσβαση στον TCNT1 ενδέχεται να είναι απαραίτητη η απενεργοποίηση των διακοπών.

Παρόμοια τεχνική χειρισμού μπορεί να χρησιμοποιηθεί και για άλλους 16-bit καταχωρητές όπως οι OCR1 και ICR1.

Διακοπές υπερχειλίσσης του Timer/Counter1(TCNT1)

Η επίτρεψη της διακοπής υπερχειλίσσης του Timer/Counter1(TCNT1)1 ρυθμίζεται δια μέσω του flag TOIE1 του καταχωρητή TIMSK1 σύμφωνα με το παρακάτω σχήμα:

TIMSK1 (Διεύθυνση: 0x6F)

Bit	7	6	5	4	3	2	1	0
			ICIE1			OCIE1B	OCIE1A	TOIE1
Access			R/W			R/W	R/W	R/W
Reset			0			0	0	0

Γράφοντας 1 στο ψηφίο TOIE1 επιτρέπονται διακοπές υπερχειλίσσης του Timer/Counter1(TCNT1), εφόσον επιτραπούν γενικά οι διακοπές με την εντολή sei.

Για παράδειγμα, με τις παρακάτω εντολές επιτρέπεται η διακοπή υπερχειλίσσης του μετρητή TCNT1.

```
ldi r24, (1<<TOIE1) ; ενεργοποίηση διακοπής υπερχειλίσσης του TCNT1
sts TIMSK1, r24 ;
```


Η θέση μνήμης του διανύσματος διακοπής του Timer1 φαίνεται στον παρακάτω κώδικα.

```
.org 0x1A
```

```
rjmp ISR_TIMER1_OVF ; ρουτίνα εξυπηρέτησης της διακοπής υπερχειλίσσης του TCNT1
```

Το flag υπερχειλίσσης του Timer/Counter1 (TOV1) βρίσκεται στον καταχωρητή TIFR1 όπως φαίνεται στο παρακάτω σχήμα:

TIFR1 (Διεύθυνση: 0x36)

Bit	7	6	5	4	3	2	1	0
			ICF1			OCF1B	OCF1A	TOV1
Access			R/W			R/W	R/W	R/W
Reset			0			0	0	0

Ρύθμιση τρόπου λειτουργίας του Timer/Counter1

Τα flags WGM10, WGM11 του καταχωρητή TCCR1A και τα flags WGM12, WGM13 του καταχωρητή TCCR1B, ο οποίος παρουσιάστηκε στη σελίδα 1 παραπάνω, χρησιμεύουν για να καθοριστεί ο τρόπος λειτουργίας του Timer/Counter1(TCNT1), σύμφωνα με τον παρακάτω πίνακα:

TCCR1A (Διεύθυνση: 0x80)

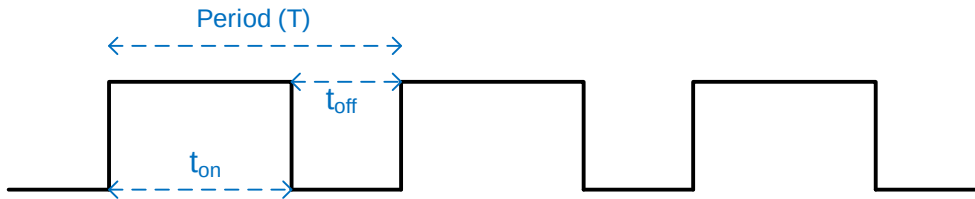
Bit	7	6	5	4	3	2	1	0
	COM1A1	COM1A0	COM1B1	COM1B0			WGM11	WGM10
Access	R/W	R/W	R/W	R/W			R/W	R/W
Reset	0	0	0	0			0	0

Mode	WGM13	WGM12	WGM11	WGM10	Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	-	-	-
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

Πίνακας 3.2: Ρύθμιση τρόπου λειτουργίας του Timer/Counter1

Διαμόρφωση εύρους παλμών (Pulse Width Modulation, PWM)

Μία PWM κυματομορφή είναι μία τετραγωνική περιοδική κυματομορφή η οποία έχει δύο τμήματα. Το τμήμα ON στο οποίο η κυματομορφή μία μέγιστη τιμή και το τμήμα OFF στο οποίο έχει μία ελάχιστη τιμή. Η περίοδος της κυματομορφής είναι σταθερή ενώ οι χρόνοι t_{on} και t_{off} μεταβάλλονται.



Σχήμα 3.1: PWM κυματομορφή

Ο βαθμός χρησιμοποίησης (Duty Cycle) συμβολίζεται με DC και ορίζεται σύμφωνα με τον τύπο:

$$DC = \frac{t_{on}}{T}$$

Διαμόρφωση εύρους παλμών με τον ATmega328PB

Ο ATmega328PB διαθέτει διάφορους μετρητές (Timer/Counters) οι οποίοι μπορούν να χρησιμοποιηθούν για την παραγωγή PWM κυματομορφών τάσης. Από αυτούς ο Timer/Counter1 (TCNT1) έχει δύο εξόδους PWM και μπορεί να ρυθμιστεί ως 8-bit ή ως 16-bit. Η μία PWM έξοδος συμβολίζεται ως OC1A και μπορεί να συνδεθεί στον ακροδέκτη PB1 ενώ η άλλη PWM έξοδος συμβολίζεται ως OC1B και μπορεί να συνδεθεί στον ακροδέκτη PB2, Οι υπόλοιποι Timers/Counters του ATmega328PB έχουν παρόμοιο τρόπο λειτουργίας.

Εδώ θα εξεταστεί ένας τρόπος λειτουργίας με τον οποίο παράγεται μια PWM κυματομορφή τάσης υψηλής συχνότητας. Στη λειτουργία αυτή (Fast PWM Mode) ο Timer/Counter1 (TCNT1) αυξάνεται ξεκινώντας από την τιμή BOTTOM και όταν φτάσει την τιμή TOP τότε παίρνει ξανά την τιμή BOTTOM και η διαδικασία επαναλαμβάνεται.

Η κυματομορφή εξόδου θα είναι παλμοί με σταθερή **συχνότητα (f_{PWM})** η τιμή της οποίας εξαρτάται από τη συχνότητα του ρολογιού του συστήματος (f_{clk}) και την αρχικοποίηση του prescaler όπως προκύπτει από τον παρακάτω τύπο:

$$f_{PWM} = \frac{f_{clk}}{N \cdot (1 + TOP)}$$

Η μεταβλητή N αντιπροσωπεύει την τιμή του prescaler (1, 8, 64, 256 ή 1024).

Το Duty Cycle ρυθμίζεται, δια μέσω της τιμής του καταχωρητή OCR1A. Η χαμηλότερη τιμή για τον καταχωρητή OCR1A είναι η BOTTOM, όπου η έξοδος θα είναι ένας εξαιρετικά μικρός παλμός στην αρχή κάθε περιόδου και η υψηλότερη τιμή είναι η TOP όπου η έξοδος θα είναι σταθερά υψηλή. Το Low byte του OCR1AL έχει διεύθυνση 0x88 και το Hi byte του OCR1AH έχει διεύθυνση 0x89.

Στην ανάστροφη PWM λειτουργία, οι κυματομορφές εξόδου είναι αντεστραμμένες.

Τα flags COM1A0 και COM1A1 του καταχωρητή TCCR1A, χρησιμεύουν για να καθοριστεί εάν η PWM έξοδος OC1A του Timer1 θα συνδεθεί ή όχι στον ακροδέκτη PB1, καθώς επίσης και η μέθοδος παραγωγής της PWM κυματομορφής σε αυτή την έξοδο. Για παραγωγή της μη ανάστροφης PWM κυματομορφής που περιεγράφηκε παραπάνω οι τιμές των (COM1A0, COM1A1) είναι (0, 1) ενώ για ανάστροφη λειτουργία οι τιμές είναι (1,1). Αντίστοιχα τα flags COM1B0 και COM1B1 χρησιμεύουν για να καθοριστεί εάν η PWM έξοδος OC1B του Timer1 θα συνδεθεί ή όχι στον ακροδέκτη PB2, καθώς επίσης και η μέθοδος παραγωγής της PWM κυματομορφής σε αυτή την έξοδο.

Για όλους τους υπολογισμούς η τιμή BOTTOM θα είναι το μηδέν και η τιμή TOP θα δίνεται από τον Πίνακα 3.2 που παρουσιάστηκε παραπάνω.

Παράδειγμα 3.1

Στη συνέχεια παρουσιάζεται ένα παράδειγμα παραγωγής παλμών PWM στον ακροδέκτη PB1, σε γλώσσα προγραμματισμού C, για την εκπαιδευτική κάρτα ntuAboard_G1. Ο κώδικας αρχικοποιεί κατάλληλα τον χρονιστή TMR1A για να λειτουργεί σε Fast PWM Mode, με μη ανάστροφη λειτουργία(non-inverting mode) και με τιμή 8 για τον prescaler. Ο TMR1B δεν χρησιμοποιείται. Στον ακροδέκτη PB1 είναι συνδεδεμένο ένα LED. Η ρουτίνα main() αυξομειώνει συνεχώς το Duty Cycle της PWM κυματομορφής αυξομειώνοντας αντίστοιχα την φωτεινότητα του LED που είναι συνδεδεμένο στον ακροδέκτη PB1.

```
#define F_CPU 16000000UL
#include "avr/io.h"
#include <util/delay.h>

int main ()
{
    unsigned char duty;

    //set TMR1A in fast PWM 8 bit mode with non-inverted output
    //prescale=8
    TCCR1A = (1<<WGM10) | (1<<COM1A1);
    TCCR1B = (1<<WGM12) | (1<<CS11);

    DDRB |= 0b00111111;    //set PB5-PB0 pins as output

    while (1)
    {
        for(duty=0; duty<255; duty++)
        {
            OCR1A=duty; //increase the LED2 light intensity
            _delay_ms(10);
        }
        for(duty=255; duty>1; duty--)
        {
            OCR1A=duty; //increase the LED2 light intensity
            _delay_ms(10);
        }
    }
}
```

Μετατροπέας αναλογικής σε ψηφιακή μορφή ADC.

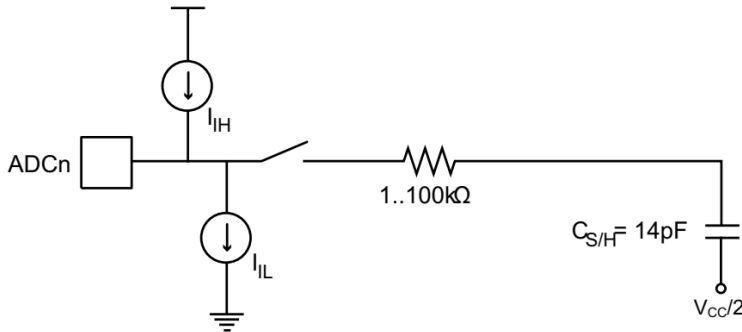
Βασικό περιφερειακό των μικροελεγκτών είναι ο μετατροπέας από Αναλογική σε Ψηφιακή μορφή (ADC). Αν V_{in} είναι η αναλογική τάση στην εισόδου του ADC με ανάλυση n-bit, V_{REF} μία τάση αναφοράς βάση της οποίας γίνεται η μετατροπή και $V_{in} \leq V_{REF}$, τότε η ψηφιακή έξοδος του ADC μετατροπέα είναι:

$$ADC = \frac{V_{in} \cdot 2^n}{V_{REF}}$$

Στην παραπάνω σχέση όλες η ποσότητες είναι ακέραιοι θετικοί αριθμοί.

Αναλογικό κύκλωμα εισόδου

Το κύκλωμα αναλογικής εισόδου του ADC απεικονίζεται στο παρακάτω σχήμα:



Σχήμα 3.2: Κύκλωμα αναλογικής εισόδου στον ADC

Μια αναλογική πηγή που εφαρμόζεται στον ακροδέκτη ADCn επηρεάζεται από τη χωρητικότητα του ακροδέκτη αυτού και από το ρεύμα διαρροής του. Όταν επιλεγεί το κανάλι αυτό για μετατροπή τότε πρέπει το σήμα του να φορτίσει τον πυκνωτή S/H δια μέσω της αντίστασης σειράς (Ολική αντίσταση στη διαδρομή εισόδου).

Το ADC είναι βελτιστοποιημένο για αναλογικά σήματα εισόδου με σύνθετη αντίσταση μέχρι 10 KΩ, όπου ο χρόνος δειγματοληψίας είναι αμελητέος. Εάν χρησιμοποιηθεί πηγή με υψηλότερη τιμή αντίστασης τότε ο χρόνος δειγματοληψίας αυξάνεται και εξαρτάται από το χρόνο που απαιτείται για να φορτιστεί ο πυκνωτής S/H.

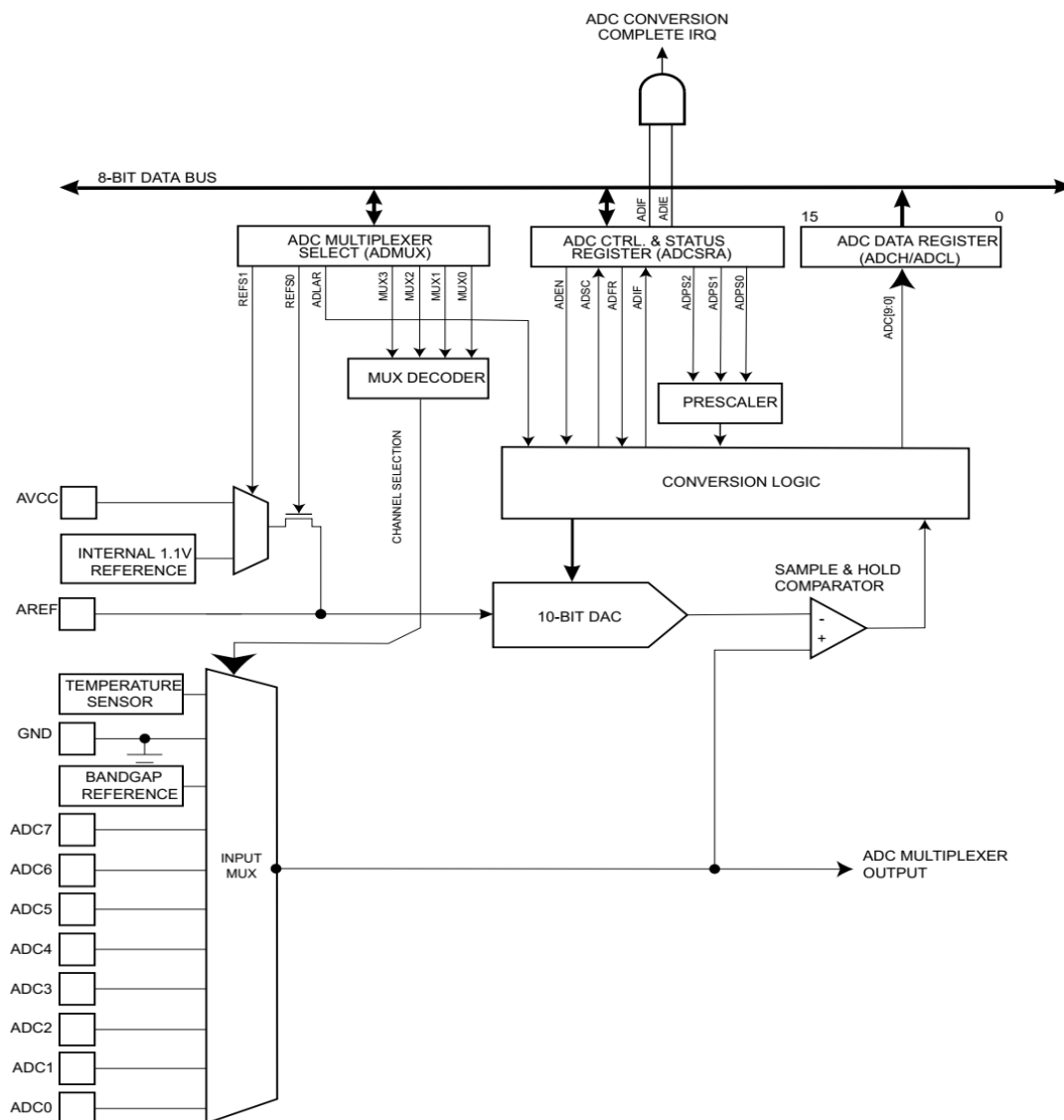
Σε κανονική λειτουργία ο χρόνος δειγματοληψίας(sampling time) είναι 1,5 κύκλος ρολογιού του ADC.

AVR ATMega328PB ADC

Ο μικροελεγκτής AVR ATMega328PB διαθέτει έναν ADC, ανάλυσης n=10 bit, που βασίζεται στη μέθοδο των διαδοχικών προσεγγίσεων.

Το κύκλωμα διαδοχικής προσέγγισης του ADC απαιτεί συχνότητα ρολογιού μεταξύ 50 kHz και 200 kHz για μέγιστη ανάλυση. Εάν απαιτείται ανάλυση χαμηλότερη από 10 bit, η συχνότητα ρολογιού εισόδου του ADC μπορεί να είναι υψηλότερη από 200 KHz.

Στο παρακάτω σχήμα φαίνεται το μπλοκ διάγραμμά του ADC μετατροπέα:



Σχήμα 3.3: Μπλοκ διάγραμμα του ADC

Αναλογικός πολυπλέκτης του ADC

Ο ADC είναι συνδεδεμένος με ένα αναλογικό πολυπλέκτη 16 εισόδων. Το κανάλι αναλογικής εισόδου που είναι κάθε φορά συνδεδεμένο στο κύκλωμα μετατροπής του ADC επιλέγεται από τα bit MUX[3:0] του καταχωρητή ADMUX, όπως φαίνεται παρακάτω:

ADMUX (Διεύθυνση: 0x7C). Multiplexer Selection Register

Bit	7	6	5	4	3	2	1	0
	REFSn	REFSn	ADLAR		MUXn	MUXn	MUXn	MUXn
Access	R/W	R/W	R/W		R/W	R/W	R/W	R/W
Reset	0	0	0		0	0	0	0

Σχήμα 3.3: Καταχωρητής ADMUX

MUX[3:0]	ΚΑΝΑΛΙ ΕΙΣΟΔΟΥ
0000	ADC0
0001	ADC1
0010	ADC2
0011	ADC3
0100	ADC4
0101	ADC5
0110	ADC6
0111	ADC7
1000	Temperature sensor
1001	Reserved
1010	Reserved
1011	Reserved
1100	Reserved
1101	Reserved
1110	1.1V (VBG)
1111	0V (GND)

Πίνακας 3.3: Επιλογή αναλογικών εισόδων του ADC

Στον παρακάτω πίνακα εμφανίζονται οι ακροδέκτες του μικροελεγκτή AVR ATMega328PB που μπορούν να δεχτούν αναλογικά σήματα και τα κανάλια του ADC που αντιστοιχούν. Όσοι από αυτούς τους ακροδέκτες θα χρησιμοποιηθούν από τον ADC πρέπει να οριστούν ως είσοδοι, κάνοντας χρήση του αντίστοιχου καταχωρητή DDRx.

ΑΚΡΟΔΕΚΤΗΣ	ΚΑΝΑΛΙ ΤΟΥ ADC
PC0	ADC0
PC1	ADC1
PC2	ADC2
PC3	ADC3
PC4	ADC4
PC5	ADC5
PE2	ADC6
PE3	ADC7

Πίνακας 3.4: Αναλογικές εισοδοι του ADC

Τα bit REFS[1:0] χρησιμοποιούνται για την επιλογή του V_{REF} σύμφωνα με τον επόμενο πίνακα:

REFS[1:0]	Voltage reference selection
00	AREF, internal V_{REF} turned OFF
01	AVCC with external capacitor at AREF pin
10	Reserved
11	Internal 1.1V voltage reference with external capacitor at AREF pin

Πίνακας 3.5: Επιλογή του ADC V_{REF}

Έλεγχος της λειτουργίας του ADC

Ο έλεγχος της λειτουργίας του ADC γίνεται δια μέσω των καταχωρητών ADCSRA και ADCSRB. Ο καταχωρητής ADCSRA φαίνεται στο παρακάτω σχήμα:

ADCSRA (Διεύθυνση: 0x7A). ADC Control and Status Register A

Bit	7	6	5	4	3	2	1	0
	ADEN	ADSC	ADATE	ADIF	ADIE	ADPSn	ADPSn	ADPSn
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Σχήμα 3.4: Καταχωρητής ADCSRA

Στη συνέχεια παρατίθενται οι λειτουργίες αυτού του καταχωρητή:

ADEN: ADC Enable

Όταν η σημαία αυτή τεθεί τότε ο ADC ενεργοποιείται.

ADSC: ADC Start Conversion

Στη λειτουργία μεμονωμένης μετατροπής (Single Conversion mode), η σημαία αυτή πρέπει να τεθεί για να ξεκινήσει μία μετατροπή. Στη λειτουργία Free Running, η σημαία αυτή πρέπει να τεθεί για να ξεκινήσει η πρώτη μετατροπή. Η πρώτη μετατροπή διαρκεί 25 κύκλους ρολογιού του ADC ενώ οι επόμενες μετατροπές διαρκούν 13 κύκλους ρολογιού. Η πρώτη μετατροπή εκτελεί την αρχικοποίηση του ADC.

ADATE: ADC Auto Trigger Enable

Όταν η σημαία αυτή τεθεί, ενεργοποιείται η λειτουργία Auto Triggering του ADC. Το ADC θα ξεκινήσει αυτόματα μια μετατροπή σε μια θετική ακμή του επιλεγμένου σήματος σκανδαλισμού. Η πηγή σκανδαλισμού επιλέγεται ρυθμίζοντας τα bit ADTS[2:0] στον καταχωρητή ADCSRB.

ADIF: ADC Interrupt Flag

Η σημαία αυτή τίθεται αυτόματα όταν ολοκληρωθεί μια μετατροπή και ενημερωθούν οι καταχωρητές δεδομένων. Αν η τιμή της σημαίας ADIE καθώς και της σημαίας I στον SREG είναι 1 τότε θα προκληθεί διακοπή ολοκλήρωσης μετατροπής του ADC. Το ADIF μηδενίζεται από το υλικό κατά την εκτέλεση του αντίστοιχου διανύσματος χειρισμού διακοπών. Εναλλακτικά, το ADIF μηδενίζεται από τον κώδικα γράφοντας σε αυτό λογικό 1.

ADIE: ADC Interrupt Enable

Αν η τιμή της σημαίας ADIE καθώς και της σημαίας I στον SREG είναι 1 τότε θα προκληθεί διακοπή ολοκλήρωσης μετατροπής του ADC.

ADPS[2:0]: ADC Prescaler Select bits

Τα bit αυτά θέτουν ένα συντελεστή διαίρεσης μεταξύ του κεντρικού ρολογιού και της συχνότητας λειτουργίας του ADC όπως φαίνεται στον επόμενο πίνακα:

ADPS[2:0]	Division Factor
000	2
001	2
010	4
011	8
100	16
101	32
110	64
111	128

Πίνακας 3.6: ADC Prescaler Select bits

Το κύκλωμα διαδοχικής προσέγγισης του ADC, για μέγιστη ανάλυση 10 Bits, απαιτεί συχνότητα ρολογιού εισόδου μεταξύ 50 KHz και 200 KHz.

Ο καταχωρητής ADCSRB φαίνεται στο παρακάτω σχήμα:

ADCSRB (Διεύθυνση: 0x7B). ADC Control and Status Register B

Bit	7	6	5	4	3	2	1	0
		ACME				ADTSn	ADTSn	ADTSn
Access		R/W				R/W	R/W	R/W
Reset		0				0	0	0

Σχήμα 3.5: Καταχωρητής ADCSRB

Στη συνέχεια παρατίθενται οι λειτουργίες αυτού του καταχωρητή:

ACME: Analog Comparator Multiplexer Enable

Χρησιμοποιείται για την επιλογή της αναλογικής εισόδου στον analog comparator, ο οποίος είναι μια περιφερειακή συσκευή, ενσωματωμένη στον μικροελεγκτή ATmega328PB.

ADTSn [2:0]: ADC Auto Trigger Source

Όταν η σημαία ADATE του καταχωρητή ADCSRA είναι 1 τότε ενεργοποιείται η λειτουργία Auto Triggering του ADC. Το ADC θα ξεκινάει αυτόματα μια μετατροπή στη θετική ακμή του σήματος σκανδαλισμού το οποίο επιλέγεται σύμφωνα με τον παρακάτω πίνακα:

ADTS[2:0]	Trigger Source
000	Free Running mode
001	Analog Comparator
010	External Interrupt Request 0
011	Timer/Counter0 Compare Match A
100	Timer/Counter0 Overflow
101	Timer/Counter1 Compare Match B
110	Timer/Counter1 Overflow
111	Timer/Counter1 Capture Event

Πίνακας 3.7: ADC Auto Trigger Source

Καταχωρητές δεδομένων του ADC

Το αποτέλεσμα του ADC έχει μήκος 10-bit και παρουσιάζεται στους καταχωρητές δεδομένων ADCH (high byte) και ADCL (low byte). Από προεπιλογή, το αποτέλεσμα παρουσιάζεται δεξιά προσαρμοσμένο(right adjusted), αλλά μπορεί προαιρετικά να τεθεί η σημαία ADLAR του καταχωρητή ADMUX και να παρουσιαστεί αριστερά προσαρμοσμένο (left adjusted).

Κατά την ανάγνωση του ADCL, οι καταχωρητές δεδομένων του ADC δεν ενημερώνονται μέχρι να διαβαστεί το ADCH. Εάν το αποτέλεσμα είναι αριστερά προσαρμοσμένο(Left adjusted) και δεν απαιτείται ακρίβεια μεγαλύτερη από 8 bit, αρκεί να διαβαστεί ο καταχωρητής ADCH. Σε διαφορετική περίπτωση, πρέπει πρώτα να διαβαστεί το ADCL και μετά το ADCH.

Οι καταχωρητές δεδομένων του ADC, ADCL και ADCH έχουν διευθύνσεις 0x78 και 0x79 αντίστοιχα. Παρουσιάζονται στο επόμενο σχήμα:

ADC Data Register Low and High Byte (ADLAR=0, Right adjusted)

Bit	15	14	13	12	11	10	9	8
							ADC9	ADC8
Access							R	R
Reset							0	0

Bit	7	6	5	4	3	2	1	0
	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

ADC Data Register Low and High Byte (ADLAR=1, Left adjusted)

Bit	15	14	13	12	11	10	9	8
	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	ADC1	ADC0						
Access	R	R						
Reset	0	0						

Σχήμα 4.6: Καταχωρητές ADCL και ADCH

DIDR0 (Διεύθυνση: 0x7E). Digital Input Disable Register 0

Bit	7	6	5	4	3	2	1	0
	ADC7D	ADC6D	ADC5D	ADC4D	ADC3D	ADC2D	ADC1D	ADC0D
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Σχήμα 3.7: Καταχωρητής DIDR0

Όταν ένα bit αυτού του καταχωρητή τίθεται, τότε ο απομονωτής του αντίστοιχου ακροδέκτη του ADC απενεργοποιείται. Εάν ένα αναλογικό σήμα εφαρμόζεται σε έναν από του ακροδέκτες ADC[7:0] και η ψηφιακή είσοδος από αυτόν τον ακροδέκτη δεν χρειάζεται τότε το αντίστοιχο bit ADCxD θα πρέπει να τεθεί για να μειωθεί η κατανάλωση ενέργειας στον απομονωτή της ψηφιακής εισόδου.

Παραγωγή αναλογικών τάσεων στο ntuAboard_G1

Η κάρτα ntuAboard_G1 διαθέτει κυκλώματα για την παραγωγή αναλογικών τάσεων, οι οποίες μπορούν να συνδεθούν στις εισόδους 0 έως 3 του ADC μετατροπέα. Ποιο συγκεκριμένα η κάρτα ntuAboard_G1 διαθέτει 4 ποτενσιόμετρα, 3 αναλογικά φίλτρα τα οποία παράγουν μεταβαλλόμενες DC τάσεις από τις PWM εξόδους των χρονιστών και ένα μετατροπέα ψηφιακού σήματος σε αναλογικό(DAC).

Τα κυκλώματα που αναφέρθηκαν απεικονίζονται στο παρακάτω σχήμα:



Σχήμα 3.4: Κυκλώματα για την παραγωγή αναλογικών τάσεων στην κάρτα ntuAboard_G1.

Η σύνδεση καθεμιάς από τις πρώτες 4 αναλογικές εισόδους του ADC με τις αναλογικές τάσεις που παράγονται στην κάρτα ntuAboard_G1 μπορεί να επιτευχθεί χρησιμοποιώντας βραχυκυκλωτήρες:

- Η αναλογική είσοδος 0 του ADC μπορεί να συνδεθεί είτε με το ποτενσιόμετρο POT1 είτε με το αναλογικό φίλτρο PD6_PWM.
- Η αναλογική είσοδος 1 του ADC μπορεί να συνδεθεί είτε με το ποτενσιόμετρο POT2 είτε με το αναλογικό φίλτρο PB1_PWM.
- Η αναλογική είσοδος 2 του ADC μπορεί να συνδεθεί είτε με το ποτενσιόμετρο POT3 είτε με το αναλογικό φίλτρο PD3_PWM.
- Η αναλογική είσοδος 3 του ADC μπορεί να συνδεθεί είτε με το ποτενσιόμετρο POT4 είτε με την έξοδο του DAC.

Να σημειωθεί ότι για να μπορέσει να λειτουργήσει το αναλογικό φίλτρο στο PD6_PWM πρέπει να συνδεθεί ένας βραχυκυκλωτήρας στον κονέκτορα J3. Παρομοίως το φίλτρο PB1_PWM απαιτεί ένα βραχυκυκλωτήρα στον κονέκτορα J5 και το φίλτρο PD3_PWM απαιτεί ένα βραχυκυκλωτήρα στον κονέκτορα J7.

Για παράδειγμα στο σχήμα 4.8 η αναλογική είσοδος A0 έχει συνδεθεί στο ποτενσιόμετρο POT1, η αναλογική είσοδος A1 έχει συνδεθεί στο PB1_PWM φίλτρο και οι αναλογικές εισόδους A2 και A3 παραμένουν ασύνδετες.

Για να λειτουργήσουν σωστά οι αναλογικές εισόδους της PORTC πρέπει να αποσυνδεθεί η θύρα αυτή από τα led δια μέσω του DIP switch.

Παράδειγμα 3.2

Στο παρακάτω παράδειγμα ο ADC έχει ρυθμιστεί έτσι ώστε να διαβάζει συνεχώς την αναλογική είσοδο 0. Το αποτέλεσμα της μετατροπής είναι αριστερά προσαρμοσμένο (Left adjusted) και τα 8 σημαντικότερα bit απεικονίζονται στα Led του PORTD.

Η τάση αναφοράς έχει ρυθμιστεί στα 5 volt.

Η συχνότητα του ADC ισούται με 125 KHz και προκύπτει από τη διαίρεση της συχνότητας λειτουργίας του μικροελεγκτή(16MHz) με το συντελεστή 128.

```

.include "m328PBdef.inc"                ;ATmega328P microcontroller definitions

.def temp = r16
.def ADC_L = r21
.def ADC_H = r22

.org 0x00
    jmp reset

.org 0x2A                                ;ADC Conversion Complete Interrupt
    reti
reset:
    ldi temp, high(RAMEND)
    out SPH,temp
    ldi temp, low(RAMEND)
    out SPL,temp

    ldi temp, 0xFF
    out DDRD, temp                       ;Set PORTD as output

    ldi temp, 0x00
    out DDRC, temp                       ;Set PORTC as input

    ; REFSn[1:0]=01 => select Vref=5V, MUXn[4:0]=0000 => select ADC0(pin PC0),
    ; ADLAR=1 => Left adjust the ADC result
    ldi temp, 0b01100000 ;
    sts ADMUX, temp

    ; ADEN=1 => ADC Enable, ADSC=0 => No Conversion,
    ; ADIE=0 => disable adc interrupt, ADPS[2:0]=111 => fADC=16MHz/128=125KHz
    ldi temp, 0b10000111
    sts ADCSRA, temp

Start_conv:
    lds temp, ADCSRA                     ;
    ori temp, (1<<ADSC)                 ; Set ADSC flag of ADCSRA
    sts ADCSRA, temp                     ;

    wait_adc:
    lds temp, ADCSRA                     ;
    sbrc temp,ADSC                       ; Wait until ADSC flag of ADCSRA becomes 0
    rjmp wait_adc                       ;

    lds ADC_L,ADCL                       ; Read ADC result(Left adjusted)
    lds ADC_H,ADCH                       ;

    out PORTD, ADC_H                     ; Output ADCH to PORTD

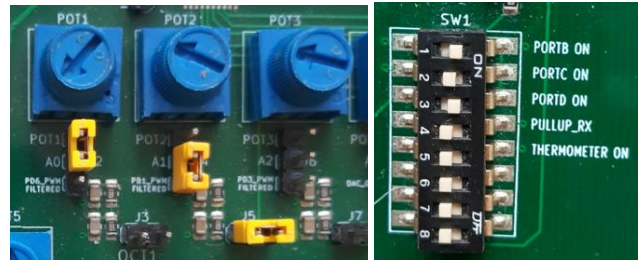
    rjmp Start_conv

```

Τα ζητούμενα της 3^{ης} εργαστηριακής άσκησης

Για την υλοποίηση των επομένων εργασιών συνδέστε την είσοδο A0 του ADC με το POT1, την είσοδο A1 του ADC με το αναλογικό φίλτρο PB1_PWM, και γεφυρώστε το J5 όπως φαίνεται στο διπλανό σχήμα.

Το PORTC περιλαμβάνει αναλογικές εισόδους του ADC, οπότε όταν γίνεται χρήση του ADC τότε τα led που ελέγχονται από το PORTC πρέπει να αποσυνδεθούν, κάνοντας χρήση των dip switches SW1.



Ζήτημα 3.1

Να δημιουργηθεί κώδικας σε γλώσσα assembly, ο οποίος να αρχικοποιεί τον TMR1A σε λειτουργία 8-bit και να παράγει μία PWM κυματομορφή στον ακροδέκτη PB1 με συχνότητα 62500 Hz. Θεωρήστε ότι BOTTOM=0. Στον ακροδέκτη PB1 είναι συνδεδεμένο ένα LED και η φωτεινότητα του μεταβάλλεται εάν μεταβληθεί το Duty Cycle της PWM κυματομορφής. Αρχικά το Duty Cycle ρυθμίζεται σε 50% αποθηκεύεται σε μία μεταβλητή με όνομα DC_VALUE. Στη συνέχεια κάθε φορά που πιέζεται το μπουτόν PD3 το Duty Cycle αυξάνεται κατά 8%. Όταν το Duty Cycle φτάσει τη μέγιστη τιμή 98% η πίεση του μπουτόν PD3 δεν το αυξάνει περαιτέρω. Κάθε φορά που πιέζεται το μπουτόν PD4 το Duty Cycle μειώνεται κατά 8%. Όταν το Duty Cycle φτάσει τη ελάχιστη τιμή 2% η πίεση του μπουτόν PD4 δεν το μειώνει περαιτέρω. Οι τιμές που πρέπει να πάρει ο καταχωρητής OCR1A, για τις διάφορες τιμές του Duty Cycle, να έχουν υπολογιστεί εκ των προτέρων και να έχουν τοποθετηθεί σε ένα πίνακα στη μνήμη του μικροελεγκτή, έτσι ώστε να μη χρειάζεται να υπολογιστούν κατά τη διάρκεια εκτέλεσης του κώδικα.

Ζήτημα 3.2

Να γραφτεί ξανά ο κώδικας του ζητήματος 3.1 σε γλώσσα C, ο οποίος θα παράγει PWM έξοδο στον ακροδέκτη PB1, με συχνότητα 62500 Hz. Για την αύξηση και μείωση του Duty Cycle επιλέξτε μόνοι σας και χρησιμοποιήστε δύο πλήκτρα που είναι διαθέσιμα και όχι τα PD3 και PD4 που είναι δεσμευμένα για τα led.

Κάθε 100 mSec (μικρές αποκλίσεις είναι αποδεκτές), ο ADC(ADCH: ADCL) θα διαβάσει την DC τάση που παράγεται στην έξοδο του αναλογικού φίλτρου PB1_PWM με ακρίβεια 10 bit.

Να υπολογίζεται η μέση τιμή του ADC αθροίζοντας 16 διαδοχικές μετρήσεις και διαιρώντας δια 16. Προκειμένου να επιτευχθεί η ταχύτερη εκτέλεση του κώδικα, η διαίρεση θα υλοποιείται με ολίσθηση των bit του αθροίσματος κατά 4 θέσεις. Ανάλογα με την τιμή μέτρησης του ADC (ADCH: ADCL) θα ανάβει ένα από τα led που είναι συνδεδεμένα στη θύρα PORTD σύμφωνα με τον παρακάτω πίνακα:

Τιμή μέτρησης του ADC(ADCvalue)	LED ON
$0 \leq \text{ADCvalue} \leq 200$	PD0
$200 \leq \text{ADCvalue} \leq 400$	PD1
$400 \leq \text{ADCvalue} \leq 600$	PD2
$600 \leq \text{ADCvalue} \leq 800$	PD3
$800 \leq \text{ADCvalue}$	PD4

Ζήτημα 3.3

Να γραφτεί ξανά ο κώδικας του ζητήματος 3.1 σε γλώσσα C.

Επιπλέον, για τη μεταβολή της φωτεινότητας του led θα υπάρχουν δυο τρόποι λειτουργίας, το mode1 και το mode2. Στο mode1 η φωτεινότητα(Duty Cycle) θα αποθηκεύεται σε μια μεταβλητή με όνομα DC_VALUE και το πάτημα του μπουτόν PD1 θα αυξάνει τη φωτεινότητα ενώ το πάτημα του μπουτόν PD2 θα μειώνει τη φωτεινότητα.

Στο mode2 η φωτεινότητα(Duty Cycle) θα ελέγχεται από το ποτενσιόμετρο POT1.

Εάν πατηθεί το μπουτόν PD6 τότε θα επιλέγεται το mode1 ενώ αν πατηθεί το μπουτόν PD7 τότε θα επιλέγεται το mode2.



Ε.Μ.Π. - ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΑΚΑΔ. ΕΤΟΣ 2024-2025

ΑΘΗΝΑ, 25 Οκτωβρίου 2024

4^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Χρήση Οθόνης 2×16 Χαρακτήρων στον AVR

Εξέταση – Επίδειξη: Παρασκευή 1/11/2024
Προθεσμία για παράδοση Έκθεσης: Τρίτη 5/11/2023 (23:59)

Αλφαριθμητική Οθόνη 2×16 Χαρακτήρων

Η οθόνη είναι υγρού κρυστάλλου (LCD) με χαμηλή κατανάλωση και έχει ενσωματωμένο ένα μικροελεγκτή. Ο ενσωματωμένος μικροελεγκτής εμπεριέχει δύο μνήμες για τη δημιουργία χαρακτήρων: την CG-ROM (Character Generation ROM) και την CG-RAM (Character Generation RAM). Επίσης έχει μία μνήμη RAM στην οποία κρατούνται προσωρινά οι χαρακτήρες που απεικονίζονται κάθε φορά στην οθόνη την DD_RAM (Display Data RAM).

Ακροδέκτες της οθόνης

Στη συνέχεια παρατίθενται οι ακροδέκτες της οθόνης και οι αντίστοιχες λειτουργίες τους:

DB[0:7]	Τρικατάστατος αμφίδρομος διάδρομος δεδομένων 8-γραμμών, με τον οποίο η οθόνη ανταλλάσσει δεδομένα με τον εξωτερικό μικροεπεξεργαστή. Υπάρχει δυνατότητα χρήσης μόνο 4 γραμμών, οπότε η ανταλλαγή των δεδομένων γίνεται σε δύο φάσεις, δια μέσω των ακροδεκτών DB[7:4] ενώ οι ακροδέκτες DB[0:3] δεν χρησιμοποιούνται. Τα 4 περισσότερο σημαντικά bit πρέπει να μεταφέρονται πρώτα.
E'	Σήμα που ενεργοποιεί την οθόνη. Η ενεργοποίηση γίνεται Αν $E'=0$.
R/W'	Σήμα που καθορίζει αν πρόκειται να γίνει εγγραφή ή διάβασμα από την οθόνη ($R/W'=0$ εγγραφή, $R/W'=1$ διάβασμα).
RS	Το σήμα RS καθορίζει αν το περιεχόμενο των DB[0:7] είναι διεύθυνση/εντολή ή αν είναι δεδομένα που πρόκειται να διαβαστούν ή να εγγραφούν ($RS=0$ διεύθυνση/εντολή, $RS=1$ δεδομένα).
V0	Ρυθμίζει τη φωτεινότητα της οθόνης.
VDD	Συνδέεται με την τάση τροφοδοσίας (3,3 volt).
VSS	Συνδέεται με τη γη (0 volt).

Καταχωρητές

Ο εσωτερικός μικροελεγκτής της LCD οθόνης ενσωματώνει δύο καταχωρητές των 8-bit, τον καταχωρητή εντολών (instruction register -IR) και τον καταχωρητή δεδομένων (data register- DR).

Ο καταχωρητής IR αποθηκεύει τον κώδικα των εντολών, ή τη διεύθυνση μιας θέσης της μνήμης DD-RAM ή της μνήμης CG-RAM. Στον καταχωρητή IR γίνεται μόνο εγγραφή και όχι ανάγνωση από τον εξωτερικό μικροελεγκτή. Ο καταχωρητής DR αποθηκεύει προσωρινά τα δεδομένα που διαβάζονται ή γράφονται στην DD-RAM ή στην CG-RAM.

Σημαία απασχόλησης (Busy Flag-BF)

Όταν η σημαία απασχόλησης είναι 1, η οθόνη είναι σε κατάσταση εσωτερικής λειτουργίας και δεν γίνεται δεκτή καμία άλλη εντολή. Όταν $RS=0$ και $R/W'=1$, η σημαία απασχόλησης εμφανίζεται στην έξοδο DB7. Η επόμενη εντολή πρέπει να δοθεί, μετά την εξασφάλιση ότι η σημαία απασχόλησης είναι 0.

Μετρητής Διεύθυνση (Address Counter -AC)

Ο μετρητής διεύθυνση (AC) παρέχει διευθύνσεις σε αμφότερες τις μνήμες DD-RAM και CG-RAM. Όταν μια διεύθυνση μιας εντολής εγγράφεται στον IR, η διεύθυνση στέλνεται από τον IR στον AC.

Μετά την εγγραφή ή την ανάγνωση από την DD-RAM ή την CG-RAM, η AC αυτόματα αυξάνεται κατά 1 (ή ελαττώνεται κατά 1). Το περιεχόμενο του καταχωρητή AC δίνεται στη έξοδο μέσω των DB[0:6] όταν RS = 0 και R/W' = 1.

RS	R/W	Operation
0	0	IR write as an internal operation (display clear, etc.)
0	1	Read busy flag (DB7) and address counter (DB0 to DB6)
1	0	DR write as an internal operation (DR to DD-RAM or CG-RAM)
1	1	DR read as an internal operation (DD-RAM or CG-RAM to DR)

Πίνακας 4.1: Επιλογή Καταχωρητή

Εσωτερική μνήμη DD-RAM

Η DD-RAM αποθηκεύει τα δεδομένα που πρόκειται να απεικονιστούν στην οθόνη. Η χωρητικότητά της είναι 80 byte. Ο κώδικας της διεύθυνσης μιας θέσης μνήμης DD-RAM είναι των επτά Bit. Οι χαρακτήρες που οι κώδικες τους βρίσκονται στις θέσεις 0x00 έως 0x0F της DD-RAM τοποθετούνται(απεικονίζονται) στη γραμμή 1 της οθόνης, ενώ οι κώδικες που βρίσκονται στις θέσεις 0x40 έως 0x4F απεικονίζονται στη γραμμή 2 όπως φαίνονται στον παρακάτω πίνακα:

Γραμμή_1	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
Γραμμή_2	40	41	42	43	44	45	46	47	48	49	4 ^A	4B	4C	4D	4E	4F

Πίνακας 4.2: Απεικόνιση χαρακτήρων

Αν πρέπει να απεικονιστούν και άλλοι χαρακτήρες που οι κώδικες τους είναι αποθηκευμένοι στη DD-RAM, εκτός από τους 32 χαρακτήρες που αναφέρθηκαν πιο πάνω, τότε πρέπει να γίνει ολίσθηση στο περιεχόμενο της οθόνης δεξιά ή αριστερά.

Εσωτερική μνήμη CG-ROM

Η CG-ROM χρησιμοποιείται για τη δημιουργία εικόνων χαρακτήρων 5×8 κουκίδων από κωδικούς χαρακτήρων των 8 bit. Η αντιστοιχία αριθμών των 8 bit και χαρακτήρων φαίνεται στον ακόλουθο πίνακα.

Upper 4 bit Lower 4 bit		LLLL	LLLH	LLHL	LLHH	LHLL	LHLH	LHHL	LHHH	HLLL	HLLH	HLHL	HLHH	HHLL	HHLH	HHHL	HHHH
LLLL	CG RAM (1)																
LLLH	CG RAM (2)																
LLHL	CG RAM (3)																
LLHH	CG RAM (4)																
LHLL	CG RAM (5)																
LHLH	CG RAM (6)																
LHHL	CG RAM (7)																
LHHH	CG RAM (8)																
HLLL	CG RAM (1)																
HLLH	CG RAM (2)																
HLHL	CG RAM (3)																
HLHH	CG RAM (4)																
HHLL	CG RAM (5)																
HHLH	CG RAM (6)																
HHHL	CG RAM (7)																
HHHH	CG RAM (8)																

Πίνακας 4.3: Αντιστοιχία αριθμών των 8 bit και χαρακτήρων.

Εντολές της οθόνης

Επειδή δεν υπάρχει δυνατότητα ανάγνωσης της σημαίας BUSY FLAG για να γνωρίζουμε πότε η οθόνη είναι έτοιμη να δεχτεί νέα εντολή πρέπει να εισάγουμε καθυστέρηση μεταξύ των διαδοχικών εντολών. Ο χρόνος που χρειάζεται για την εκτέλεση κάθε εντολής είναι μεταβλητός και εξαρτάται από την κάθε εντολή. Το σύνολο εντολών της οθόνης περιέχεται στον ακόλουθο πίνακα.

Instruction	Instruction Code										Description	Execution time (fosc=270Khz)
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0		
Clear Display	0	0	0	0	0	0	0	0	0	1	Write "00H" to DDRAM and set DDRAM address to "00H" from AC	1.53ms
Return Home	0	0	0	0	0	0	0	0	1	—	Set DDRAM address to "00H" from AC and return cursor to its original position if shifted. The contents of DDRAM are not changed.	1.53ms
Entry Mode Set	0	0	0	0	0	0	0	1	I/D	SH	Assign cursor moving direction and enable the shift of entire display.	39 μ s
Display ON/OFF Control	0	0	0	0	0	0	1	D	C	B	Set display (D), cursor (C), and blinking of cursor (B) on/off control bit.	39 μ s
Cursor or Display Shift	0	0	0	0	0	1	S/C	R/L	—	—	Set cursor moving and display shift control bit, and the direction, without changing of DDRAM data.	39 μ s
Function Set	0	0	0	0	1	DL	N	F	—	—	Set interface data length (DL:8-bit/4-bit), numbers of display line (N:2-line/1-line)and, display font type (F:5×11 dots/5×8 dots)	39 μ s
Set CGRAM Address	0	0	0	1	AC5	AC4	AC3	AC2	AC1	AC0	Set CGRAM address in address counter.	39 μ s
Set DDRAM Address	0	0	1	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Set DDRAM address in address counter.	39 μ s
Read Busy Flag and Address	0	1	BF	AC6	AC5	AC4	AC3	AC2	AC1	AC0	Whether during internal operation or not can be known by reading BF. The contents of address counter can also be read.	0 μ s
Write Data to RAM	1	0	D7	D6	D5	D4	D3	D2	D1	D0	Write data into internal RAM (DDRAM/CGRAM).	43 μ s
Read Data from RAM	1	1	D7	D6	D5	D4	D3	D2	D1	D0	Read data from internal RAM (DDRAM/CGRAM).	43 μ s

* "—" : don't care

Πίνακας 4.4: Το σύνολο εντολών της οθόνης

Παρατήρηση: Πειραματικά έχει φανεί ότι οι οθόνες που χρησιμοποιούνται στην κάρτα ntuAboard G1 δεν μπορούν να ανταποκριθούν στους χρόνους που φαίνονται στον παραπάνω πίνακα. Για το σκοπό αυτό, στα τμήματα του κώδικα που ακολουθούν, έχουν χρησιμοποιηθεί μεγαλύτερες καθυστερήσεις. Αυτό πρέπει να ληφθεί υπόψιν και στον κώδικα που θα δημιουργήσετε.

Τέλος για να γίνει δεκτή μια εντολή από τον ελεγκτή πρέπει να τηρούνται οι χρόνοι του παρακάτω πίνακα:

Ta=25°C, VDD=5.0V

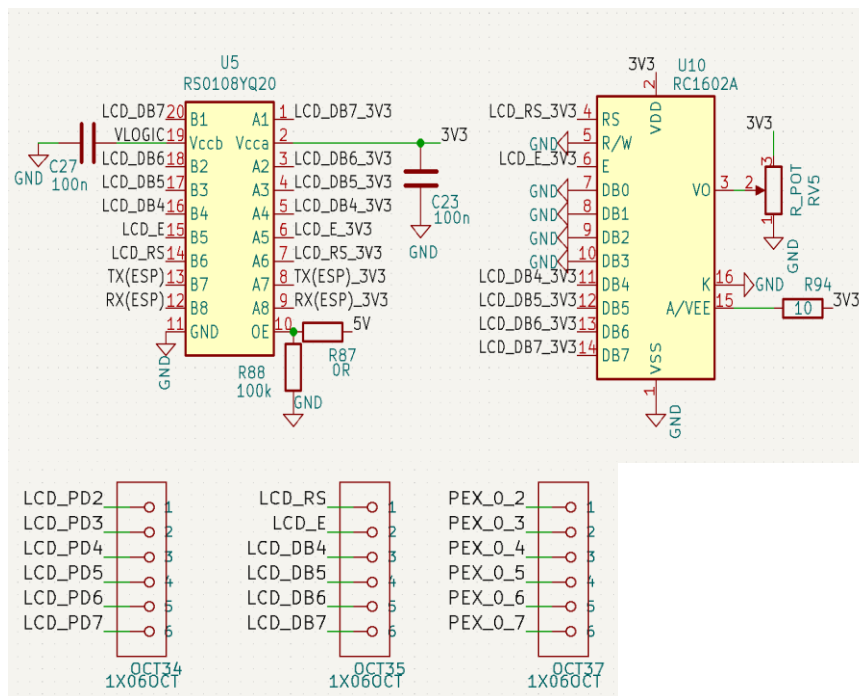
Item	Symbol	Min	Typ	Max	Unit
Enable cycle time	T _C	1200	—	—	ns
Enable pulse width	T _{PW}	140	—	—	ns
Enable rise/fall time	T _R , T _F	—	—	25	ns
Address set-up time (RS, R/W to E)	t _{AS}	0	—	—	ns
Address hold time	t _{AH}	10	—	—	ns
Data set-up time	t _{DSW}	40	—	—	ns
Data hold time	t _H	10	—	—	ns

Πίνακας 4.5: Χρονισμός εντολών οθόνης

Χρήση της οθόνης LCD 2x16 της κάρτας ntuAboard_G1.

Η κάρτα ntuAboard_G1 διαθέτει μία LCD οθόνη 2x16 χαρακτήρων. Μεταξύ της οθόνης και του μικροελεγκτή παρεμβάλλεται το ολοκληρωμένο RS0108YQ20 (8-Bit Bidirectional Voltage Level Translator) για προσαρμογή του επιπέδου της τάσης του μικροελεγκτή (5Volt) με το επίπεδο της τάσης της οθόνης (3.3 Volt).

Η συνδεσμολογία της οθόνης LCD με τον Μικροελεγκτή ATmega328PB φαίνεται στο παρακάτω σχήμα:



Σχήμα 4.1: Συνδεσμολογία της οθόνης LCD

Κάνοντας χρήση των κονεκτόρων OCT34, OCT35 και OCT37 μπορούμε να επιλέξουμε εάν οι ακροδέκτες ελέγχου της LCD οθόνης (OCT35) θα συνδεθούν στο PORTD του ATmega328PB (OCT34) ή στο PORT EXPANDER (OCT37). Ανάλογα με τη σύνδεση, οι ακροδέκτες [LCD_PD2 : LCD_PD7] ή οι ακροδέκτες [PEX_0_7 :

PEX_0_2] πρέπει να είναι ρυθμισμένοι για έξοδο. Η επικοινωνία μεταξύ οθόνης και του μικροελεγκτή γίνεται με λέξεις των 4 bit.

Κώδικας επικοινωνίας μεταξύ μικροελεγκτή και οθόνης

Αρχικά χρειαζόμαστε μια ρουτίνα που θα μεταφέρει τα δύο τμήματα των 4 bit κάθε εντολής. Η ρουτίνα θα πρέπει να αφήνει ανεπηρέαστους τους ακροδέκτες που επιλέγουν μεταξύ καταχωρητή εντολών και καταχωρητή δεδομένων, ώστε να μπορεί να χρησιμοποιηθεί και για τις δύο λειτουργίες.

Ρουτίνα: write_2_nibbles

write_2_nibbles:

```
push r24                ; save r24(LCD_Data)

in r25 ,PIND            ; read PIND

andi r25 ,0x0f          ;
andi r24 ,0xf0          ; r24[3:0] Holds previus PORTD[3:0]
add r24 ,r25            ; r24[7:4] <-- LCD_Data_High_Byte
out PORTD ,r24          ;

sbi PORTD ,PD3          ; Enable Pulse
nop
nop
cbi PORTD ,PD3

pop r24                 ; Recover r24(LCD_Data)
swap r24                ;
andi r24 ,0xf0          ; r24[3:0] Holds previus PORTD[3:0]
add r24 ,r25            ; r24[7:4] <-- LCD_Data_Low_Byte
out PORTD ,r24

sbi PORTD ,PD3          ; Enable Pulse
nop
nop
cbi PORTD ,PD3

ret
```

Στη συνέχεια, με βάση την προηγούμενη ρουτίνα θα δημιουργήσουμε δύο άλλες. Η μία θα στέλνει εντολές στην οθόνη και η άλλη δεδομένα.

Ρουτίνα: lcd_data

Αποστολή ενός byte δεδομένων στον ελεγκτή της οθόνης lcd. Ο ελεγκτής πρέπει να βρίσκεται σε 4 bit mode. Το byte που μεταδίδεται είναι αποθηκευμένο στον καταχωρητή r24

lcd_data:

```
sbi PORTD ,PD2          ; LCD_RS=1(PD2=1), Data
rcall write_2_nibbles    ; send data
ldi r24 ,250            ;
ldi r25 ,0              ; Wait 250uSec
rcall wait_usec
ret
```

Ρουτίνα: lcd_command

Αποστολή μιας εντολής στον ελεγκτή της οθόνης lcd.

Ο ελεγκτής πρέπει να βρίσκεται σε 4 bit mode.

Η εντολή που μεταδίδεται είναι αποθηκευμένη στον καταχωρητή r24

lcd_command:

```
cbi PORTD ,PD2           ; LCD_RS=0(PD2=0), Instruction  
rcall write_2_nibbles    ; send Instruction  
ldi r24 ,250             ;  
ldi r25 ,0               ; Wait 250uSec  
rcall wait_usec  
ret
```

Ρουτίνα: lcd_clear_disply

Καθαρισμός της οθόνης LCD :

lcd_clear_display:

```
ldi r24 ,0x01             ; clear display command  
rcall lcd_command  
  
ldi r24 ,low(5)           ;  
ldi r25 ,high(5)          ; Wait 5 mSec  
rcall wait_msec          ;  
  
ret
```

Για να μπορεί να χρησιμοποιηθεί οθόνη πρέπει να αρχικοποιηθεί στην επιθυμητή κατάσταση. Όταν η οθόνη τροφοδοτείται με ρεύμα για πρώτη φορά πραγματοποιείται μια εσωτερική αρχικοποίηση και για αυτό απαιτείται χρονική καθυστέρηση 200 ms πριν την τελική αρχικοποίηση.

Κάθε φορά που προγραμματίζουμε τον Μικροελεγκτή, ξεκινάει η εκτέλεση του κώδικα από την αρχή. Η οθόνη όμως βρίσκεται στην κατάσταση που την αφήσαμε την προηγούμενη φορά. Ο κώδικας που θα κάνει την αρχικοποίηση της οθόνης δεν γνωρίζει αν ο ελεγκτής της οθόνης βρίσκεται σε 8 bit mode ή σε 4 bit mode. Για το λόγο αυτό αρχικά στέλνεται 3 φορές η εντολή 0x30 (function set) για 8 bit mode. Αν ο ελεγκτής είναι σε 8 bit mode δεν θα αλλάξει κάτι, αν όμως είναι σε 4 bit mode θα μεταβεί σε 8 bit mode. Στη συνέχεια στέλνεται η εντολή 0x20 για να οδηγηθεί η οθόνη σε 4 bit mode. Μόλις είμαστε βέβαιοι για την μορφή που πρέπει να στέλνουμε τις εντολές μπορούμε να προχωρήσουμε με την αρχικοποίηση.

Ρουτίνα: lcd_init

Αρχικοποίηση και ρυθμίσεις της οθόνης LCD όπως παρουσιάζεται παρακάτω:

DL = 0 4 bit mode

N = 1 2 lines

F = 0 5×8 dots

D = 1 display on

C = 0 cursor off

B = 0 blinking off

I/D = 1 DD-RAM address auto increment

SH = 0 shift of entire display off

lcd_init:

```
ldi r24 ,low(200)      ;
ldi r25 ,high(200)     ; Wait 200 mSec
rcall wait_msec        ;

ldi r24 ,0x30           ; command to switch to 8 bit mode
out PORTD ,r24         ;
sbi PORTD ,PD3         ; Enable Pulse
nop
nop
cbi PORTD ,PD3
ldi r24 ,250           ;
ldi r25 ,0             ; Wait 250uSec
rcall wait_usec        ;

ldi r24 ,0x30           ; command to switch to 8 bit mode
out PORTD ,r24         ;
sbi PORTD ,PD3         ; Enable Pulse
nop
nop
cbi PORTD ,PD3
ldi r24 ,250           ;
ldi r25 ,0             ; Wait 250uSec
rcall wait_usec        ;

ldi r24 ,0x30           ; command to switch to 8 bit mode
out PORTD ,r24         ;
sbi PORTD ,PD3         ; Enable Pulse
nop
nop
cbi PORTD ,PD3
ldi r24 ,250           ;
ldi r25 ,0             ; Wait 250uSec
rcall wait_usec        ;

ldi r24 ,0x20           ; command to switch to 4 bit mode
out PORTD ,r24         ;
sbi PORTD ,PD3         ; Enable Pulse
nop
nop
cbi PORTD ,PD3
ldi r24 ,250           ;
ldi r25 ,0             ; Wait 250uSec
rcall wait_usec        ;

ldi r24 ,0x28           ; 5x8 dots, 2 lines
rcall lcd_command

ldi r24 ,0x0c           ; display on, cursor off
rcall lcd_command
```

```
rcall lcd_clear_display
```

```
ldi r24 ,0x06          ; Increase address, no display shift  
rcall lcd_command      ;  
ret
```

Παράδειγμα

Ο κώδικας που ακολουθεί προξενεί επαναληπτικά:

Καθαρισμό της οθόνης - Καθυστέρηση 1 sec - Απεικόνιση του χαρακτήρα 'Α' στην οθόνη - Καθυστέρηση 1 sec.

```
.include "m328Pbdef.inc" ;ATmega328P microcontroller definition
```

```
.equ PD0=0  
.equ PD1=1  
.equ PD2=2  
.equ PD3=3  
.equ PD4=4  
.equ PD5=5  
.equ PD6=6  
.equ PD7=7
```

main:

```
ldi r24, low(RAMEND)  
out SPL, r24  
ldi r24, high(RAMEND)  
out SPH, r24          ; init stack pointer
```

```
ser r24  
out DDRD, r24         ; set PORTD as output
```

```
clr r24
```

```
rcall lcd_init
```

```
ldi r24, low(100)  
ldi r25, high(100)    ; delay 100 mS  
rcall wait_msec
```

main1:

```
rcall lcd_clear_display
```

```
ldi r24, low(1000)  
ldi r25, high(1000)   ; delay 1 Sec  
rcall wait_msec
```

```
ldi r24, 'A'  
call lcd_data
```

```
ldi r24, low(1000)  
ldi r25, high(1000)   ; delay 1 Sec  
rcall wait_msec  
jmp main1
```

Παραδείγματα ρουτινών χρονοκαθυστερήσεων που χρησιμοποιούνται στα τμήματα κώδικα παραπάνω:

wait_msec:

```
push r24          ; 2 cycles
push r25          ; 2 cycles
ldi r24 , low(999) ; 1 cycle
ldi r25 , high(999) ; 1 cycle
rcall wait_usec   ; 998.375 usec
pop r25           ; 2 cycles
pop r24           ; 2 cycles
nop               ; 1 cycle
nop               ; 1 cycle

sbiw r24 , 1      ; 2 cycles
brne wait_msec    ; 1 or 2 cycles

ret               ; 4 cycles
```

wait_usec:

```
sbiw r24 , 1      ; 2 cycles (2/16 usec)
call delay_8cycles ; 4+8=12 cycles
brne wait_usec    ; 1 or 2 cycles
ret
```

delay_8cycles:

```
nop
nop
nop
```

Τα ζητούμενα της 4^{ης} εργαστηριακής άσκησης

Ζήτημα 4.1

Στην κάρτα ntuAboard_G1 να γίνει η σύνδεση του POT2 με την είσοδο A1 του ADC, μέσω κατάλληλου συνδετήρα(Jumper) και να γραφεί πρόγραμμα **σε assembly** για τον ATmega328PB το οποίο θα ξεκινάει μια μετατροπή του ADC κάθε 1Sec. Ο ADC Θα μετατρέπει την τάση που υπάρχει κάθε φορά την είσοδο A1. Η ανάγνωση των δεδομένων του ADC πρέπει να γίνεται μέσα στην ρουτίνα εξυπηρέτησης της διακοπής ολοκλήρωσης μετατροπής του ADC. Η διακοπή αυτή (ADC) μεταφέρει τον έλεγχο στην διεύθυνση **0x02A**, αν είναι ενεργοποιημένη η αντίστοιχη διακοπή (από το bit ADIE του ADCSRA) καθώς και το flag επίτρεψης όλων των διακοπών. Η τιμή μέτρησης του ADC να μετατρέπεται σε τάση και να εκτυπώνονται στην LCD οθόνη, αρχίζοντας κάθε φορά από τον πρώτο χαρακτήρα της πρώτης γραμμής, με ακρίβεια δύο δεκαδικών ψηφίων (δεν χρειάζεται στρογγυλοποίηση).

Η τάση δίνεται από τον τύπο:

$$V_{IN} = \frac{ADC}{1024} V_{REF}$$

Όπου:

- V_{IN} η τάση στην αναλογική είσοδο A2 του μικροελεγκτή.
- ADC η τιμή που διαβάζεται από τον ADC (10bit, από 0-1023)
- V_{REF} η τάση αναφοράς που έχει οριστεί στα 5V.

Ζήτημα 4.2

Να γραφεί σε γλώσσα C, πρόγραμμα για τον ATmega328PB το οποίο θα ξεκινάει μια ADC μετατροπή, όπως στο Ζήτημα 4.1. Δεν θα κάνει χρήση της διακοπής ολοκλήρωσης μετατροπής του ADC και θα περιμένει μέχρι να ολοκληρωθεί η ADC μετατροπή ελέγχοντας το bit ADSC του ADCSRA το οποίο γίνεται 0 μόλις ολοκληρωθεί η μετατροπή (Polling). Θα εκτυπώνει την τάση στην LCD οθόνη με ακρίβεια δύο δεκαδικών ψηφίων.

Ζήτημα 4.3

Να δημιουργηθεί κώδικας **σε C** για την επιτήρηση ενός χώρου όπου υπάρχει αυξημένος κίνδυνος ύπαρξης μονοξειδίου του άνθρακα (CO). Ο αισθητήρας CO είναι συνδεδεμένος στην αναλογική είσοδο A2 του μικροελεγκτή.

Καθ' όλη την διάρκεια πρέπει να διαβάζεται η τιμή του αισθητήρα ανά 100 ms (μικρές αποκλίσεις είναι αποδεκτές) και να εμφανίζεται μια ένδειξη του επιπέδου του αερίου στα LED PB0-PB5.

Αν οποιαδήποτε στιγμή η συγκέντρωση του CO ξεπεράσει τα 70ppm να τυπώνεται στην LCD το μήνυμα GAS DETECTED και να αναβοσβήνουν τα αντίστοιχα LED στα PB0-PB5 αναλόγως του επιπέδου του αερίου.

Το επίπεδο θα πρέπει να συνεχίζει να διαβάζεται (και να εμφανίζεται στα LED) και αν επανέλθει σε επίπεδο χαμηλότερο των 70ppm τα LEDs να σταματούν να αναβοσβήνουν και να τυπώνεται στην LCD το μήνυμα CLEAR. Είναι καλή πρακτική να διατηρείτε τις ρουτίνες εξυπηρέτησης διακοπών όσο το δυνατόν μικρότερες σε χρονική διάρκεια. Παρόλα αυτά αν το πρόγραμμα σας είναι λειτουργικό δεν θα υπάρξει αρνητική επίπτωση στην βαθμολογία.

Για την μέτρηση της συγκέντρωσης του CO χρησιμοποιείται αισθητήρας του τύπου ULPSM-CO-968-001:

(https://helios.ntua.gr/pluginfile.php/250446/mod_folder/content/0/ULPSM-CO-968-001.pdf?forcedownload=1),

ο οποίος βρίσκεται σε σταθερές συνθήκες θερμοκρασίας (20°C) και υγρασίας (40%). Δίνεται ότι $V_{gas0}=0.1V$ και sensitivity code 129nA/ppm. Για να προσομοιώσετε την έξοδο που θα είχε ο αισθητήρας CO να χρησιμοποιήσετε το ποτενσιόμετρο POT3 στην κάρτα ntuAboard_G1 το οποίο παράγει μια DC τάση στο εύρος 0-5V.



ΑΘΗΝΑ, 1 Νοεμβρίου 2024

5^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Χρήση εξωτερικών Θυρών Επέκτασης στον AVR

Εξέταση – Επίδειξη: Παρασκευή 8/11/2024
Προθεσμία για παράδοση Έκθεσης: Τρίτη 12/11/2024 (23:59)

Διεπαφή TWI (Two wire Serial Interface)

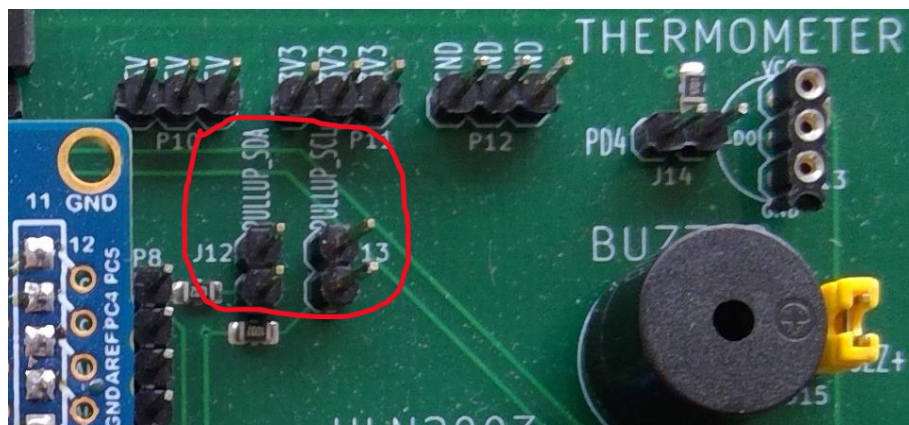
Το πρωτόκολλο TWI επιτρέπει τη διασύνδεση έως και 127 διαφορετικών συσκευών χρησιμοποιώντας μόνο δύο αμφίδρομες γραμμές διαύλου, μία για ρολόι (SCL) και μία για δεδομένα (SDA). Τα μόνα εξωτερικά υλικά που χρειάζονται για την υλοποίηση του διαύλου είναι δύο αντιστάσεις πρόσδεσης (pull-up resistors) για τις γραμμές του διαύλου TWI.

- Όλες οι συσκευές που είναι συνδεδεμένες στο δίαυλο TWI έχουν ατομικές διευθύνσεις.
- Το πρωτόκολλο TWI ενσωματώνει μηχανισμό αποφυγής σύγκρουσης δεδομένων στο δίαυλο.
- Το πρωτόκολλο TWI είναι συμβατό με το πρωτόκολλο I²C της Philips.

Διεπαφές TWI στο ntuAboard_G1

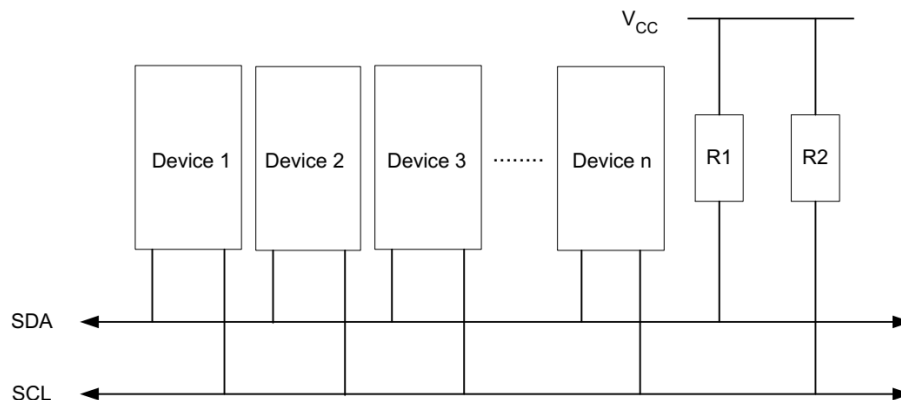
Στο ntuAboard_G1 υπάρχουν δύο διεπαφές TWI, η TWI0 και η TWI1. Οι πληροφορίες που παρουσιάζονται στη συνέχεια αναφέρονται στη διεπαφή TWI0. Ακριβώς τα ίδια ισχύουν και για διεπαφή TWI1.

Οι δυο γραμμές του TWI0 διαύλου συνδέονται στους ακροδέκτες PC4 (SDA) και PC5 (SCL) του ATmega328PB. Οι αντιστάσεις πρόσδεσης (pull-up resistors) έχουν τιμή 10 K Ω και συνδέονται στο δίαυλο με την τοποθέτηση βραχυκυκλωτήρων στους κονέκτορες J12 και J13.



Σχήμα 5.1: Κονέκτορες J12 και J13 για σύνδεση των TWI pull-up resistors

Όταν ο διάυλος TWI είναι ενεργός τότε οι ακροδέκτες PC4 (SDA) και PC5 (SCL) δεν μπορούν να χρησιμοποιηθούν για άλλο σκοπό. Στο παρακάτω σχήμα φαίνεται η συνδεσμολογία του διαύλου TWI



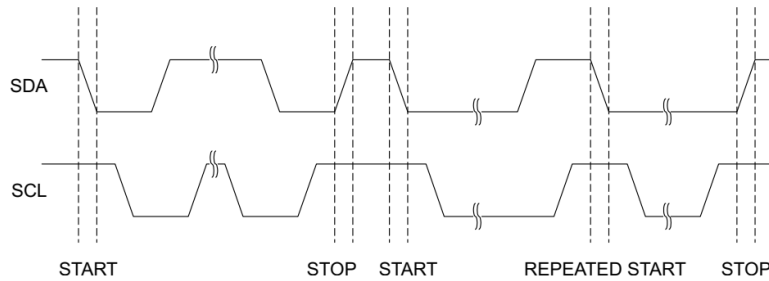
Σχήμα 5.2: Συνδεσμολογία TWI διαύλου.

Ορολογία διαύλου TWI:

- **Master:** Η συσκευή που αρχίζει μια μεταφορά, παράγει το σήμα του ρολογιού (SCL) και τερματίζει τη μεταφορά.
- **Slave:** Η συσκευή που καλείται κάθε φορά από τον Master κάνοντας χρήση της ατομικής διεύθυνσης της.
- **Transmitter:** Η συσκευή που στέλνει δεδομένα στο δίαυλο.
- **Receiver:** Η συσκευή που λαμβάνει δεδομένα από το δίαυλο.
- **Multi-master:** Η δυνατότητα ύπαρξης πολλών Master στο δίαυλο, χωρίς απώλεια δεδομένων λόγω σύγκρουσης. Ωστόσο, κάθε χρονική στιγμή, υπάρχει ενεργός μόνο ένας Master στο δίαυλο.
- **Arbitration:** Η διαδικασία που εγκρίνει κάθε φορά, μόνο έναν Master, να πάρει τον έλεγχο του διαύλου.
- **Synchronization:** Η διαδικασία που συγχρονίζει τα σήματα ρολογιών που παρέχονται από δύο ή περισσότερους Master.
- **SDA:** Η γραμμή του σήματος δεδομένων.
- **SCL:** Η γραμμή του σήματος ρολογιού

Συνθήκες START και STOP

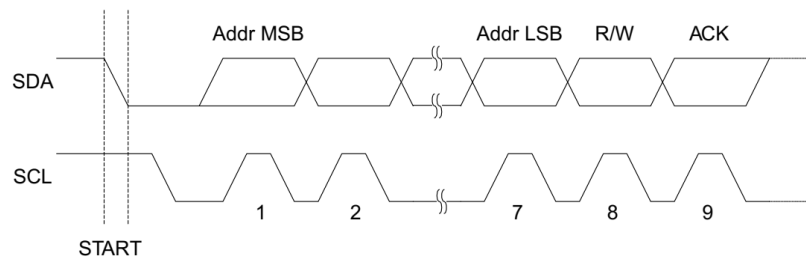
Η μετάδοση ξεκινά όταν ο Master εκδίδει μια συνθήκη START στο δίαυλο και τερματίζεται όταν ο Master εκδίδει μια συνθήκη STOP. Μεταξύ μιας συνθήκης START και STOP, ο δίαυλος θεωρείται απασχολημένος και κανένας άλλος Master δεν πρέπει να προσπαθήσει να πάρει τον έλεγχο του διαύλου. Μια ειδική περίπτωση παρουσιάζεται όταν εκδίδεται μια νέα συνθήκη START μεταξύ μιας συνθήκης START και STOP. Αυτό αναφέρεται ως συνθήκη REPEATED START και χρησιμοποιείται όταν ο Master επιθυμεί να ξεκινήσει μια νέα μεταφορά χωρίς να παραιτηθεί από τον έλεγχο του διαύλου.



Σχήμα 5.3 Συνθήκες START, REPEATED START, και STOP

Διευθυνσιοδότηση TWI διαύλου

Όλα τα πακέτα διευθύνσεων που μεταδίδονται στον δίαυλο TWI έχουν μήκος εννέα bit, όπως φαίνεται στο παρακάτω σχήμα:

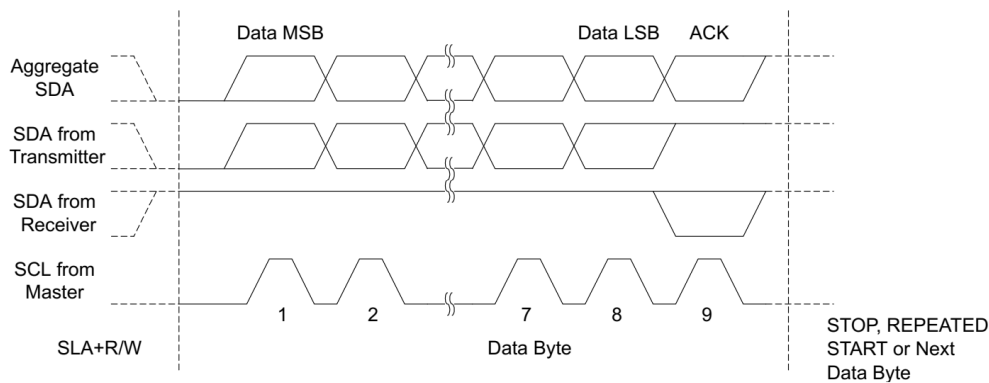


Σχήμα 5.4 Μορφή πακέτου διευθύνσεων

Αποτελούνται από επτά bit διεύθυνσης, ένα bit ελέγχου READ/WRITE και ένα bit επιβεβαίωσης (ACK). Εάν το bit READ/WRITE είναι λογικό 1 πρέπει να εκτελεστεί μια λειτουργία ανάγνωσης, διαφορετικά θα πρέπει να εκτελεστεί μια λειτουργία εγγραφής. Το MSBit του byte διεύθυνσης μεταδίδεται πρώτο. Ένα πακέτο διευθύνσεων που αποτελείται από μια διεύθυνση slave και ένα bit READ ή WRITE ονομάζεται SLA+R ή SLA+W αντίστοιχα. Όταν ένας Slave αναγνωρίσει ότι έχει διευθυνσιοδοτηθεί, τότε πρέπει να στείλει σήμα αναγνώρισης (ACK) κρατώντας το SDA χαμηλά στον ένατο κύκλο ρολογιού (κύκλος ACK). Εάν ο Slave είναι απασχολημένος ή για κάποιο άλλο λόγο δεν μπορεί να εξυπηρετήσει το αίτημα του Master, η γραμμή SDA θα πρέπει να παραμείνει ψηλά στον κύκλο ρολογιού ACK. Το Master μπορεί στη συνέχεια να μεταδώσει μια συνθήκη STOP ή μια συνθήκη REPEATED START για να ξεκινήσει μια νέα μετάδοση. Οι διευθύνσεις των συσκευών του διαύλου μπορούν να εκχωρηθούν ελεύθερα, αλλά η διεύθυνση '0000 000' προορίζεται για μια γενική κλήση. Όταν εκδίδεται μια γενική κλήση, όλοι οι Slave θα πρέπει να ανταποκρίνονται τραβώντας τη γραμμή SDA χαμηλά στον κύκλο ACK. Μια γενική κλήση χρησιμοποιείται όταν ένας Master επιθυμεί να μεταδώσει το ίδιο μήνυμα σε πολλούς Slave στο σύστημα. Όταν η διεύθυνση γενικής κλήσης ακολουθούμενη από ένα bit εγγραφής μεταδίδεται στο δίαυλο, όλοι οι Slave που έχουν ρυθμιστεί για να επιβεβαιώνουν τη γενική κλήση θα τραβήξουν τη γραμμή SDA χαμηλά στον κύκλο ACK. Τα ακόλουθα πακέτα δεδομένων θα ληφθούν από όλους τους Slave που αναγνώρισαν τη γενική κλήση. Όλες οι διευθύνσεις της μορφής «1111xxxx» θα πρέπει να δεσμευτούν για μελλοντικούς σκοπούς.

Πακέτα δεδομένων

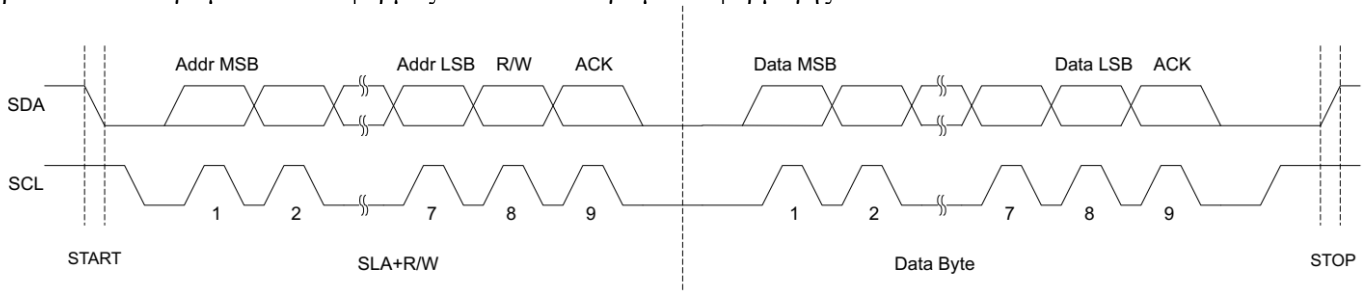
Όλα τα πακέτα δεδομένων που μεταδίδονται στο δίαυλο TWI έχουν μήκος εννέα bit, 8 bit δεδομένων και ένα bit επιβεβαίωσης. Το MSBit του byte δεδομένων μεταδίδεται πρώτο. Κατά τη μεταφορά δεδομένων, ο Master δημιουργεί το ρολόι, το START και το STOP, ενώ ο δέκτης είναι υπεύθυνος για την επιβεβαίωση της παραλαβής (ACK). Εάν ο δέκτης παραλείψει το ACK τότε αυτό ισοδυναμεί με μη επιβεβαίωση (Not Acknowledge, NACK). Όταν ο δέκτης λάβει το τελευταίο byte ή για κάποιο λόγο δεν μπορεί να λάβει άλλα byte, θα πρέπει να ενημερώσει τον πομπό στέλνοντας ένα NACK μετά το τελευταίο byte.



Σχήμα 5.5 Μορφή πακέτου δεδομένων

Τυπική μετάδοση δεδομένων

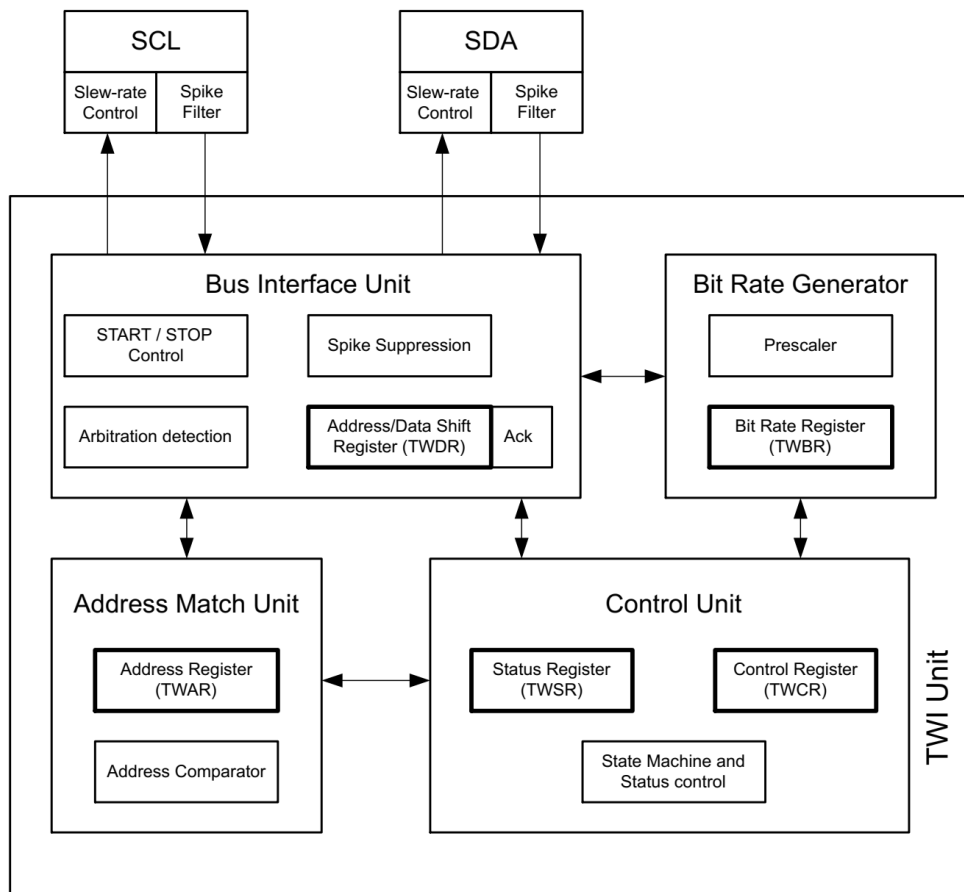
Μια μετάδοση αποτελείται από μια συνθήκη START, ένα πακέτο διεύθυνσης (Slave address(SLA) +R/W), ένα ή περισσότερα πακέτα δεδομένων και μια συνθήκη STOP. Ένα κενό μήνυμα, που αποτελείται από ένα START ακολουθούμενο από ένα STOP, είναι μη έγκυρο. Ο Slave μπορεί να παρατείνει τη χαμηλή περίοδο του ρολογιού κρατώντας τη γραμμή SCL χαμηλά. Αυτό είναι χρήσιμο εάν η ταχύτητα ρολογιού που έχει ρυθμιστεί από τον Master είναι πολύ γρήγορη για το Slave ή εάν το Slave χρειάζεται επιπλέον χρόνο για επεξεργασία μεταξύ των μεταδόσεων δεδομένων. Το παρακάτω σχήμα απεικονίζει μια τυπική μετάδοση δεδομένων. Σημειώστε ότι πολλά byte δεδομένων μπορούν να μεταδοθούν μεταξύ του SLA+R/W και της συνθήκης STOP, ανάλογα με το πρωτόκολλο λογισμικού που εφαρμόζεται από το λογισμικό εφαρμογής.



Σχήμα 5.6 Τυπική μετάδοση δεδομένων

Διεπαφή TWI στον ATmega328PB

Η Διεπαφή TWI στον ATmega328PB αποτελείται από πολλά υποσυστήματα, όπως φαίνεται στο παρακάτω σχήμα.



Σχήμα 5.7 Διάγραμμα διεπαφής TWI στον ATmega328PB

Στη συνέχεια παρουσιάζονται οι καταχωρητές του ATmega328PB που σχετίζονται με τη λειτουργία του TWI.

Register	Offset	Reset Value	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TWBR0	0xB8	0b00000000	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn
TWSR0	0xB9	0b11111X00	TWS7	TWS6	TWS5	TWS4	TWS3		TWPS[1:0]	
TWAR0	0xBA	0b00000010	TWA[6:0]							TWGCE
TWDR0	0xBB	0b00000001	TWD[7:0]							
TWCR0	0xBC	0b000000X0	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN		TWIE
TWAMR0	0xBD	0b0000000X	TWAM[6:0]							

Πίνακας 5.1 Οι καταχωρητές του ATmega328PB που σχετίζονται με τη λειτουργία του TWI

TWBR0 (Offset 0xB8), TWI Bit Rate Register

Bit	7	6	5	4	3	2	1	0
	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Σχήμα 5.8 TWBR0, TWI Bit Rate Register

TWBR0[7:0] Ρυθμίζουν τη συχνότητα του σήματος SCL σύμφωνα με την ακόλουθη εξίσωση:

$$\text{Συχνότητα SCL} = \frac{\text{Συχνότητα ρολογιού CPU}}{16 + 2 \cdot (\text{TWBR0}) \cdot \text{PrescalerValue}}$$

Η τιμή του Prescaler καθορίζεται από τα bits TWPS[1:0] του καταχωρητή TWSR0.

TWSR0(Offset 0xB9), TWI Status Register

Bit	7	6	5	4	3	2	1	0
	TWS7	TWS6	TWS5	TWS4	TWS3		TWPS[1:0]	
Access	R	R	R	R	R		R/W	R/W
Reset	1	1	1	1	1		0	0

Σχήμα 5.9 TWSR0, TWI Status Register

TWPS[1:0] Η τιμή του Prescaler για τη ρύθμιση της συχνότητας του σήματος SCL καθορίζεται από τα bits TWPS[1:0] σύμφωνα με τον παρακάτω πίνακα:

TWPS[1:0]	Prescaler Value
00	1
01	4
10	16
11	64

Πίνακας 5.2 Bit rate Prescaler Values

TWS[7:3] αντικατοπτρίζουν την κατάσταση λειτουργίας του σειριακού διαύλου TWI για καθένα από τους τέσσερις κύριους τρόπους λειτουργίας:

1. Status Codes for Master Transmitter Mode

TWSR	Status of the Bus and Interface Hardware
0x08	A START has been transmitted
0x10	A repeated START has been transmitted
0x18	SLA+W has been transmitted; ACK has been received
0x20	SLA+W has been transmitted; NOT ACK has been received
0x28	Data byte has been transmitted; ACK has been received
0x30	Data byte has been transmitted; NOT ACK has been received
0x38	Arbitration lost in SLA+W or data bytes

2. Status codes for Master Receiver Mode

TWSR	Status of the Bus and Interface Hardware
0x08	A START has been transmitted
0x10	A repeated START has been transmitted
0x38	Arbitration lost in SLA+R or NOT ACK bit
0x40	SLA+R has been transmitted; ACK has been received
0x48	SLA+R has been transmitted; NOT ACK has been received
0x50	Data byte has been received; ACK has been returned
0x58	Data byte has been received; NOT ACK has been returned

3. Status Codes for Slave Transmitter Mode

TWSR	Status of the Bus and Interface Hardware
0xA8	Own SLA+R has been received; ACK has been returned
0xB0	Arbitration lost in SLA+R/W; Own SLA+R has been received; ACK has been returned;
0xB8	Data byte has been transmitted; ACK has been received;
0xC0	Data byte has been transmitted; NOT ACK has been received;
0xC8	Last data byte has been transmitted; ACK has been received;

4. Status codes for Slave Receiver Mode

TWSR	Status of the Bus and Interface Hardware
0x60	Own SLA+W has been received; ACK has been returned;
0x68	Arbitration lost in SLA+R/W; Own SLA+W has been received; ACK has been returned;
0x70	General call address has been received; ACK has been returned;
0x78	Arbitration lost in SLA+R/W; General call address has been received; ACK has been returned;
0x80	Data has been received; ACK has been returned;
0x88	Data has been received; NOT ACK has been returned;
0x90	General call data has been received; ACK has been returned;
0x98	General call data has data has been received; NOT ACK has been returned;
0xA0	A STOP condition or repeated START condition has been received while still addressed as Slave;

TWAR0 (Offset 0xBA), TWI (Slave) Address Register

Bit	7	6	5	4	3	2	1	0
	TWA[6:0]							TWGCE
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	1	0

Σχήμα 5.10 TWAR0, TWI (Slave) Address

TWA[6:0]: είναι η 7-bit διεύθυνση του slave

TWGCE: Χρησιμοποιείται για να επιτρέψει την αναγνώριση της γενικής διεύθυνσης κλήσης (0x00).

TWDR0 (Offset 0xBB), TWI Data Register

Bit	7	6	5	4	3	2	1	0
	TWD[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	1

Σχήμα 5.11 TWDR0, TWI Data Register

TWD[7:0] Byte δεδομένων που θα μεταδοθεί ή το τελευταίο byte δεδομένων που ελήφθη.

TWCR0 (Offset 0xBC), TWI Control Register

Bit	7	6	5	4	3	2	1	0
	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN		TWIE
Access	R/W	R/W	R/W	R/W	R	R/W		R/W
Reset	0	0	0	0	0	0		0

Σχήμα 5.12 TWCR0, TWI Control Register

TWINT: Σημαία διακοπής TWI. Τίθεται όταν το TWI ολοκληρώσει την τρέχουσα εργασία του και αναμένει την απόκριση του λογισμικού εφαρμογής. Αν TWINT=1 τότε η χαμηλή περίοδος του SCL επιμηκύνεται. Η σημαία TWINT μηδενίζεται μόνο από το λογισμικό γράφοντας λογικό 1 σε αυτήν. Αν το TWINT μηδενιστεί τότε ξεκινά τη λειτουργία του TWI. Όλες οι προσβάσεις στα TWAR0, TWSR0 και TWDR0 πρέπει να έχουν ολοκληρωθεί πριν από την εκκαθάριση αυτής της σημαίας.

TWEA: Αυτό το bit ελέγχει τη δημιουργία του παλμού επιβεβαίωσης. Εάν TWEA=1 το ACK δημιουργείται στον διάυλο εάν έχει ληφθεί η ατομική διεύθυνση του slave ή έχει ληφθεί μια γενική κλήση και ταυτόχρονα TWAR0.TWGCE =1 ή έχει ληφθεί ένα byte δεδομένων σε λειτουργία Master Receiver ή Slave Receiver.

Γράφοντας το bit TWEA στο μηδέν, η συσκευή μπορεί ουσιαστικά να αποσυνδεθεί από το δίαυλο προσωρινά. Στη συνέχεια, η αναγνώριση διεύθυνσης μπορεί να ενεργοποιηθεί ξανά, θέτοντας ξανά το bit TWEA.

TWSTA: Συνθήκη START. Η εφαρμογή θέτει το TWSTA όταν επιθυμεί να γίνει Master. Το υλικό TWI ελέγχει εάν ο δίαυλος είναι διαθέσιμος και δημιουργεί μια συνθήκη START στο δίαυλο εάν είναι ελεύθερος. Ωστόσο, εάν ο δίαυλος δεν είναι ελεύθερος, το TWI περιμένει μέχρι να εντοπιστεί μια συνθήκη STOP και στη συνέχεια δημιουργεί μια νέα συνθήκη START για να δώσει το δικαίωμα του Master. Το TWSTA πρέπει να μηδενιστεί από το λογισμικό όταν έχει γίνει εκπομπή του START.

TWSTO: Συνθήκη STOP. Σε λειτουργία Master θα δημιουργήσει μια συνθήκη STOP στον δίαυλο. Όταν η συνθήκη STOP εκτελεστεί στο δίαυλο, το TWSTO μηδενίζεται αυτόματα. Στη λειτουργία Slave το TWSTO τίθεται για την ανάκτηση από μια συνθήκη σφάλματος. Αυτό δεν θα δημιουργήσει μια συνθήκη STOP, αλλά το TWI επιστρέφει σε μια προκαθορισμένη λειτουργία μη διευθυνσιοδοτούμενου Slave και απελευθερώνει τις γραμμές SCL και SDA.

TWWC: Σημαία σύγκρουσης εκπομπής στο δίαυλο. Το bit TWWC τίθεται όταν γίνεται προσπάθεια εγγραφής του TWDR0 ενώ TWCR0.TWINT = 0. Αυτή η σημαία μηδενίζεται γράφοντας στον καταχωρητή TWDR0 ενώ TWINT=1.

TWEN: Ενεργοποίηση του TWI. Όταν TWEN=1 το TWI η αναλαμβάνει τον έλεγχο των ακροδεκτών που είναι συνδεδεμένες με τα σήματα SCL και SDA. Επίσης ενεργοποιεί τους περιοριστές του ρυθμού μεταβολής της τάσης και τα φίλτρα θορύβου σε αυτούς τους ακροδέκτες. Εάν αυτό το TWEN μηδενιστεί το TWI απενεργοποιείται και όλες οι μεταδόσεις του τερματίζονται.

TWIE: Ενεργοποίηση της διακοπής του TWI. Όταν TWIE =1 και SREG.I=1 το αίτημα διακοπής του TWI θα ενεργοποιηθεί για όσο διάστημα η σημαία TWCR0.TWINT είναι υψηλή.

TWAMR0 (Offset 0xBD), TWI Slave Address Mask Register

Bit	7	6	5	4	3	2	1	0
	TWAM[6:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Reset	0	0	0	0	0	0	0	

Σχήμα 5.13 TWAMR0 , TWI Slave Address Mask Register

TWAM[6:0]: Το TWAMR0 μπορεί να φορτωθεί με μια μάσκα για τη διεύθυνση του Slave (7 bit). Κάθε ένα από τα bit TWAMR[6:0] μπορεί να κρύψει(απενεργοποιήσει) τα αντίστοιχα bit διεύθυνσης στον καταχωρητή διευθύνσεων TWAR0. Εάν το bit μάσκας έχει οριστεί σε λογικό 1, τότε η διάταξη αντιστοίχισης διεύθυνσης αγνοεί τη σύγκριση μεταξύ του εισερχόμενου bit διεύθυνσης και του αντίστοιχου bit στο TWAR0.

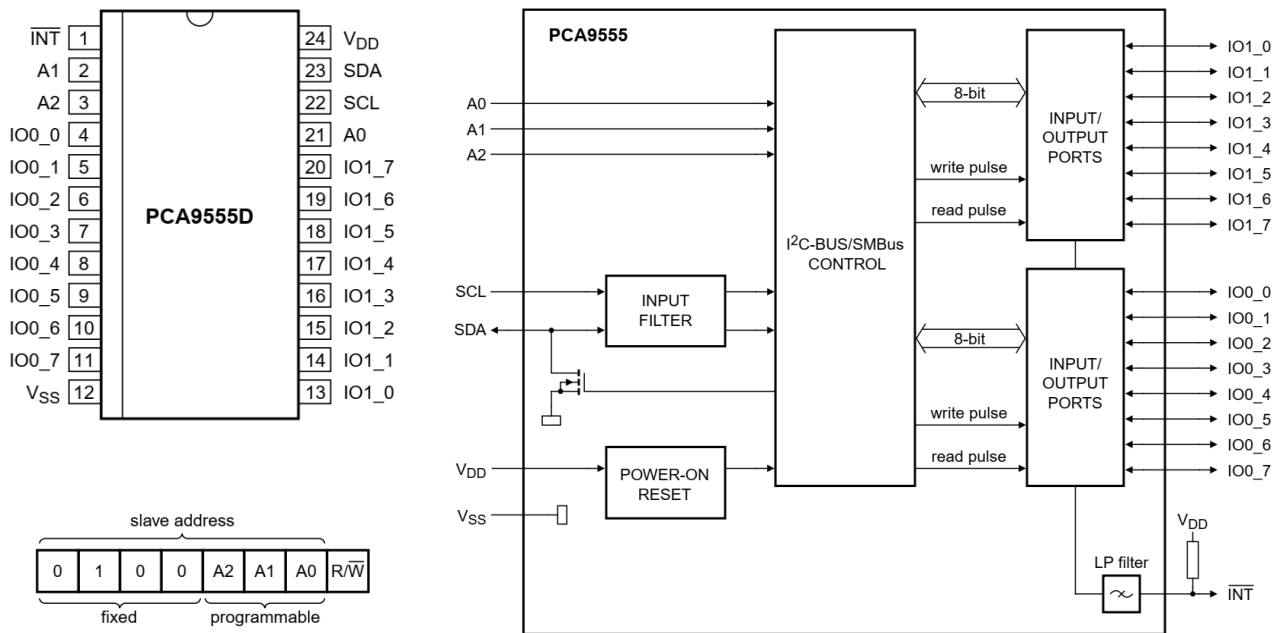
Παράδειγμα μετάδοσης ενός byte δεδομένων σε ένα slave.

Το TWI βασίζεται σε διακοπές. Όταν τεθεί το TWINT, τότε το TWI έχει ολοκληρώσει μια λειτουργία και ο καταχωρητής κατάστασης TWSR0 περιέχει μια τιμή που υποδεικνύει την τρέχουσα κατάσταση του TWI διαύλου. Το λογισμικό εφαρμογής ρυθμίζει τους καταχωρητές TWCR0 και TWDR0 για τον επόμενο κύκλο του διαύλου TWI.

	ASSEMBLY CODE	C CODE	ΣΧΟΛΙΑ
1	<code>Ldi r16,1<<TWINT) (1<<TWSTA) (1<<TWEN) out TWCR0, r16</code>	<code>TWCR0 = (1<<TWINT) (1<<TWSTA) (1<<TWEN)</code>	Αποστολή START
2	<code>Call wait1 Wait1: in r16, TWCR0 sbrs r16, TWINT rjmp wait1 ret</code>	<code>while (!(TWCR0 & (1<<TWINT)));</code>	Αναμονή έως TWINT=1 που σημαίνει ότι το START μεταδόθηκε.
3a	<code>in r16, TWSR0 andi r16, 0xF8 cpi r16, START brne ERROR</code>	<code>if ((TWSR0 & 0xF8) != START) ERROR();</code>	Αν TWSR0.TWS[7:3] δείχνει κωδικό διαφορετικό από START τότε υπάρχει σφάλμα.
3b	<code>ldi r16, SLA_W out TWDR0, r16 ldi r16, (1<<TWINT) (1<<TWEN) out TWCR0, r16</code>	<code>TWDR0 = SLA_W; TWCR0 = (1<<TWINT) (1<<TWEN);</code>	Φορτώστε το SLA_W στον καταχωρητή TWDR0. Μηδένισε το TWCR.WINT για να ξεκινήσει η μετάδοση.
4	<code>Call wait1</code>	<code>while (!(TWCR0 & (1<<TWINT)));</code>	
5a	<code>in r16, TWSR0 andi r16, 0xF8 cpi r16, MT_SLA_ACK brne ERROR</code>	<code>if ((TWSR0 & 0xF8) != MT_SLA_ACK) ERROR();</code>	Αν TWSR0.TWS[7:3] δείχνει κωδικό διαφορετικό από SLA_ACK τότε υπάρχει σφάλμα.
5b	<code>ldi r16, DATA out TWDR0, r16 ldi r16, (1<<TWINT) (1<<TWEN) out TWCR, r16</code>	<code>TWDR0 = DATA; TWCR0 = (1<<TWINT) (1<<TWEN);</code>	Φορτώστε τα DATA στον καταχωρητή TWDR0. Μηδένισε το TWCR.WINT για να ξεκινήσει η μετάδοση.
6	<code>Call wait1</code>	<code>while (!(TWCR0 & (1<<TWINT)));</code>	
7a	<code>in r16, TWSR0 andi r16, 0xF8 cpi r16, MT_DATA_ACK brne ERROR</code>	<code>if ((TWSR0 & 0xF8) != MT_DATA_ACK) ERROR();</code>	Αν TWSR0.TWS[7:3] δείχνει κωδικό διαφορετικό από DATA_ACK τότε υπάρχει σφάλμα.
7b	<code>ldi r16, (1<<TWINT) (1<<TWEN) (1<<TWSTO) out TWCR0, r16</code>	<code>TWCR0 = (1<<TWINT) (1<<TWEN) (1<<TWSTO);</code>	Αποστολή STOP

Ολοκληρωμένο PCA9555

Το PCA9555 είναι ένα ολοκληρωμένο που παρέχει δύο πόρτες εισόδου/εξόδου γενικής χρήσης, IO0 και IO1, των 8-bit η κάθε μια, για εφαρμογές I2C-bus. Διαθέτει μία έξοδο διακοπής (open-drain) που ενεργοποιείται όταν η κατάσταση εισόδου διαφέρει από την αντίστοιχη κατάσταση του καταχωρητή θύρας εισόδου και χρησιμοποιείται για να υποδείξει ότι μια κατάσταση εισόδου έχει μεταβληθεί. Κατά την εφαρμογή της τάσεις τροφοδοσίας γίνεται επαναφορά της συσκευής και οι καταχωρητές τίθενται στις προεπιλεγμένες τιμές τους. Η I²C διεύθυνση του PCA9555 είναι 7-bit εκ των οποίων τα 4bit είναι σταθερά και τα υπόλοιπα τρία ρυθμίζονται από τους ακροδέκτες A0, A1 και A2. Στο ntuAboard_G1 οι ακροδέκτες A0, A1 και A2 είναι συνδεδεμένοι στη γη (0 Volt) οπότε η διεύθυνση του ενσωματωμένου PCA9555 είναι 0b0100000. Το διάγραμμα, οι ακροδέκτες και η I²C διεύθυνση του PCA9555 φαίνονται στο παρακάτω σχήμα:



Σχήμα 5.14 Διάγραμμα, ακροδέκτες και I2C διεύθυνση του PCA9555

Καταχωρητές του PCA9555

Κατά τη διάρκεια μιας μετάδοσης εγγραφής, το πρώτο byte που ακολουθεί το byte της διεύθυνσης(command byte) χρησιμοποιείται ως δείκτης για να προσδιορίσει ποιος από τους παρακάτω καταχωρητές θα γραφεί ή θα διαβαστεί σύμφωνα με τον παρακάτω πίνακα:

Command	Register
0	Input port 0
1	Input port 1
2	Output port 0
3	Output port 1
4	Polarity Inversion port 0
5	Polarity Inversion port 1
6	Configuration port 0
7	Configuration port 1

Πίνακας 5.3 Command byte

Οι καταχωρητές Input port 0 και Input port 1 είναι θύρες μόνο εισόδου. Αντικατοπτρίζουν τα εισερχόμενα λογικά επίπεδα των ακροδεκτών, ανεξάρτητα από το εάν ένας ακροδέκτης ορίζεται ως είσοδος ή έξοδος. Οι εγγραφές σε αυτούς τους καταχωρητές δεν έχουν κανένα αποτέλεσμα. Η προεπιλεγμένη τιμή «X» καθορίζεται από το εξωτερικά εφαρμοζόμενο λογικό επίπεδο.

Input port 0 Register

Bit	7	6	5	4	3	2	1	0
Symbol	I0.7	I0.6	I0.5	I0.4	I0.3	I0.2	I0.1	I0.0
Default	X	X	X	X	X	X	X	X

Input port 1 Register

Bit	7	6	5	4	3	2	1	0
Symbol	I1.7	I1.6	I1.5	I1.4	I1.3	I1.2	I1.1	I1.0
Default	X	X	X	X	X	X	X	X

Οι καταχωρητές Output port 0 και Output port 1 είναι θύρες μόνο εξόδου. Αντικατοπτρίζουν τα εξερχόμενα λογικά επίπεδα των ακροδεκτών, που ορίζονται ως εξόδοι. Οι αναγνώσεις από αυτούς τους καταχωρητές αντικατοπτρίζουν την τιμή που βρίσκεται στα flip-flop που ελέγχουν την επιλογή εξόδου, όχι την πραγματική τιμή των ακροδεκτών.

Output port 0 register

Bit	7	6	5	4	3	2	1	0
Symbol	O0.7	O0.6	O0.5	O0.4	O0.3	O0.2	O0.1	O0.0
Default	1	1	1	1	1	1	1	1

Output port 1 register

Bit	7	6	5	4	3	2	1	0
Symbol	O1.7	O1.6	O1.5	O1.4	O1.3	O1.2	O1.1	O1.0
Default	1	1	1	1	1	1	1	1

Οι καταχωρητές Polarity Inversion port 0 και Polarity Inversion port 1 επιτρέπουν την αντιστροφή της πολικότητας των δεδομένων των καταχωρητών της θύρας εισόδου. Εάν έχει τεθεί κάποιο bit σε έναν από αυτούς τους καταχωρητές, η πολικότητα δεδομένων της θύρας εισόδου αντιστρέφεται.

Polarity Inversion port 0 register

Bit	7	6	5	4	3	2	1	0
Symbol	N0.7	N0.6	N0.5	N0.4	N0.3	N0.2	N0.1	N0.0
Default	0	0	0	0	0	0	0	0

Polarity Inversion port 1 register

Bit	7	6	5	4	3	2	1	0
Symbol	N1.7	N1.6	N1.5	N1.4	N1.3	N1.2	N1.1	N1.0
Default	0	0	0	0	0	0	0	0

Οι καταχωρητές Configuration port 0 και Configuration port 1 διαμορφώνουν τις κατευθύνσεις των ακροδεκτών I/O. Εάν έχει τεθεί ένα bit σε έναν από αυτούς τους καταχωρητές, ο αντίστοιχος ακροδέκτης της θύρας ενεργοποιείται

ως είσοδος. Υπάρχει μια αντίσταση υψηλής τιμής (pull-up resistor) για πρόσδεσης κάθε ακροδέκτη στο VDD. Κατά την επαναφορά, οι θύρες της συσκευής είναι είσοδοι με αντιστάσεις πρόσδεσης στο VDD.

Configuration port 0 register

Bit	7	6	5	4	3	2	1	0
Symbol	C0.7	C0.6	C0.5	C0.4	C0.3	C0.2	C0.1	C0.0
Default	1	1	1	1	1	1	1	1

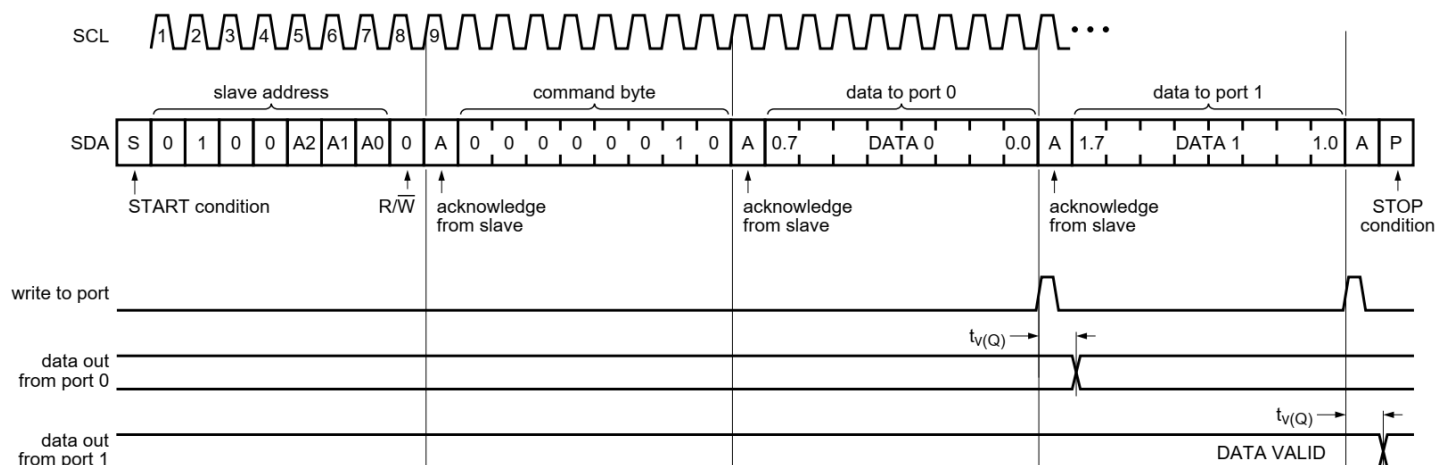
Configuration port 1 register

Bit	7	6	5	4	3	2	1	0
Symbol	C1.7	C1.6	C1.5	C1.4	C1.3	C1.2	C1.1	C1.0
Default	1	1	1	1	1	1	1	1

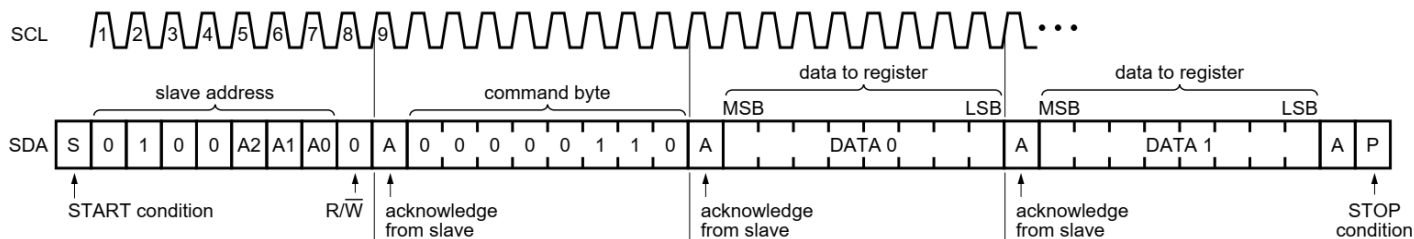
Τα δεδομένα μεταδίδονται στο PCA9555 στέλνοντας τη διεύθυνση της συσκευής και μηδενίζοντάς το λιγότερο σημαντικό bit. Το byte εντολής αποστέλλεται μετά τη διεύθυνση και καθορίζει ποιος καταχωρητής θα λάβει τα δεδομένα που ακολουθούν. Οι οκτώ καταχωρητές του PCA9555 έχουν ρυθμιστεί να λειτουργούν ως τέσσερα ζεύγη καταχωρητών. Τα τέσσερα ζεύγη είναι θύρες εισόδου, θύρες εξόδου, θύρες αναστροφής πολικότητας και θύρες διαμόρφωσης.

Εγγραφή στους καταχωρητές θυρών

Μετά την αποστολή δεδομένων σε έναν καταχωρητή, το επόμενο byte δεδομένων θα σταλεί στον άλλο καταχωρητή του ζεύγους. Για παράδειγμα, εάν το πρώτο byte αποσταλεί στην Output Port 1 (register 3) τότε το επόμενο byte θα αποθηκευτεί στη Output Port 0 (register 2). Δεν υπάρχει περιορισμός στον αριθμό των byte δεδομένων που αποστέλλονται σε μία μετάδοση εγγραφής. Με αυτόν τον τρόπο, κάθε καταχωρητής 8-bit μπορεί να ενημερώνεται ανεξάρτητα από τους άλλους καταχωρητές. Στα επόμενα δύο σχήματα απεικονίζεται η διαδικασία εγγραφής στα Output port registers και στα Configuration registers.



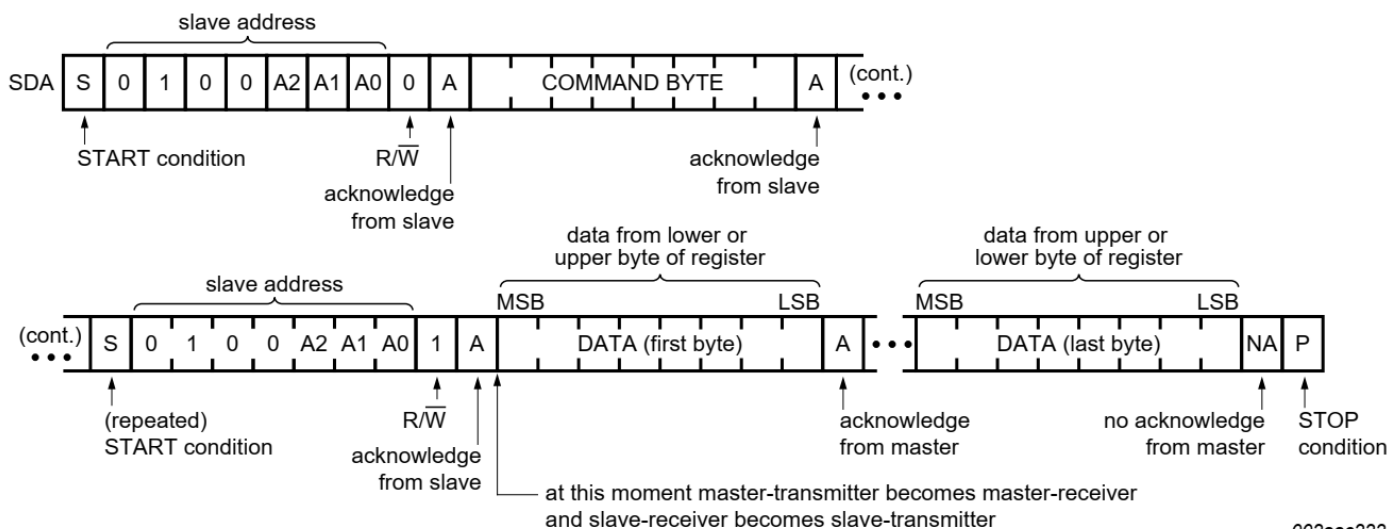
Σχήμα 5.15 Write to Output port registers



Σχήμα 5.16 Write to Configuration registers

Ανάγνωση των καταχωρητών των θυρών

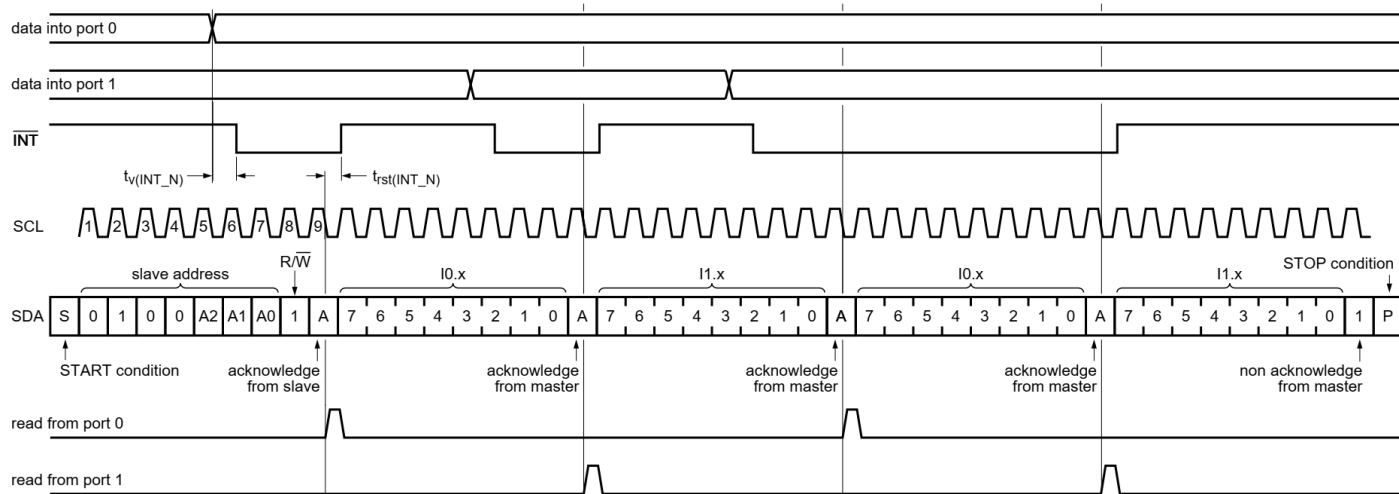
Για να διαβαστούν τα δεδομένα από το PCA9555, ο master του I2C διαύλου πρέπει πρώτα να στείλει τη διεύθυνση του PCA9555 με το λιγότερο σημαντικό bit να έχει οριστεί σε λογικό 0. Μετά τη διεύθυνση αποστέλλεται το command byte και καθορίζει σε ποιον καταχωρητή θα γίνει προσπέλαση. Μετά από επανεκκίνηση, η διεύθυνση της συσκευής αποστέλλεται ξανά, αλλά αυτή τη φορά το λιγότερο σημαντικό bit ορίζεται σε ένα λογικό 1. Τα δεδομένα από τον καταχωρητή που ορίζεται από command byte θα σταλούν στη συνέχεια από το PCA9555. Μετά την ανάγνωση του πρώτου byte, μπορούν να διαβαστούν επιπλέον byte, αλλά τα δεδομένα θα αντικατοπτρίζουν τώρα τις πληροφορίες στον άλλο καταχωρητή του ζεύγους. Δεν υπάρχει περιορισμός στον αριθμό των byte δεδομένων που λαμβάνονται σε μία μετάδοση ανάγνωσης, αλλά στο τελικό byte που λαμβάνεται, ο master του διαύλου δεν πρέπει να στείλει ACK. Στα επόμενα σχήματα απεικονίζεται η διαδικασία ανάγνωσης από τους διάφορους καταχωρητές.



002aac222

Σχήμα 5.17 Read from register

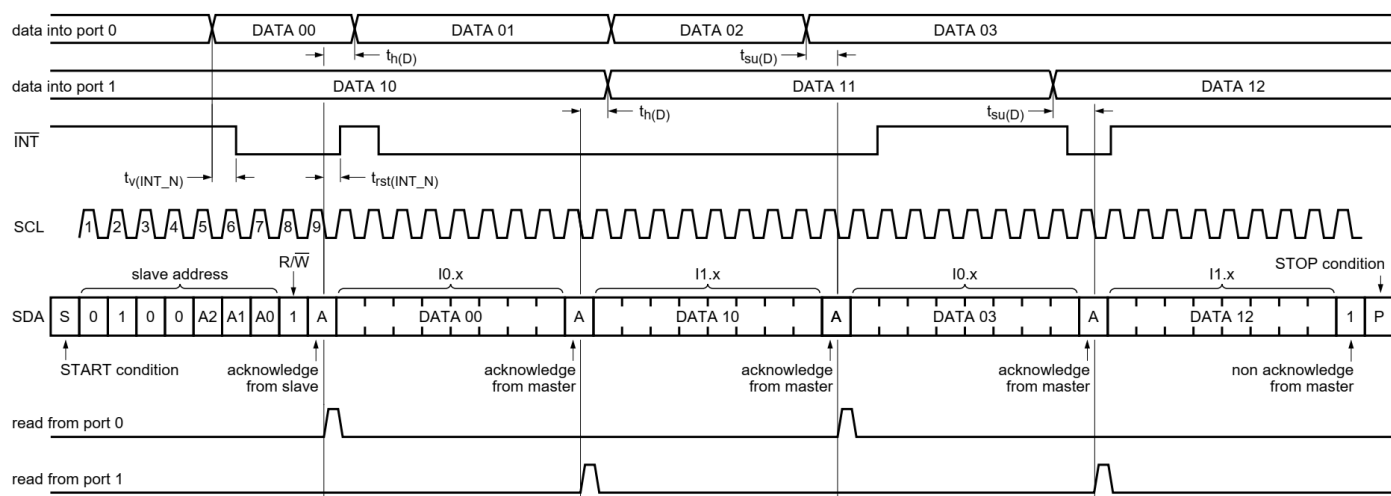
Η μεταφορά μπορεί να διακοπεί ανά πάσα στιγμή με αποστολή STOP.



Σχήμα 5.18 Read Input port register, scenario 1

Η μεταφορά δεδομένων μπορεί να διακοπεί ανά πάσα στιγμή με μια συνθήκη STOP. Όταν συμβεί αυτό, τα δεδομένα που υπάρχουν στην τελευταία φάση επιβεβαίωσης είναι έγκυρα.

Υποτίθεται ότι το command byte έχει προηγουμένως ρυθμιστεί σε «00» (Διάβασμα Input Port register).



Σχήμα 5.19 Read Input port register, scenario 2

Η μεταφορά δεδομένων μπορεί να διακοπεί ανά πάσα στιγμή με μια συνθήκη STOP. Όταν συμβεί αυτό, τα δεδομένα που υπάρχουν στην τελευταία φάση επιβεβαίωσης είναι έγκυρα.

Υποτίθεται ότι το command byte έχει προηγουμένως ρυθμιστεί σε «00» (Διάβασμα Input Port register).

Παράδειγμα 5.1

Στο παρακάτω παράδειγμα ρυθμίζεται η θύρα επέκτασης 0 του ολοκληρωμένου PCA9555 έτσι ώστε η λογική ή κατάσταση των ακροδεκτών να εναλλάσσεται μεταξύ λογικού 0 και λογικού 1. Ο χρόνος παραμονής σε καθεμία από τις δύο αυτές λογικές καταστάσεις είναι ένα δευτερόλεπτο. Το ολοκληρωμένο PCA9555 επικοινωνεί με τον μικροελεγκτή ATmega328PB μέσω της σειριακής Θύρας TWI.

```
#define F_CPU 16000000UL

#include<avr/io.h>
#include<avr/interrupt.h>
#include<util/delay.h>

#define PCA9555_0_ADDRESS 0x40          //A0=A1=A2=0 by hardware
#define TWI_READ  1                     // reading from twi device
#define TWI_WRITE  0                    // writing to twi device
#define SCL_CLOCK  100000L              // twi clock in Hz

//Fsc1=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0          = 0,
    REG_INPUT_1          = 1,
    REG_OUTPUT_0         = 2,
    REG_OUTPUT_1         = 3,
    REG_POLARITY_INV_0   = 4,
    REG_POLARITY_INV_1   = 5,
    REG_CONFIGURATION_0  = 6,
    REG_CONFIGURATION_1  = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START          0x08
#define TW_REP_START      0x10

//----- Master Transmitter -----
#define TW_MT_SLA_ACK     0x18
#define TW_MT_SLA_NACK    0x20
#define TW_MT_DATA_ACK    0x28

//----- Master Receiver -----
#define TW_MR_SLA_ACK     0x40
#define TW_MR_SLA_NACK    0x48
#define TW_MR_DATA_NACK   0x58
```

```

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

//initialize TWI clock
void twi_init(void)
{
    TWSR0 = 0;           // PRESCALER_VALUE=1
    TWBR0 = TWBR0_VALUE; // SCL_CLOCK 100KHz
}

// Read one byte from the twi device (request more data from device)
unsigned char twi_readAck(void)
{
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while(!(TWCR0 & (1<<TWINT)));
    return TWDR0;
}

//Read one byte from the twi device, read is followed by a stop condition
unsigned char twi_readNak(void)
{
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while(!(TWCR0 & (1<<TWINT)));
    return TWDR0;
}

// Issues a start condition and sends address and transfer direction.
// return 0 = device accessible, 1= failed to access device
unsigned char twi_start(unsigned char address)
{
    uint8_t twi_status;

    // send START condition
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // wait until transmission completed
    while(!(TWCR0 & (1<<TWINT)));

    // check value of TWI Status Register.
    twi_status = TW_STATUS & 0xF8;
    if ( (twi_status != TW_START) && (twi_status != TW_REP_START)) return 1;

    // send device address
    TWDR0 = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // wait until transmission completed and ACK/NACK has been received
    while(!(TWCR0 & (1<<TWINT)));
}

```



```

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK) )
        {
            return 1;
        }

        return 0;
    }

// Send start condition, address, transfer direction.
// Use ack polling to wait until device is ready
void twi_start_wait(unsigned char address)
{
    uint8_t twi_status;

    while ( 1 )
    {
        // send START condition
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // wait until transmission completed
        while(!(TWCR0 & (1<<TWINT)));

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status != TW_START) && (twi_status != TW_REP_START)) continue;

        // send device address
        TWDR0 = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // wait until transmission completed
        while(!(TWCR0 & (1<<TWINT)));

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status == TW_MT_SLA_NACK )||(twi_status ==TW_MR_DATA_NACK) )
        {
            /* device busy, send stop condition to terminate write operation */
            TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

            // wait until stop condition is executed and bus released
            while(TWCR0 & (1<<TWSTO));

            continue;
        }
        break;
    }
}

```

```
// Send one byte to twi device, Return 0 if write successful or 1 if write failed
unsigned char twi_write( unsigned char data )
```

```
{
    // send data to the previously addressed device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // wait until transmission completed
    while(!(TWCR0 & (1<<TWINT)));
    if( (TW_STATUS & 0xF8) != TW_MT_DATA_ACK) return 1;
    return 0;
}
```

```
// Send repeated start condition, address, transfer direction
```

```
//Return: 0 device accessible
```

```
//      1 failed to access device
```

```
unsigned char twi_rep_start(unsigned char address)
```

```
{
    return twi_start( address );
}
```

```
// Terminates the data transfer and releases the twi bus
```

```
void twi_stop(void)
```

```
{
    // send stop condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // wait until stop condition is executed and bus released
    while(TWCR0 & (1<<TWSTO));
}
```

```
void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value)
```

```
{
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}
```

```
uint8_t PCA9555_0_read(PCA9555_REGISTERS reg)
```

```
{
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();
}
```

```

    return ret_val;
}

int main(void) {

    twi_init();
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00); //Set EXT_PORT0 as output

    while(1)
    {
        PCA9555_0_write(REG_OUTPUT_0, 0x00);
        _delay_ms(1000);

        PCA9555_0_write(REG_OUTPUT_0, 0xFF);
        _delay_ms(1000);
    }
}

```

Τα ζητούμενα της 5^{ης} εργαστηριακής άσκησης

Ζήτημα 5.1

Να υλοποιηθούν για το μικροελεγκτή ATmega328PB, σε γλώσσα C, οι λογικές συναρτήσεις:

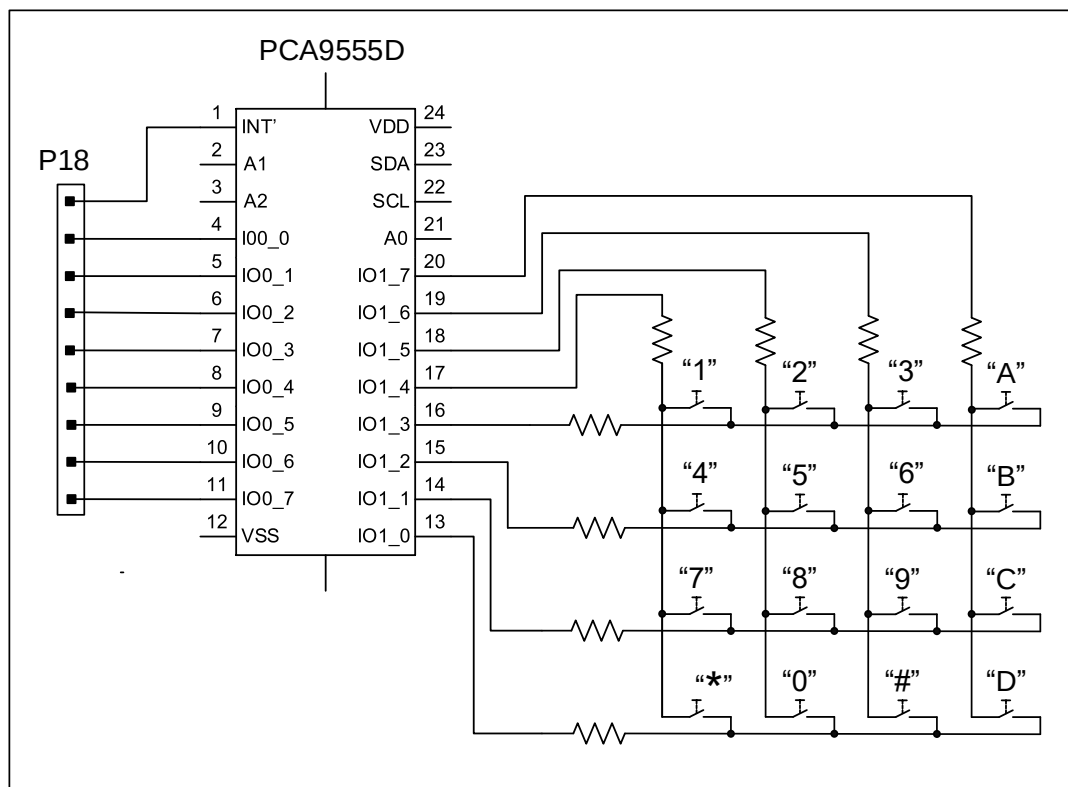
$$F0 = (A'BC + B'D)'$$

$$F1 = (A+B+C) \cdot (B \cdot D)'$$

Οι μεταβλητές εισόδου δίνονται στα bit **PORTB [3:0]** ($A = PORTB.0$, $B = PORTB.1$, $C = PORTB.2$, $D = PORTB.3$) του ntuAboard_G1. Οι τιμές των $F0$ - $F1$ να εμφανίζονται αντίστοιχα στους ακροδέκτες IO0_0 και IO0_1 του ολοκληρωμένου επέκτασης θυρών PCA9555, στο ntuAboard_G1. Οι ακροδέκτες IO0_0 και IO0_1 του κονέκτορα P18 να συνδεθούν, μέσω καλωδίων, με τους ακροδέκτες LED_PD2 και LED_PD3 του κονέκτορα J18 αντίστοιχα, έτσι ώστε οι τιμές των συναρτήσεων $F0$ και $F1$ να εμφανίζονται στα led PD2 και PD3 αντίστοιχα.

Ζήτημα 5.2

Στο παρακάτω σχήμα εμφανίζεται η διασύνδεση του ολοκληρωμένου PCA9555D στο ntuAboard_G1.



Προκειμένου να μπορέσει να υπάρξει οπτική απεικόνιση της λογικής κατάστασης των ακροδεκτών IO0_0 έως IO0_3, της θύρας επέκτασης 0, να ρυθμιστούν ως έξοδοι και να συνδεθούν, μέσω καλωδίων, με τους ακροδέκτες LED_PD4 έως LED_PD7 του κονέκτορα J18 αντίστοιχα.

Ο ακροδέκτης IO1_0 της θύρας επέκτασης 1 να ρυθμιστεί ως έξοδος ενώ οι ακροδέκτες IO1_4 έως IO0_7 να ρυθμιστούν ως είσοδοι.

Να υλοποιηθεί κώδικας για το μικροελεγκτή ATmega328PB, σε γλώσσα C, ο οποίος όταν πιέζεται το πλήκτρο “*” να ανάβει το led PD4, όταν πιέζεται το πλήκτρο “0” να ανάβει το led PD 5, όταν πιέζεται το πλήκτρο “#” να ανάβει το led PD6 και όταν πιέζεται το πλήκτρο “D” να ανάβει το led PD7. Αν δεν πιέζεται κανένα πλήκτρο τότε όλα τα led να παραμένουν σβηστά. Θεωρείστε δεδομένο ότι κάθε φορά πιέζεται μόνο ένα πλήκτρο.

Ζήτημα 5.3

Να συνδεθεί η LCD οθόνη 2x16 χαρακτήρων, δια μέσω του κονέκτορα J19, στην θύρα επέκτασης 1 του ολοκληρωμένου PCA9555 και να υλοποιηθεί κώδικας για το μικροελεγκτή ATmega328PB, σε γλώσσα C, ο οποίος θα απεικονίζει στην οθόνη το όνομα και το επίθετο σας.



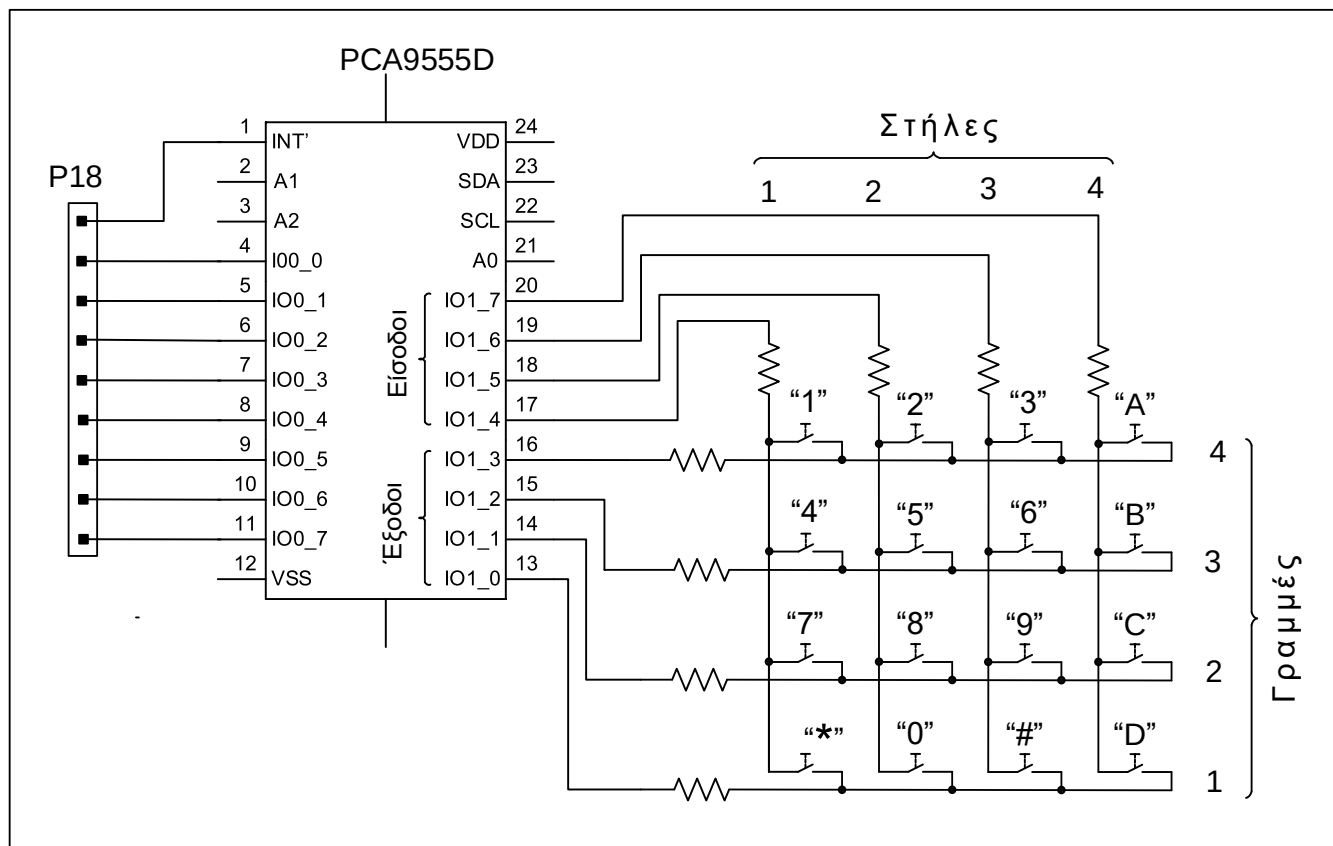
ΑΘΗΝΑ, 8 Νοεμβρίου 2024

6^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Χρήση πληκτρολογίου 4×4 σε θύρα επέκτασης στον AVR

Εξέταση – Επίδειξη: Παρασκευή 22/11/2024
Προθεσμία για παράδοση Έκθεσης: Τρίτη 26/11/2024 (23:59)

Πληκτρολόγιο 4×4

Το πληκτρολόγιο 4x4 της κάρτας ntuAboard έχει τέσσερις γραμμές και τέσσερις στήλες δηλαδή συνολικά 16 πλήκτρα. Όταν πατηθεί κάποιο πλήκτρο ενώνονται η γραμμή και στήλη που αντιστοιχούν σε αυτό το πλήκτρο. Η ακριβής διάταξη του πληκτρολογίου φαίνεται στο παρακάτω σχήμα:



Σχήμα 6.1 Σύνδεση του πληκτρολογίου 4×4 στην εξωτερική θύρα IO1 του PCA9555

Το πληκτρολόγιο είναι συνδεδεμένο στην εξωτερική θύρα εισόδου/εξόδου γενικής χρήσης IO1 του PCA9555, το οποίο επικοινωνεί με τον μικροελεγκτή ATmega328PB δια μέσω της διεπαφής TWI0 και η διεύθυνση του είναι 0b0100000. Οι δυο γραμμές του διαύλου TWI0 συνδέονται στους ακροδέκτες PC4 (SDA) και PC5 (SCL) του ATmega328PB. Οι αντιστάσεις πρόσδεσης (pull-up resistors) συνδέονται στο δίαυλο με την τοποθέτηση βραχυκυκλωτήρων στους κονέκτορες J12 και J13. Όταν ο δίαυλος TWI είναι ενεργός τότε οι ακροδέκτες PC4(SDA) και PC5(SCL) δεν μπορούν να χρησιμοποιηθούν για άλλο σκοπό. Όταν ένας ακροδέκτης, οποιασδήποτε από τις δύο θύρες του PCA9555, ρυθμίζεται ως είσοδος τότε ο ακροδέκτης αυτός προσδένεται στην τάση VDD μέσω μιας αντίσταση υψηλής τιμής(pull-up resistor).

Ανάγνωση του πληκτρολογίου

Μία μέθοδος ανάγνωσης του πληκτρολογίου είναι η εξής:

Διαδοχικά κάθε μία από τις γραμμές του πληκτρολογίου, οι οποίες έχουν ρυθμιστεί ως έξοδοι και είναι συνδεδεμένες με τους ακροδέκτες IO1[3:0] της θύρας IO1 του PCA9555, τίθεται σε λογικό 0 (χαμηλή στάθμη τάσης) ενώ ταυτόχρονα οι υπόλοιπες γραμμές έχουν τεθεί σε λογικό 1. Για κάθε μία από τις γραμμές, που έχει τεθεί σε λογικό 0, εκτελείται μία μικρή χρονοκαθυστέρηση και μετά διαβάζονται οι ακροδέκτες IO1[7:4] οι οποίοι αντιστοιχούν στις στήλες του πληκτρολογίου και έχουν ρυθμιστεί ως είσοδοι. Αν δεν υπάρχει πιεσμένος διακόπτης, λόγω των pull-up αντιστάσεων, οι ακροδέκτες εισόδου βρίσκονται σε κατάσταση λογικού 1. Τα πλήκτρα που είναι πατημένα κάθε φορά μπορούν να εντοπιστούν γνωρίζοντας πια γραμμή έχει τεθεί σε λογικό 0 και διαβάζοντας ταυτόχρονα ποιες από τις στήλες βρίσκονται σε λογικό 0.

Όταν ο χρήστης πατάει ένα πλήκτρο, αυτό μπορεί να μείνει πιεσμένο για μεγάλο χρονικό διάστημα. Για το λόγο αυτό απαιτείται ιδιαίτερος χειρισμός για να εντοπιστούν τα πλήκτρα που έχουν πατηθεί ως ολοκληρωμένη ενέργεια και όχι απλά αυτά που είναι κάθε φορά πατημένα. Αυτό επιτυγχάνεται εάν κληθεί μία ρουτίνα που διαβάζει την κατάσταση των πλήκτρων και αποθηκεύει το αποτέλεσμα στη μνήμη του μικροελεγκτή. Στη συνέχεια καλείτε ξανά η ίδια ρουτίνα, για νέο έλεγχο της κατάστασης των πλήκτρων. Μια σύγκριση των δύο καταστάσεων του πληκτρολογίου θα αποκαλύψει τις διαφορές στην κατάσταση των διακοπών. Το χρονικό διάστημα ανάμεσα στις διαδοχικές κλήσεις της συνάρτησης διαβάσματος του πληκτρολογίου είναι κρίσιμο, διότι καθορίζει το χρόνο που θα πρέπει να μείνει πιεσμένος ένας διακόπτης για να καταγραφεί από τον μικροελεγκτή. Αυτό σημαίνει ότι ένα μεγάλο χρονικό διάστημα μεταξύ των διαδοχικών κλήσεων της ρουτίνας θα αναγκάσει τον χρήστη να κρατά πατημένο το πλήκτρο για αντίστοιχα μεγάλο χρονικό διάστημα (ώστε να μπορεί να αναγνωριστεί). Αντίθετα, ένα πολύ μικρό χρονικό διάστημα θα δημιουργήσει προβλήματα λόγω του σπινθηρισμού που παρουσιάζουν οι διακόπτες. Οι τυπικές τιμές καθυστέρησης είναι της τάξης των 10 έως 20 msec.

Ο παρακάτω κώδικας μπορεί να χρησιμοποιηθεί για να γίνει εγγραφή ή ανάγνωση ενός από τους καταχωρητές ελέγχου του ολοκληρωμένου PCA9555:

```
#define F_CPU 16000000UL

#include<avr/io.h>
#include<avr/interrupt.h>
#include<util/delay.h>

#define PCA9555_0_ADDRESS 0x40          //A0=A1=A2=0 by hardware
#define TWI_READ  1                     // reading from twi device
#define TWI_WRITE 0                     // writing to twi device
#define SCL_CLOCK 100000L               // twi clock in Hz

//Fscl=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0      = 0,
    REG_INPUT_1      = 1,
    REG_OUTPUT_0     = 2,
    REG_OUTPUT_1     = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START      0x08
#define TW_REP_START  0x10

//----- Master Transmitter -----
#define TW_MT_SLA_ACK  0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28

//----- Master Receiver -----
#define TW_MR_SLA_ACK  0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b11111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

//initialize TWI clock
```

```

void twi_init(void)
{
    TWSR0 = 0;           // PRESCALER_VALUE=1
    TWBR0 = TWBR0_VALUE; // SCL_CLOCK 100KHz
}

// Read one byte from the twi device (request more data from device)
unsigned char twi_readAck(void)
{
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while(!(TWCR0 & (1<<TWINT)));
    return TWDR0;
}

//Read one byte from the twi device, read is followed by a stop condition
unsigned char twi_readNak(void)
{
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while(!(TWCR0 & (1<<TWINT)));
    return TWDR0;
}

// Issues a start condition and sends address and transfer direction.
// return 0 = device accessible, 1= failed to access device
unsigned char twi_start(unsigned char address)
{
    uint8_t twi_status;

    // send START condition
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // wait until transmission completed
    while(!(TWCR0 & (1<<TWINT)));

    // check value of TWI Status Register.
    twi_status = TW_STATUS & 0xF8;
    if ( (twi_status != TW_START) && (twi_status != TW_REP_START)) return 1;

    // send device address
    TWDR0 = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // wait until transmission completed and ACK/NACK has been received
    while(!(TWCR0 & (1<<TWINT)));

    // check value of TWI Status Register.
    twi_status = TW_STATUS & 0xF8;
    if ( (twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK) )
    {
        return 1;
    }
}

```



```

        return 0;
    }

// Send start condition, address, transfer direction.
// Use ack polling to wait until device is ready
void twi_start_wait(unsigned char address)
{
    uint8_t twi_status;

    while ( 1 )
    {
        // send START condition
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // wait until transmission completed
        while(!(TWCR0 & (1<<TWINT)));

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status != TW_START) && (twi_status != TW_REP_START)) continue;

        // send device address
        TWDR0 = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);

        // wait until transmission completed
        while(!(TWCR0 & (1<<TWINT)));

        // check value of TWI Status Register.
        twi_status = TW_STATUS & 0xF8;
        if ( (twi_status == TW_MT_SLA_NACK) || (twi_status == TW_MR_DATA_NACK) )
        {
            /* device busy, send stop condition to terminate write operation */
            TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

            // wait until stop condition is executed and bus released
            while(TWCR0 & (1<<TWSTO));

            continue;
        }
        break;
    }
}

// Send one byte to twi device, Return 0 if write successful or 1 if write failed
unsigned char twi_write( unsigned char data )
{
    // send data to the previously addressed device
    TWDR0 = data;

```

```

    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // wait until transmission completed
    while(!(TWCR0 & (1<<TWINT)));
    if( (TW_STATUS & 0xF8) != TW_MT_DATA_ACK) return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
//Return: 0 device accessible
//      1 failed to access device
unsigned char twi_rep_start(unsigned char address)
{
    return twi_start( address );
}

// Terminates the data transfer and releases the twi bus
void twi_stop(void)
{
    // send stop condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // wait until stop condition is executed and bus released
    while(TWCR0 & (1<<TWSTO));
}

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value)
{
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg)
{
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();

    return ret_val;
}

```

Τα ζητούμενα της 6^{ης} εργαστηριακής άσκησης

Ζήτημα 6.1

Να υλοποιηθεί κώδικας για το μικροελεγκτή ATmega328PB, σε γλώσσα C, ο οποίος να διαβάζει το πληκτρολόγιο της ntuAboard_G1, και να έχει τη δομή που περιγράφεται ακολούθως:

α) Να δημιουργηθεί μια συνάρτηση με όνομα `scan_row` η οποία να ελέγχει μια γραμμή του πληκτρολογίου για πιεσμένους διακόπτες.

β) Να δημιουργηθεί μια συνάρτηση με όνομα `scan_keypad` η οποία θα καλεί διαδοχικά την `scan_row` 4 φορές και θα ελέγχει ολόκληρο το πληκτρολόγιο.

γ) Να δημιουργηθεί μια συνάρτηση με όνομα `scan_keypad_rising_edge` η οποία θα καλείτε συνεχόμενα και θα κρατάει σε μια μεταβλητή με όνομα `pressed_keys` μία καταγραφή των πατημένων πλήκτρων.

Η συνάρτηση `scan_keypad_rising_edge` θα καλεί τη συνάρτηση `scan_keypad` και θα αποθηκεύει τα πλήκτρα που είναι πατημένα σε μια 16μπιτη μεταβλητή με όνομα `pressed_keys_tempo`. Προκειμένου να αντιμετωπιστεί το πρόβλημα του σπινθηρισμού, που παρουσιάζουν οι διακόπτες, θα καλεί ξανά την `scan_keypad`, μετά από χρονικό διάστημα της τάξης των 10 έως 20 mSec και θα συγκρίνει την τρέχουσα εικόνα των πατημένων πλήκτρων με αυτή που είναι αποθηκευμένη στη μεταβλητή `pressed_keys_tempo` και όσα πλήκτρα διαφέρουν θα απορρίπτονται τροποποιώντας ανάλογα τη μεταβλητή `pressed_keys_tempo`.

Η τροποποιημένη τιμή της μεταβλητής `pressed_keys_tempo` θα συγκρίνεται με την καταγραφή που είναι αποθηκευμένη στη μεταβλητή `pressed_keys` από προηγούμενη κλήση έτσι ώστε να εντοπίσει ποια πλήκτρα έχουν πατηθεί μόλις τώρα, απορρίπτοντας τα πλήκτρα που ήταν πατημένα και στην προηγούμενη κλήση της `scan_keypad_rising_edge`. Η τωρινή καταγραφή των πατημένων πλήκτρων στη μεταβλητή `pressed_keys_tempo` θα αντικαθιστά την καταγραφή στην μεταβλητή `pressed_keys` για χρήση στην επόμενη κλήση.

δ) Να δημιουργηθεί μια συνάρτηση με όνομα `keypad_to_ascii` που εντοπίζει τον διακόπτη που έχει πατηθεί και επιστρέφει τον κωδικό `ascii` του χαρακτήρα που αντιστοιχεί στον διακόπτη. Αν δεν είναι πιεσμένος κανένας διακόπτης επιστρέφει την τιμή 0. Θεωρείστε δεδομένο ότι κάθε φορά μπορεί να είναι πατημένο μόνο ένα πλήκτρο.

ε) Να αντιστοιχήσετε τέσσερα από τα πλήκτρα του πληκτρολογίου σε τέσσερα led σύμφωνα με τον παρακάτω πίνακα:

“A”	“8”	“6”	“*”
Led στο PB0	Led στο PB1	Led στο PB2	Led στο PB3

Κάθε φορά που θα πιέζεται κάποιο από τα παραπάνω τέσσερα πλήκτρα να ανάβει το αντίστοιχο led. Το led θα παραμένει αναμμένο όσο διάστημα το πλήκτρο είναι πατημένο.

Θεωρείστε δεδομένο ότι κάθε φορά μπορεί να είναι πατημένο μόνο ένα πλήκτρο.

Ζήτημα 6.2

Να υλοποιηθεί κώδικας για το μικροελεγκτή ATmega328PB, σε γλώσσα C, ο οποίος θα κάνει χρήση των συναρτήσεων του ζητήματος 6.1 και θα απεικονίζει στην οθόνη LCD 2x16 το χαρακτήρα που αντιστοιχεί στο πλήκτρο που πατήθηκε τελευταίο.

Θεωρείστε δεδομένο ότι κάθε φορά μπορεί να πατηθεί μόνο ένα πλήκτρο.

Ζήτημα 6.3

Γράψτε ένα πρόγραμμα «ηλεκτρονικής κλειδαριάς» το οποίο να ανάβει τα 6 leds PB0 έως PB5 για 3 sec, όταν δοθεί από το keypad 4×4 ο διψήφιος αριθμός της ομάδας σας (π.χ. 09). Εάν κάποιος σπουδαστής δεν έχει αριθμό ομάδας να κάνει χρήση του αριθμού 99. Αν δεν έχουν δοθεί σωστά τα δύο ψηφία του αριθμού της ομάδας τότε τα leds PB0 έως PB5 να αναβοσβήνουν(500 mSec ανάμεσα, 500 mSec σβηστά) για 5 sec. Ανεξάρτητα από το χρονικό διάστημα για το οποίο θα μείνει πατημένο ένα πλήκτρο, το πρόγραμμά θα πρέπει να θεωρεί ότι πατήθηκε μόνο μια φορά. Μετά το πάτημα δύο αριθμών το πρόγραμμα να μην δέχεται για 5 sec άλλον αριθμό. Το πρόγραμμα να είναι συνεχόμενης λειτουργίας. Δώστε το διάγραμμα ροής και το πρόγραμμα σε γλώσσα C.



ΑΘΗΝΑ, 8 Νοεμβρίου 2024

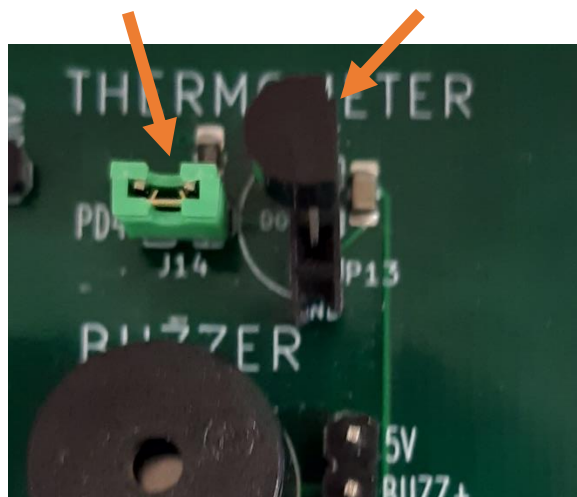
7^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Αισθητήρας Θερμοκρασίας DS1820 στην κάρτα ntuAboard_G1

Εξέταση – Επίδειξη: Παρασκευή 22/11/2024
Προθεσμία για παράδοση Έκθεσης: Τρίτη 26/11/2024 (23:59)

Χρήση Αισθητήρα Θερμοκρασίας DS1820 και Σειριακή Επικοινωνία 1-wire

Η σειριακή επικοινωνία ενός καλωδίου (1-wire serial communication) είναι ένα πρωτόκολλο που επιτρέπει τη μεταφορά δεδομένων από μία μόνο καλωδιακή σύνδεση. Η διαδικασία συντονίζεται από ένα μικροελεγκτή σε ρόλο master. Το πλεονέκτημα του συγκεκριμένου πρωτοκόλλου είναι ότι απαιτεί έναν μόνο ακροδέκτη του μικροελεγκτή. Όλες οι συσκευές που συνδέονται σε αυτόν τον ακροδέκτη έχουν έναν προκαθορισμένο κωδικό slave, που επιτρέπει στο μικροελεγκτή να αναγνωρίζει εύκολα με ποια συσκευή επικοινωνεί κάθε φορά.

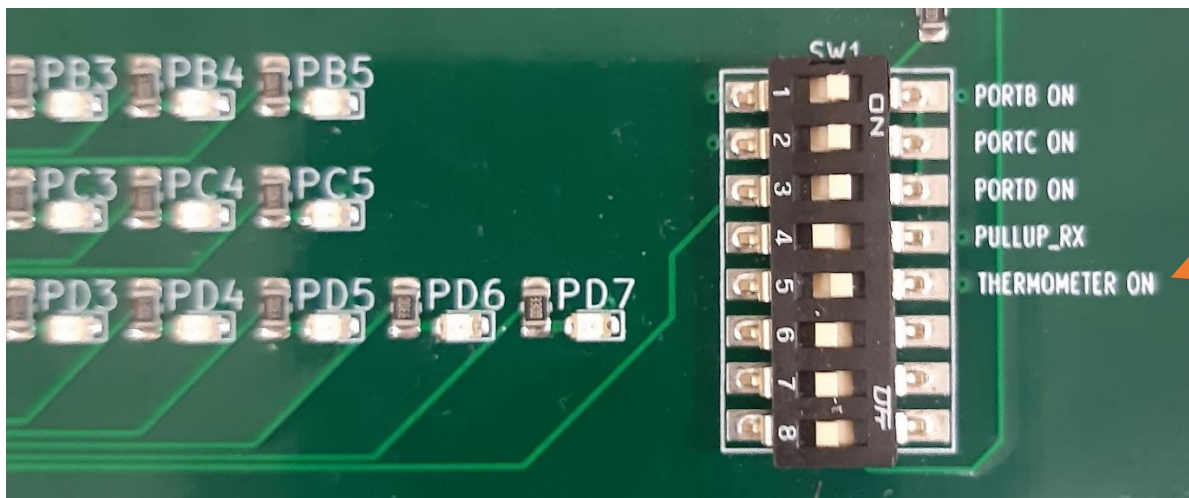
Η αναπτυξιακή πλακέτα *ntuAboard_G1* είναι εφοδιασμένη με τον αισθητήρα θερμοκρασίας DS1820 (σχήμα 7.1) που χρησιμοποιεί το πρωτόκολλο σειριακής επικοινωνίας ενός καλωδίου. Μετράει θερμοκρασίες από -55°C έως 125°C και παρέχει ακρίβεια $\pm 0.5^\circ\text{C}$ στην περιοχή θερμοκρασιών από -10°C έως 85°C. Για την τροφοδοσία του απαιτείται συνεχής τάση από 3V έως 5V. Ο αισθητήρας χρειάζεται μέγιστο χρόνο 750msec για να υπολογίσει τη θερμοκρασία με ακρίβεια 9 bit.



Σχήμα 7.1 Αισθητήρας θερμοκρασίας DS1820.

Στην αναπτυξιακή πλακέτα *ntuAboard_G1* ο αισθητήρας θερμοκρασίας DS1820 τοποθετείται στη δεξιά πάνω γωνία και για να συνδεθεί με τον ακροδέκτη PD4 του μικροελεγκτή, απαιτείται η χρήση ενός βραχυκυκλωτήρα στο κονέκτορα J14, όπως φαίνεται στο σχήμα 7.1.

Η σύνδεση της τροφοδοσίας στον αισθητήρα θερμοκρασίας DS1820 γίνεται δια μέσω του DIP Switch SW1-5 (THERMOMETER ON) όπως φαίνεται στο Σχήμα 7.2 παρακάτω



Σχήμα 7.2 Σύνδεση τροφοδοσίας στον αισθητήραDS1820

Η επικοινωνία του μικροελεγκτή με τον αισθητήρα θερμοκρασίας πραγματοποιείται δια μέσω του ακροδέκτη PD4. Η τοποθέτηση του αισθητήρα στην πλακέτα *ntuAboard_G1* γίνεται σε ειδική βάση, προσέχοντας το τυπωμένο ημικύκλιο στην πλακέτα να συμπίπτει με την ημικυκλική πλευρά του αισθητήρα, όπως στο σχήμα 7.1. Το ολοκληρωμένο DS1820 δεν είναι ένας απλός αισθητήρας αλλά μια σύνθετη συσκευή με ενσωματωμένο μετατροπέα ADC και πρωτόκολλο επικοινωνίας. Αν συνδεθεί λανθασμένα θα πάθει βλάβη.

Η επικοινωνία ενός καλωδίου είναι ένα πολύ απλό πρωτόκολλο που διαβάζει και γράφει σειριακά δεδομένα τηρώντας συγκεκριμένα χρονικά παράθυρα, όπως φαίνεται στο τεχνικό εγχειρίδιο του DS1820. Η σωστή λειτουργία του πρωτοκόλλου απαιτεί και την ύπαρξη μιας αντίστασης πρόσδεσης (pull-up) στον ακροδέκτη PD4.

Στη συνέχεια περιγράφεται ο κώδικας για την επικοινωνία του DS1820 με το μικροελεγκτή ATmega328PB που βρίσκεται πάνω στην αναπτυξιακή πλακέτα *ntuAboard_G1*.

Η ρουτίνα αρχικοποίησης **one_wire_reset** εκπέμπει έναν παλμό αρχικοποίησης στη γραμμή και ανιχνεύει εάν υπάρχει κάποια συνδεδεμένη συσκευή. Ο ακροδέκτης PD4 αρχικά ρυθμίζεται ως έξοδος και τίθεται σε κατάσταση λογικού 0, για 480usec. Στη συνέχεια μετά από 100usec, διαβάζεται η κατάσταση της θύρας PD, και ακολουθεί μία επιπλέον χρονική καθυστέρηση 380usec ώστε να ολοκληρωθεί ο κύκλος αρχικοποίησης. Αν κάποια συνδεδεμένη συσκευή αποκριθεί και θέσει το PD4 σε κατάσταση λογικού 0 τότε επιστρέφεται η τιμή 1 στον καταχωρητή r24, διαφορετικά επιστρέφεται η τιμή 0.

one_wire_reset:

```

sbi DDRD, PD4      ; set PD4 as output

cbi PORTD, PD4     ;
ldi r24, low(480)   ; 480 usec reset pulse
ldi r25, high(480)  ;
rcall wait_usec    ;

cbi DDRD, PD4      ; set PD4 as input
cbi PORTD, PD4     ; disable pull-up

ldi r24, 100        ; wait 100 usec for connected devices
ldi r25, 0          ; to transmit the presence pulse
rcall wait_usec    ;

in r24, PIND        ; read PORTD

push r24            ; save PORTD

ldi r24, low(380)   ;
ldi r25, high(380)  ; wait for 380 usec
rcall wait_usec    ;

pop r25             ; retrieve PORTD

clr r24             ; if a connected device is
sbrs r25, PD4       ; detected(PD4=0) return 1
ldi r24, 0x01       ; else return 0
ret

```

Η ρουτίνα **one_wire_receive_bit** έχει ως αντικείμενο το διάβασμα ενός bit, το οποίο αποθηκεύεται στον καταχωρητή **r24**, από τον αισθητήρα DS1820 δια μέσω του ακροδέκτη PD4, σύμφωνα με το πρωτόκολλο επικοινωνίας 1-wire serial communication.. Ξεκινά με εντολές που ρυθμίζουν τη θύρα PD ως έξοδο και ο ακροδέκτης PD4 τίθεται σε λογικό 0 για 2usec. Στη συνέχεια ρυθμίζει τη θύρα PD ως είσοδο, περιμένει 10usec, διαβάζει τον ακροδέκτη PD4 και γράφει την τιμή που διάβασε στον καταχωρητή r24. Τέλος δημιουργείται μια ακόμη χρονική καθυστέρηση 49usec (48usec για ολοκλήρωση του κύκλου ανάγνωσης και 1usec για αποκατάσταση) και η ρουτίνα επιστέφει.

one_wire_receive_bit:

```

sbi DDRD, PD4      ; set PD4 as output

cbi PORTD, PD4     ;
ldi r24, 0x02      ; time slot 2 usec
ldi r25, 0x00      ;
rcall wait_usec    ;

cbi DDRD, PD4      ; set PD4 as input
cbi PORTD, PD4     ; disable pull-up

ldi r24, 10        ;
ldi r25, 0          ; wait 10 usec
rcall wait_usec    ;

clr r24            ;
sbic PIND, PD4     ; r24 = PD4
ldi r24, 1         ;
push r24

ldi r24, 49        ;
ldi r25, 0          ; delay 49 usec to meet the standards
rcall wait_usec    ;

pop r24
ret

```


Αντίστοιχα η ρουτίνα **one_wire_transmit_bit**, έχει ως αντικείμενο την εγγραφή ενός bit, το οποίο βρίσκεται καταχωρητή **r24**, στον αισθητήρα DS1820 δια μέσω του ακροδέκτη PD4. Ξεκινά με εντολές που ρυθμίζουν τη θύρα PORTD ως έξοδο και ο ακροδέκτης PD4 τίθεται σε λογικό 0 για 2usec. Στη συνέχεια ο ακροδέκτης PD4 διατηρείται σε λογικό 0 ή μεταβαίνει σε λογικό 1 ανάλογα με την τιμή του καταχωρητή r24 και περιμένει 58usec μέχρι ο αισθητήρας θερμοκρασίας να διαβάσει αυτή την τιμή. Τέλος, η θύρα PORTD ρυθμίζεται ως είσοδος, δημιουργείται μια ακόμη χρονική καθυστέρηση 1usec για αποκατάσταση και η ρουτίνα επιστρέφει.

one_wire_transmit_bit:

```

push r24                ; save output bit

sbi DDRD, PD4           ; set PD4 as output

cbi PORTD, PD4          ;
ldi r24, 0x02           ; time slot 2 usec
ldi r25, 0x00           ;
rcall wait_usec         ;

pop r24                 ; retrieve output bit

sbrc r24, 0             ;
sbi PORTD, PD4          ; PD4 = r24[0]
sbrs r24, 0             ;
cbi PORTD, PD4          ;

ldi r24, 58             ; wait 58 usec for connected
ldi r25, 0              ; device to sample the line
rcall wait_usec         ;

cbi DDRD, PD4           ; set PD4 as input
cbi PORTD, PD4          ; disable pull-up

ldi r24, 0x01           ;
ldi r25, 0x00           ; recovery time 1 usec
rcall wait_usec         ;

ret

```

Η ρουτίνα **one_wire_receive_byte** έχει ως αντικείμενο τη λήψη ενός byte από τον αισθητήρα DS1820, καλώντας τη ρουτίνα `one_wire_receive_bit` 8 φορές ξεκινώντας από το λιγότερο σημαντικό bit. Το λαμβανόμενο byte αποθηκεύεται στον καταχωρητή **r24**.

```
one_wire_receive_byte:
    ldi r27,8                ; 8 repetitions
    clr r26

loop_:
    rcall one_wire_receive_bit ; r24[0] holds the received bit

    lsr r26

    sbrc r24,0                ; logical or with 0x00 if returned bit is 0,
    ldi r24,0x80                ; or
    or r26,r24                 ; with 0x80 if returned bit is 1

    dec r27
    brne loop_
    mov r24,r26
    ret
```

Η ρουτίνα **one_wire_transmit_byte** έχει ως αντικείμενο την εγγραφή ενός byte στον αισθητήρα DS1820, καλώντας τη `one_wire_transmit_bit` 8 φορές ξεκινώντας από το λιγότερο σημαντικό bit. Το εκπεμπόμενο byte αποθηκεύεται στον καταχωρητή **r24**.

```
one_wire_transmit_byte:
    mov r26,r24
    ldi r27,8                ; 8 repetitions

_one_more_:
    clr r24                    ;
    sbrc r26,0                ; load r24[0] with the transmitted bit
    ldi r24,0x01                ;

    rcall one_wire_transmit_bit

    lsr r26
    dec r27
    brne _one_more_
    ret
```

Τα ζητούμενα της 4^{ης} εργαστηριακής άσκησης

Ζήτημα 4.1 (α) Να υλοποιηθεί ένα πρόγραμμα σε γλώσσα C, για την επικοινωνία του μικροελεγκτή ATmega328PB με τον αισθητήρα θερμοκρασίας DS18B20 στην κάρτα *NtuAboard_G1*.

Το πρόγραμμα να περιλαμβάνει μόνο τις στοιχειώδεις λειτουργίες, λαμβάνοντας υπόψη ότι στον ακροδέκτη PD4 είναι συνδεδεμένος μόνο ένας αισθητήρας. Οι στοιχειώδεις αυτές λειτουργίες στηρίζονται στις εντολές που δέχεται ο αισθητήρας και παρουσιάζονται στο τεχνικό εγχειρίδιου του DS18B20.

Οι εντολές αυτές μπορούν να χωριστούν σε **εντολές μνήμης**, με τις οποίες επιλέγεται η συσκευή με την οποία θα γίνει η επικοινωνία, και **εντολές λειτουργίας**, με τις οποίες υλοποιούνται λειτουργίες που έχουν σχέση με τη **μέτρηση θερμοκρασίας**. Ενδιαμέσως, ο αισθητήρας βρίσκεται σε κατάσταση **χαμηλής κατανάλωσης ισχύος** και επανέρχεται σε κατάσταση λειτουργίας με μια **εντολή αρχικοποίησης**.

Ειδικότερα να γραφεί ρουτίνα που να επιστρέφει στον διπλό καταχωρητή r25:r24 την τιμή της θερμοκρασίας και σε περίπτωση που δεν υπάρχει συνδεδεμένη συσκευή να επιστρέφει την τιμή 0x8000.

Ποιο αναλυτικά η ρουτίνα να περιλαμβάνει τα εξής:

- Αρχικοποίηση και έλεγχος αν υπάρχει συνδεδεμένη συσκευή με την εκτέλεση της ρουτίνας *one_wire_reset*.
- Χρήση της ρουτίνας *one_wire_transmit_byte* για αποστολή της εντολής **0xCC**, η οποία παρακάμπτει την επιλογή συσκευής από διάδρομο πολλών συσκευών, μιας και είναι δεδομένο ότι στην κάρτα *NtuAboard_G1* υπάρχει μόνο μία συνδεδεμένη συσκευή.
- Χρήση της ρουτίνας *one_wire_transmit_byte* για αποστολή της εντολής **0x44**, η οποία ξεκινάει μια μέτρηση θερμοκρασίας. Όταν η μέτρηση θερμοκρασίας ολοκληρωθεί, το DS18B20 σηματοδοτεί τον τερματισμό της μετατροπής με την αποστολή ενός bit με τιμή 1, το οποίο ελέγχεται με την εκτέλεση της ρουτίνας *one_wire_receive_bit*.
- Νέα αρχικοποίηση της συνδεδεμένης συσκευής με την εκτέλεση της ρουτίνας *one_wire_reset*.
- Χρήση της ρουτίνας *one_wire_transmit_byte* για αποστολή της εντολής **0xCC**.
- Χρήση της ρουτίνας *one_wire_transmit_byte* για αποστολή της εντολής **0xBE** για ανάγνωση των 16 bit της μετρημένης θερμοκρασίας, η οποία πραγματοποιείται με κλίση της ρουτίνας *one_wire_receive_byte* δύο φορές και αποθήκευση της τιμής στο ζεύγος καταχωρητών r25:r24. Η τιμή που διαβάζεται, σύμφωνα με όσα αναφέρονται στο τεχνικό εγχειρίδιο, είναι ένας αριθμός σε μορφή συμπληρώματος ως προς 2. Τα 8 λιγότερα σημαντικά bit αντιστοιχούν σε θερμοκρασία βαθμών Κελσίου, με **ακρίβεια 0.5°C** ανά bit, και τα 8 περισσότερο σημαντικά αποτελούν επέκταση προσήμου (στην προκαθορισμένη λειτουργία του αισθητήρα). Για παράδειγμα, ο αριθμός 0x0032 αντιστοιχεί σε θερμοκρασία 25°C και ο 0xFFFF σε -0.5°C.
- Η ρουτίνα συμπληρώνεται από μια εντολή που επιστρέφει την τιμή 0x8000 σε περίπτωση που δεν υπάρχει συσκευή συνδεδεμένη στον ακροδέκτη PD4.

Το πρόγραμμα πρέπει να έχει παρόμοια δομή με την assembly, δηλ. οι συναρτήσεις της C να αντιστοιχούν υποχρεωτικά στις ρουτίνες που σας δίνονται.

Αντί για τον αισθητήρα DS18B20 μπορεί να χρησιμοποιηθεί και ο αισθητήρας DS18B20 με τη διαφορά ότι στην προεπιλεγμένη ρύθμιση της λειτουργίας του DS18B20, η τιμή της θερμοκρασίας μετριέται με ακρίβεια 0.0625 °C και δίνεται σε μορφή συμπληρώματος ως προς 2 σύμφωνα με τον παρακάτω πίνακα:

	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LS BYTE	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴
	BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
MS BYTE	S	S	S	S	S	2 ⁶	2 ⁵	2 ⁴

S = SIGN

Για παράδειγμα, ο αριθμός 0x0190 αντιστοιχεί σε θερμοκρασία 25°C και ο 0xFFFF σε -0.5°C.

Ζήτημα 4.2 Γράψτε πρόγραμμα σε C που με την χρήση της προηγούμενης ρουτίνας να απεικονίζει τη θερμοκρασία σε °C στο LCD display σε δεκαδική τιμή τριών ψηφίων με το πρόσημο (**-55°C** έως **+125°C**).

Επίσης στην περίπτωση που δεν υπάρχει συσκευή συνδεδεμένη να εμφανίζει το μήνυμα **“NO Device”**.



8^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΓΙΑ ΤΟ ΜΑΘΗΜΑ "Εργαστήριο Μικροϋπολογιστών"
Συνδυαστική/Επαναληπτική άσκηση – Εφαρμογή Internet of Things (στο ntuAboard)
Εξέταση – Επίδειξη: Παρασκευή 6/12/2024
Προθεσμία για παράδοση Έκθεσης: Τρίτη 10/12/2024 23:59

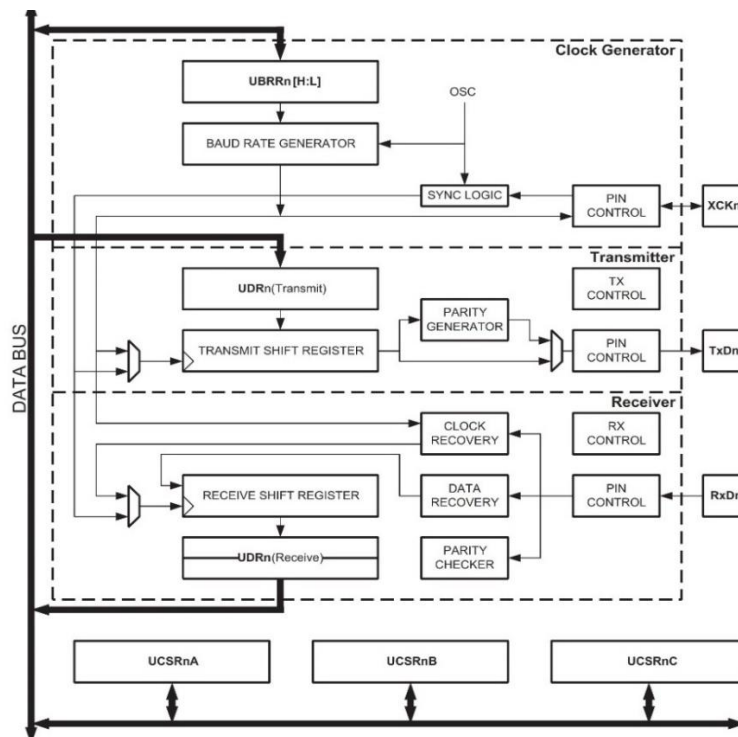
Ασύγχρονη σειριακή επικοινωνία UART

Ο ATmega328PB είναι εξοπλισμένος με δυο μονάδες USART (Universal Synchronous and Asynchronous serial Receiver and Transmitter) οι οποίες μπορούν να χρησιμοποιηθούν για Σύγχρονη αλλά και για Ασύγχρονη επικοινωνία. Στο πλαίσιο αυτής της άσκησης θα μελετηθεί ο Ασύγχρονος τρόπος επικοινωνίας (UART) και η μονάδα USART1. Οι δυο μονάδες είναι ισοδύναμες και για αυτό υπάρχει σε όλα τα σχηματικά το γράμμα n, όπου n=0 για την πρώτη μονάδα και n=1 για την δεύτερη.

Η μονάδα UART μπορεί να μεταδώσει 30 συνδυασμούς πλαισίου:

- 1 bit εκκίνησης
- 5,6,7,8 ή 9 bits
- Ένα η δύο bit λήξης
- Καμία, άρτια ή περιττή ισοτιμία (parity)

Η βασική δομή φαίνεται στο παρακάτω σχήμα:



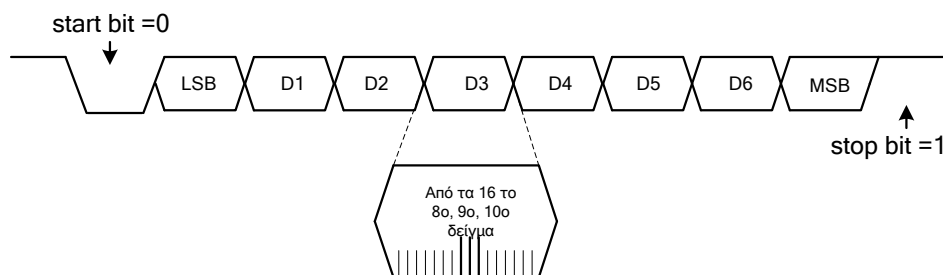
Σχήμα 8.1. Κυκλωματικό διάγραμμα Μονάδας UART.

Ο ακροδέκτης XCKn (Transfer Clock) χρησιμοποιείται μόνο στο σύγχρονο τρόπο επικοινωνίας.

Στο ρυθμό μετάδοσης υπάρχει διαίρεση με το 16. Ο πομπός χρησιμοποιεί την διαιρεμένη συχνότητα για την αποστολή ενός bit ενώ ο δέκτης χρησιμοποιεί το 16πλάσιο της συχνότητας αυτής ώστε να δειγματοληπτεί το σήμα. Από τα δείγματα που λήφθηκαν χρησιμοποιούνται το 8^ο, το 9^ο και το 10^ο και με τον κανόνα της πλειοψηφίας αποφασίζεται αν λήφθηκε λογικό 1, λογικό 0 ή αν πρόκειται για θόρυβο στην περίπτωση που αναμένεται start bit (που πρέπει οπωσδήποτε να είναι 0).

Όταν αναγνωριστεί το start bit πραγματοποιείται ένας "συγχρονισμός" του δέκτη με τον πομπό, αφού αναγνωρίζεται η μετάβαση από λογικό 1 σε λογικό 0 και στη συνέχεια το επόμενο bit θα καταφθάσει σε συγκεκριμένο χρόνο (1/BAUD).

Ο ρυθμός μετάδοσης, ο αριθμός των bit που αναμένονται, ο αριθμός των stop bit και το είδος του parity bit πρέπει να είναι όλα γνωστά στον δέκτη για να λειτουργήσει σωστά η επικοινωνία.



Σχήμα 8.2 Δειγματοληψία δεδομένου

Περισσότερες πληροφορίες σχετικά με την λειτουργία των καταχωρητών αλλά και γενικότερα της UART μπορείτε να βρείτε στο datasheet [ATmega328PB](#).

Καταχωρητές

Καταχωρητής δεδομένων (UDRn)

Bit	7	6	5	4	3	2	1	0	
	RXB[7:0]								UDRn (Read)
	TXB[7:0]								UDRn (Write)
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Από το σχήμα φαίνεται ότι πομπός και δέκτης μοιράζονται τον καταχωρητή δεδομένων UDRn. Στην πραγματικότητα όμως πρόκειται για δύο διαφορετικούς καταχωρητές που έχουν κοινή φυσική διεύθυνση 0xC6 (UDR0). Ο ένας είναι ο καταχωρητής δεδομένων εκπομπής όπου μεταφέρονται τα δεδομένα που γράφουμε στη διεύθυνση 0xC6. Ο άλλος είναι ο καταχωρητής δεδομένων λήψης όπου μεταφέρονται τα δεδομένα που διαβάζουμε από τη διεύθυνση 0xC6. Ο καταχωρητής δεδομένων λήψης είναι δομημένος σε FIFO 2 επιπέδων και η κατάσταση του αλλάζει σε κάθε προσπέλαση. Γι αυτό το λόγω δε πρέπει να χρησιμοποιούνται οι εντολές SBI και CBI και χρειάζεται ιδιαίτερη προσοχή στη χρήση των SBIC και SBIS αφού και αυτές αλλάζουν την κατάσταση του καταχωρητή.

Καταχωρητής κατάστασης (UCSRnA)

Bit	7	6	5	4	3	2	1	0	
	RXCn	TXCn	UDREN	FEn	DORn	UPEn	U2Xn	MPCMn	UCSRnA
Read/Write	R	R/W	R	R	R	R	R/W	R/W	
Initial Value	0	0	1	0	0	0	0	0	

RXCn: Αν είναι λογικό 1 τότε υπάρχουν δεδομένα προς ανάγνωση. Μετά την ανάγνωση τους η τιμή μηδενίζεται.

TXCn: Τίθεται σε λογικό 1 όταν έχει ολοκληρωθεί η ολίσθηση όλων των δεδομένων και δεν υπάρχουν καινούρια δεδομένα στον καταχωρητή UDRn.

UDREN: Αν είναι λογικό 1 ο καταχωρητής αποστολής δεδομένων UDRn είναι έτοιμος να δεχθεί νέα δεδομένα.

Καταχωρητής ελέγχου (UCSRnB)

Bit	7	6	5	4	3	2	1	0	
	RXCIEn	TXCIEn	UDRIEn	RXENn	TXENn	UCSZn2	RXB8n	TXB8n	UCSRnB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

RXENn: Αν είναι λογικό 1 ενεργοποιείται το τμήμα λήψης της μονάδας UART

TXENn: Αν είναι λογικό 1 ενεργοποιείται το τμήμα εκπομπής της μονάδας UART

Καταχωρητής ελέγχου (UCSRnC)

Bit	7	6	5	4	3	2	1	0	
	UMSELn1	UMSELn0	UPMn1	UPMn0	USBSn	UCSZn1	UCSZn0	UCPOLn	UCSRnC
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	1	1	0	

UMSELn1:0: Επιλογή του τρόπου λειτουργίας της USART. 00 για Ασύγχρονη λειτουργία.

UCSZn1:0: Τα bit UCSZn1:0 σε συνδυασμό με το bit UCSZn2 του καταχωρητή UCSRnB καθορίζουν τον αριθμό των bit δεδομένων (Character Size) που ανταλλάσσουν πομπός και δέκτης. Συγκεκριμένα:

Table 66. UCSZ Bits Settings

UCSZ2	UCSZ1	UCSZ0	Character Size
0	0	0	5-bit
0	0	1	6-bit
0	1	0	7-bit
0	1	1	8-bit
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	9-bit

Καταχωρητής του ρυθμού μετάδοσης (UBRRn)

Bit	15	14	13	12	11	10	9	8	
	—	—	—	—	UBRRn[11:8]				UBRRnH
	UBRRn[7:0]								UBRRnL
	7	6	5	4	3	2	1	0	
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	0	

UBRR11:0: Πρόκειται για έναν καταχωρητή 12-bit που περιέχει το ρυθμό μετάδοσης. Ο καταχωρητής UBRRnH περιέχει τα 4 MSB, και ο UBRRnL περιλαμβάνει τα 8 LSB. Εγγραφή στον UBRRnL ανανεώνει άμεσα τον ρυθμό και θα προκαλέσει αλλοίωση των δεδομένων σε περίπτωση που γίνει κατά τη διάρκεια εκπομπής.

Για τον υπολογισμό του ρυθμού μετάδοσης BAUD από την τιμή του καταχωρητή ισχύει ο τύπος

$$BAUD = \frac{f_{osc}}{16(UBRR + 1)}$$

όπου f_{osc} η συχνότητα λειτουργίας του μικροελεγκτή. Λύνοντας ως προς UBRR έχουμε:

$$UBRR = \frac{f_{osc}}{16BAUD} - 1$$

Για να έχουμε BAUD=9600 και αφού ο μικροελεγκτής μας λειτουργεί στα 16MHz πρέπει UBRR=103.1667. Στρογγυλοποιούμε στο 103, εισάγοντας ένα σφάλμα 0,16%.

Για να αρχικοποιήσουμε την USART μπορούμε να χρησιμοποιήσουμε την εξής ρουτίνα:

```
/* Routine: usart_init
Description:
This routine initializes the
usart as shown below.
----- INITIALIZATIONS -----

Baud rate: 9600 (Fck= 8MH)
Asynchronous mode
Transmitter on
Reciever on
Communication parameters: 8 Data ,1 Stop, no Parity
-----
parameters: ubrr to control the BAUD.
return value: None.*/

void usart_init(unsigned int ubrr){
    UCSRA=0;
    UCSRB=(1<<RXEN0)|(1<<TXEN0);
    UBRR0H=(unsigned char)(ubrr>>8);
    UBRR0L=(unsigned char)ubrr;
    UCSRC=(3 << UCSZ00);
    return;
}

/* Routine: usart_transmit
Description:
This routine sends a byte of data
```

```

using usart.
parameters:
data: the byte to be transmitted
return value: None. */

void usart_transmit(uint8_t data){
    while(!(UCSR0A&(1<<UDRE0)));
    UDR0=data;
}

/* Routine: usart_receive
Description:
This routine receives a byte of data
from usart.
parameters: None.
return value: the received byte */

uint8_t usart_receive(){
    while(!(UCSR0A&(1<<RXC0)));
    return UDR0;
}

```

Σύντομη εισαγωγή στο Internet of Things

Η ραγδαία ανάπτυξη του Internet of Things (IoT) έχει επιφέρει σημαντική αύξηση των συσκευών που είναι συνδεδεμένες στο διαδίκτυο. Ο αριθμός αυτός εξακολουθεί να αυξάνεται με ταχύτατους ρυθμούς. Συνέπεια αυτού είναι η παραγωγή τεράστιου όγκου πληροφοριών προς επεξεργασία.

Τι είναι οι Συσκευές IoT:

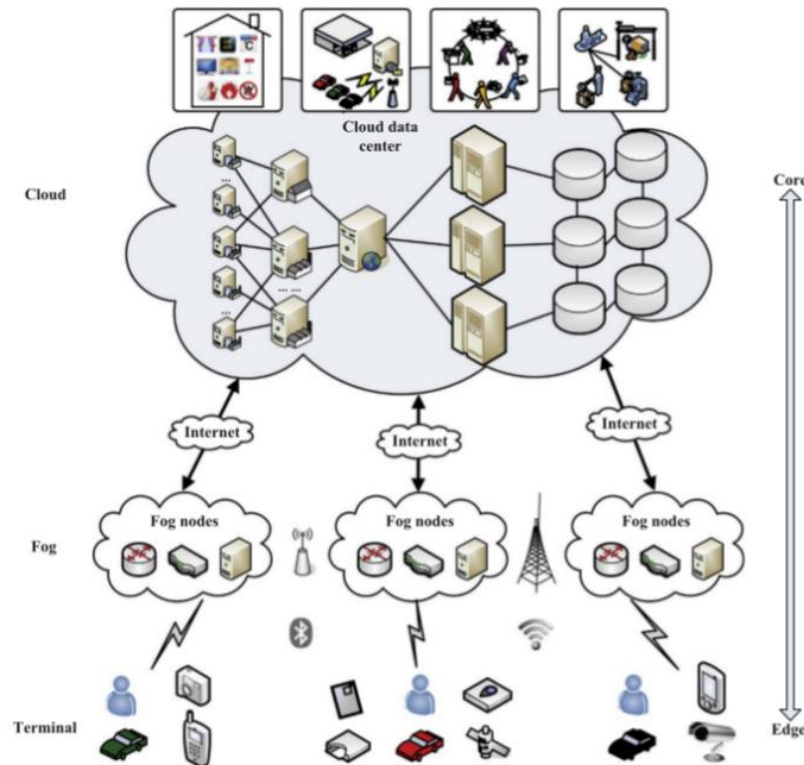
Ως συσκευές IoT ορίζουμε κάθε είδους φυσική συσκευή, οχήματα, οικιακές συσκευές και άλλα αντικείμενα, στα οποία υπάρχουν ενσωματωμένα αισθητήρες, λογισμικό, ενεργοποιητές και σύνδεση στο διαδίκτυο, που τους επιτρέπει να επικοινωνούν και να ανταλλάσσουν δεδομένα είτε μεταξύ τους, είτε με το υπόλοιπο διαδίκτυο. Αυτές οι συσκευές έχουν περιορισμένους πόρους σε ό,τι αφορά την cpu, τη μνήμη και τις δικτυακές τους ικανότητες.

Τι είναι IoT Gateway:

Καθώς δισεκατομύρια συσκευές συνδέονται πλέον στο δίκτυο, το «Gateway» χρειάζεται προκειμένου να συνδέει υπο-δίκτυα διαφορετικών αρχιτεκτονικών και σε διαφορετικά περιβάλλοντα. Στην κλασική δομή του IoT χρησιμοποιείται ως διαμεσολαβητής/γέφυρα μεταξύ των τελικών συσκευών και του Cloud διαμορφώνοντας κατάλληλα τα δεδομένα που στέλνονται και λαμβάνονται. Τα Gateways συνδέουν επίσης στο Cloud υπο-δίκτυα που στο εσωτερικό τους μπορεί να χρησιμοποιούν διαφορετικά πρωτόκολλα επικοινωνίας. Τα Gateways εποπτεύουν την διαδικασία επικοινωνίας εντός και εκτός του υπο-δικτύου στο οποίο βρίσκονται. Μπορούν να εκτελούν διεργασίες ελέγχου, ασφάλειας, φιλτραρίσματος των δεδομένων καθώς επίσης να κάνουν σε χαμηλότερο επίπεδο επεξεργασία των δεδομένων που λαμβάνουν (πιο κοντά στις συσκευές και όχι στο Cloud). Τέλος, μεγάλος αριθμός δεδομένων που λαμβάνονται από τους αισθητήρες των υπο-δικτύων IoT, χρειάζεται το Gateway ως σύστημα συντονισμού και ελέγχου της μεταφοράς τους όχι μόνο από μία συσκευή στο Cloud αλλά και από συσκευή σε συσκευή.

Edge / Fog Computing:

Ωστόσο, τα τελευταία χρόνια, λόγω του γεγονότος ότι οι υποδομές του Cloud είναι γεωγραφικά κεντριοποιημένες και απομακρυσμένες από τους χρήστες, οι εφαρμογές που απαιτούν επεξεργασία σε πραγματικό χρόνο, με χαμηλό latency αδυνατούν να εκτελεστούν επιτυχώς. Αυτό συμβαίνει λόγω μεγάλου χρόνου μεταφοράς των δεδομένων, συμφόρησης του δικτύου και υποβάθμισης της ποιότητας. Επιπλέον, σε εφαρμογές που απαιτείται ιδιωτικότητα των δεδομένων (π.χ. ιατρικές εφαρμογές), αυτή δε μπορεί να διασφαλιστεί απόλυτα σε απομακρυσμένες υποδομές. Αυτοί οι λόγοι οδήγησαν στη δημιουργία του Edge και του Fog Computing, όπου οι απαιτούμενοι υπολογισμοί γίνονται κοντά στις IoT συσκευές. Το σχήμα της αρχιτεκτονικής αυτής παρουσιάζεται στην Εικόνα 1. Έτσι οι χρήστες ανεξαρτητοποιούνται από τις υποδομές του Cloud και, ταυτόχρονα, ενισχύεται η προστασία των προσωπικών τους δεδομένων.



Εικόνα 1 Αρχιτεκτονική Fog και Edge Computing¹

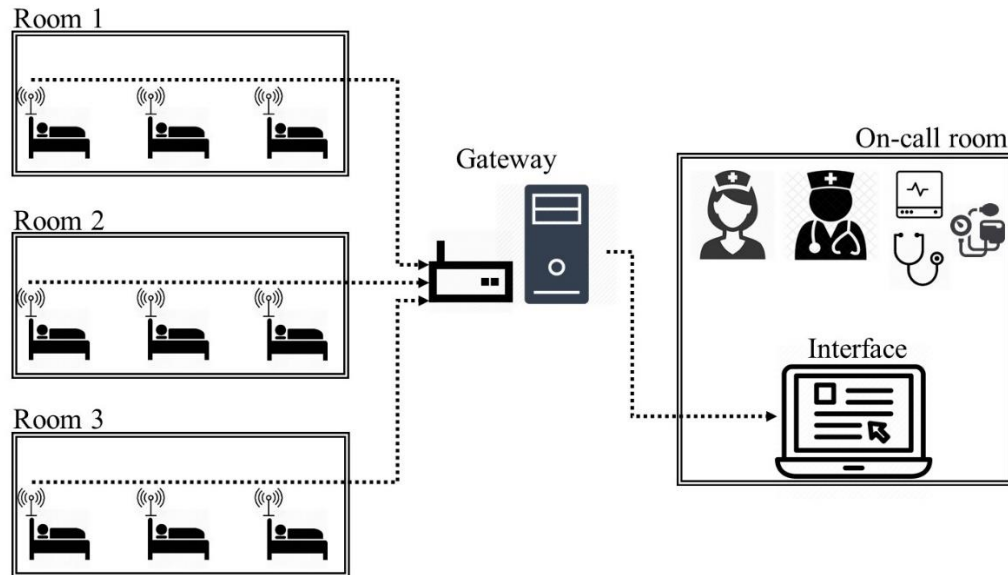
Από τη μία πλευρά, οι συσκευές IoT βρίσκονται κοντά στους καθημερινούς χρήστες και πιο συγκεκριμένα, στο terminal layer της αρχιτεκτονικής που φαίνεται στην Εικόνα 1. Από την άλλη πλευρά ο Edge Node (Gateways του Edge Computing, τα οποία συνεισφέρουν στην ανάπτυξη υπηρεσιών Edge και στη παροχή υπολογιστικών, αποθηκευτικών και δικτυακών πόρων στις συσκευές IoT) βρίσκεται στη διεπαφή του Fog layer και του terminal layer. Επιπροσθέτως, οι συσκευές IoT είναι συνδεδεμένες, είτε ενσύρματα, είτε ασύρματα, με το Gateway κόμβο και μπορούν να επικοινωνούν και να ανταλλάσσουν δεδομένα με εκείνον.

Η αρχιτεκτονική του συστήματος που θα μελετήσουμε στα πλαίσια της παρούσας εργασίας αποτελείται από ένα σύνολο n IoT συσκευών (npuAboard πλακέτες) και μίας συσκευής Gateway. Δεδομένα από αισθητήρες συνδεδεμένους στις συσκευές μπορούν να σταλούν προς επεξεργασία στο Gateway, ενώ τον Gateway μπορεί να στέλνει δεδομένα πίσω στις συσκευές.

¹ P. Hu, S. Dhelim, H. Ning, and T. Qiu. Survey on fog computing: architecture, key technologies, applications and open issues. Journal of Network and Computer Applications, 2017

Περιγραφή της εφαρμογής

Στην παρούσα άσκηση θα μελετηθεί μια εφαρμογή νοσοκομείου. Η Εικόνα 2 δίνει το γενικό σχήμα της εφαρμογής. Το κάθε κρεβάτι (IoT node) έχει συνδεδεμένους αισθητήρες στον ασθενή, τα δεδομένα των οποίων στέλνονται στο Gateway. Το Gateway επεξεργάζεται τα δεδομένα και είτε στέλνει κάποια απάντηση στο IoT node προκειμένου να γίνει κάποια αυτόματη λειτουργία, είτε στέλνει τα δεδομένα στο δωμάτιο των νοσηλευτών και των γιατρών. Τα δεδομένα παρουσιάζονται σε μια οθόνη (Interface) ούτως ώστε να μελετηθούν από το προσωπικό του νοσοκομείου προκειμένου να προβεί στις κινήσεις που απαιτούνται.



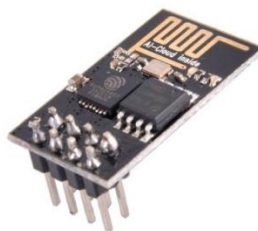
Εικόνα 2 Η εφαρμογή IoT που θα μελετηθεί στην παρούσα άσκηση

Η κάθε συσκευή IoT που βρίσκεται σε κάθε κρεβάτι θα είναι μια πλακέτα ntuAboard, το πρόγραμμα της οποίας θα γραφτεί από την κάθε ομάδα. Ο αριθμός της ομάδας θα είναι ο αριθμός του κρεβατιού. Για λόγους απλότητας ο ένας αισθητήρας που θα χρησιμοποιήσετε θα είναι ο αισθητήρας θερμοκρασίας DS18B20 που μελετήθηκε σε προηγούμενη Εργαστηριακή άσκηση. Για τον δεύτερο αισθητήρα θα χρησιμοποιηθεί ένα ποτενσιόμετρο για να προσομοιώσει την λειτουργία ενός αισθητήρα κεντρικής φλεβικής πίεσης 0-20 cm H₂O.

Το Gateway θα είναι ήδη προγραμματισμένο και το Interface του δωματίου προσωπικού του νοσοκομείου θα παρουσιάζεται στον προβολέα του εργαστηρίου. Αν κάποια ομάδα το επιθυμεί, υπάρχει δυνατότητα να αναπτύξει δικό της πρόγραμμα για το Gateway.

Ο πομποδέκτης WiFi ESP8266:

Ο πομποδέκτης Wi-Fi που θα χρησιμοποιήσουμε για την παραπάνω εφαρμογή είναι ο ESP8266 (Εικόνα 3). Πρόκειται για έναν προγραμματίσιμο πομποδέκτη ο οποίος έχει προγραμματιστεί να δέχεται εντολές μέσω σειριακής και να στέλνει αντίστοιχη απάντηση.



Εικόνα 3 Ο πομποδέκτης WiFi ESP8266

Συγκεκριμένα, κάποιες από τις εντολές που δέχεται φαίνονται στον παρακάτω πίνακα:

command	usage
baudrate: "Your baudrate"	Resets the Serial to the required speed
configuration	Reports your current configuration
connect	Uses the provided SSID and Password to connect to the WiFi network
debug: "true or false"	Every command will send extra debug information if set to true
help	Lists all commands
password: "Your Password"	Password to be used for the connection
payload: [Your payload]	The payload to post to the Host. Must be in brackets
restart	Restarts the ESP
ssid: "Your SSID"	SSID to be used for the connection
transmit	Transmits the specified payload to the url
url: "Your url"	The url to transmit to. Example: http://192.168.1.250:5000/data"

Προκειμένου το ESP8266 να αναγνωρίσει ότι πρέπει να διαβάσει τα δεδομένα της σειριακής, οτιδήποτε στείλετε πρέπει να ξεκινάει με ESP: (π.χ. για σύνδεση θα στείλετε ESP:connect).

Το ESP μπορεί να συνδεθεί στον server σαν client και να στέλνει αιτήματα στον server για να γίνει ανταλλαγή δεδομένων.

Δώστε προσοχή στα εξής:

- 1) Σε όποια εντολή υπάρχουν τα "" πρέπει να τα στέλνετε. Κάθε εντολή που στέλνετε καθώς και κάθε απάντηση που λαμβάνετε θα έχει στο τέλος τον χαρακτήρα αλλαγής γραμμής '\n'.
- 2) Μερικές φορές οι client αποσυνδέονται οπότε να ελέγχετε κάθε φορά με την εντολή connect και μπορείτε σε περίπτωση που αποτύχει να δοκιμάζετε άλλη μια φορά (σε περίπτωση αποτυχίας υπάρχει timeout 20sec όπου το ESP δεν θα αποκρίνεται).
- 3) Μπορείτε στην αρχή του προγράμματος σας να στέλνετε την εντολή restart για να σιγουρευτείτε ότι το ESP βρίσκεται στις default επιλογές του. Μετά την επανεκκίνηση το ESP τυπώνει μια γραμμή με κάποιες πληροφορίες και την γραμμή ESP8266: Waiting for command.
- 4) Το payload που δέχεται ο server πρέπει να είναι σε μορφή json και συγκεκριμένα η εντολή πρέπει να είναι στην μορφή:

```
payload:
[{"name": "temperature", "value": "36.0"}, {"name": "pressure", "value": "60.0"}, {"name": "team", "value": "5"}, {"name": "status", "value": "OK"}]
```

- 5) Η μεταβλητή team είναι απαραίτητη στο payload και σε περίπτωση που δεν την συμπεριλάβετε θα λάβετε ειδικό μήνυμα λάθους από το gateway. Σε περίπτωση που λείπει κάποια από τις άλλες μεταβλητές η έχετε κάνει τυπογραφικό θα συμπληρωθεί με None.
- 6) Τα ονόματα των μεταβλητών στο payload είναι αυστηρά. Σε όποια εντολή υπάρχουν τα "" πρέπει να τα στέλνετε. Κάθε εντολή πρέπει να τελειώνει με τον χαρακτήρα αλλαγής γραμμής '\n'. Τα ssid, password και baudrate(9600) **έχουν ήδη ρυθμιστεί για εσάς.**

Default τιμές:

baudrate	9600
debug	false
password	Microlab_IoT
ssid	Micro_IoT

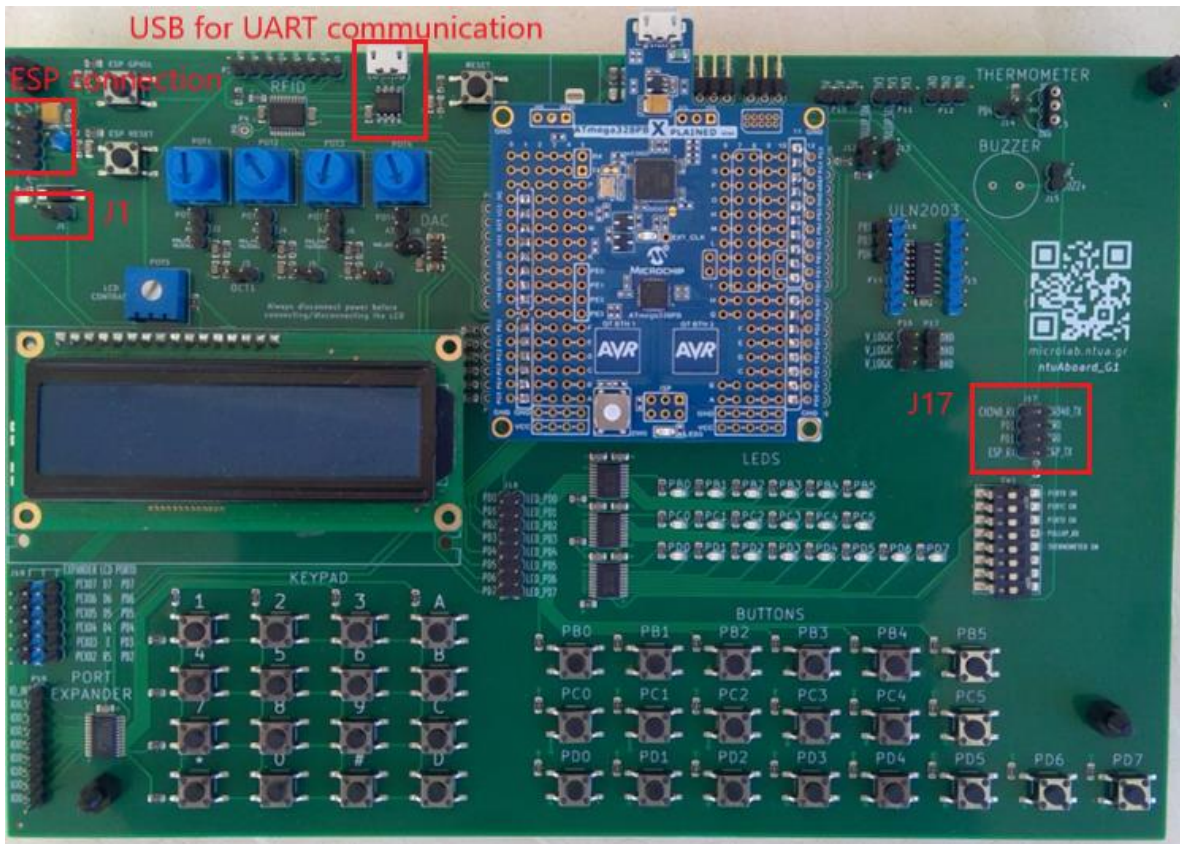
Σας δίνεται παράδειγμα επικοινωνίας με το ESP:

SENDER	MESSAGE
AVR	ESP:connect
ESP	"Success"
AVR	ESP:url:"http://192.168.1.250:5000/data"
ESP	"Success"
AVR	ESP:payload:[{"name": "team", "value": "5"}]
ESP	"Success"
AVR	ESP:transmit
ESP	200 OK

Jumpers

Η UART μπορεί μέσω του J17 να συνδεθεί είτε με το ESP8266 είτε με το CH340N το οποίο είναι ένα ολοκληρωμένο υπεύθυνο για την μετατροπή των σημάτων του διαύλου USB σε σήματα σειριακής επικοινωνίας και το αντίστροφο. Αυτό δίνει την δυνατότητα στον υπολογιστή σας να επικοινωνήσει με τη θύρα UART του ATmega328 αν συνδέσετε ένα επιπλέον καλώδιο USB από τον υπολογιστή σας στο αριστερό μέρος του ntuAboard που φαίνεται στην εικόνα. Για την σύνδεση με το ESP8266 χρειάζεται να συνδέσετε το ESP8266 στο ntuAboard, τα jumper στα J17 και J1 όπως φαίνονται στις παρακάτω εικόνες.

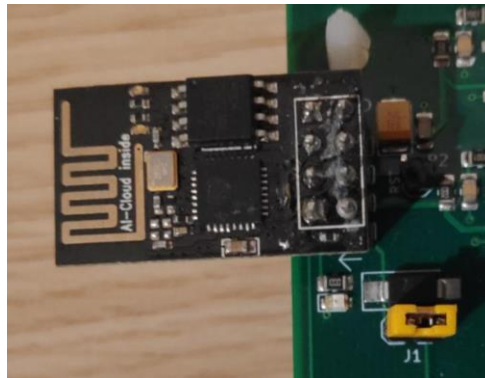
ΠΡΟΣΟΧΗ: Συνδέστε το ESP8266 μόνο με τη φορά που φαίνεται στην εικόνα!



Εικόνα 4 ESP8266 και UART στο ntuAboard



Εικόνα 5 Jumper στο J17 για επικοινωνία με ESP8266 (αριστερά) ή το CH340 (δεξιά)



Εικόνα 6 Σύνδεση ESP8266 και J1 jumper

Μπορείτε να χρησιμοποιήσετε την UART και να προσομοιώνετε εσείς τις απαντήσεις του ESP8266 από τον υπολογιστή για να σιγουρευτείτε ότι λειτουργεί σωστά η σειριακή επικοινωνία. Για την επικοινωνία με τον υπολογιστή σας μπορείτε να χρησιμοποιήσετε οποιοδήποτε πρόγραμμα σειριακής επικοινωνίας επιθυμείτε. Ένα πρόγραμμα είναι το Arduino IDE 2.0.3 που μπορείτε να κατεβάσετε στον υπολογιστή σας από εδώ (<https://www.arduino.cc/en/software>) και αφού το ανοίξετε επιλέξτε Tools->Port και επιλέξτε τη θύρα που είναι συνδεδεμένο το USB σειριακής επικοινωνίας (το αριστερά) και στη συνέχεια Tools->Serial Monitor. Επιλέξτε το σωστό baud από δεξιά κάτω.

Για τα πρώτα ερωτήματα θα χρησιμοποιήσετε την UART και θα προσομοιώνεται εσείς τις απαντήσεις του ESP8266 και μόνο στο τελευταίο ερώτημα θα συνδέσετε το ESP8266 στο ntuAboard.

Ζήτημα 8.1 Να γραφεί πρόγραμμα σε C για τον ATmega328 στο ntuAboard το οποίο να στέλνει στην UART κατάλληλη εντολή για να συνδεθεί το ESP8266 στο δίκτυο. Στη συνέχεια, θα διαβάζει από την UART και θα περιμένει την απάντηση του ESP8266 και αν αυτή είναι "Success" θα απεικονίζει στην LCD το μήνυμα 1.Success ενώ αν είναι "Fail" θα τυπώνει στην LCD το μήνυμα 1.Fail. Επαναλάβετε το ίδιο για την εντολή url ώστε να θέσετε το url σε <http://192.168.1.250:5000/data> και αντίστοιχα να εμφανίζεται μηνύματα 2.Success και 2.Fail. Μπορείτε να προσθέσετε μια καθυστέρηση πριν την αποστολή της δεύτερης εντολής για να προλάβετε να δείτε τα μηνύματα στην LCD.

Ζήτημα 8.2 Επεκτείνετε το παραπάνω πρόγραμμα ώστε μετά την εκτύπωση του 2^{ου} μηνύματος για επαρκή χρόνο

(α) να διαβάσει μια μέτρηση από τον αισθητήρα θερμοκρασίας DS18B20 με σκοπό να ενσωματώσετε την τιμή στο payload που θα στείλετε. Προσθέστε στην τιμή του αισθητήρα μια τιμή ώστε το αποτέλεσμα σας να είναι σε συνθήκες θερμοκρασίας δωματίου κοντά στο 36 για να προσομοιάζει πραγματική θερμοκρασία ασθενούς.

(β) να διαβάζει μια μέτρηση από το POT0 και να την μετατρέπετε σε κλίμακα 0-20 cm H2O για να συμπεριφέρεται σαν αισθητήρας κεντρικής φλεβικής πίεσης. Και αυτή η τιμή πρέπει να ενσωματωθεί στο payload.

(γ) να διαμορφώνει το status ως εξής:

1) Αν πατηθεί στο πληκτρολόγιο το πλήκτρο που αντιστοιχεί στο τελευταίο ψηφίο της ομάδας σας τότε το status να γίνεται NURSE CALL. Αν πατηθεί στη συνέχεια η δίεση # τότε το status να γίνεται OK εκτός αν συμβαίνει κάτι από τα 2 η 3.

2) Αν η πίεση είναι πάνω από 12 η κάτω από 4 τότε το status να γίνεται CHECK PRESSURE.

3) Αν η θερμοκρασία είναι κάτω από 34 η πάνω από 37 τότε το status να γίνεται CHECK TEMP.

4) Αν δεν έχει συμβεί κανένα από τα παραπάνω τότε το status να γίνεται OK.

Την τελική τιμή του status ενσωματώστε την στο payload.

Εμφανίστε τις τιμές της θερμοκρασίας και του αισθητήρα κεντρικής φλεβικής πίεσης στην 1^η γραμμή της LCD και το status στην 2^η γραμμή για επαρκή χρόνο.

Ζήτημα 8.3 Επεκτείνετε το παραπάνω πρόγραμμα ώστε να στέλνει στην UART κατάλληλες εντολές για την αποστολή του payload στον server. Αυτές είναι η εντολή payload για την οποία αντίστοιχα να εμφανίζεται μηνύματα 3.Success και 3.Fail και η εντολή transmit. Για την transmit δεν στέλνει το ESP Success η Fail αλλά στέλνει κατευθείαν την απάντηση του Server. Να τυπώνετε στην οθόνη την απάντηση του server στη μορφή 4. Απάντηση server. Αν όλα έχουν γίνει σωστά πρέπει να λάβετε 200 OK και να δείτε τα στοιχεία που στείλατε στον προβολέα του εργαστηρίου.

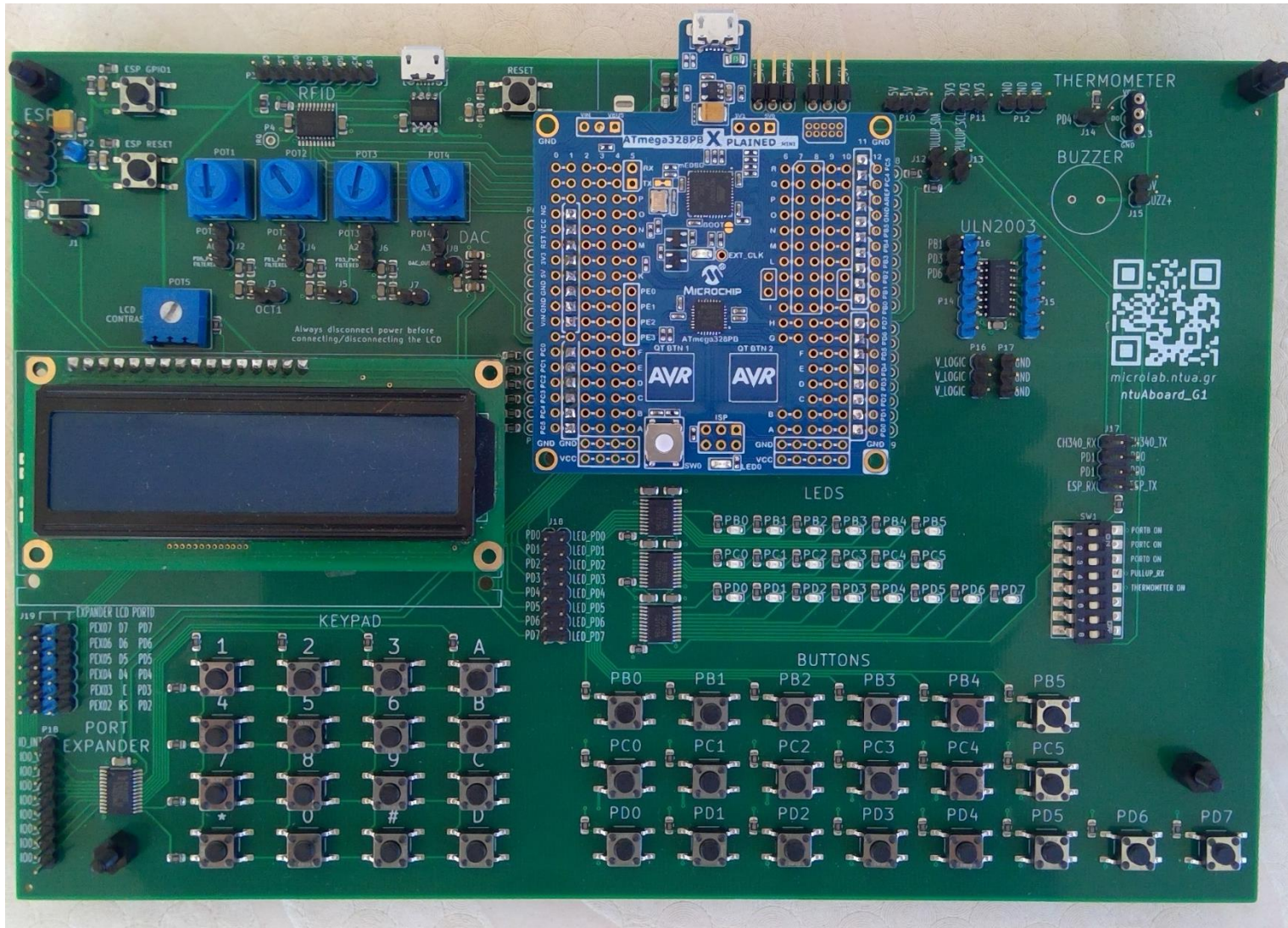
Το πρόγραμμα πρέπει να επαναλαμβάνει την λειτουργία του από το 8.1-8.3.

ΟΔΗΓΙΕΣ ΧΡΗΣΗΣ ΚΑΡΤΑΣ ntuAboard_G1

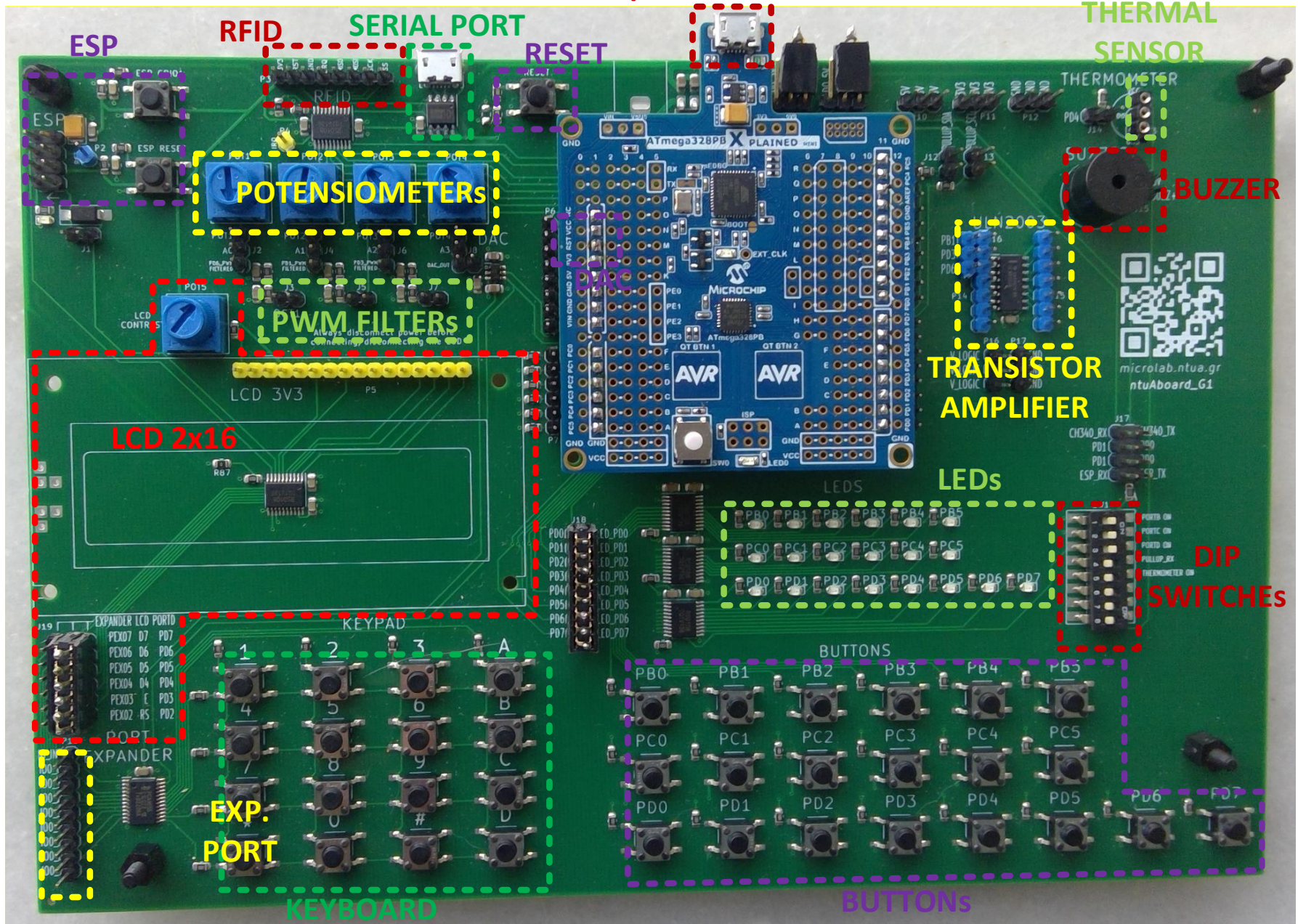


Ε.Μ.Π. - ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΑΚΑΔ. ΕΤΟΣ 2022-2023

Παρουσίαση της κάρτας ntuAboard_G1



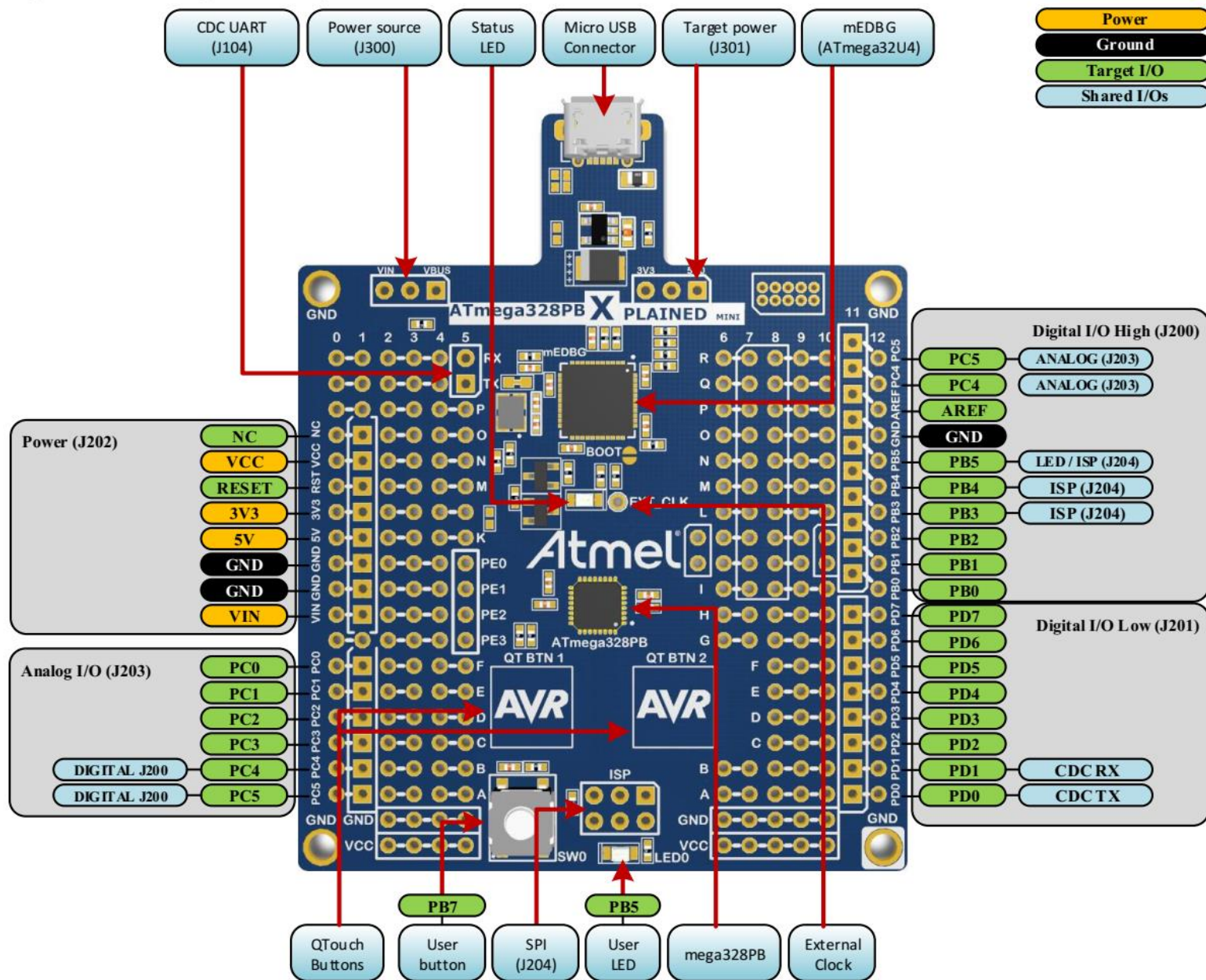
DEBUGGER/PROGRAMMER PORT



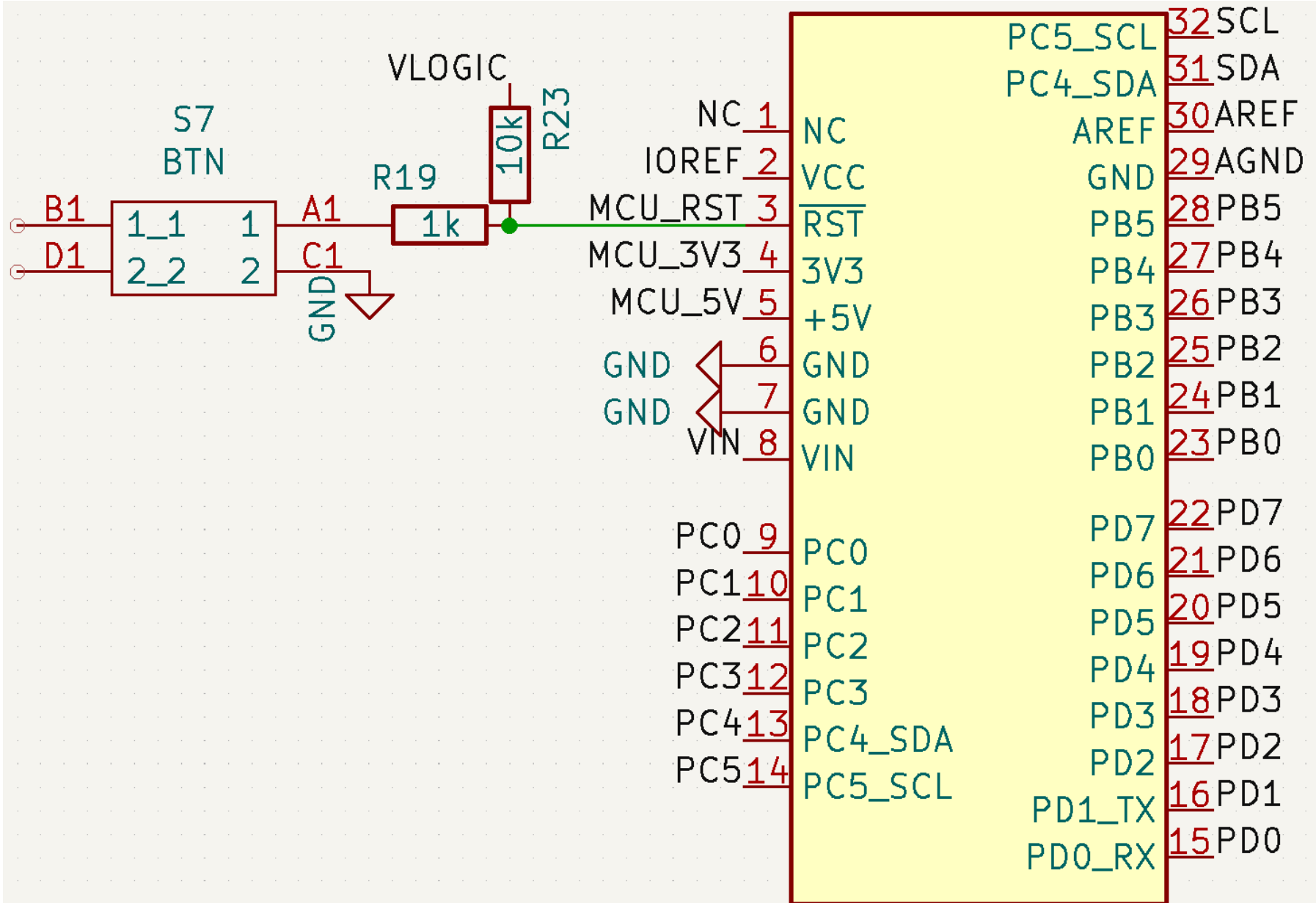
Συνοπτική περιγραφή ntuAboard_G1

- Η ntuAboard_G1 διαθέτει επαφές διασύνδεσης συμβατές με Arduino shield και μπορεί να υποδεχτεί μικροελεγκτές διαφόρων τεχνολογιών.
- Οι κυριότερες περιφερειακές συσκευές που εμπεριέχει είναι:
 - Ενδεικτικά Leds.
 - Μπουτόν.
 - Θύρα επέκτασης I/O 16 pins με σύνδεση I2C.
 - LCD οθόνη 2 x 16 χαρακτήρων.
 - Πληκτρολόγιο 4 X 4 πλήκτρων.
 - Buzzer για παραγωγή ήχου.
 - 4 Ποτενσιόμετρα για παραγωγή ρυθμιζόμενων αναλογικών τάσεων.
 - Μετατροπέα ψηφιακού σήματος σε αναλογικό (DAC) με σύνδεση I2C.
 - 3 αναλογικά φίλτρα για δημιουργία αναλογικών τάσεων από PWM κυματομορφές.
 - 7 Darlington Transistor Drivers για οδήγηση κινητήρων, Led ισχύος, Buzzer κλπ.
 - Αισθητήρα θερμοκρασίας.
 - Προσαρμογέα σειριακής θύρας USART σε USB για σύνδεση με προσωπικό υπολογιστή.
 - Υποδοχή διασύνδεσης του ESP-01 WiFi Module.
 - Υποδοχή διασύνδεσης του RC522 RFID Development Kit.
 - Υποδοχές διασύνδεσης σειριακών θυρών I2C, SPI, USART.

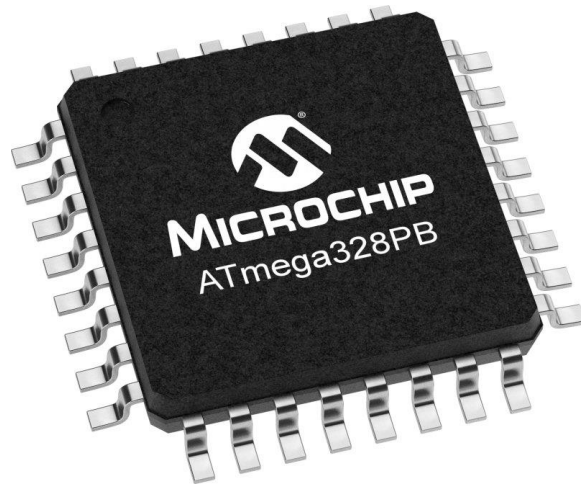
ATmega328PB Xplained Mini Headers and Connectors



MicrochipXplainedMini Pinout



Μικροελεγκτής ATmega328PB



- Ο ATmega328PB χρησιμοποιείται σε πληθώρα εμπορικών εφαρμογών.
- Είναι ο μικροελεγκτής που χρησιμοποιείται στο Arduino UNO.
- Πληροφορίες στη διεύθυνση:

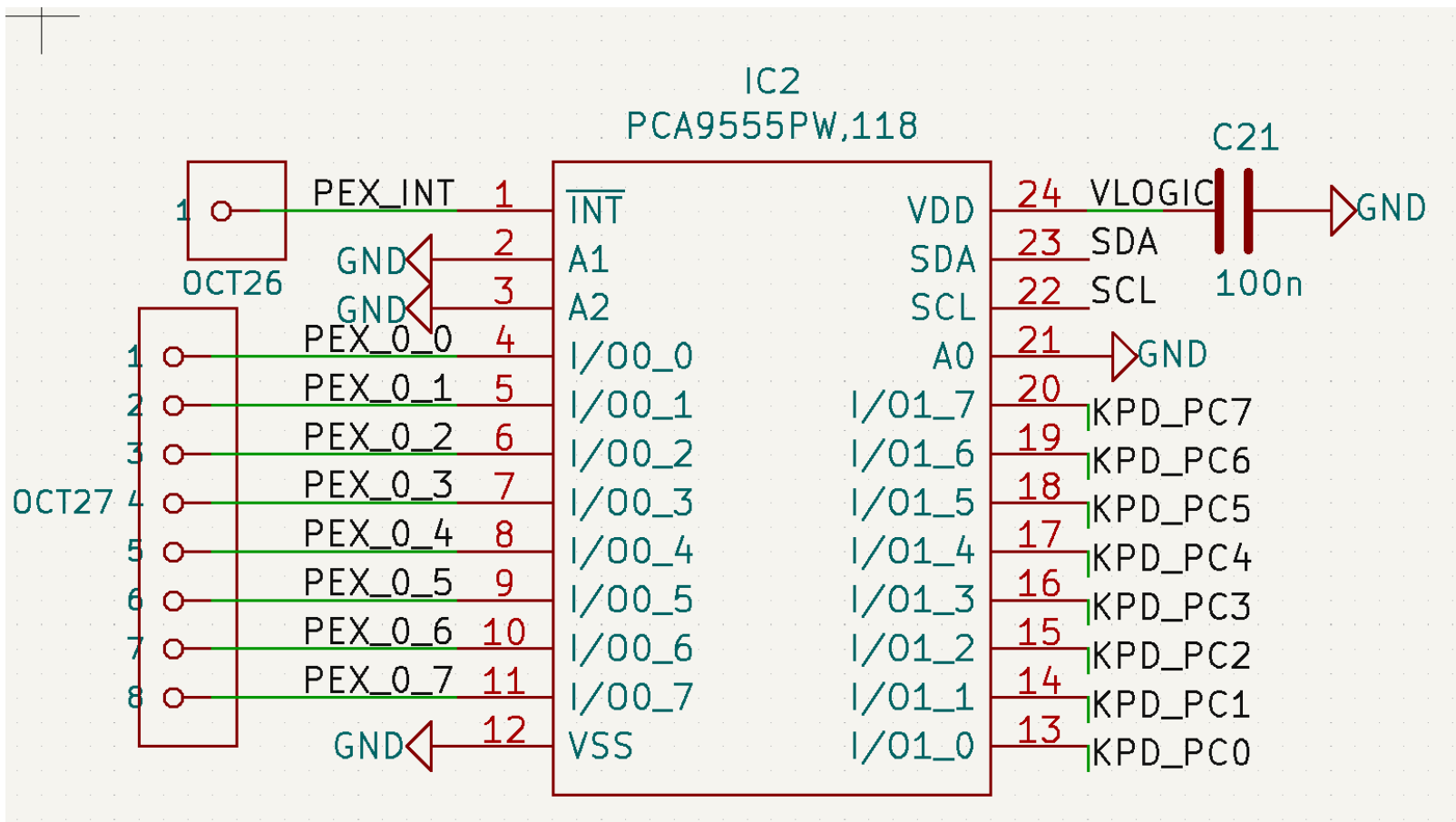
<https://www.microchip.com/en-us/product/ATmega328PB>

Βασικά χαρακτηριστικά ATmega328PB Xplained Mini

- Ενσωματωμένο mini programmer/debugger (mEDBG).
- Auto-ID για εντοπισμό της κάρτας από το MPLAB® X.
- Πρόσβαση σε όλους τους ακροδέκτες (pins) του μικροελεγκτή ATmega328PB.
- Ένα πράσινο LED ένδειξης της κατάστασης του mEDBG.
- Ένα κίτρινο LED γενικής χρήσης.
- Ένα πιεστικό διακόπτη γενικής χρήσης (Push Button).
- Δύο αισθητήρες αφής (QTouch®).
- Εικονική σειριακή θύρα COM (Virtual COM Port - CDC).
- Ενσωματωμένο κρύσταλλο 16 MHz(5V) ή 8 MHz(3,3V).
- Τροφοδοσία δια μέσω της θύρας USB.
- Ρυθμιστής τάσης 3,3V.
- Συμβατοί ακροδέκτες για Arduino Shields.
- Επαφές για SPI Bus Header.

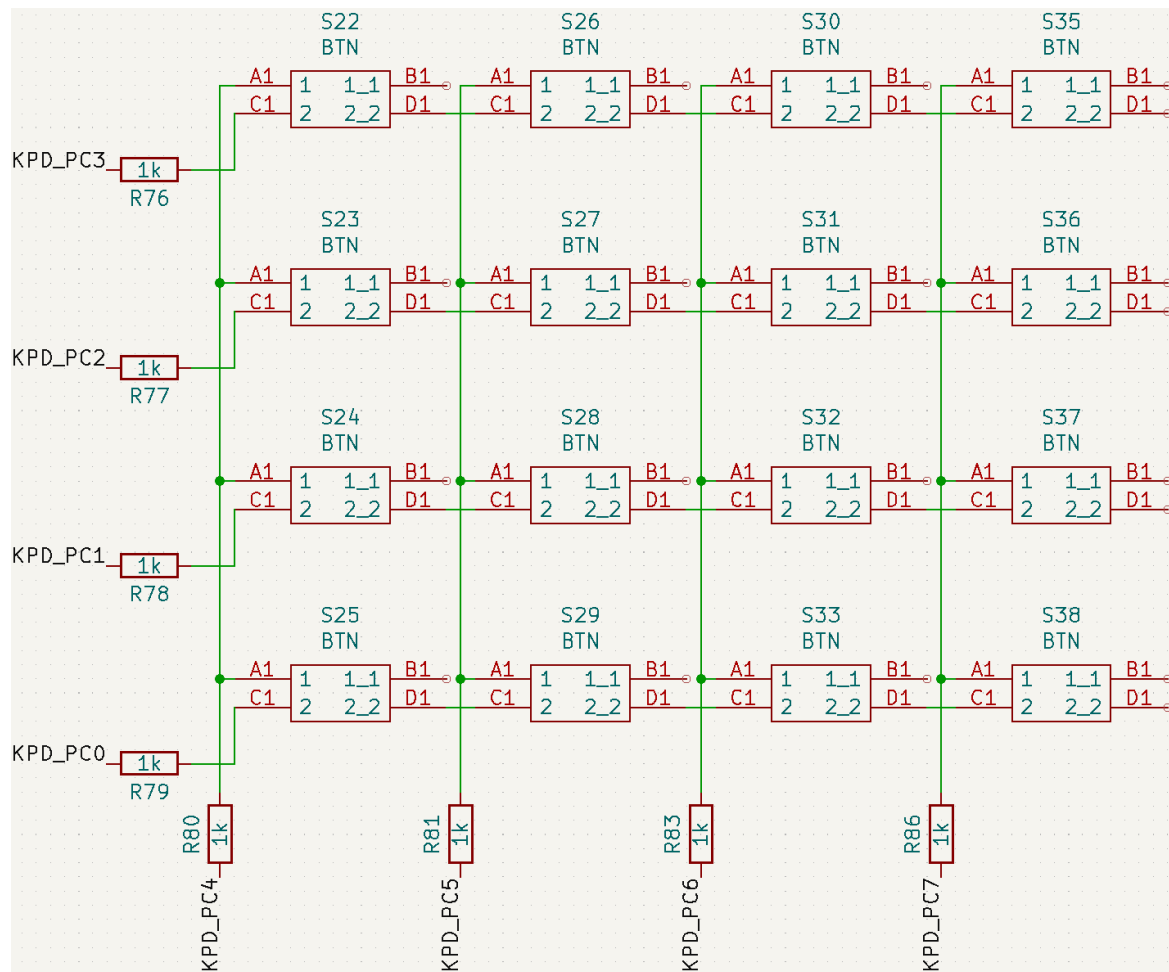
PORT EXPANDER

- Συνδέεται στον μικροελεγκτή δια μέσω της I2C διεπαφής (2 pins).
- Παράγει δύο θύρες εισόδου/εξόδου των 8 bit η καθεμία.



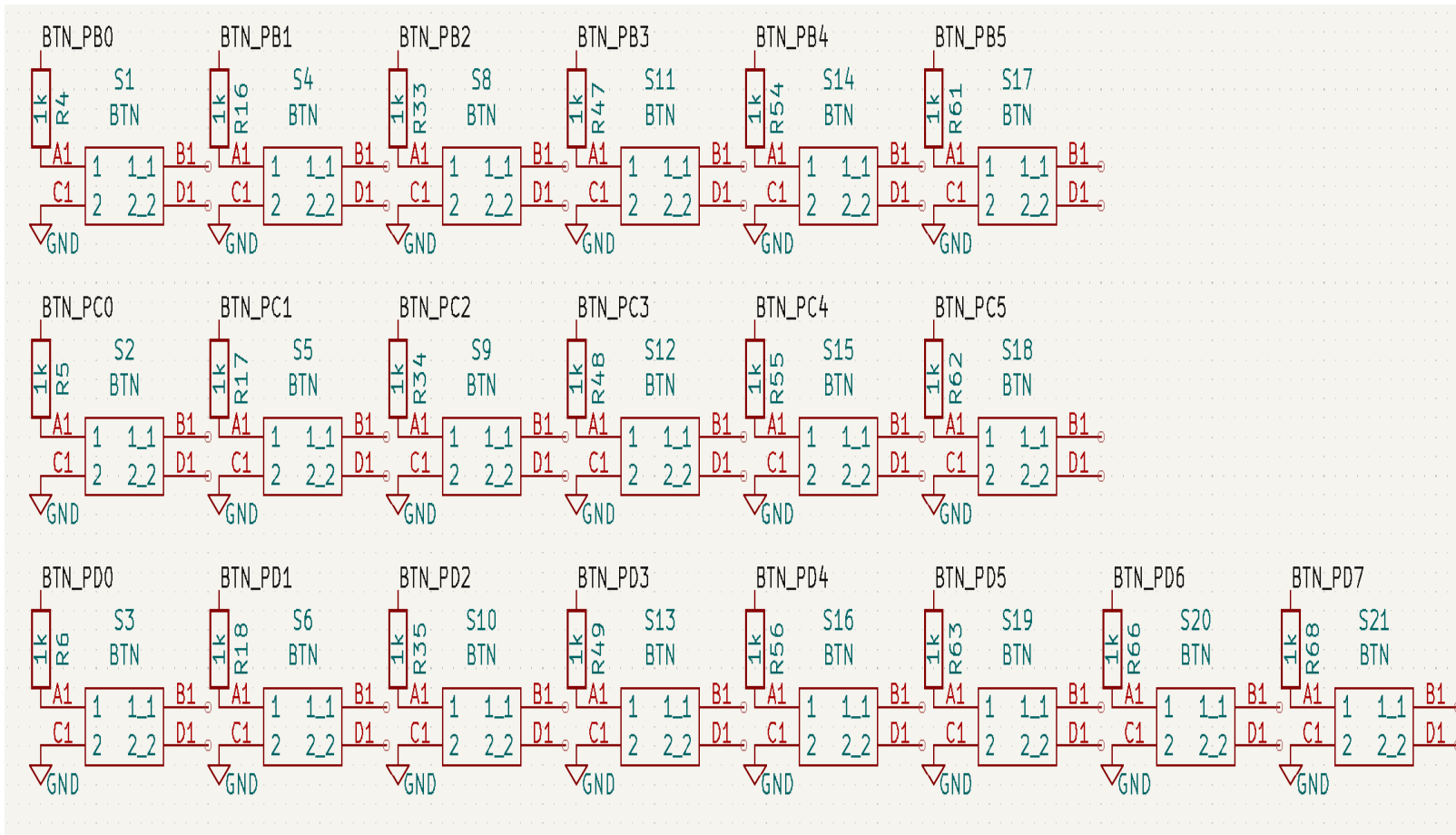
KEYPAD

- Είναι ένα πληκτρολόγιο 16 πλήκτρων
- Συνδέεται σε μία από τις δύο πόρτες του PORT EXPANDER.
- Η συνδεσμολογία του φαίνεται στο παρακάτω σχήμα:



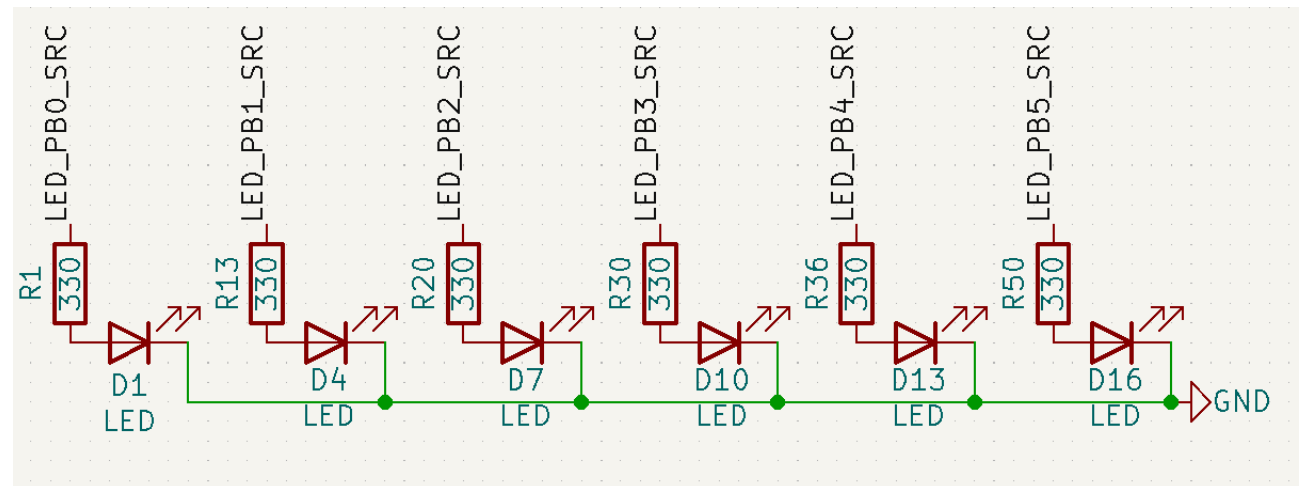
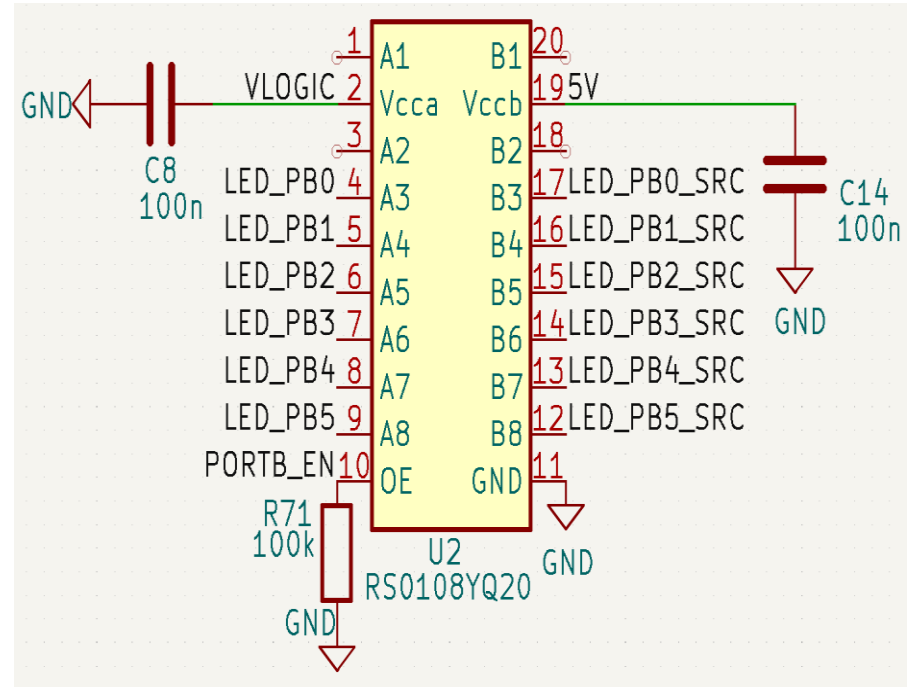
BUTTONS

- Υπάρχει ένα μπουτόν για κάθε ακροδέκτη του μικροελεγκτή.
- Ο ένας ακροδέκτης κάθε μπουτόν συνδέεται στο GND και ο άλλος ακροδέκτης συνδέεται σε ένα pin του μικροελεγκτή.
- Σε κάθε ακροδέκτη παρεμβάλλεται μία αντίσταση για προστασία από υψηλό ρεύμα στα pins του μικροελεγκτή.



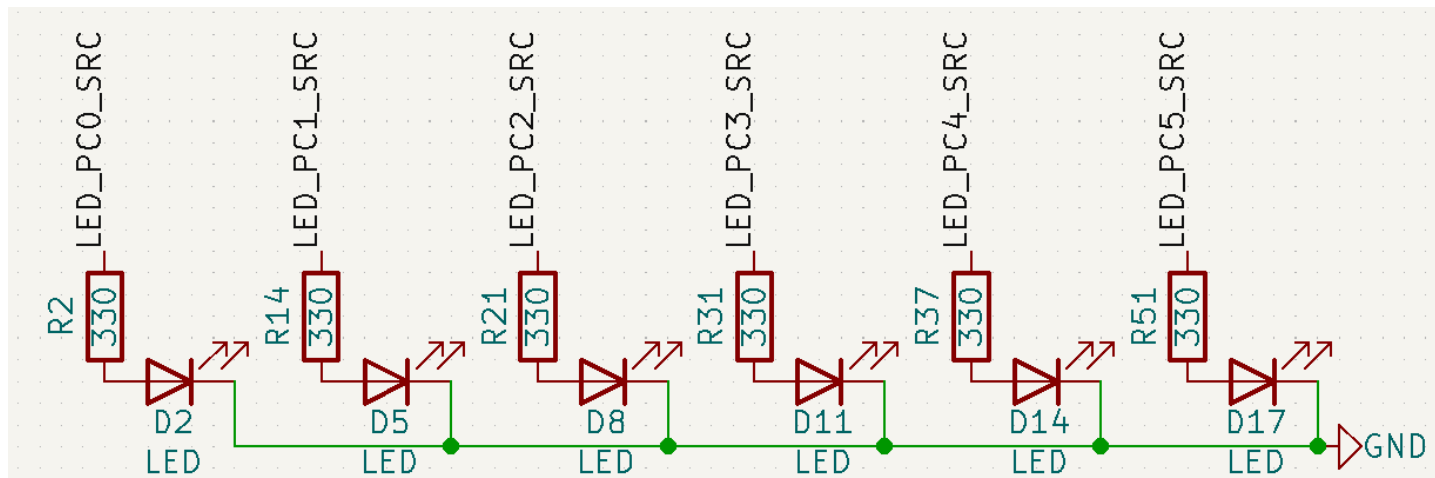
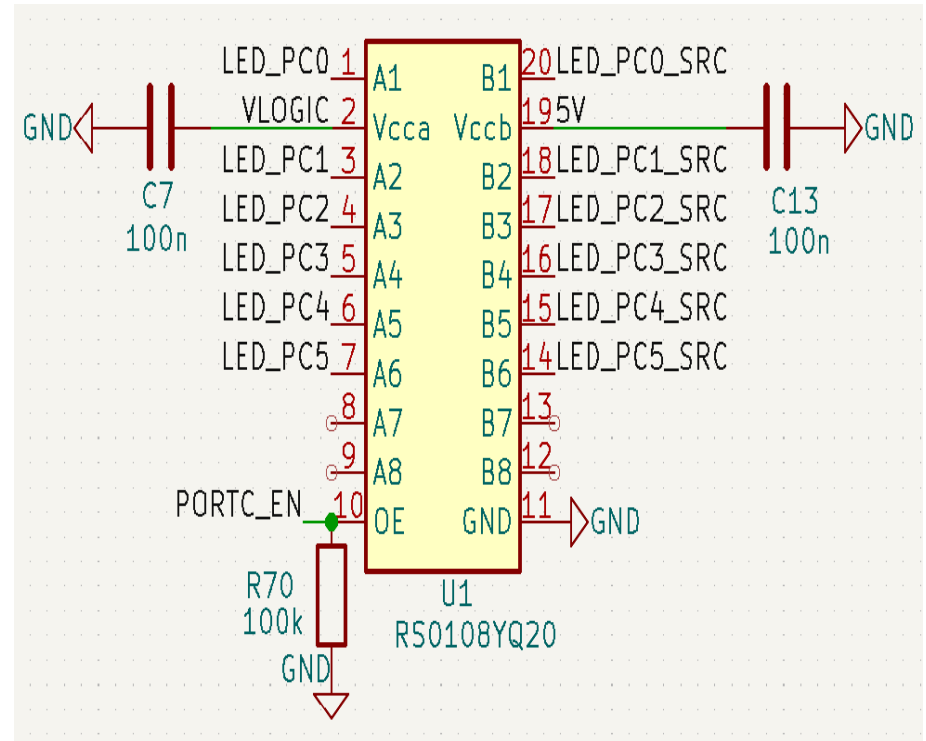
PORTB LEDS

- Σε κάθε ακροδέκτη του PORTB υπάρχει συνδεδεμένο ένα led.
- Μεταξύ του PORTB και των leds παρεμβάλλεται ένα ολοκληρωμένο για προσαρμογή της τάσης του μικροελεγκτή και για απομόνωση των leds από το PORTB όταν απαιτείται.



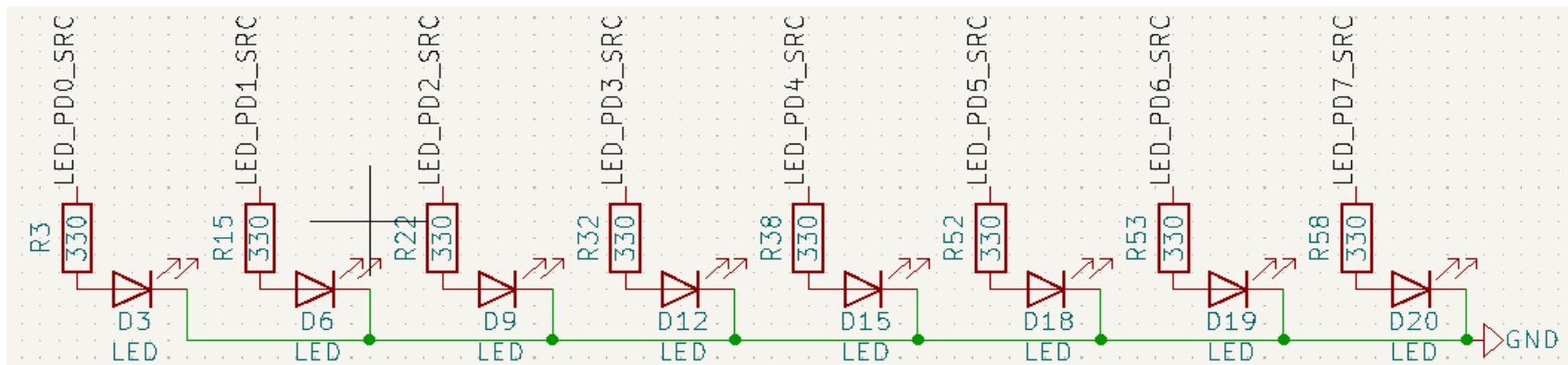
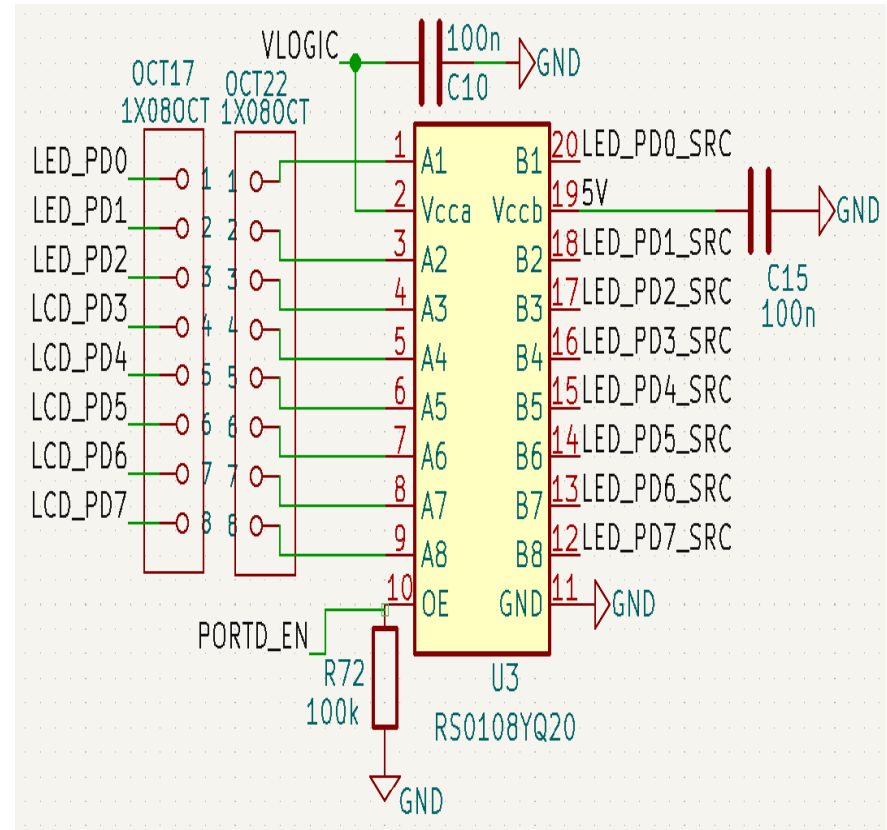
PORTC LEDS

- Σε κάθε ακροδέκτη του PORTC υπάρχει συνδεδεμένο ένα led.
- Μεταξύ του PORTC και των leds παρεμβάλλεται ένα ολοκληρωμένο για προσαρμογή της τάσης του μικροελεγκτή και για απομόνωση των leds από το PORTC όταν απαιτείται.



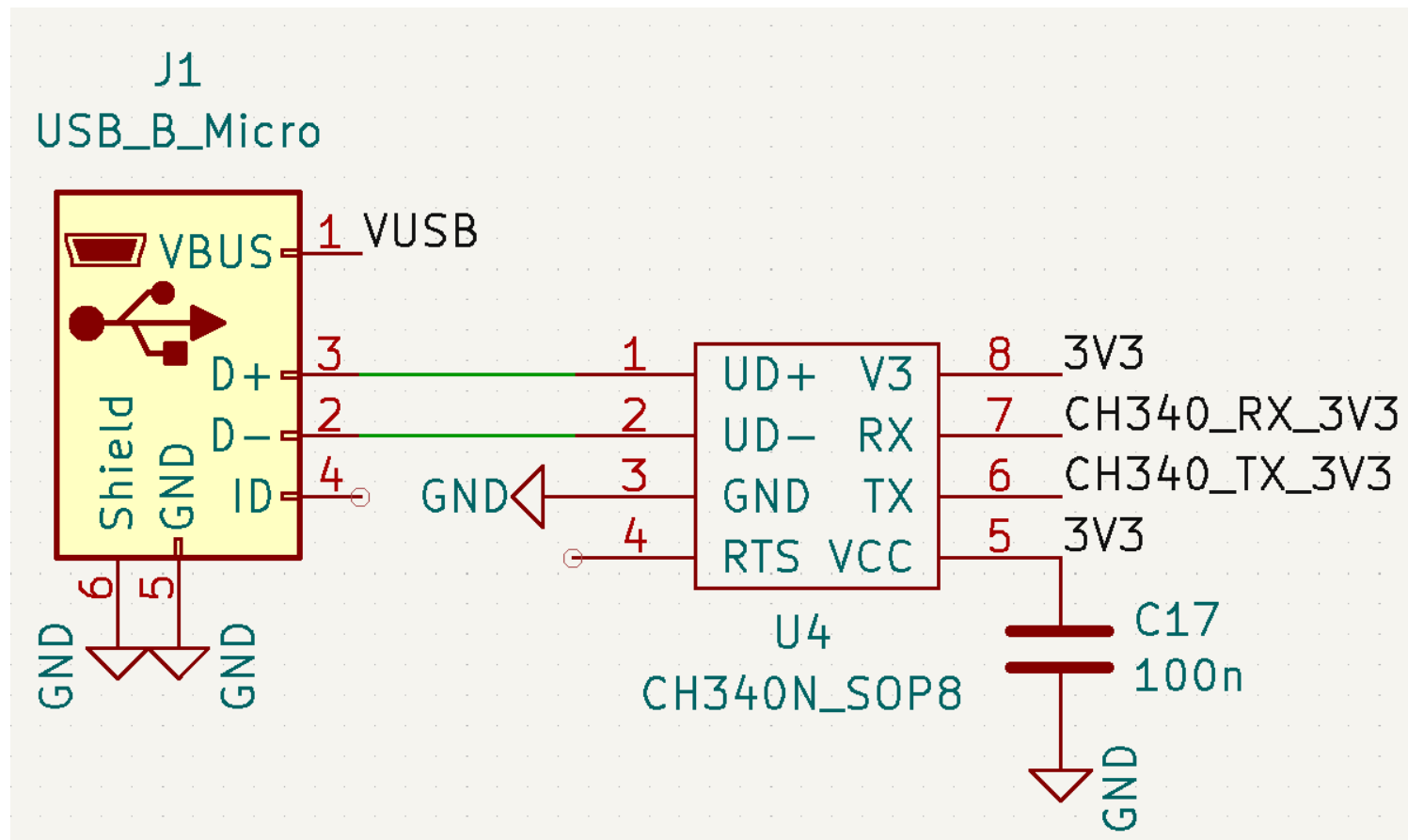
PORTD LEDS

- Σε κάθε ακροδέκτη του PORTD μπορεί να συδεθεί ένα led.
- Μεταξύ του PORTD και των leds παρεμβάλλεται ένα ολοκληρωμένο για προσαρμογή της τάσης και για απομόνωση των leds όταν απαιτείται.
- Τα leds συνδέονται στον κονέκτορα OCT22. Το PORTD συνδέεται στον κονέκτορα OCT17, οπότε κάνοντας χρήση βραχυκυκλωτήρων γίνονται οι συνδέσεις που απαιτούνται.
- Εναλλακτικά, στα led αυτά μπορούν να συνδεθούν με καλώδιο και οι ακροδέκτες του extended port.



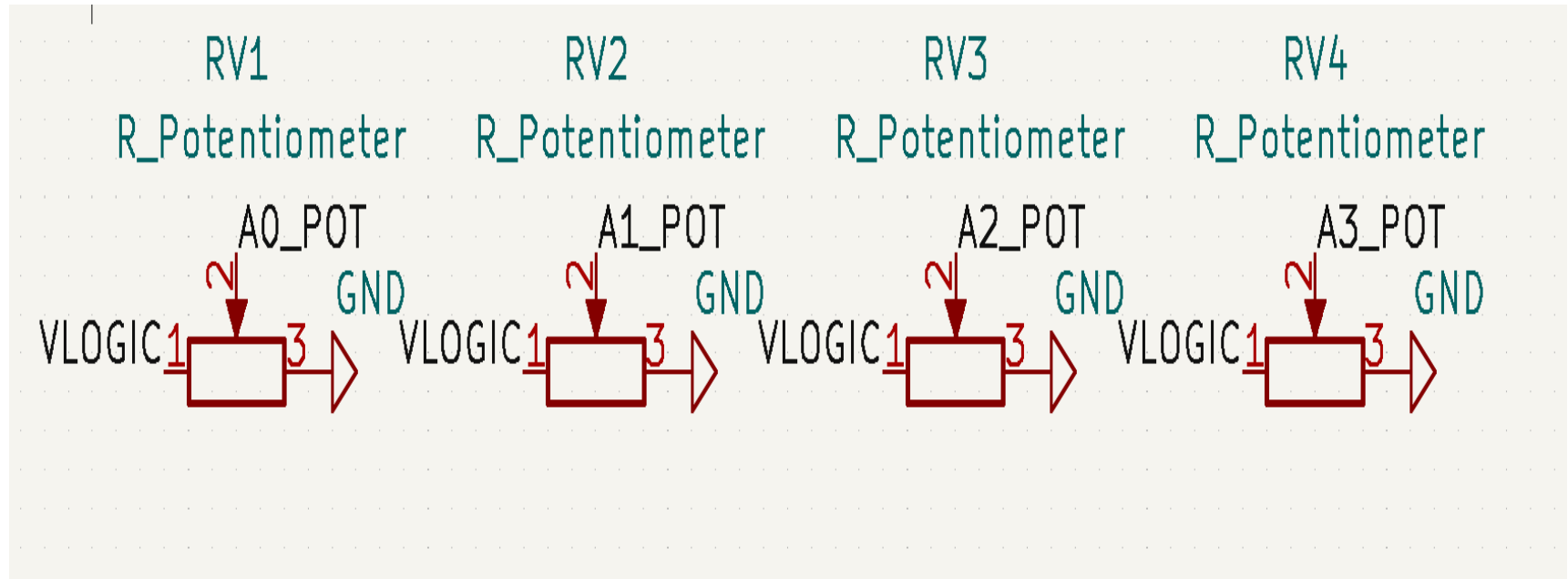
USB TO SERIAL

- Η διάταξη αυτή χρησιμοποιείται για σύνδεση της σειριακής Θύρας του μικροελεγκτή σε μία USB θύρα ενός PC.



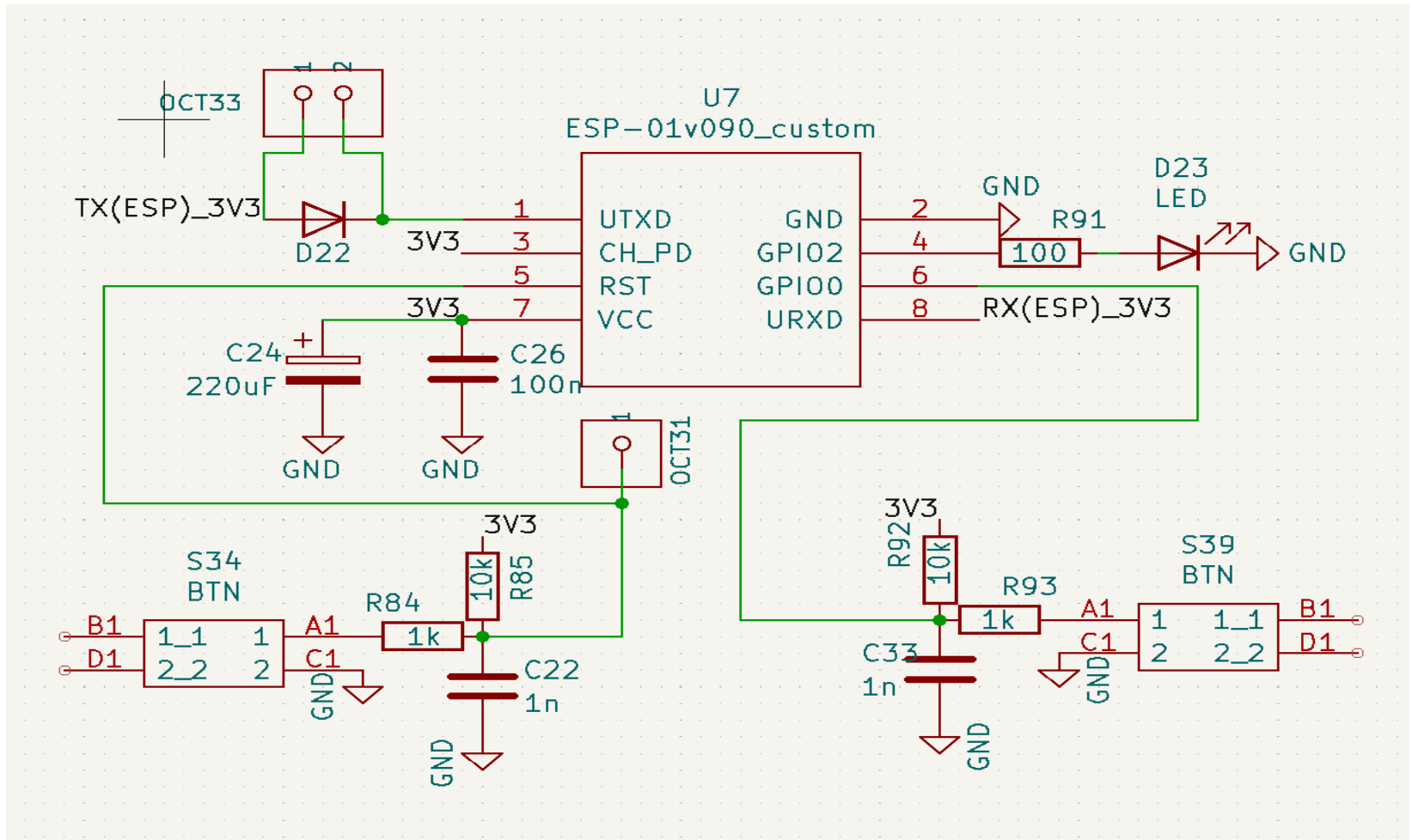
POTENSIOMETERS

- Το ntuAboard_G1 διαθέτει 4 ποτενσιόμετρα για παραγωγή 4 αναλογικών τάσεων, οι οποίες μπορούν να χρησιμοποιηθούν για τον έλεγχο του ADC μετατροπέα.



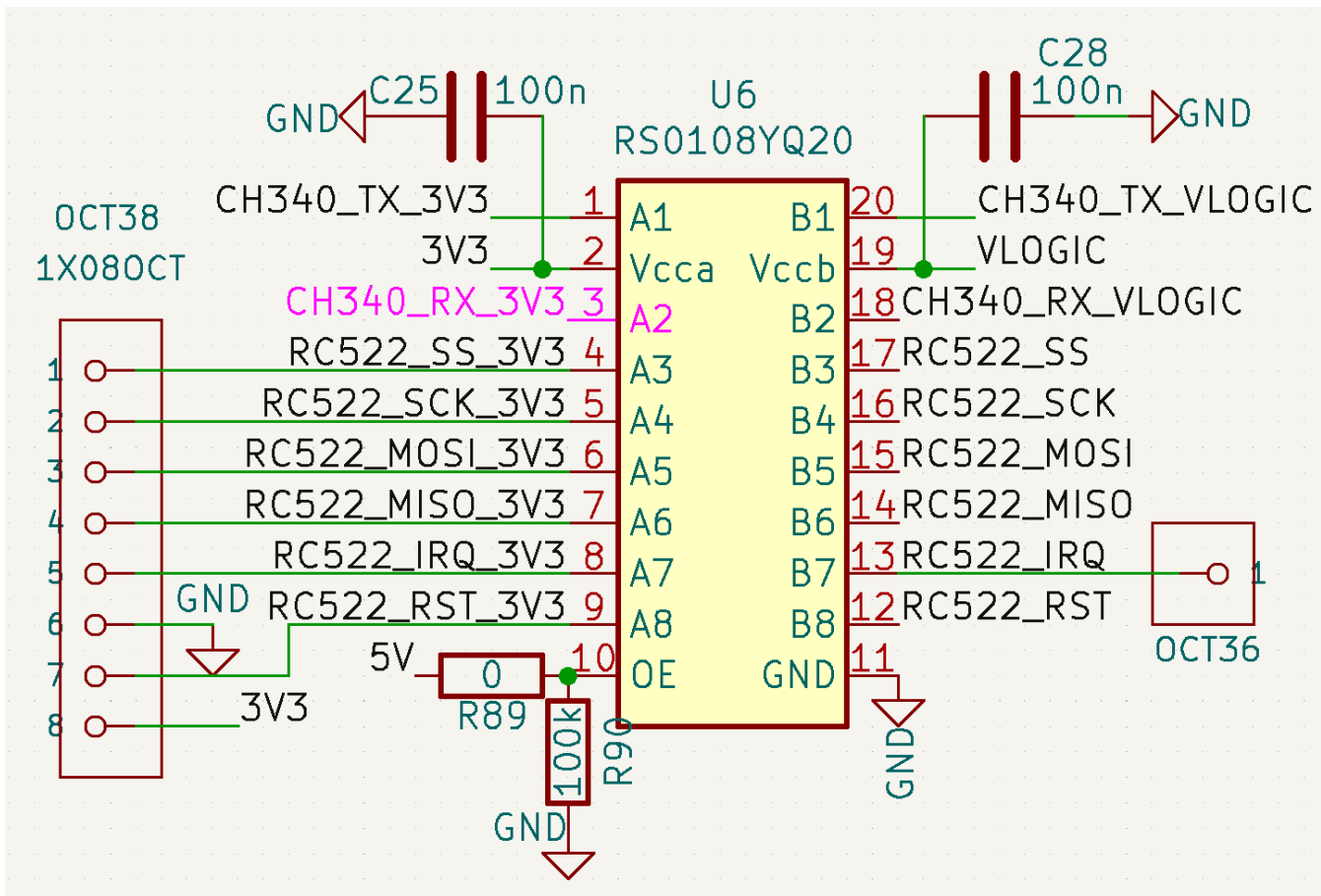
ESP8266 WiFi Module

- Το ntuAboard_G1 διαθέτει 1 κονέκτορα με 8 ακροδέκτες (P1) στον οποίο μπορεί να συνδεθεί το ESP8266 WiFi Module για εγκαθίδρυση ασύρματης επικοινωνίας.



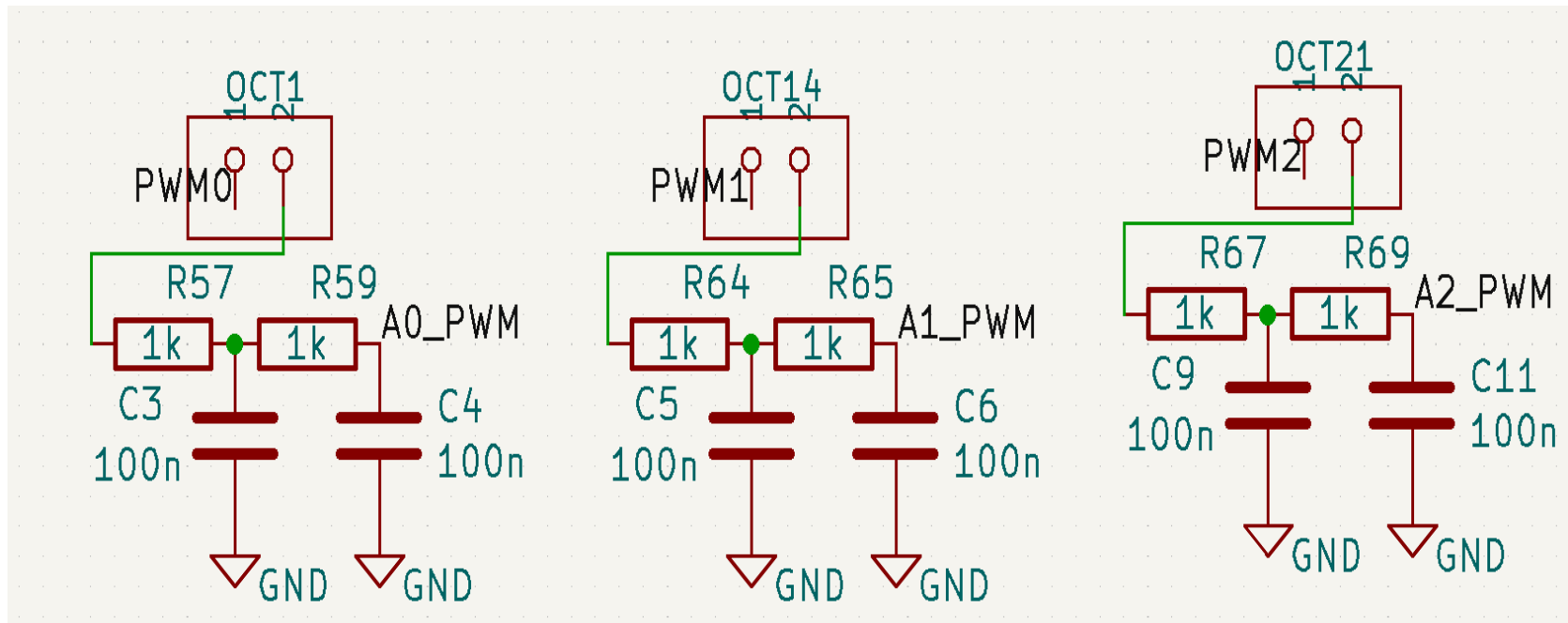
RFID

- Το ntuAboard_G1 διαθέτει 1 κονέκτορα με 8 ακροδέκτες (OCT38) στον οποίο μπορεί να συνδεθεί το RC522 RFID Development Kit.
- Μεταξύ του μικροελεγκτή και του RC522 RFID παρεμβάλλεται ένα ολοκληρωμένο για προσαρμογή της τάσης για απομόνωση όταν απαιτείται.



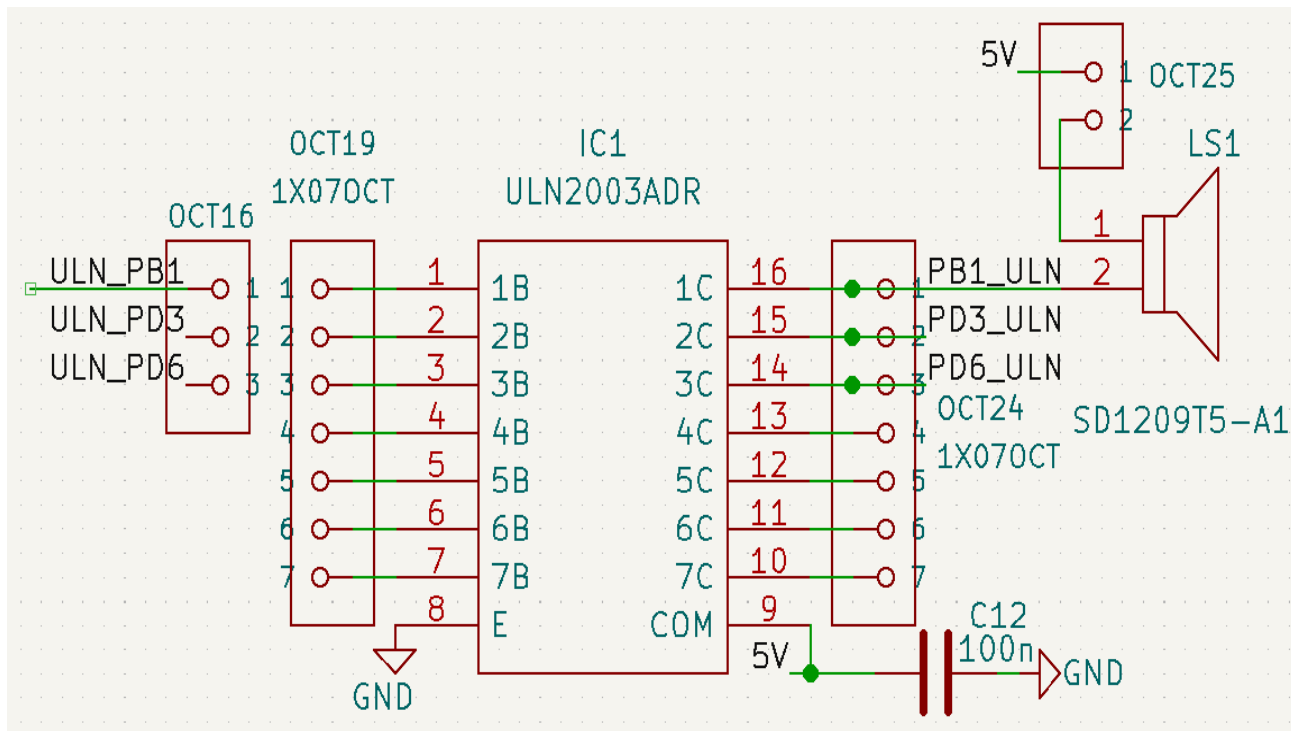
PWM FILTERS

- Ο μικροελεγκτής μπορεί να παράγει PWM κυματομορφές με μεταβλητό duty cycle.
- Το ntuAboard_G1 διαθέτει 3 αναλογικά φίλτρα τα οποία χρησιμεύουν για παραγωγή μεταβαλλόμενων DC τάσεων από τις PWM εξόδους.



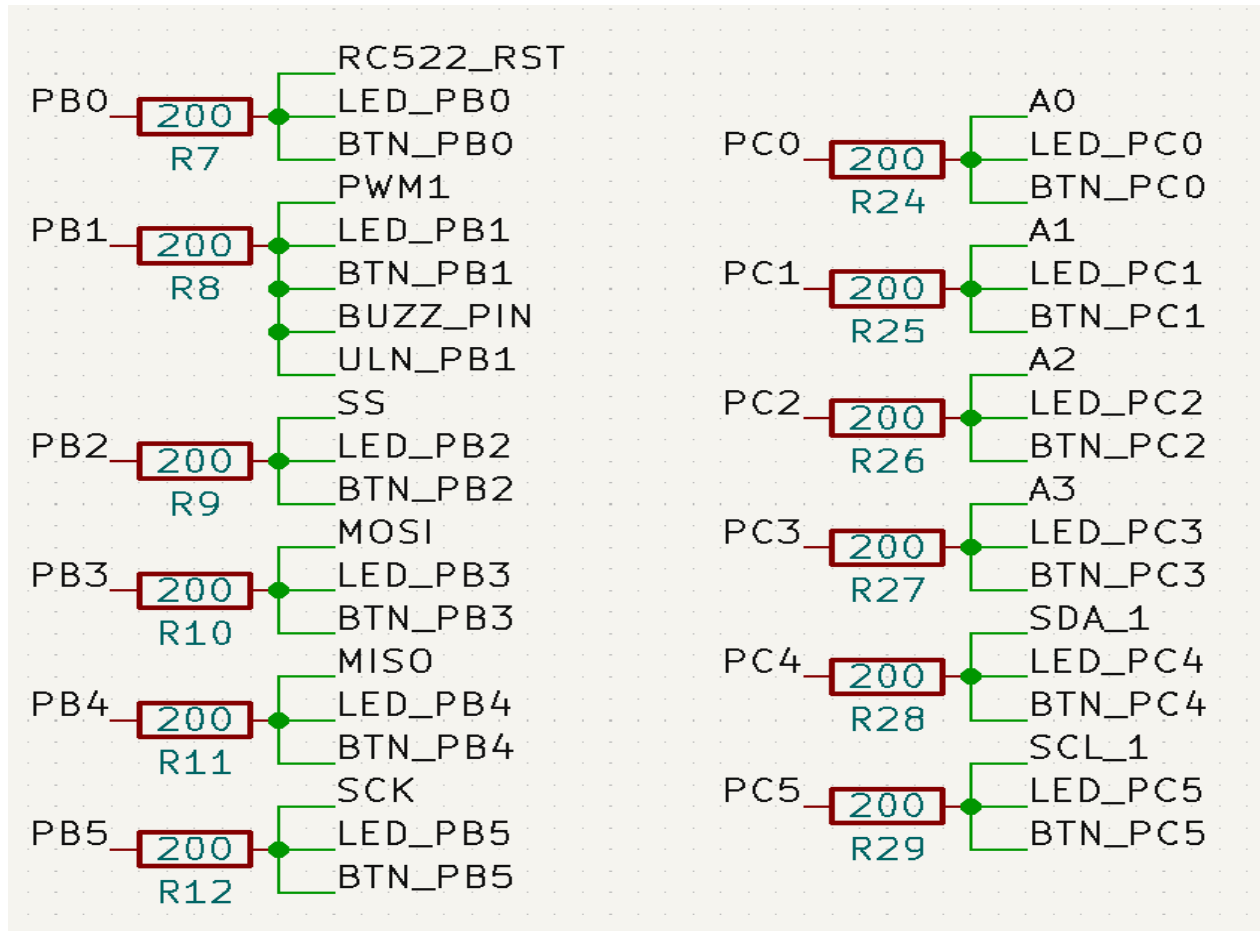
BUZZER and TRANSISTOR ARRAY IC

- Το ntuAboard_G1 έχει ενσωματωμένο ένα ολοκληρωμένο 7 καναλιών ενίσχυσης με darlington transistors
- Σε ένα από αυτά τα κανάλια μπορεί να συνδεθεί ένα buzzer.



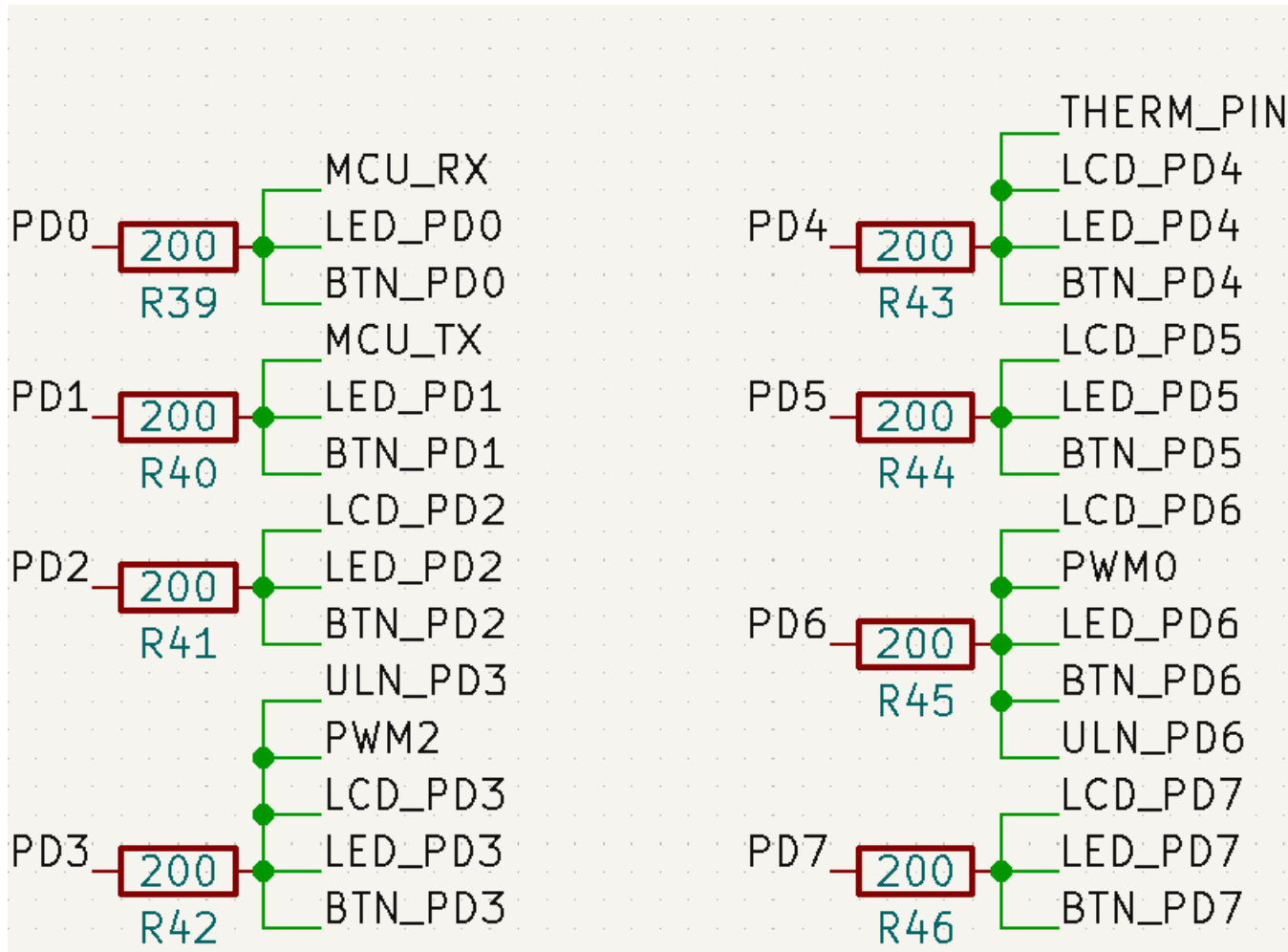
PORTB and PORTC CONNECTIONS

- Στο παρακάτω σχήμα φαίνονται οι διάφορες συνδέσεις που είναι διαθέσιμες για καθέναν από τους ακροδέκτες του PORTB και του PORTC.



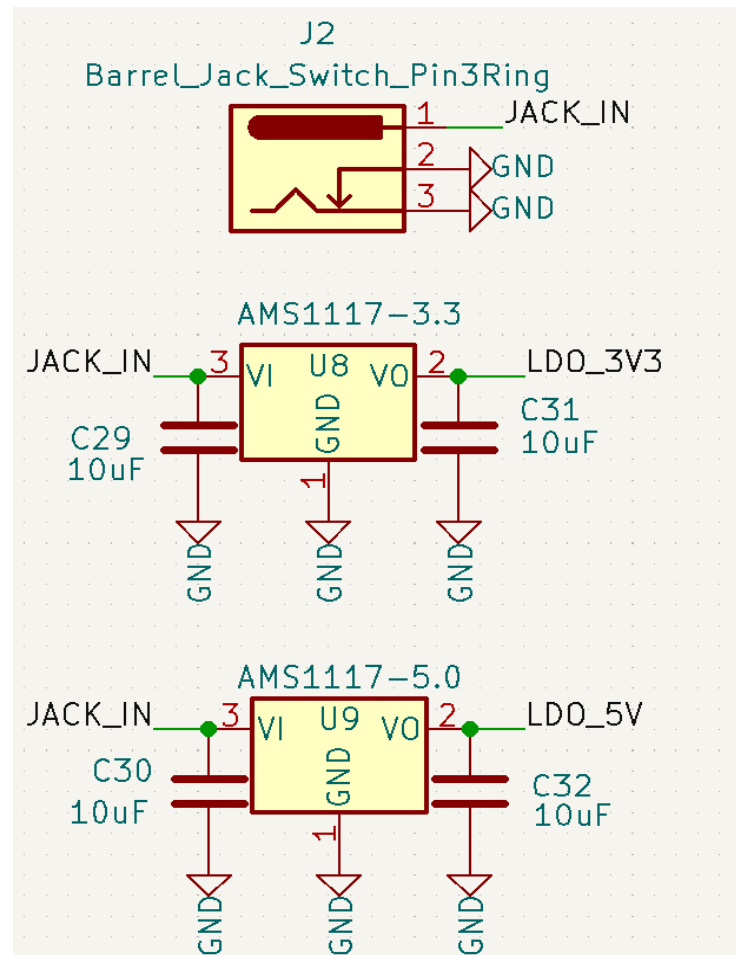
PORTD CONNECTIONS

- Στο παρακάτω σχήμα φαίνονται οι διάφορες συνδέσεις που είναι διαθέσιμες για καθέναν από τους ακροδέκτες του PORTD.



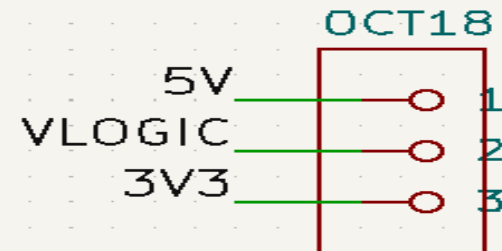
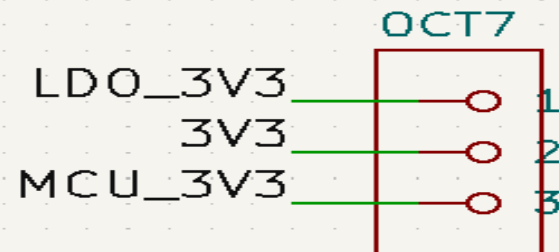
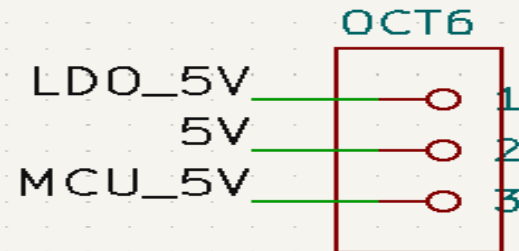
VOLTAGE REGULATORS

- Το ntuAboard_G1 διαθέτει 2 voltage regulators για παραγωγή DC τάσεων 3,3Volt και 5 Volt.
- Η τροφοδοσία των voltage regulators γίνεται από μία εξωτερική DC τάση που εφαρμόζεται στον connector J2 και η τιμή της κυμαίνεται από 7 έως 13 volt.



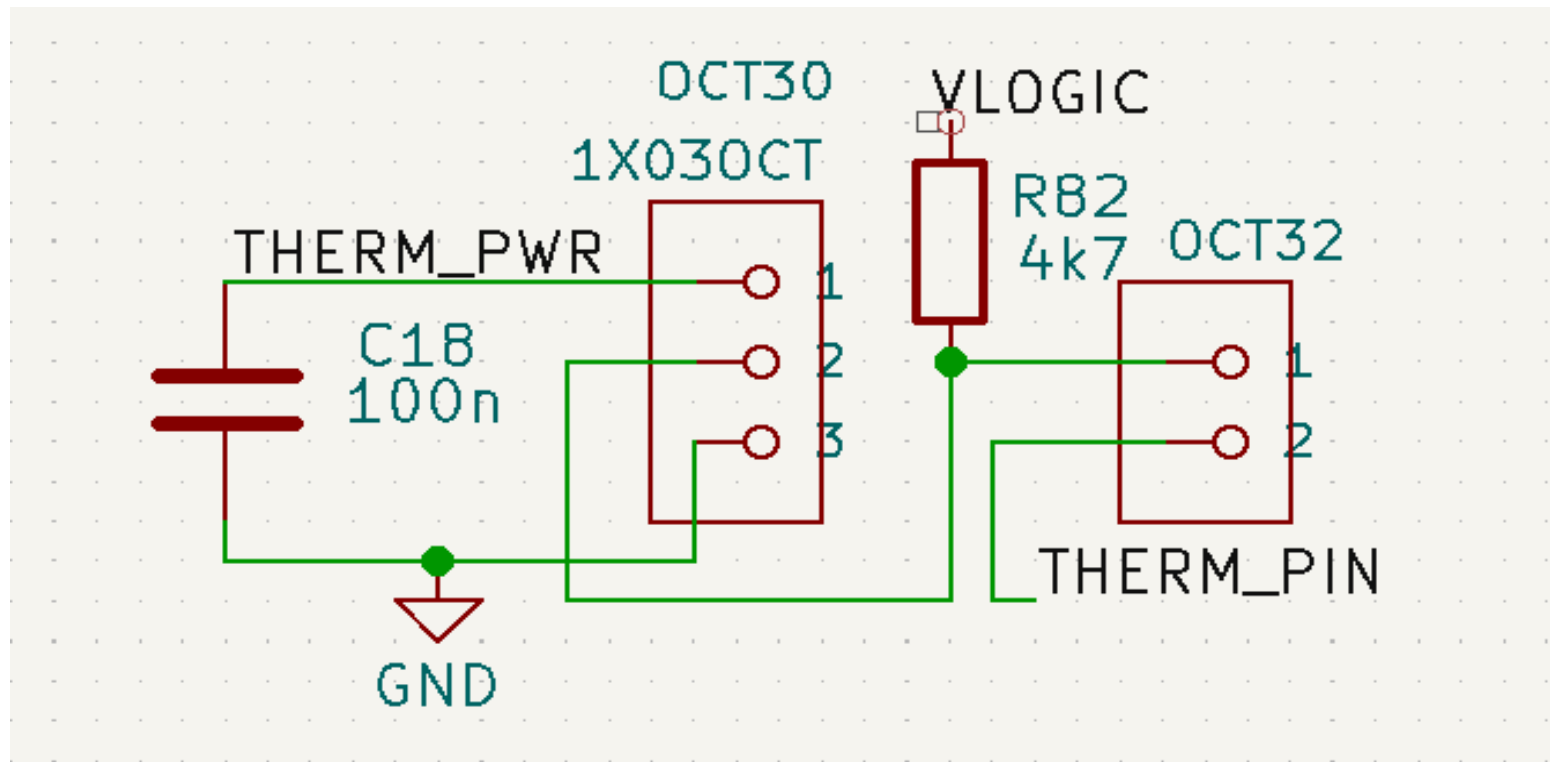
VOLTAGE SELECTORS

- Το ntuAboard_G1 παρέχει τη δυνατότητα επιλογής της τάσης τροφοδοσίας του είτε από την κάρτα ATmega328PB Χρplained Mini είτε από τα 2 voltage regulators που ενσωματώνει.
- Η επιλογή αυτή γίνεται από τους κονέκτορες OCT6 και OCT7 με χρήση κατάλληλων βραχυκυκλωτήρων, όπως φαίνεται στο διπλανό σχήμα.
- Με τον κονέκτορα OCT18 επιλέγετε εάν η τάση VLOGIC του ntuAboard_G1 είναι 3,3Volt ή 5Volt.



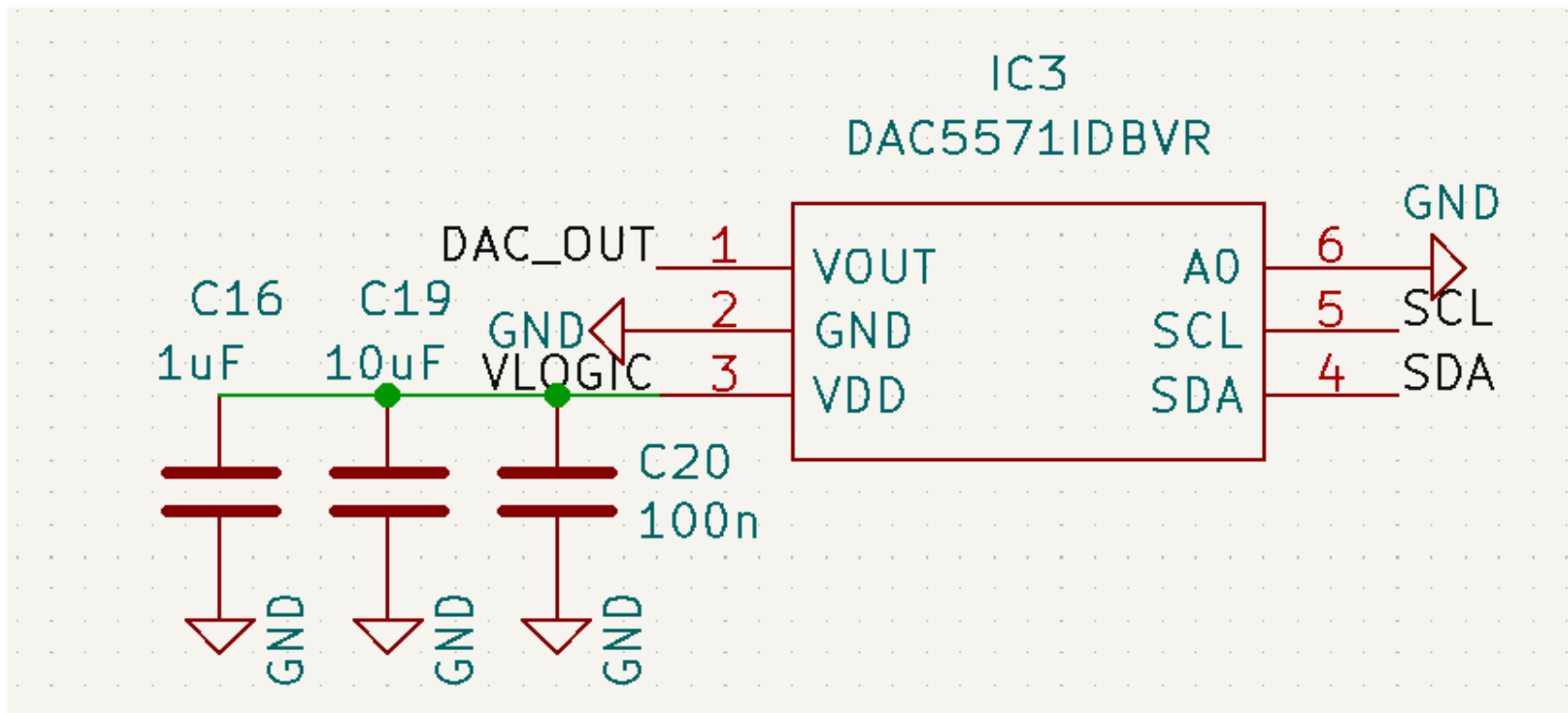
TEMPERATURE SENSOR

- Στον κόνεκτορα OCT32 μπορεί να συνδεθεί ένας αισθητήρας θερμοκρασίας και με τον κόνεκτορα OCT30 επιλέγεται εάν αυτός ο αισθητήρας είναι συνδεδεμένος ή όχι.



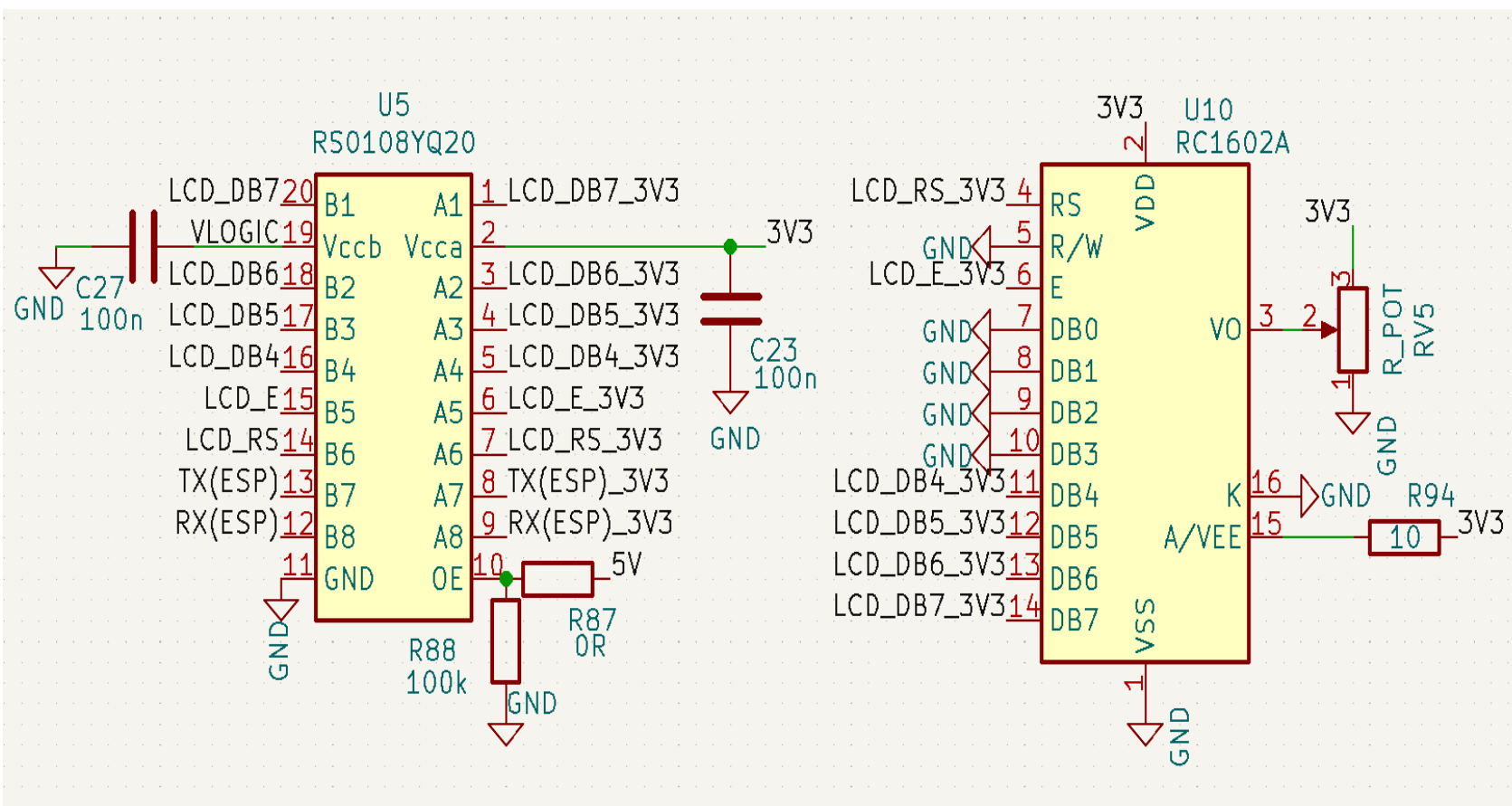
DIGITAL TO ANALOG CONVERTER (DAC)

- Το ntuAboard_G1 διαθέτει ένα μετατροπέα ψηφιακού σήματος σε αναλογικό(DAC).
- Ο DAC μετατροπέας συνδέεται στη I2C θύρα, χρησιμοποιώντας μόνο δύο ακροδέκτες του μικροελεγκτή, όπως φαίνεται στο παρακάτω σχήμα:



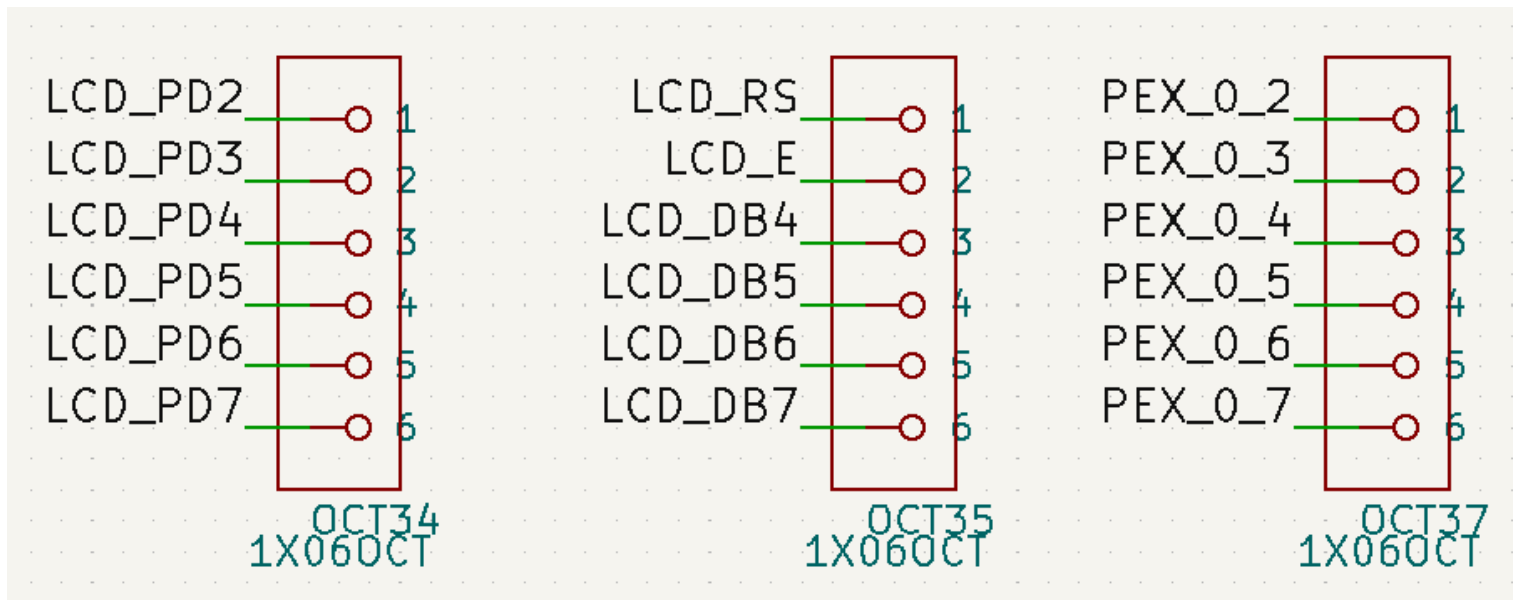
2x16 LCD

- Το ntuAboard_G1 διαθέτει μία lcd οθόνη 2x16 χαρακτήρων.
- Μεταξύ της οθόνης και του μικροελεγκτή παρεμβάλλεται ένα ολοκληρωμένο για προσαρμογή της τάσης.



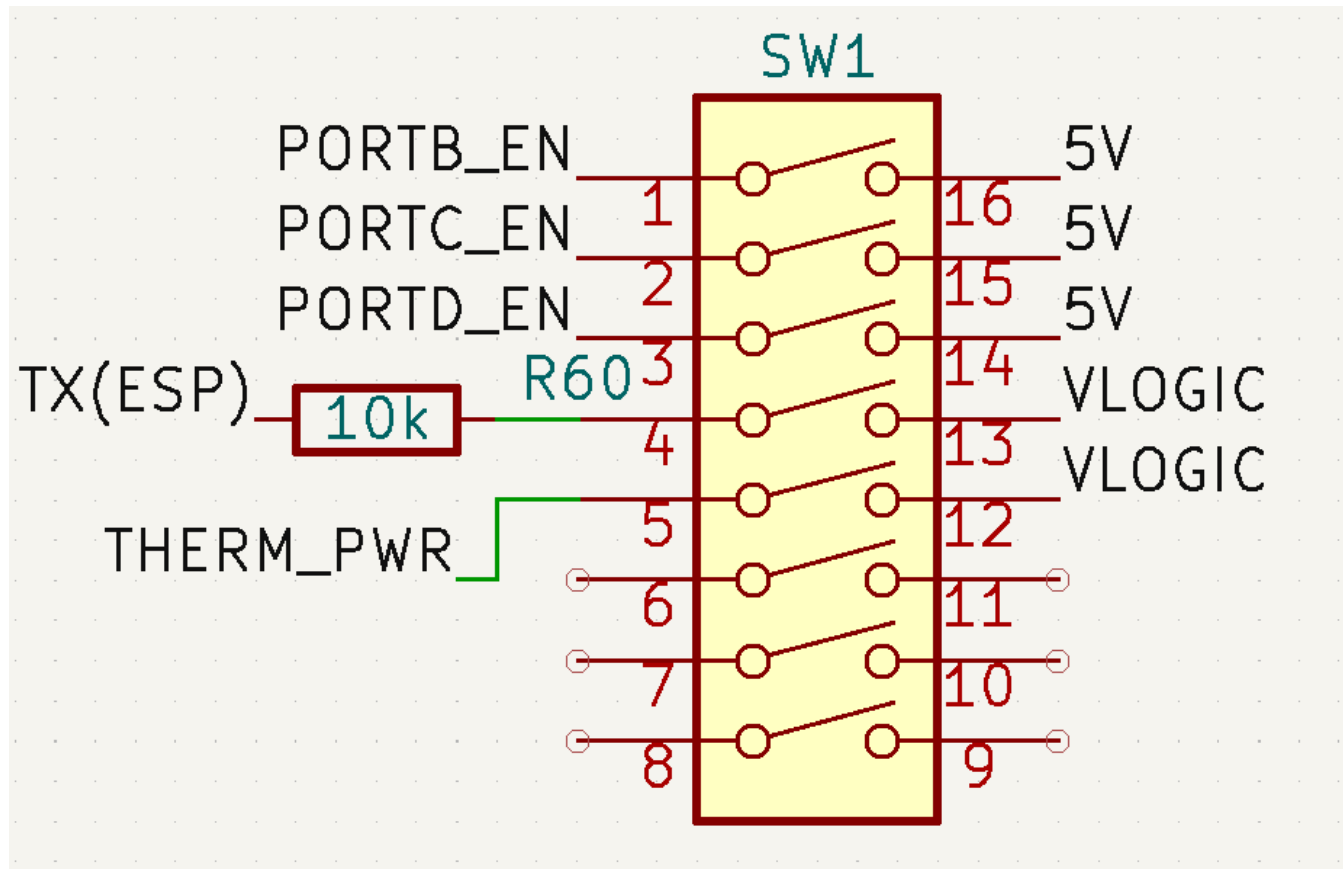
2x16 LCD

- Κάνοντας χρήση των koneκτόρων OCT34, OCT35 και OCT37 μπορούμε να επιλέξουμε εάν η lcd οθόνη θα συνδεθεί στο PORTD ή στο PORT EXPANDER.



DIP SWITCHES

- Με τα dip switches SW1 επιλέγουμε τη σύνδεση ή όχι για τα leds των τριών θυρών, για τον ακροδέκτη TX(ESP) και για την τροφοδοσία του αισθητήρα θερμοκρασίας.



MPLAB X Kit Window

MPLAB X IDE v6.00 - LCD_KEYPAD_I2C_TEST : default

File Edit View Navigate Source Refactor Production Debug Team Tools Window Help

default

PC: 0x0 ithsvnzc

Kit Window

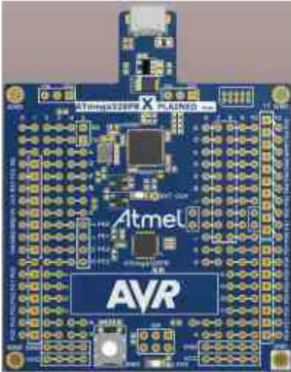
MCU Boards

- ✓ ATmega328PB Xplained Mini

Extension

None

ATmega328PB Xplained Mini



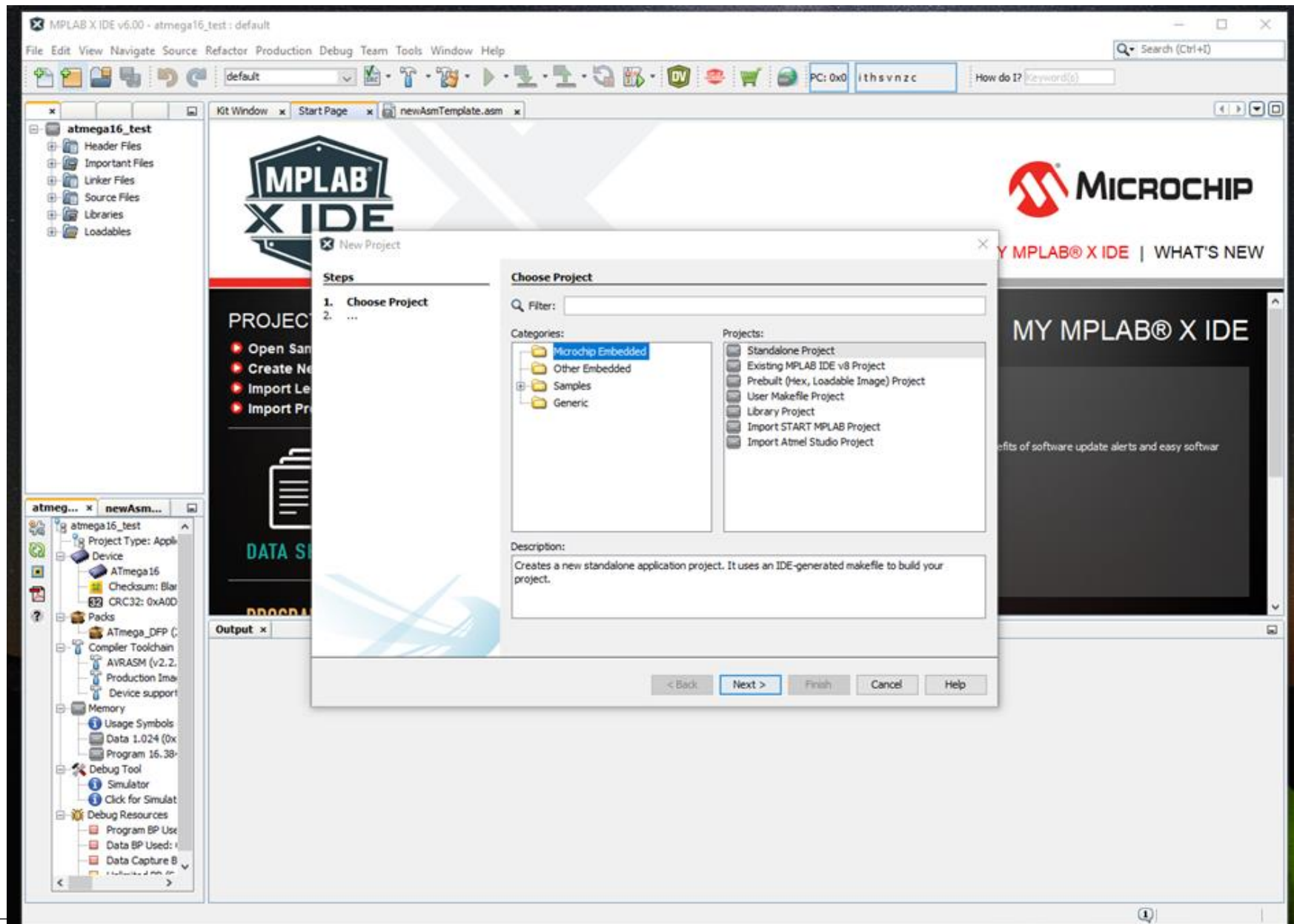
The Microchip ATmega328PB Xplained Mini evaluation kit is a hardware platform to evaluate the Microchip AVR microcontrollers. The evaluation kit comes with a fully integrated debugger that provides seamless integration with Microchip Studio. The kit provides access to the features of the ATmega328PB enabling easy integration of the device in a custom design.

External Links

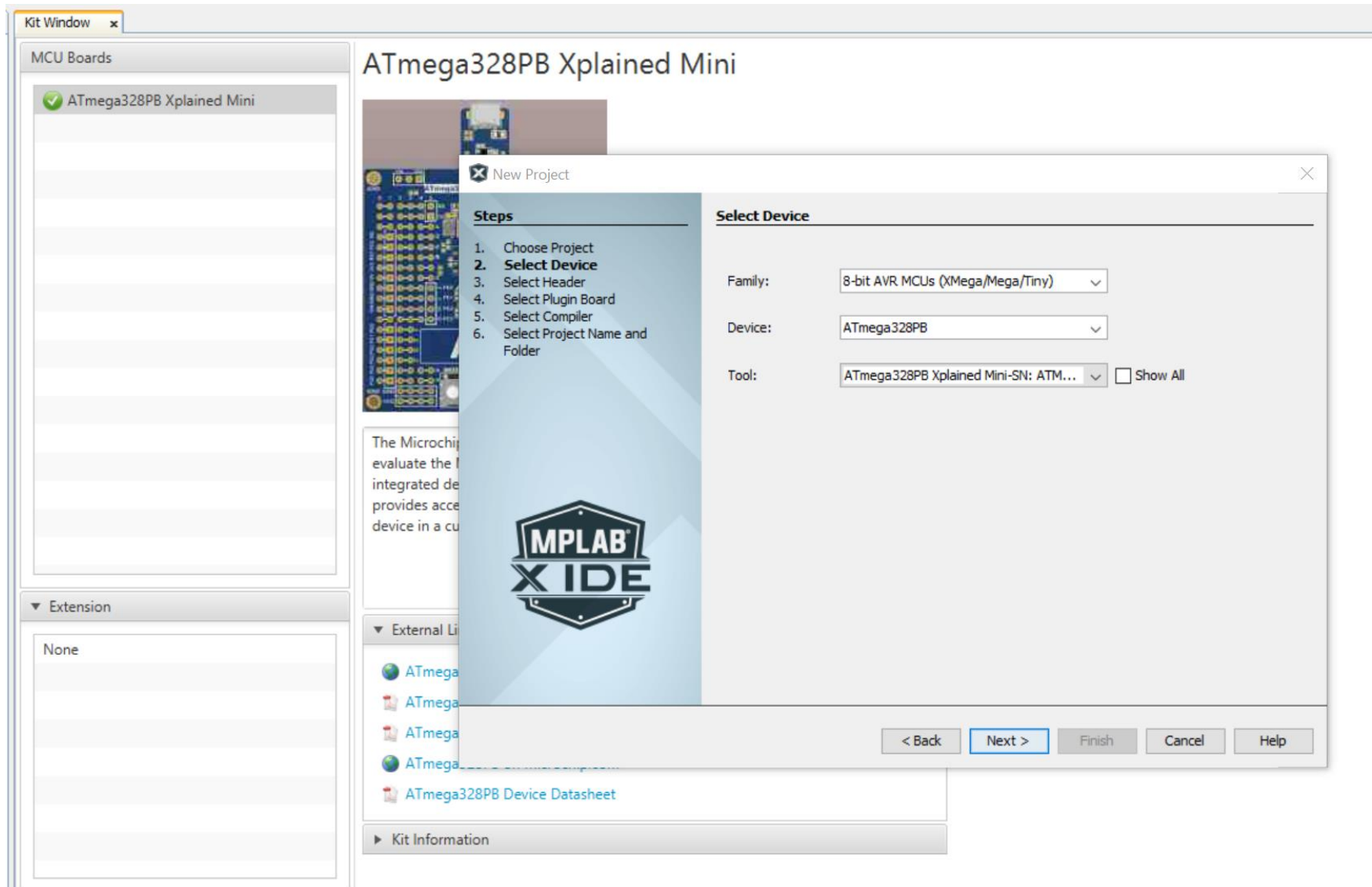
- [ATmega328PB Xplained Mini on microchip.com](#)
- [ATmega328PB Xplained Mini User Guide](#)
- [ATmega328PB Xplained Mini Schematics](#)
- [ATmega328PB on microchip.com](#)
- [ATmega328PB Device Datasheet](#)

Kit Information

MPLAB X: File>New Project



Select Device and Tool



MPLAB X: Select assembler

 New Project 

Steps

1. Choose Project
2. Select Device
3. Select Header
4. Select Plugin Board
5. **Select Compiler**
6. Select Project Name and Folder



Select Compiler

Compiler Toolchains

- + XC8
- + AVR-GCC
- avrasm2
 - avrasm2 (v2.2.8) [C:\Program Files (x86)\Atmel\Studio\7.0\toolchain\avr8\avrasm2]
- IAR for AVR

< Back

Next >

Finish

Cancel

Help

MPLAB X: New File

New File

Steps

1. Choose File Type

2. ...

Choose File Type

Project: ATMEGA328PB_EXAMPLE_1

Filter:

Categories:

Microchip Embedded

C

C++

Assembler

Shell Scripts

Makefiles

Python

XML

Other

File Types:

AssemblyFile.asm

AssemblyFile.s

AssemblyFile.inc

AssemblyFile.as

Description:

An Assembler file with no content.

< Back

Next >

Finish

Cancel

Help

MPLAB X: File Name and Location

New AssemblyFile.asm

Steps

1. Choose File Type
2. **Name and Location**

Name and Location

File Name:

Project:

Folder:

Created File:

< Back Next > **Finish** Cancel Help

Παράδειγμα assembly

```
.include "m328PBdef.inc"      ;ATmega328P microcontroller definitions

; delay = (1000*F1+14) cycles (about DEL_mS in mSeconds)
.equ FOSC_MHZ=16              ;MHz
.equ DEL_mS=1                 ;mS
.equ F1=FOSC_MHZ*DEL_mS

;Init Stack Pointer
    ldi r24, LOW(RAMEND)
    out SPL, r24
    ldi r24, HIGH(RAMEND)
    out SPH, r24

;Init PORTD as output
    ser r26
    out DDRD , r26

    ldi r24, low(F1)          ;
    ldi r25, high(F1)         ;Set delay

loop1:
    ser r26
    out PORTD, r26
    rcall delay_outer        ; mS
    clr r26
    out PORTD, r26
    rcall delay_outer        ; mS
    rjmp loop1
```

Παράδειγμα assembly (Συνέχεια)

;this routine is used to produce a delay 993 cycles

delay_inner:

ldi r23, 247 ; (1 cycle)

loop3:

dec r23 ; 1 cycle

nop ; 1 cycle

brne loop3 ; 1 or 2 cycles

nop ; 1 cycle

ret ; 4 cycles

;this routine is used to produce a delay of (1000*F1+14) cycles

delay_outer:

push r24 ; (2 cycles)

push r25 ; (2 cycles) Save r24:r25

loop4:

rcall delay_inner ; (3+993)=996 cycles

sbiw r24 ,1 ; 2 cycles

brne loop4 ; 1 or 2 cycles

pop r25 ; (2 cycles)

pop r24 ; (2 cycles) Restore r24:r25

ret ;4 cycles

Παράδειγμα σε γλώσσα C

```
#define F_CPU 16000000UL
#include <util/delay.h>
#include <avr/io.h>
```

```
#define DEL 1000U
```

```
int main(void)
{
    DDRD = 0xFF;
    while (1)
    {
        PORTD = 0x00;
        _delay_ms(DEL);
        PORTD = 0xFF;
        _delay_ms(DEL);
    }
}
```

DS18B20

Programmable Resolution 1-Wire Digital Thermometer

General Description

The DS18B20 digital thermometer provides 9-bit to 12-bit Celsius temperature measurements and has an alarm function with nonvolatile user-programmable upper and lower trigger points. The DS18B20 communicates over a 1-Wire bus that by definition requires only one data line (and ground) for communication with a central microprocessor. In addition, the DS18B20 can derive power directly from the data line ("parasite power"), eliminating the need for an external power supply.

Each DS18B20 has a unique 64-bit serial code, which allows multiple DS18B20s to function on the same 1-Wire bus. Thus, it is simple to use one microprocessor to control many DS18B20s distributed over a large area. Applications that can benefit from this feature include HVAC environmental controls, temperature monitoring systems inside buildings, equipment, or machinery, and process monitoring and control systems.

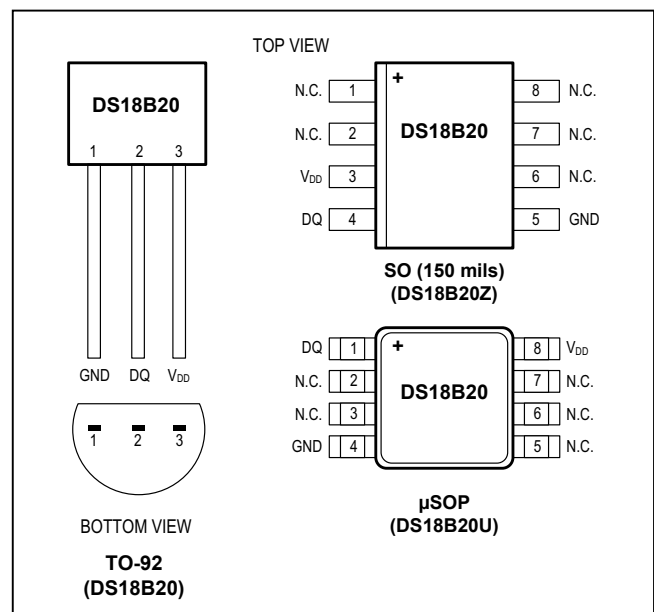
Applications

- Thermostatic Controls
- Industrial Systems
- Consumer Products
- Thermometers
- Thermally Sensitive Systems

Benefits and Features

- Unique 1-Wire® Interface Requires Only One Port Pin for Communication
- Reduce Component Count with Integrated Temperature Sensor and EEPROM
 - Measures Temperatures from -55°C to +125°C (-67°F to +257°F)
 - ±0.5°C Accuracy from -10°C to +85°C
 - Programmable Resolution from 9 Bits to 12 Bits
 - No External Components Required
- Parasitic Power Mode Requires Only 2 Pins for Operation (DQ and GND)
- Simplifies Distributed Temperature-Sensing Applications with Multidrop Capability
 - Each Device Has a Unique 64-Bit Serial Code Stored in On-Board ROM
- Flexible User-Definable Nonvolatile (NV) Alarm Settings with Alarm Search Command Identifies Devices with Temperatures Outside Programmed Limits
- Available in 8-Pin SO (150 mils), 8-Pin µSOP, and 3-Pin TO-92 Packages

Pin Configurations



Ordering Information appears at end of data sheet.

1-Wire is a registered trademark of Maxim Integrated Products, Inc.

Absolute Maximum Ratings

Voltage Range on Any Pin Relative to Ground -0.5V to +6.0V
Operating Temperature Range..... -55°C to +125°C

Storage Temperature Range -55°C to +125°C
Solder Temperature Refer to the IPC/JEDEC
J-STD-020 Specification.

These are stress ratings only and functional operation of the device at these or any other conditions above those indicated in the operation sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods of time may affect reliability.

DC Electrical Characteristics

(-55°C to +125°C; V_{DD} = 3.0V to 5.5V)

PARAMETER	SYMBOL	CONDITIONS	MIN	TYP	MAX	UNITS
Supply Voltage	V_{DD}	Local power (Note 1)	+3.0		+5.5	V
Pullup Supply Voltage	V_{PU}	Parasite power	+3.0		+5.5	V
		Local power	+3.0		V_{DD}	
Thermometer Error	t_{ERR}	-10°C to +85°C			±0.5	°C
		-30°C to +100°C			±1	
		-55°C to +125°C			±2	
Input Logic-Low	V_{IL}	(Notes 1, 4, 5)	-0.3		+0.8	V
Input Logic-High	V_{IH}	Local power	+2.2	The lower of 5.5 or $V_{DD} + 0.3$		V
		Parasite power	+3.0			
Sink Current	I_L	$V_{I/O} = 0.4V$	4.0			mA
Standby Current	I_{DDs}	(Notes 7, 8)		750	1000	nA
Active Current	I_{DD}	$V_{DD} = 5V$ (Note 9)		1	1.5	mA
DQ Input Current	I_{DQ}	(Note 10)		5		μA
Drift		(Note 11)		±0.2		°C

Note 1: All voltages are referenced to ground.

Note 2: The Pullup Supply Voltage specification assumes that the pullup device is ideal, and therefore the high level of the pullup is equal to V_{PU} . In order to meet the V_{IH} spec of the DS18B20, the actual supply rail for the strong pullup transistor must include margin for the voltage drop across the transistor when it is turned on; thus: $V_{PU_ACTUAL} = V_{PU_IDEAL} + V_{TRANSISTOR}$.

Note 3: See typical performance curve in [Figure 1](#). Thermometer Error limits are 3-sigma values.

Note 4: Logic-low voltages are specified at a sink current of 4mA.

Note 5: To guarantee a presence pulse under low voltage parasite power conditions, V_{ILMAX} may have to be reduced to as low as 0.5V.

Note 6: Logic-high voltages are specified at a source current of 1mA.

Note 7: Standby current specified up to +70°C. Standby current typically is 3μA at +125°C.

Note 8: To minimize I_{DDs} , DQ should be within the following ranges: $GND \leq DQ \leq GND + 0.3V$ or $V_{DD} - 0.3V \leq DQ \leq V_{DD}$.

Note 9: Active current refers to supply current during active temperature conversions or EEPROM writes.

Note 10: DQ line is high ("high-Z" state).

Note 11: Drift data is based on a 1000-hour stress test at +125°C with $V_{DD} = 5.5V$.

AC Electrical Characteristics–NV Memory(-55°C to +125°C; $V_{DD} = 3.0V$ to 5.5V)

PARAMETER	SYMBOL	CONDITIONS	MIN	TYP	MAX	UNITS
NV Write Cycle Time	t_{WR}			2	10	ms
EEPROM Writes	N_{EEWR}	-55°C to +55°C	50k			writes
EEPROM Data Retention	t_{EEDR}	-55°C to +55°C	10			years

AC Electrical Characteristics(-55°C to +125°C; $V_{DD} = 3.0V$ to 5.5V)

PARAMETER	SYMBOL	CONDITIONS	MIN	TYP	MAX	UNITS
Temperature Conversion Time	t_{CONV}	9-bit resolution	(Note 12)		93.75	ms
		10-bit resolution			187.5	
		11-bit resolution			375	
		12-bit resolution			750	
Time to Strong Pullup On	t_{SPON}	Start convert T command issued			10	μs
Time Slot	t_{SLOT}	(Note 12)	60		120	μs
Recovery Time	t_{REC}	(Note 12)	1			μs
Write 0 Low Time	t_{LOW0}	(Note 12)	60		120	μs
Write 1 Low Time	t_{LOW1}	(Note 12)	1		15	μs
Read Data Valid	t_{RDV}	(Note 12)			15	μs
Reset Time High	t_{RSTH}	(Note 12)	480			μs
Reset Time Low	t_{RSTL}	(Notes 12, 13)	480			μs
Presence-Detect High	t_{PDHIGH}	(Note 12)	15		60	μs
Presence-Detect Low	t_{PDLOW}	(Note 12)	60		240	μs
Capacitance	$C_{IN/OUT}$				25	pF

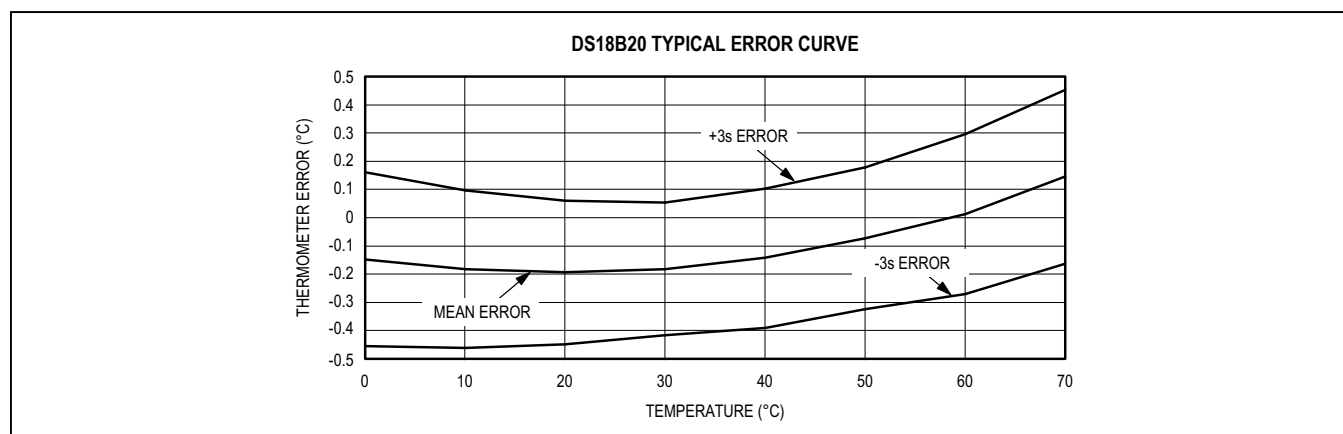
Note 12: See the timing diagrams in [Figure 2](#).**Note 13:** Under parasite power, if $t_{RSTL} > 960\mu s$, a power-on reset can occur.

Figure 1. Typical Performance Curve

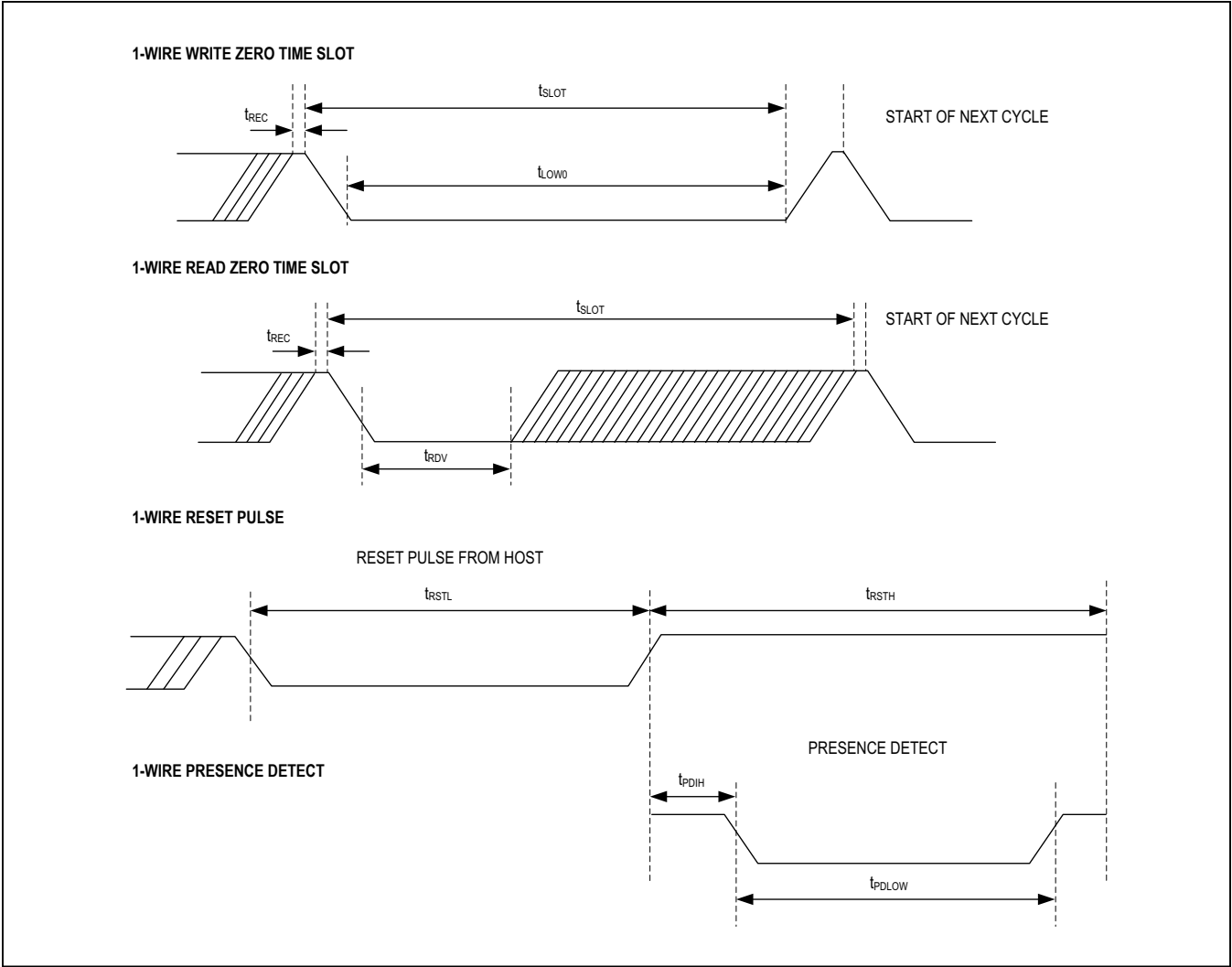


Figure 2. Timing Diagrams

Pin Description

PIN			NAME	FUNCTION
SO	μSOP	TO-92		
1, 2, 6, 7, 8	2, 3, 5, 6, 7	—	N.C.	No Connection
3	8	3	V _{DD}	Optional V _{DD} . V _{DD} must be grounded for operation in parasite power mode.
4	1	2	DQ	Data Input/Output. Open-drain 1-Wire interface pin. Also provides power to the device when used in parasite power mode (see the <i>Powering the DS18B20</i> section.)
5	4	1	GND	Ground

Overview

Figure 3 shows a block diagram of the DS18B20, and pin descriptions are given in the *Pin Description* table. The 64-bit ROM stores the device's unique serial code. The scratchpad memory contains the 2-byte temperature register that stores the digital output from the temperature sensor. In addition, the scratchpad provides access to the 1-byte upper and lower alarm trigger registers (T_H and T_L) and the 1-byte configuration register. The configuration register allows the user to set the resolution of the temperature-to-digital conversion to 9, 10, 11, or 12 bits. The T_H , T_L , and configuration registers are nonvolatile (EEPROM), so they will retain data when the device is powered down.

The DS18B20 uses Maxim's exclusive 1-Wire bus protocol that implements bus communication using one control signal. The control line requires a weak pullup resistor since all devices are linked to the bus via a 3-state or open-drain port (the DQ pin in the case of the DS18B20). In this bus system, the microprocessor (the master device) identifies and addresses devices on the bus using each device's unique 64-bit code. Because each device has a unique code, the number of devices that can be addressed on one bus is virtually unlimited. The 1-Wire bus protocol, including detailed explanations of the commands and "time slots," is covered in the [1-Wire Bus System](#) section.

Another feature of the DS18B20 is the ability to operate without an external power supply. Power is instead supplied through the 1-Wire pullup resistor through the

DQ pin when the bus is high. The high bus signal also charges an internal capacitor (C_{PP}), which then supplies power to the device when the bus is low. This method of deriving power from the 1-Wire bus is referred to as "parasite power." As an alternative, the DS18B20 may also be powered by an external supply on V_{DD} .

Operation—Measuring Temperature

The core functionality of the DS18B20 is its direct-to-digital temperature sensor. The resolution of the temperature sensor is user-configurable to 9, 10, 11, or 12 bits, corresponding to increments of 0.5°C , 0.25°C , 0.125°C , and 0.0625°C , respectively. The default resolution at power-up is 12-bit. The DS18B20 powers up in a low-power idle state. To initiate a temperature measurement and A-to-D conversion, the master must issue a Convert T [44h] command. Following the conversion, the resulting thermal data is stored in the 2-byte temperature register in the scratchpad memory and the DS18B20 returns to its idle state. If the DS18B20 is powered by an external supply, the master can issue "read time slots" (see the [1-Wire Bus System](#) section) after the Convert T command and the DS18B20 will respond by transmitting 0 while the temperature conversion is in progress and 1 when the conversion is done. If the DS18B20 is powered with parasite power, this notification technique cannot be used since the bus must be pulled high by a strong pullup during the entire temperature conversion. The bus requirements for parasite power are explained in detail in the [Powering the DS18B20](#) section.

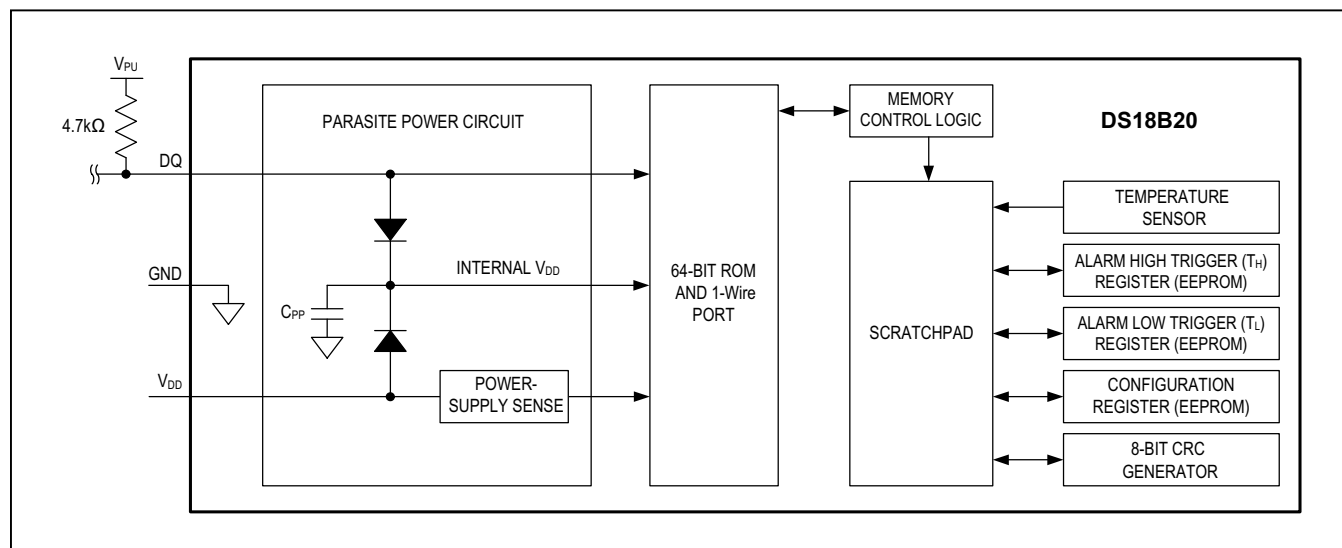


Figure 3. DS18B20 Block Diagram

The DS18B20 output temperature data is calibrated in degrees Celsius; for Fahrenheit applications, a lookup table or conversion routine must be used. The temperature data is stored as a 16-bit sign-extended two's complement number in the temperature register (see [Figure 4](#)). The sign bits (S) indicate if the temperature is positive or negative: for positive numbers S = 0 and for negative numbers S = 1. If the DS18B20 is configured for 12-bit resolution, all bits in the temperature register will contain valid data. For 11-bit resolution, bit 0 is undefined. For 10-bit resolution, bits 1 and 0 are undefined, and for 9-bit resolution bits 2, 1, and 0 are undefined. [Table 1](#) gives examples of digital output data and the corresponding temperature reading for 12-bit resolution conversions.

Operation—Alarm Signaling

After the DS18B20 performs a temperature conversion, the temperature value is compared to the user-defined two's complement alarm trigger values stored in the 1-byte T_H and T_L registers (see [Figure 5](#)). The sign bit (S) indicates if the value is positive or negative: for positive numbers S = 0 and for negative numbers S = 1. The T_H and T_L registers are nonvolatile (EEPROM) so they will retain data when the device is powered down. T_H and T_L can be accessed through bytes 2 and 3 of the scratchpad as explained in the [Memory](#) section.

Only bits 11 through 4 of the temperature register are used in the T_H and T_L comparison since T_H and T_L are 8-bit registers. If the measured temperature is lower than

	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LS BYTE	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴
	BIT 15	BIT 14	BIT 13	BIT 12	BIT 11	BIT 10	BIT 9	BIT 8
MS BYTE	S	S	S	S	S	2 ⁶	2 ⁵	2 ⁴

S = SIGN

Figure 4. Temperature Register Format

Table 1. Temperature/Data Relationship

TEMPERATURE (°C)	DIGITAL OUTPUT (BINARY)	DIGITAL OUTPUT (HEX)
+125	0000 0111 1101 0000	07D0h
+85*	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Fh
-55	1111 1100 1001 0000	FC90h

*The power-on reset value of the temperature register is +85°C.

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
S	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰

Figure 5. T_H and T_L Register Format

or equal to T_L or higher than or equal to T_H , an alarm condition exists and an alarm flag is set inside the DS18B20. This flag is updated after every temperature measurement; therefore, if the alarm condition goes away, the flag will be turned off after the next temperature conversion.

The master device can check the alarm flag status of all DS18B20s on the bus by issuing an Alarm Search [ECh] command. Any DS18B20s with a set alarm flag will respond to the command, so the master can determine exactly which DS18B20s have experienced an alarm condition. If an alarm condition exists and the T_H or T_L settings have changed, another temperature conversion should be done to validate the alarm condition.

Powering the DS18B20

The DS18B20 can be powered by an external supply on the V_{DD} pin, or it can operate in “parasite power” mode, which allows the DS18B20 to function without a local external supply. Parasite power is very useful for applications that require remote temperature sensing or that are very space constrained. Figure 3 shows the DS18B20's parasite-power control circuitry, which “steals” power from the 1-Wire bus via the DQ pin when the bus is high. The stolen charge powers the DS18B20 while the bus is high, and some of the charge is stored on the parasite power capacitor (C_{PP}) to provide power when the bus is low. When the DS18B20 is used in parasite power mode, the V_{DD} pin must be connected to ground.

In parasite power mode, the 1-Wire bus and CPP can provide sufficient current to the DS18B20 for most operations as long as the specified timing and voltage requirements are met (see the [DC Electrical Characteristics](#) and [AC Electrical Characteristics](#)). However, when the DS18B20 is performing temperature conversions or copying data from the scratchpad memory to EEPROM, the operating current can be as high as 1.5mA. This current can cause an unacceptable voltage drop across the weak 1-Wire pullup resistor and is more current than can be supplied

by C_{PP} . To assure that the DS18B20 has sufficient supply current, it is necessary to provide a strong pullup on the 1-Wire bus whenever temperature conversions are taking place or data is being copied from the scratchpad to EEPROM. This can be accomplished by using a MOSFET to pull the bus directly to the rail as shown in Figure 6. The 1-Wire bus must be switched to the strong pullup within 10 μ s (max) after a Convert T [44h] or Copy Scratchpad [48h] command is issued, and the bus must be held high by the pullup for the duration of the conversion (t_{CONV}) or data transfer ($t_{WR} = 10$ ms). No other activity can take place on the 1-Wire bus while the pullup is enabled.

The DS18B20 can also be powered by the conventional method of connecting an external power supply to the V_{DD} pin, as shown in Figure 7. The advantage of this method is that the MOSFET pullup is not required, and the 1-Wire bus is free to carry other traffic during the temperature conversion time.

The use of parasite power is not recommended for temperatures above +100°C since the DS18B20 may not be able to sustain communications due to the higher leakage currents that can exist at these temperatures. For applications in which such temperatures are likely, it is strongly recommended that the DS18B20 be powered by an external power supply.

In some situations the bus master may not know whether the DS18B20s on the bus are parasite powered or powered by external supplies. The master needs this information to determine if the strong bus pullup should be used during temperature conversions. To get this information, the master can issue a Skip ROM [CCh] command followed by a Read Power Supply [B4h] command followed by a “read time slot”. During the read time slot, parasite powered DS18B20s will pull the bus low, and externally powered DS18B20s will let the bus remain high. If the bus is pulled low, the master knows that it must supply the strong pullup on the 1-Wire bus during temperature conversions.

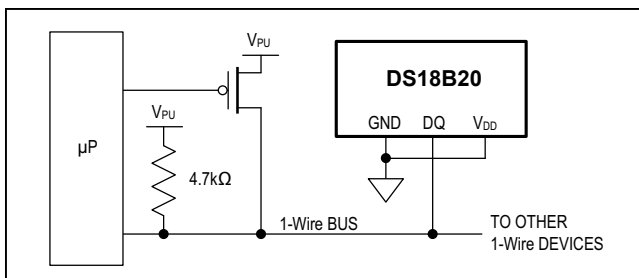


Figure 6. Supplying the Parasite-Powered DS18B20 During Temperature Conversions

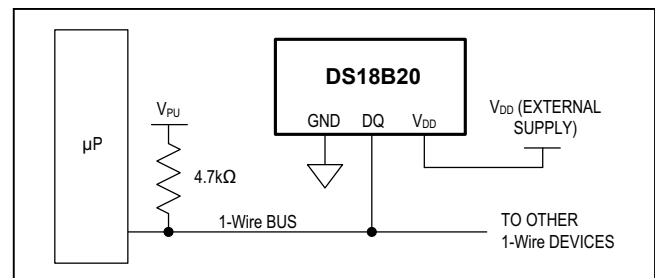


Figure 7. Powering the DS18B20 with an External Supply

64-BIT Lasered ROM code

Each DS18B20 contains a unique 64-bit code (see [Figure 8](#)) stored in ROM. The least significant 8 bits of the ROM code contain the DS18B20's 1-Wire family code: 28h. The next 48 bits contain a unique serial number. The most significant 8 bits contain a cyclic redundancy check (CRC) byte that is calculated from the first 56 bits of the ROM code. A detailed explanation of the CRC bits is provided in the [CRC Generation](#) section. The 64-bit ROM code and associated ROM function control logic allow the DS18B20 to operate as a 1-Wire device using the protocol detailed in the [1-Wire Bus System](#) section.

Memory

The DS18B20's memory is organized as shown in [Figure 9](#). The memory consists of an SRAM scratchpad with nonvolatile EEPROM storage for the high and low alarm trigger registers (T_H and T_L) and configuration register. Note that if the DS18B20 alarm function is not used, the TH and TL registers can serve as general-purpose memory. All memory commands are described in detail in the [DS18B20 Function Commands](#) section.

Byte 0 and byte 1 of the scratchpad contain the LSB and the MSB of the temperature register, respectively. These bytes are read-only. Bytes 2 and 3 provide access to TH and TL registers. Byte 4 contains the configuration regis-

ter data, which is explained in detail in the [Configuration Register](#) section. Bytes 5, 6, and 7 are reserved for internal use by the device and cannot be overwritten.

Byte 8 of the scratchpad is read-only and contains the CRC code for bytes 0 through 7 of the scratchpad. The DS18B20 generates this CRC using the method described in the [CRC Generation](#) section.

Data is written to bytes 2, 3, and 4 of the scratchpad using the Write Scratchpad [4Eh] command; the data must be transmitted to the DS18B20 starting with the least significant bit of byte 2. To verify data integrity, the scratchpad can be read (using the Read Scratchpad [BEh] command) after the data is written. When reading the scratchpad, data is transferred over the 1-Wire bus starting with the least significant bit of byte 0. To transfer the T_H, T_L and configuration data from the scratchpad to EEPROM, the master must issue the Copy Scratchpad [48h] command.

Data in the EEPROM registers is retained when the device is powered down; at power-up the EEPROM data is reloaded into the corresponding scratchpad locations. Data can also be reloaded from EEPROM to the scratchpad at any time using the Recall E² [B8h] command. The master can issue read time slots following the Recall E² command and the DS18B20 will indicate the status of the recall by transmitting 0 while the recall is in progress and 1 when the recall is done.

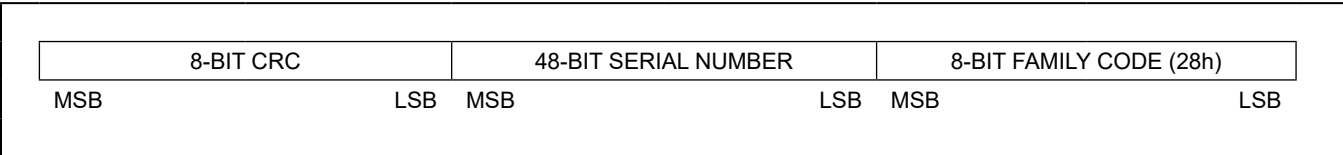


Figure 8. 64-Bit Lasered ROM Code

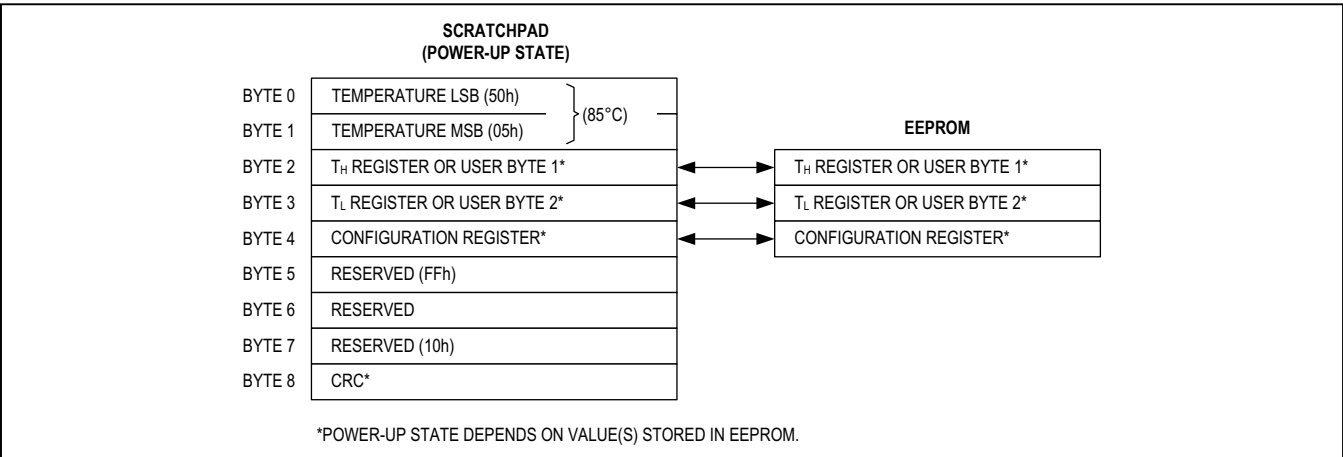


Figure 9. DS18B20 Memory Map

Configuration Register

Byte 4 of the scratchpad memory contains the configuration register, which is organized as illustrated in [Figure 10](#). The user can set the conversion resolution of the DS18B20 using the R0 and R1 bits in this register as shown in [Table 2](#). The power-up default of these bits is R0 = 1 and R1 = 1 (12-bit resolution). Note that there is a direct tradeoff between resolution and conversion time. Bit 7 and bits 0 to 4 in the configuration register are reserved for internal use by the device and cannot be overwritten.

CRC Generation

CRC bytes are provided as part of the DS18B20's 64-bit ROM code and in the 9th byte of the scratchpad memory. The ROM code CRC is calculated from the first 56 bits of the ROM code and is contained in the most significant byte of the ROM. The scratchpad CRC is calculated from the data stored in the scratchpad, and therefore it changes when the data in the scratchpad changes. The CRCs provide the bus master with a method of data validation when data is read from the DS18B20. To verify that data has been read correctly, the bus master must re-calculate the CRC from the received data and then compare this value to either the ROM code CRC (for ROM reads) or to the scratchpad CRC (for scratchpad reads). If the calculated CRC matches the read CRC, the data has been

received error free. The comparison of CRC values and the decision to continue with an operation are determined entirely by the bus master. There is no circuitry inside the DS18B20 that prevents a command sequence from proceeding if the DS18B20 CRC (ROM or scratchpad) does not match the value generated by the bus master.

The equivalent polynomial function of the CRC (ROM or scratchpad) is:

$$CRC = X^8 + X^5 + X^4 + 1$$

The bus master can re-calculate the CRC and compare it to the CRC values from the DS18B20 using the polynomial generator shown in [Figure 11](#). This circuit consists of a shift register and XOR gates, and the shift register bits are initialized to 0. Starting with the least significant bit of the ROM code or the least significant bit of byte 0 in the scratchpad, one bit at a time should be shifted into the shift register. After shifting in the 56th bit from the ROM or the most significant bit of byte 7 from the scratchpad, the polynomial generator will contain the recalculated CRC. Next, the 8-bit ROM code or scratchpad CRC from the DS18B20 must be shifted into the circuit. At this point, if the re-calculated CRC was correct, the shift register will contain all 0s. Additional information about the Maxim 1-Wire cyclic redundancy check is available in *Application Note 27: Understanding and Using Cyclic Redundancy Checks with Maxim iButton Products*.

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
0	R1	R0	1	1	1	1	1

Figure 10. Configuration Register

Table 2. Thermometer Resolution Configuration

R1	R0	RESOLUTION (BITS)	MAX CONVERSION TIME	
0	0	9	93.75ms	(t _{CONV} /8)
0	1	10	187.5ms	(t _{CONV} /4)
1	0	11	375ms	(t _{CONV} /2)
1	1	12	750ms	(t _{CONV})

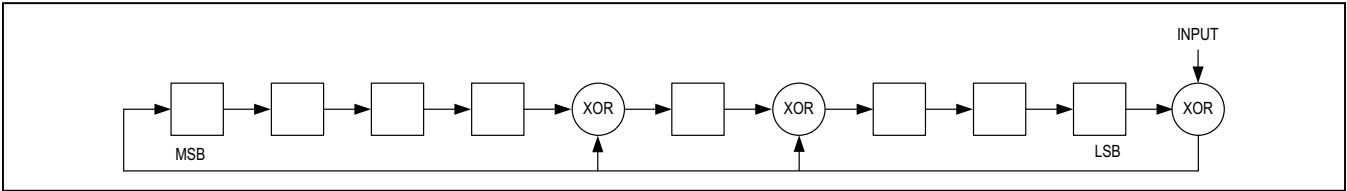


Figure 11. CRC Generator

1-Wire Bus System

The 1-Wire bus system uses a single bus master to control one or more slave devices. The DS18B20 is always a slave. When there is only one slave on the bus, the system is referred to as a “single-drop” system; the system is “multidrop” if there are multiple slaves on the bus.

All data and commands are transmitted least significant bit first over the 1-Wire bus.

The following discussion of the 1-Wire bus system is broken down into three topics: hardware configuration, transaction sequence, and 1-Wire signaling (signal types and timing).

Hardware Configuration

The 1-Wire bus has by definition only a single data line. Each device (master or slave) interfaces to the data line via an open-drain or 3-state port. This allows each device to “release” the data line when the device is not transmitting data so the bus is available for use by another device. The 1-Wire port of the DS18B20 (the DQ pin) is open drain with an internal circuit equivalent to that shown in [Figure 12](#).

The 1-Wire bus requires an external pullup resistor of approximately 5k Ω ; thus, the idle state for the 1-Wire bus is high. If for any reason a transaction needs to be suspended, the bus MUST be left in the idle state if the transaction is to resume. Infinite recovery time can occur between bits so long as the 1-Wire bus is in the inactive (high) state during the recovery period. If the bus is held low for more than 480 μ s, all components on the bus will be reset.

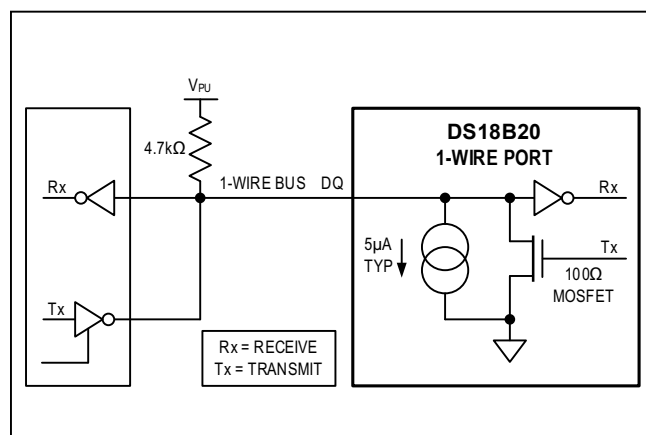


Figure 12. Hardware Configuration

Transaction Sequence

The transaction sequence for accessing the DS18B20 is as follows:

- Step 1. Initialization
- Step 2. ROM Command (followed by any required data exchange)
- Step 3. DS18B20 Function Command (followed by any required data exchange)

It is very important to follow this sequence every time the DS18B20 is accessed, as the DS18B20 will not respond if any steps in the sequence are missing or out of order. Exceptions to this rule are the Search ROM [F0h] and Alarm Search [ECh] commands. After issuing either of these ROM commands, the master must return to Step 1 in the sequence.

Initialization

All transactions on the 1-Wire bus begin with an initialization sequence. The initialization sequence consists of a reset pulse transmitted by the bus master followed by presence pulse(s) transmitted by the slave(s). The presence pulse lets the bus master know that slave devices (such as the DS18B20) are on the bus and are ready to operate. Timing for the reset and presence pulses is detailed in the [1-Wire Signaling](#) section.

ROM Commands

After the bus master has detected a presence pulse, it can issue a ROM command. These commands operate on the unique 64-bit ROM codes of each slave device and allow the master to single out a specific device if many are present on the 1-Wire bus. These commands also allow the master to determine how many and what types of devices are present on the bus or if any device has experienced an alarm condition. There are five ROM commands, and each command is 8 bits long. The master device must issue an appropriate ROM command before issuing a DS18B20 function command. A flowchart for operation of the ROM commands is shown in [Figure 13](#).

Search Rom [F0h]

When a system is initially powered up, the master must identify the ROM codes of all slave devices on the bus, which allows the master to determine the number of slaves and their device types. The master learns the ROM codes through a process of elimination that requires the master to perform a Search ROM cycle (i.e., Search ROM command followed by data exchange) as many times as necessary to identify all of the slave devices.

If there is only one slave on the bus, the simpler Read ROM [33h] command can be used in place of the Search ROM process. For a detailed explanation of the Search ROM procedure, refer to *Application Note 937: Book of iButton® Standards*. After every Search ROM cycle, the bus master must return to Step 1 (Initialization) in the transaction sequence.

Read Rom [33h]

This command can only be used when there is one slave on the bus. It allows the bus master to read the slave's 64-bit ROM code without using the Search ROM procedure. If this command is used when there is more than one slave present on the bus, a data collision will occur when all the slaves attempt to respond at the same time.

Match Rom [55H]

The match ROM command followed by a 64-bit ROM code sequence allows the bus master to address a specific slave device on a multidrop or single-drop bus. Only the slave that exactly matches the 64-bit ROM code sequence will respond to the function command issued by the master; all other slaves on the bus will wait for a reset pulse.

Skip Rom [CCh]

The master can use this command to address all devices on the bus simultaneously without sending out any ROM code information. For example, the master can make all DS18B20s on the bus perform simultaneous temperature conversions by issuing a Skip ROM command followed by a Convert T [44h] command.

Note that the Read Scratchpad [BEh] command can follow the Skip ROM command only if there is a single slave device on the bus. In this case, time is saved by allowing the master to read from the slave without sending the device's 64-bit ROM code. A Skip ROM command followed by a Read Scratchpad command will cause a data collision on the bus if there is more than one slave since multiple devices will attempt to transmit data simultaneously.

Alarm Search [ECh]

The operation of this command is identical to the operation of the Search ROM command except that only slaves with a set alarm flag will respond. This command allows the master device to determine if any DS18B20s experienced an alarm condition during the most recent temperature conversion. After every Alarm Search cycle (i.e., Alarm Search command followed by data exchange), the bus

master must return to Step 1 (Initialization) in the transaction sequence. See the [Operation—Alarm Signaling](#) section for an explanation of alarm flag operation.

DS18B20 Function Commands

After the bus master has used a ROM command to address the DS18B20 with which it wishes to communicate, the master can issue one of the DS18B20 function commands. These commands allow the master to write to and read from the DS18B20's scratchpad memory, initiate temperature conversions and determine the power supply mode. The DS18B20 function commands, which are described below, are summarized in [Table 3](#) and illustrated by the flowchart in [Figure 14](#).

Convert T [44h]

This command initiates a single temperature conversion. Following the conversion, the resulting thermal data is stored in the 2-byte temperature register in the scratchpad memory and the DS18B20 returns to its low-power idle state. If the device is being used in parasite power mode, within 10 μ s (max) after this command is issued the master must enable a strong pullup on the 1-Wire bus for the duration of the conversion (t_{CONV}) as described in the [Powering the DS18B20](#) section. If the DS18B20 is powered by an external supply, the master can issue read time slots after the Convert T command and the DS18B20 will respond by transmitting a 0 while the temperature conversion is in progress and a 1 when the conversion is done. In parasite power mode this notification technique cannot be used since the bus is pulled high by the strong pullup during the conversion.

Write Scratchpad [4Eh]

This command allows the master to write 3 bytes of data to the DS18B20's scratchpad. The first data byte is written into the T_H register (byte 2 of the scratchpad), the second byte is written into the T_L register (byte 3), and the third byte is written into the configuration register (byte 4). Data must be transmitted least significant bit first. All three bytes MUST be written before the master issues a reset, or the data may be corrupted.

Read Scratchpad [BEh]

This command allows the master to read the contents of the scratchpad. The data transfer starts with the least significant bit of byte 0 and continues through the scratchpad until the 9th byte (byte 8 – CRC) is read. The master may issue a reset to terminate reading at any time if only part of the scratchpad data is needed.

iButton is a registered trademark of Maxim Integrated Products, Inc.

Copy Scratchpad [48h]

This command copies the contents of the scratchpad T_H , T_L and configuration registers (bytes 2, 3 and 4) to EEPROM. If the device is being used in parasite power mode, within 10 μ s (max) after this command is issued the master must enable a strong pullup on the 1-Wire bus for at least 10ms as described in the [Powering the DS18B20](#) section.

Recall E² [B8h]

This command recalls the alarm trigger values (T_H and T_L) and configuration data from EEPROM and places the data in bytes 2, 3, and 4, respectively, in the scratchpad memory. The master device can issue read time slots

following the Recall E² command and the DS18B20 will indicate the status of the recall by transmitting 0 while the recall is in progress and 1 when the recall is done. The recall operation happens automatically at power-up, so valid data is available in the scratchpad as soon as power is applied to the device.

Read Power Supply [B4h]

The master device issues this command followed by a read time slot to determine if any DS18B20s on the bus are using parasite power. During the read time slot, parasite powered DS18B20s will pull the bus low, and externally powered DS18B20s will let the bus remain high. See the [Powering the DS18B20](#) section for usage information for this command.

Table 3. DS18B20 Function Command Set

COMMAND	DESCRIPTION	PROTOCOL	1-Wire BUS ACTIVITY AFTER COMMAND IS ISSUED	NOTES
TEMPERATURE CONVERSION COMMANDS				
Convert T	Initiates temperature conversion.	44h	DS18B20 transmits conversion status to master (not applicable for parasite-powered DS18B20s).	1
MEMORY COMMANDS				
Read Scratchpad	Reads the entire scratchpad including the CRC byte.	BEh	DS18B20 transmits up to 9 data bytes to master.	2
Write Scratchpad	Writes data into scratchpad bytes 2, 3, and 4 (T_H , T_L , and configuration registers).	4Eh	Master transmits 3 data bytes to DS18B20.	3
Copy Scratchpad	Copies T_H , T_L , and configuration register data from the scratchpad to EEPROM.	48h	None	1
Recall E ²	Recalls T_H , T_L , and configuration register data from EEPROM to the scratchpad.	B8h	DS18B20 transmits recall status to master.	
Read Power Supply	Signals DS18B20 power supply mode to the master.	B4h	DS18B20 transmits supply status to master.	

Note 1: For parasite-powered DS18B20s, the master must enable a strong pullup on the 1-Wire bus during temperature conversions and copies from the scratchpad to EEPROM. No other bus activity may take place during this time.

Note 2: The master can interrupt the transmission of data at any time by issuing a reset.

Note 3: All three bytes must be written before a reset is issued.

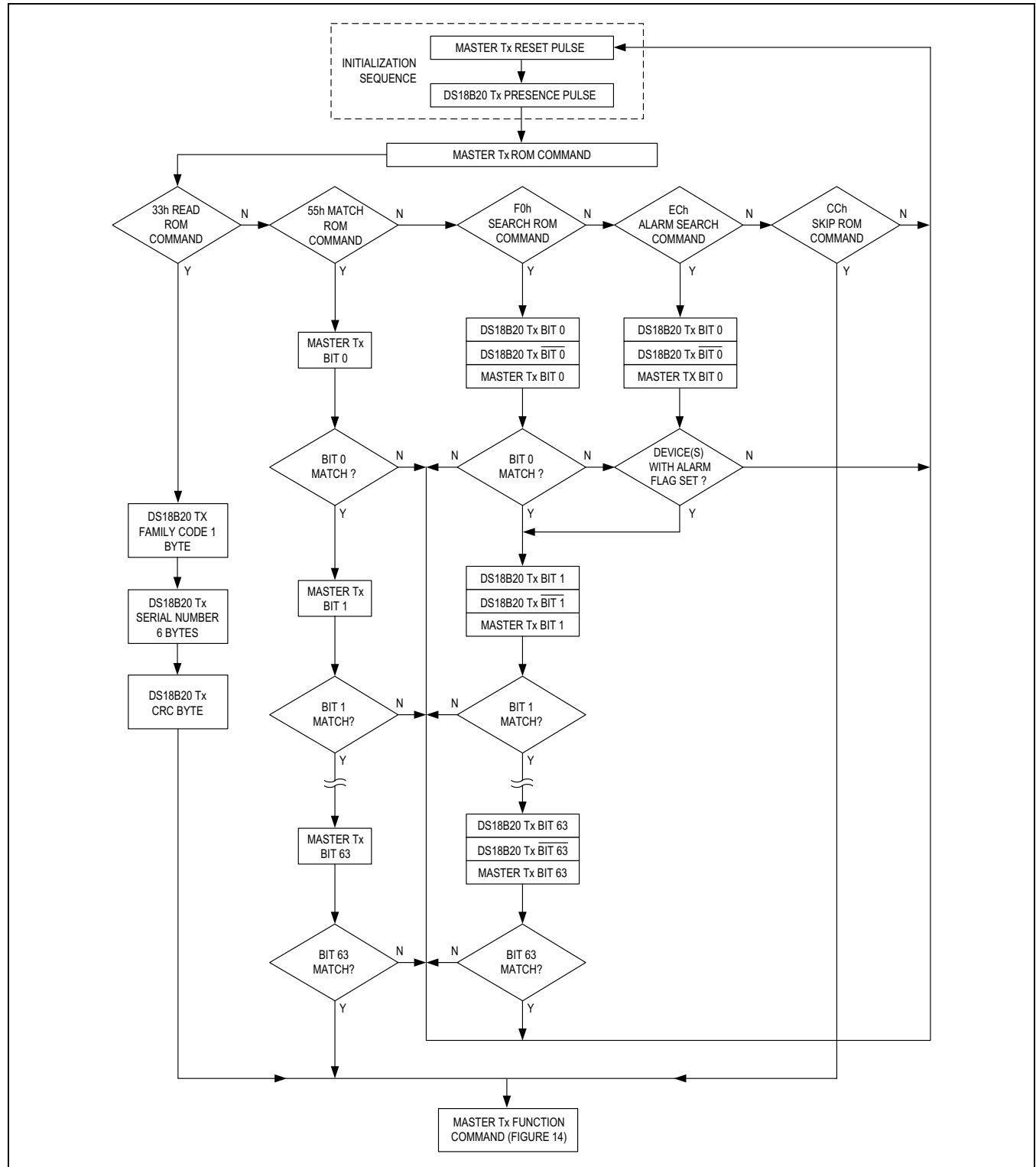


Figure 13. ROM Commands Flowchart

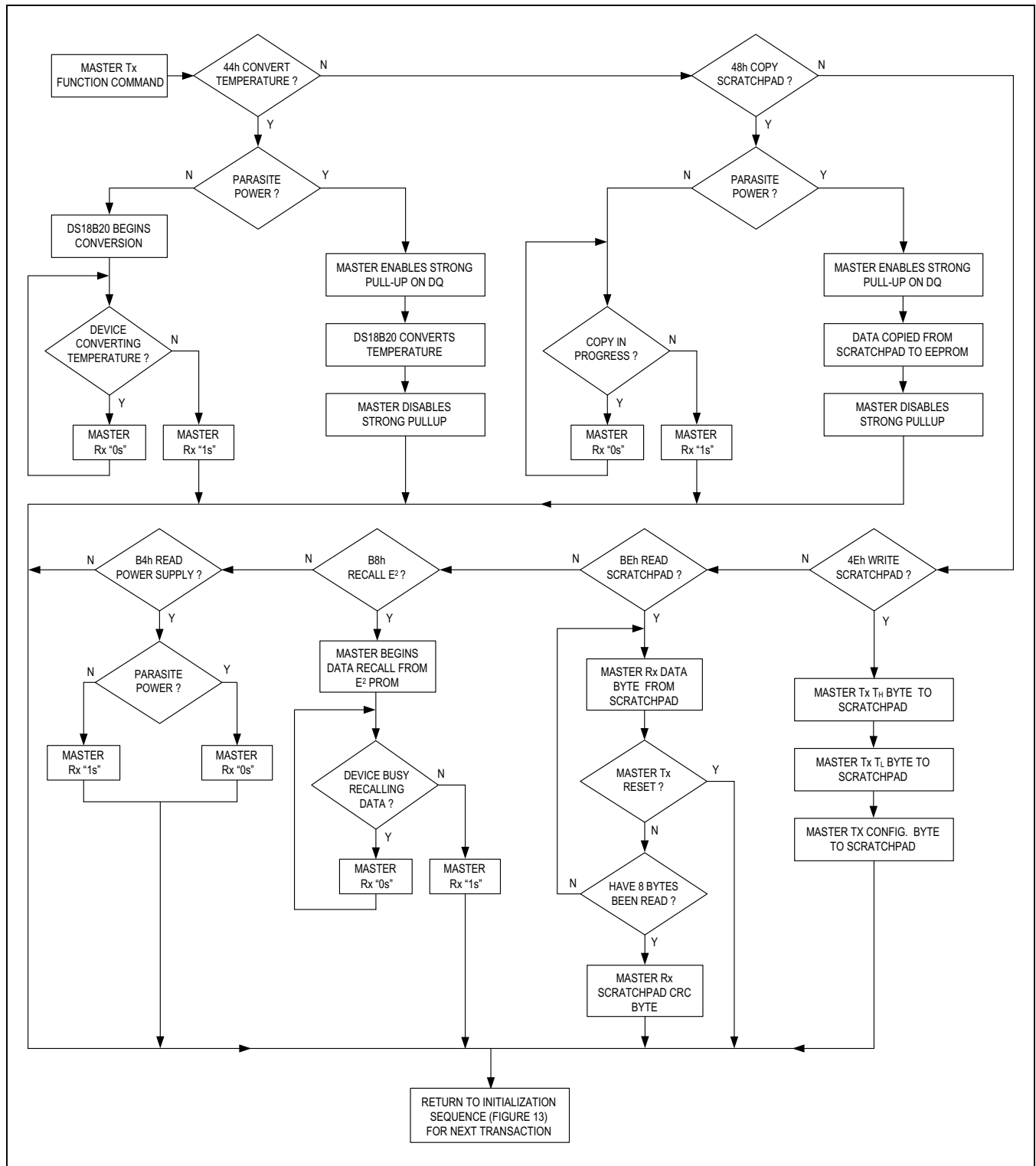


Figure 14. DS18B20 Function Commands Flowchart

1-Wire Signaling

The DS18B20 uses a strict 1-Wire communication protocol to ensure data integrity. Several signal types are defined by this protocol: reset pulse, presence pulse, write 0, write 1, read 0, and read 1. The bus master initiates all these signals, with the exception of the presence pulse.

Initialization Procedure—Reset And Presence Pulses

All communication with the DS18B20 begins with an initialization sequence that consists of a reset pulse from the master followed by a presence pulse from the DS18B20. This is illustrated in [Figure 15](#). When the DS18B20 sends the presence pulse in response to the reset, it is indicating to the master that it is on the bus and ready to operate.

During the initialization sequence the bus master transmits (T_X) the reset pulse by pulling the 1-Wire bus low for a minimum of 480 μ s. The bus master then releases the bus and goes into receive mode (R_X). When the bus is released, the 5k Ω pullup resistor pulls the 1-Wire bus high. When the DS18B20 detects this rising edge, it waits 15 μ s to 60 μ s and then transmits a presence pulse by pulling the 1-Wire bus low for 60 μ s to 240 μ s.

Read/Write Time Slots

The bus master writes data to the DS18B20 during write time slots and reads data from the DS18B20 during read time slots. One bit of data is transmitted over the 1-Wire bus per time slot.

Write Time Slots

There are two types of write time slots: “Write 1” time slots and “Write 0” time slots. The bus master uses a Write 1 time slot to write a logic 1 to the DS18B20 and a Write 0 time slot to write a logic 0 to the DS18B20. All write time slots must be a minimum of 60 μ s in duration with a minimum of a 1 μ s recovery time between individual write slots. Both types of write time slots are initiated by the master pulling the 1-Wire bus low (see [Figure 14](#)).

To generate a Write 1 time slot, after pulling the 1-Wire bus low, the bus master must release the 1-Wire bus within 15 μ s. When the bus is released, the 5k Ω pullup resistor will pull the bus high. To generate a Write 0 time slot, after pulling the 1-Wire bus low, the bus master must continue to hold the bus low for the duration of the time slot (at least 60 μ s).

The DS18B20 samples the 1-Wire bus during a window that lasts from 15 μ s to 60 μ s after the master initiates the write time slot. If the bus is high during the sampling window, a 1 is written to the DS18B20. If the line is low, a 0 is written to the DS18B20.

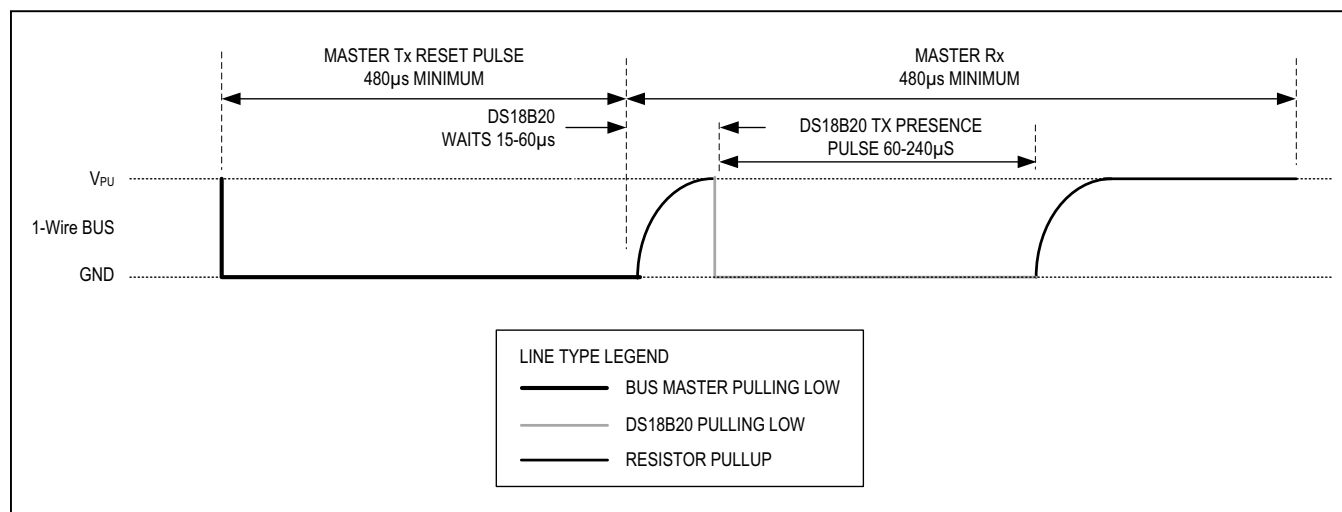


Figure 15. Initialization Timing

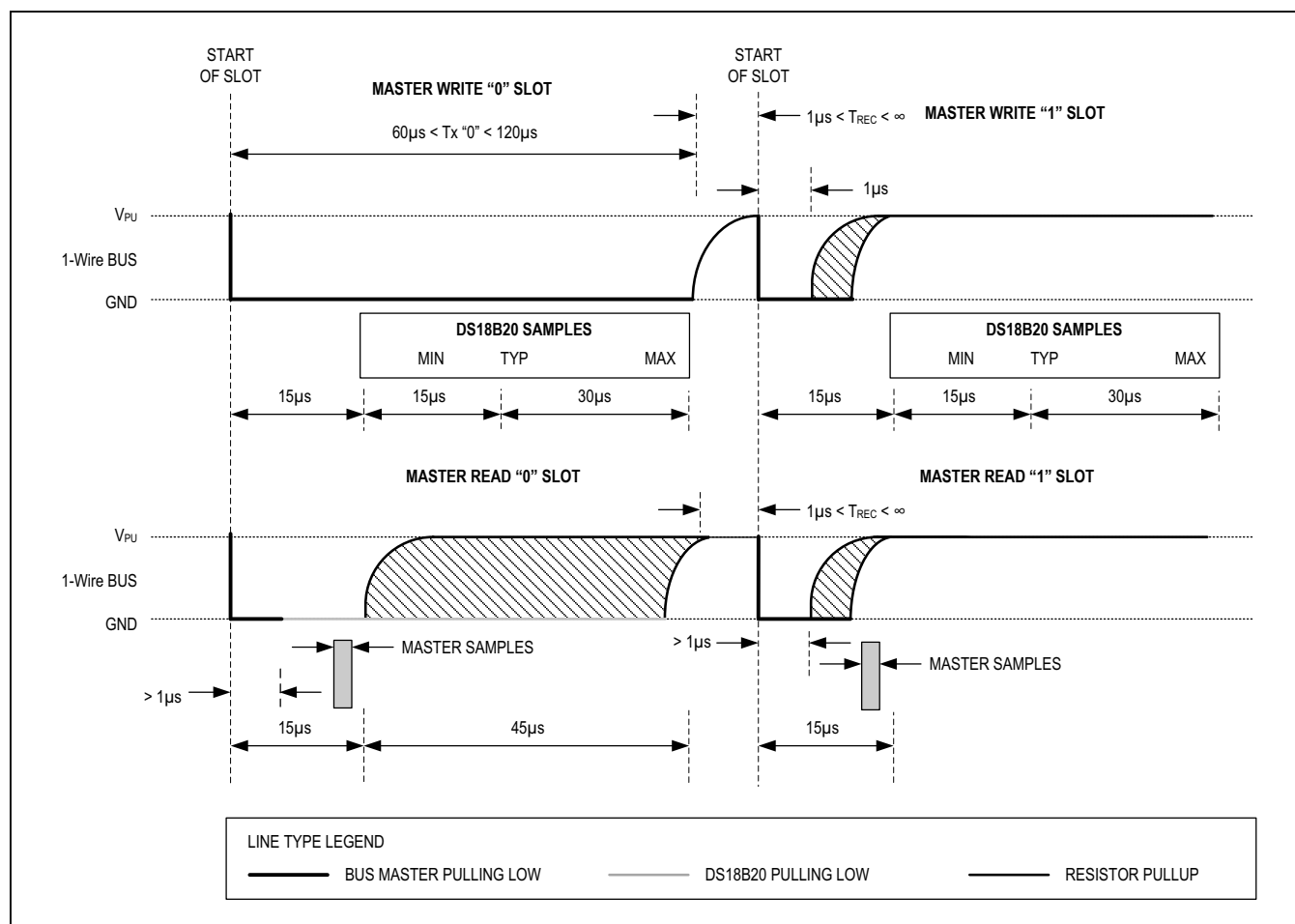


Figure 16. Read/Write Time Slot Timing Diagram

Read Time Slots

The DS18B20 can only transmit data to the master when the master issues read time slots. Therefore, the master must generate read time slots immediately after issuing a Read Scratchpad [BEh] or Read Power Supply [B4h] command, so that the DS18B20 can provide the requested data. In addition, the master can generate read time slots after issuing Convert T [44h] or Recall E2 [B8h] commands to find out the status of the operation as explained in the [DS18B20 Function Commands](#) section.

All read time slots must be a minimum of $60\mu s$ in duration with a minimum of a $1\mu s$ recovery time between slots. A read time slot is initiated by the master device pulling the 1-Wire bus low for a minimum of $1\mu s$ and then releasing the bus (see [Figure 16](#)). After the master initiates the

read time slot, the DS18B20 will begin transmitting a 1 or 0 on bus. The DS18B20 transmits a 1 by leaving the bus high and transmits a 0 by pulling the bus low. When transmitting a 0, the DS18B20 will release the bus by the end of the time slot, and the bus will be pulled back to its high idle state by the pullup resistor. Output data from the DS18B20 is valid for $15\mu s$ after the falling edge that initiated the read time slot. Therefore, the master must release the bus and then sample the bus state within $15\mu s$ from the start of the slot.

[Figure 17](#) illustrates that the sum of T_{INIT} , T_{RC} , and T_{SAMPLE} must be less than $15\mu s$ for a read time slot. [Figure 18](#) shows that system timing margin is maximized by keeping T_{INIT} and T_{RC} as short as possible and by locating the master sample time during read time slots towards the end of the $15\mu s$ period.

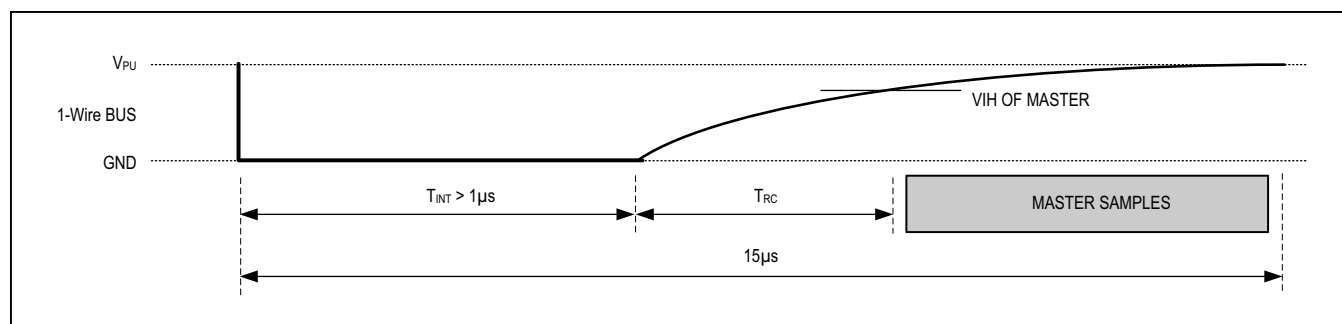


Figure 17. Detailed Master Read 1 Timing

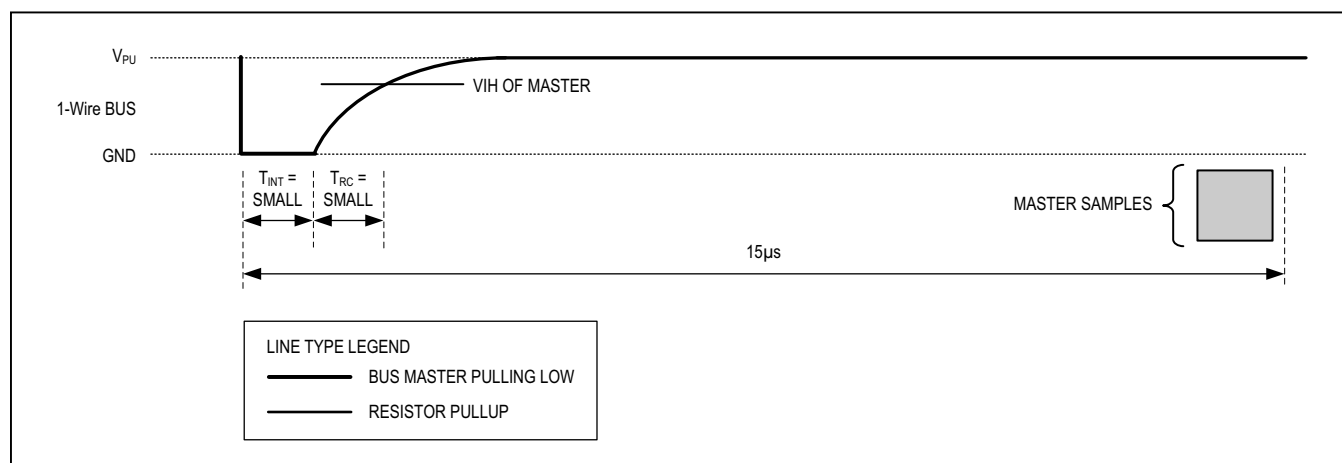


Figure 18. Recommended Master Read 1 Timing

Related Application Notes

The following application notes can be applied to the DS18B20 and are available at www.maximintegrated.com.

Application Note 27: Understanding and Using Cyclic Redundancy Checks with Maxim iButton Products

Application Note 122: Using Dallas' 1-Wire ICs in 1-Cell Li-Ion Battery Packs with Low-Side N-Channel Safety FETs Master

Application Note 126: 1-Wire Communication Through Software

Application Note 162: Interfacing the DS18x20/DS1822 1-Wire Temperature Sensor in a Microcontroller Environment

Application Note 208: Curve Fitting the Error of a Bandgap-Based Digital Temperature Sensor

Application Note 2420: 1-Wire Communication with a Microchip PICmicro Microcontroller

Application Note 3754: Single-Wire Serial Bus Carries Isolated Power and Data

Sample 1-Wire subroutines that can be used in conjunction with *Application Note 74: Reading and Writing iButtons via Serial Interfaces* can be downloaded from the Maxim website.

DS18B20 Operation Example 1

In this example there are multiple DS18B20s on the bus and they are using parasite power. The bus master initiates a temperature conversion in a specific DS18B20 and then reads its scratchpad and recalculates the CRC to verify the data.

MASTER MODE	DATA (LSB FIRST)	COMMENTS
Tx	Reset	Master issues reset pulse.
Rx	Presence	DS18B20s respond with presence pulse.
Tx	55h	Master issues Match ROM command.
Tx	64-bit ROM code	Master sends DS18B20 ROM code.
Tx	44h	Master issues Convert T command.
Tx	DQ line held high by strong pullup	Master applies strong pullup to DQ for the duration of the conversion (t_{CONV}).
Tx	Reset	Master issues reset pulse.
Rx	Presence	DS18B20s respond with presence pulse.
Tx	55h	Master issues Match ROM command.
Tx	64-bit ROM code	Master sends DS18B20 ROM code.
Tx	BEh	Master issues Read Scratchpad command.
Rx	9 data bytes	Master reads entire scratchpad including CRC. The master then recalculates the CRC of the first eight data bytes from the scratchpad and compares the calculated CRC with the read CRC (byte 9). If they match, the master continues; if not, the read operation is repeated.

DS18B20 Operation Example 2

In this example there is only one DS18B20 on the bus and it is using parasite power. The master writes to the TH, TL, and configuration registers in the DS18B20 scratchpad and then reads the scratchpad and recalculates the CRC to verify the data. The master then copies the scratchpad contents to EEPROM.

MASTER MODE	DATA (LSB FIRST)	COMMENTS
Tx	Reset	Master issues reset pulse.
Rx	Presence	DS18B20 responds with presence pulse.
Tx	CCh	Master issues Skip ROM command.
Tx	4Eh	Master issues Write Scratchpad command.
Tx	3 data bytes	Master sends three data bytes to scratchpad (T_H , T_L , and config).
Tx	Reset	Master issues reset pulse.
Rx	Presence	DS18B20 responds with presence pulse.
Tx	CCh	Master issues Skip ROM command.
Tx	BEh	Master issues Read Scratchpad command.
Rx	9 data bytes	Master reads entire scratchpad including CRC. The master then recalculates the CRC of the first eight data bytes from the scratchpad and compares the calculated CRC with the read CRC (byte 9). If they match, the master continues; if not, the read operation is repeated.
Tx	Reset	Master issues reset pulse.
Rx	Presence	DS18B20 responds with presence pulse.
Tx	CCh	Master issues Skip ROM command.
Tx	48h	Master issues Copy Scratchpad command.
Tx	DQ line held high by strong pullup	Master applies strong pullup to DQ for at least 10ms while copy operation is in progress.

Ordering Information

PART	TEMP RANGE	PIN-PACKAGE	TOP MARK
DS18B20	-55°C to +125°C	3 TO-92	18B20
DS18B20+	-55°C to +125°C	3 TO-92	18B20
DS18B20/T&R	-55°C to +125°C	3 TO-92 (2000 Piece)	18B20
DS18B20+T&R	-55°C to +125°C	3 TO-92 (2000 Piece)	18B20
DS18B20-SL/T&R	-55°C to +125°C	3 TO-92 (2000 Piece)*	18B20
DS18B20-SL+T&R	-55°C to +125°C	3 TO-92 (2000 Piece)*	18B20
DS18B20U	-55°C to +125°C	8 FSOP	18B20
DS18B20U+	-55°C to +125°C	8 FSOP	18B20
DS18B20U/T&R	-55°C to +125°C	8 FSOP (3000 Piece)	18B20
DS18B20U+T&R	-55°C to +125°C	8 FSOP (3000 Piece)	18B20
DS18B20Z	-55°C to +125°C	8 SO	DS18B20
DS18B20Z+	-55°C to +125°C	8 SO	DS18B20
DS18B20Z/T&R	-55°C to +125°C	8 SO (2500 Piece)	DS18B20
DS18B20Z+T&R	-55°C to +125°C	8 SO (2500 Piece)	DS18B20

+Denotes a lead-free package. A "+" will appear on the top mark of lead-free packages.

T&R = Tape and reel.

*TO-92 packages in tape and reel can be ordered with straight or formed leads. Choose "SL" for straight leads. Bulk TO-92 orders are straight leads only.

Revision History

REVISION DATE	DESCRIPTION	PAGES CHANGED
3/1/07	In the Absolute Maximum Ratings section, removed the reflow oven temperature value of +220°C. Reference to JEDEC specification for reflow remains.	19
10/12/07	In the <i>Operation—Alarm Signaling</i> section, added “or equal to” in the description for a TH alarm condition	5
	In the <i>Memory</i> section, removed incorrect text describing memory.	7
	In the <i>Configuration Register</i> section, removed incorrect text describing configuration register.	8
4/22/08	In the <i>Ordering Information</i> table, added TO-92 straight-lead packages and included a note that the TO-92 package in tape and reel can be ordered with either formed or straight leads.	2
1/15	Updated <i>Benefits and Features</i> section	1
09/18	Updated <i>DC Electrical Characteristics</i> table	2
7/19	Updated Figure 12	10

For pricing, delivery, and ordering information, please visit Maxim Integrated's online storefront at <https://www.maximintegrated.com/en/storefront/storefront.html>.

Maxim Integrated cannot assume responsibility for use of any circuitry other than circuitry entirely embodied in a Maxim Integrated product. No circuit patent licenses are implied. Maxim Integrated reserves the right to change the circuitry and specifications without notice at any time. The parametric values (min and max limits) shown in the Electrical Characteristics table are guaranteed. Other parametric values quoted in this data sheet are provided for guidance.

DALLAS
SEMICONDUCTOR

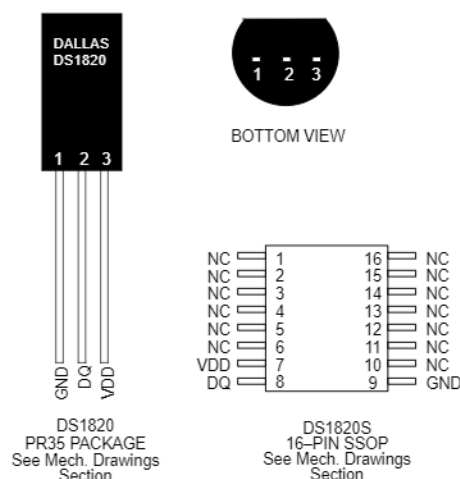
DS1820

1-Wire™ Digital Thermometer

FEATURES

- Unique 1-Wire™ interface requires only one port pin for communication
- Multidrop capability simplifies distributed temperature sensing applications
- Requires no external components
- Can be powered from data line
- Zero standby power required
- Measures temperatures from -55°C to $+125^{\circ}\text{C}$ in 0.5°C increments. Fahrenheit equivalent is -67°F to $+257^{\circ}\text{F}$ in 0.9°F increments
- Temperature is read as a 9-bit digital value.
- Converts temperature to digital word in 200 ms (typ.)
- User-definable, nonvolatile temperature alarm settings
- Alarm search command identifies and addresses devices whose temperature is outside of programmed limits (temperature alarm condition)
- Applications include thermostatic controls, industrial systems, consumer products, thermometers, or any thermally sensitive system

PIN ASSIGNMENT



PIN DESCRIPTION

GND	– Ground
DQ	– Data In/Out
V _{DD}	– Optional V _{DD}
NC	– No Connect

DESCRIPTION

The DS1820 Digital Thermometer provides 9-bit temperature readings which indicate the temperature of the device.

Information is sent to/from the DS1820 over a 1-Wire interface, so that only one wire (and ground) needs to be connected from a central microprocessor to a DS1820. Power for reading, writing, and performing temperature conversions can be derived from the data line itself with no need for an external power source.

Because each DS1820 contains a unique silicon serial number, multiple DS1820s can exist on the same 1-Wire bus. This allows for placing temperature sensors in many different places. Applications where this feature is useful include HVAC environmental controls, sensing temperatures inside buildings, equipment or machinery, and in process monitoring and control.

DETAILED PIN DESCRIPTION

PIN 16-PIN SSOP	PIN PR35	SYMBOL	DESCRIPTION
9	1	GND	Ground.
8	2	DQ	Data Input/Output pin. For 1-Wire operation: Open drain. (See "Parasite Power" section.)
7	3	V _{DD}	Optional V_{DD} pin. See "Parasite Power" section for details of connection.

DS1820S (16-pin SSOP): All pins not specified in this table are not to be connected.

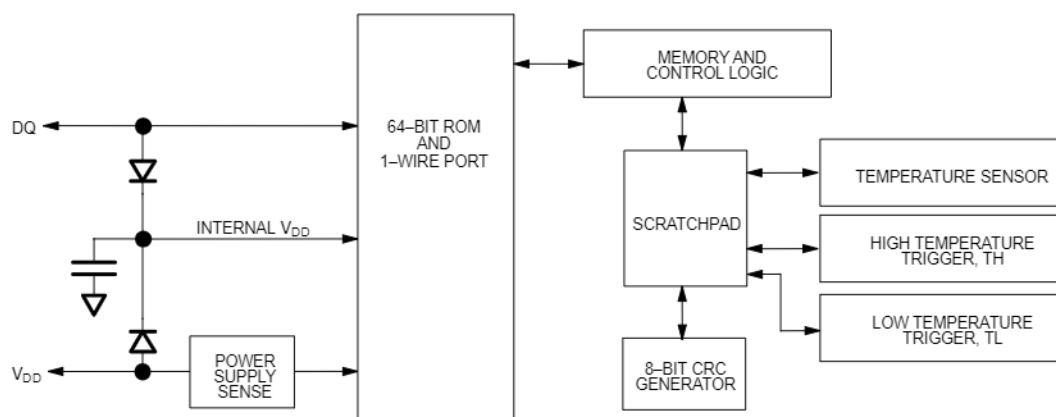
OVERVIEW

The block diagram of Figure 1 shows the major components of the DS1820. The DS1820 has three main data components: 1) 64-bit lasered ROM, 2) temperature sensor, and 3) nonvolatile temperature alarm triggers TH and TL. The device derives its power from the 1-Wire communication line by storing energy on an internal capacitor during periods of time when the signal line is high and continues to operate off this power source during the low times of the 1-Wire line until it returns high to replenish the parasite (capacitor) supply. As an alternative, the DS1820 may also be powered from an external 5 volts supply.

Communication to the DS1820 is via a 1-Wire port. With the 1-Wire port, the memory and control functions will not be available before the ROM function protocol has been established. The master must first provide one of five ROM function commands: 1) Read ROM, 2) Match ROM, 3) Search ROM, 4) Skip ROM, or 5) Alarm Search. These commands operate on the 64-bit lasered ROM portion of each device and can single out

a specific device if many are present on the 1-Wire line as well as indicate to the Bus Master how many and what types of devices are present. After a ROM function sequence has been successfully executed, the memory and control functions are accessible and the master may then provide any one of the six memory and control function commands.

One control function command instructs the DS1820 to perform a temperature measurement. The result of this measurement will be placed in the DS1820's scratchpad memory, and may be read by issuing a memory function command which reads the contents of the scratchpad memory. The temperature alarm triggers TH and TL consist of one byte EEPROM each. If the alarm search command is not applied to the DS1820, these registers may be used as general purpose user memory. Writing TH and TL is done using a memory function command. Read access to these registers is through the scratchpad. All data is read and written least significant bit first.

DS1820 BLOCK DIAGRAM Figure 1

PARASITE POWER

The block diagram (Figure 1) shows the parasite powered circuitry. This circuitry “steals” power whenever the I/O or V_{DD} pins are high. I/O will provide sufficient power as long as the specified timing and voltage requirements are met (see the section titled “1-Wire Bus System”). The advantages of parasite power are two-fold: 1) by parasiting off this pin, no local power source is needed for remote sensing of temperature, and 2) the ROM may be read in absence of normal power.

In order for the DS1820 to be able to perform accurate temperature conversions, sufficient power must be provided over the I/O line when a temperature conversion is taking place. Since the operating current of the DS1820 is up to 1 mA, the I/O line will not have sufficient drive due to the 5K pull-up resistor. This problem is particularly acute if several DS1820's are on the same I/O and attempting to convert simultaneously.

There are two ways to assure that the DS1820 has sufficient supply current during its active conversion cycle. The first is to provide a strong pull-up on the I/O line whenever temperature conversions or copies to the E^2 memory are taking place. This may be accomplished by using a MOSFET to pull the I/O line directly to the power supply as shown in Figure 2. The I/O line must be switched over to the strong pull-up within 10 μ s maximum after issuing any protocol that involves copying to the E^2 memory or initiates temperature conversions. When using the parasite power mode, the V_{DD} pin must be tied to ground.

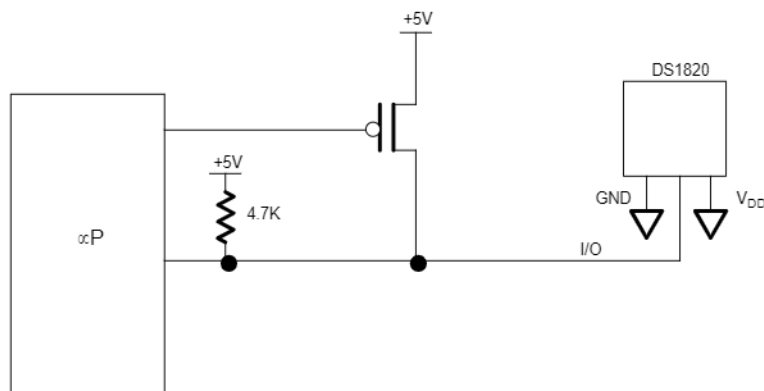
Another method of supplying current to the DS1820 is through the use of an external power supply tied to the

V_{DD} pin, as shown in Figure 3. The advantage to this is that the strong pull-up is not required on the I/O line, and the bus master need not be tied up holding that line high during temperature conversions. This allows other data traffic on the 1-Wire bus during the conversion time. In addition, any number of DS1820's may be placed on the 1-Wire bus, and if they all use external power, they may all simultaneously perform temperature conversions by issuing the Skip ROM command and then issuing the Convert T command. Note that as long as the external power supply is active, the GND pin may not be floating.

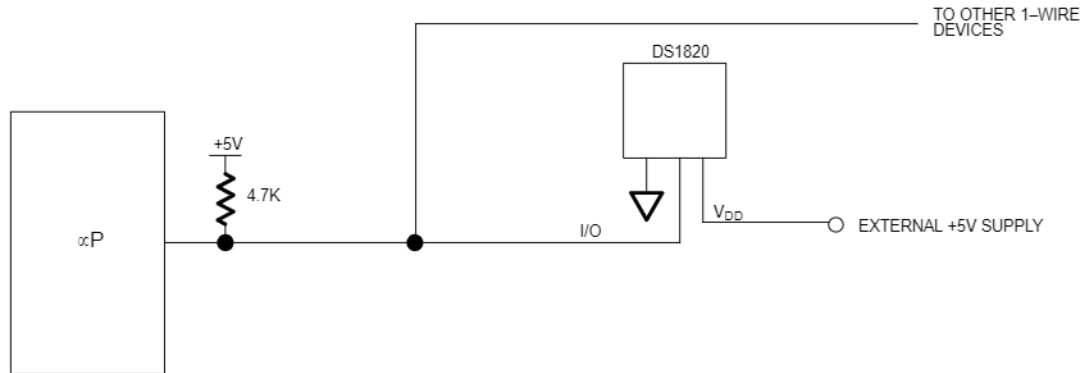
The use of parasite power is not recommended above 100°C, since it may not be able to sustain communications given the higher leakage currents the DS1820 exhibits at these temperatures. For applications in which such temperatures are likely, it is strongly recommended that V_{DD} be applied to the DS1820.

For situations where the bus master does not know whether the DS1820's on the bus are parasite powered or supplied with external V_{DD} , a provision is made in the DS1820 to signal the power supply scheme used. The bus master can determine if any DS1820's are on the bus which require the strong pull-up by sending a Skip ROM protocol, then issuing the read power supply command. After this command is issued, the master then issues read time slots. The DS1820 will send back “0” on the 1-Wire bus if it is parasite powered; it will send back a “1” if it is powered from the V_{DD} pin. If the master receives a “0”, it knows that it must supply the strong pull-up on the I/O line during temperature conversions. See “Memory Command Functions” section for more detail on this command protocol.

STRONG PULL-UP FOR SUPPLYING DS1820 DURING TEMPERATURE CONVERSION Figure 2



USING V_{DD} TO SUPPLY TEMPERATURE CONVERSION CURRENT Figure 3



OPERATION – MEASURING TEMPERATURE

The DS1820 measures temperature through the use of an on-board proprietary temperature measurement technique. A block diagram of the temperature measurement circuitry is shown in Figure 4.

The DS1820 measures temperature by counting the number of clock cycles that an oscillator with a low temperature coefficient goes through during a gate period determined by a high temperature coefficient oscillator. The counter is preset with a base count that corresponds to -55°C . If the counter reaches zero before the gate period is over, the temperature register, which is also preset to the -55°C value, is incremented, indicating that the temperature is higher than -55°C .

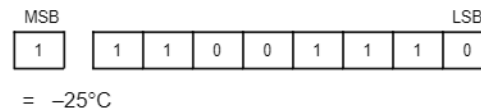
At the same time, the counter is then preset with a value determined by the slope accumulator circuitry. This circuitry is needed to compensate for the parabolic behavior of the oscillators over temperature. The counter is then clocked again until it reaches zero. If the gate period is still not finished, then this process repeats.

The slope accumulator is used to compensate for the non-linear behavior of the oscillators over temperature, yielding a high resolution temperature measurement. This is done by changing the number of counts necessary for the counter to go through for each incremental degree in temperature. To obtain the desired resolution, therefore, both the value of the counter and the number of counts per degree C (the value of the slope accumulator) at a given temperature must be known.

Internally, this calculation is done inside the DS1820 to provide 0.5°C resolution. The temperature reading is

provided in a 16-bit, sign-extended two's complement reading. Table 1 describes the exact relationship of output data to measured temperature. The data is transmitted serially over the 1-Wire interface. The DS1820 can measure temperature over the range of -55°C to $+125^{\circ}\text{C}$ in 0.5°C increments. For Fahrenheit usage, a lookup table or conversion factor must be used.

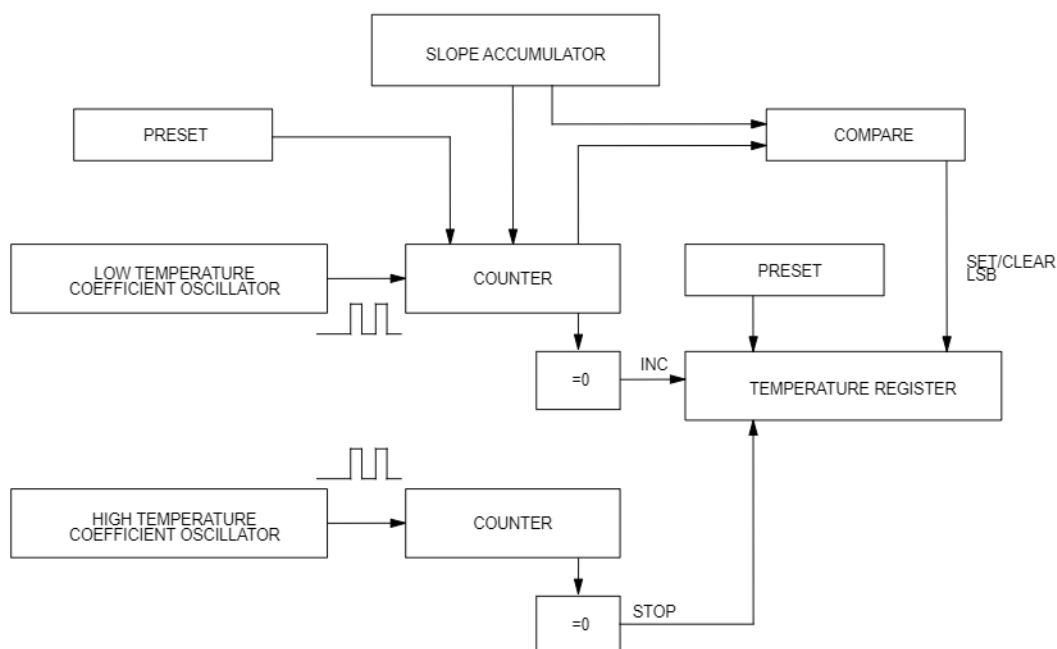
Note that temperature is represented in the DS1820 in terms of a $1/2^{\circ}\text{C}$ LSB, yielding the following 9-bit format:



The most significant (sign) bit is duplicated into all of the bits in the upper MSB of the two-byte temperature register in memory. This "sign-extension" yields the 16-bit temperature readings as shown in Table 1.

Higher resolutions may be obtained by the following procedure. First, read the temperature, and truncate the 0.5°C bit (the LSB) from the read value. This value is TEMP_READ. The value left in the counter may then be read. This value is the count remaining (COUNT_REMAIN) after the gate period has ceased. The last value needed is the number of counts per degree C (COUNT_PER_C) at that temperature. The actual temperature may be then be calculated by the user using the following:

$$\text{TEMPERATURE} = \text{TEMP_READ} - 0.25 + \frac{(\text{COUNT_PER_C} - \text{COUNT_REMAIN})}{\text{COUNT_PER_C}}$$

TEMPERATURE MEASURING CIRCUITRY Figure 4**TEMPERATURE/DATA RELATIONSHIPS** Table 1

TEMPERATURE	DIGITAL OUTPUT (Binary)	DIGITAL OUTPUT (Hex)
+125°C	00000000 11111010	00FA
+25°C	00000000 00110010	0032h
+1/2°C	00000000 00000001	0001h
+0°C	00000000 00000000	0000h
-1/2°C	11111111 11111111	FFFFh
-25°C	11111111 11001110	FFCEh
-55°C	11111111 10010010	FF92h

OPERATION – ALARM SIGNALING

After the DS1820 has performed a temperature conversion, the temperature value is compared to the trigger values stored in TH and TL. Since these registers are 8-bit only, the 0.5°C bit is ignored for comparison. The most significant bit of TH or TL directly corresponds to the sign bit of the 16-bit temperature register. If the result of a temperature measurement is higher than TH or lower than TL, an alarm flag inside the device is set.

This flag is updated with every temperature measurement. As long as the alarm flag is set, the DS1820 will respond to the alarm search command. This allows many DS1820s to be connected in parallel doing simultaneous temperature measurements. If somewhere the temperature exceeds the limits, the alarming device(s) can be identified and read immediately without having to read non-alarming devices.

64-BIT LASERED ROM

Each DS1820 contains a unique ROM code that is 64-bits long. The first eight bits are a 1-Wire family code (DS1820 code is 10h). The next 48 bits are a unique serial number. The last eight bits are a CRC of the first 56 bits. (See Figure 5.) The 64-bit ROM and ROM Function Control section allow the DS1820 to operate as a 1-Wire device and follow the 1-Wire protocol detailed in the section "1-Wire Bus System". The functions required to control sections of the DS1820 are not accessible until the ROM function protocol has been satisfied. This protocol is described in the ROM function protocol flowchart (Figure 6). The 1-Wire bus master must first provide one of five ROM function commands: 1) Read ROM, 2) Match ROM, 3) Search ROM, 4) Skip ROM, or 5) Alarm Search. After a ROM functions sequence has been successfully executed, the functions specific to the DS1820 are accessible and the bus master may then provide one of the six memory and control function commands.

CRC GENERATION

The DS1820 has an 8-bit CRC stored in the most significant byte of the 64-bit ROM. The bus master can compute a CRC value from the first 56-bits of the 64-bit ROM and compare it to the value stored within the DS1820 to determine if the ROM data has been received error-free by the bus master. The equivalent polynomial function of this CRC is:

CRC = X⁸ + X⁵ + X⁴ + 1

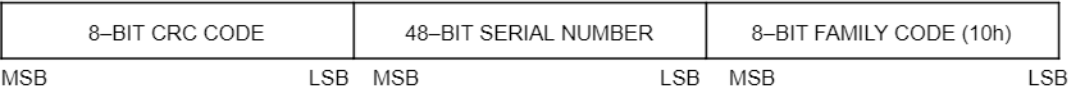
The DS1820 also generates an 8-bit CRC value using the same polynomial function shown above and provides this value to the bus master to validate the transfer of data bytes. In each case where a CRC is used for data transfer validation, the bus master must calculate a CRC value using the polynomial function given above and compare the calculated value to either the 8-bit CRC value stored in the 64-bit ROM portion of the DS1820 (for ROM reads) or the 8-bit CRC value computed within the DS1820 (which is read as a ninth byte when the scratchpad is read). The comparison of CRC values and decision to continue with an operation are determined entirely by the bus master. There is no circuitry inside the DS1820 that prevents a command sequence from proceeding if the CRC stored in or calculated by the DS1820 does not match the value generated by the bus master.

The 1-Wire CRC can be generated using a polynomial generator consisting of a shift register and XOR gates as shown in Figure 7. Additional information about the Dallas 1-Wire Cyclic Redundancy Check is available in Application Note 27 entitled "Understanding and Using Cyclic Redundancy Checks with Dallas Semiconductor Touch Memory Products".

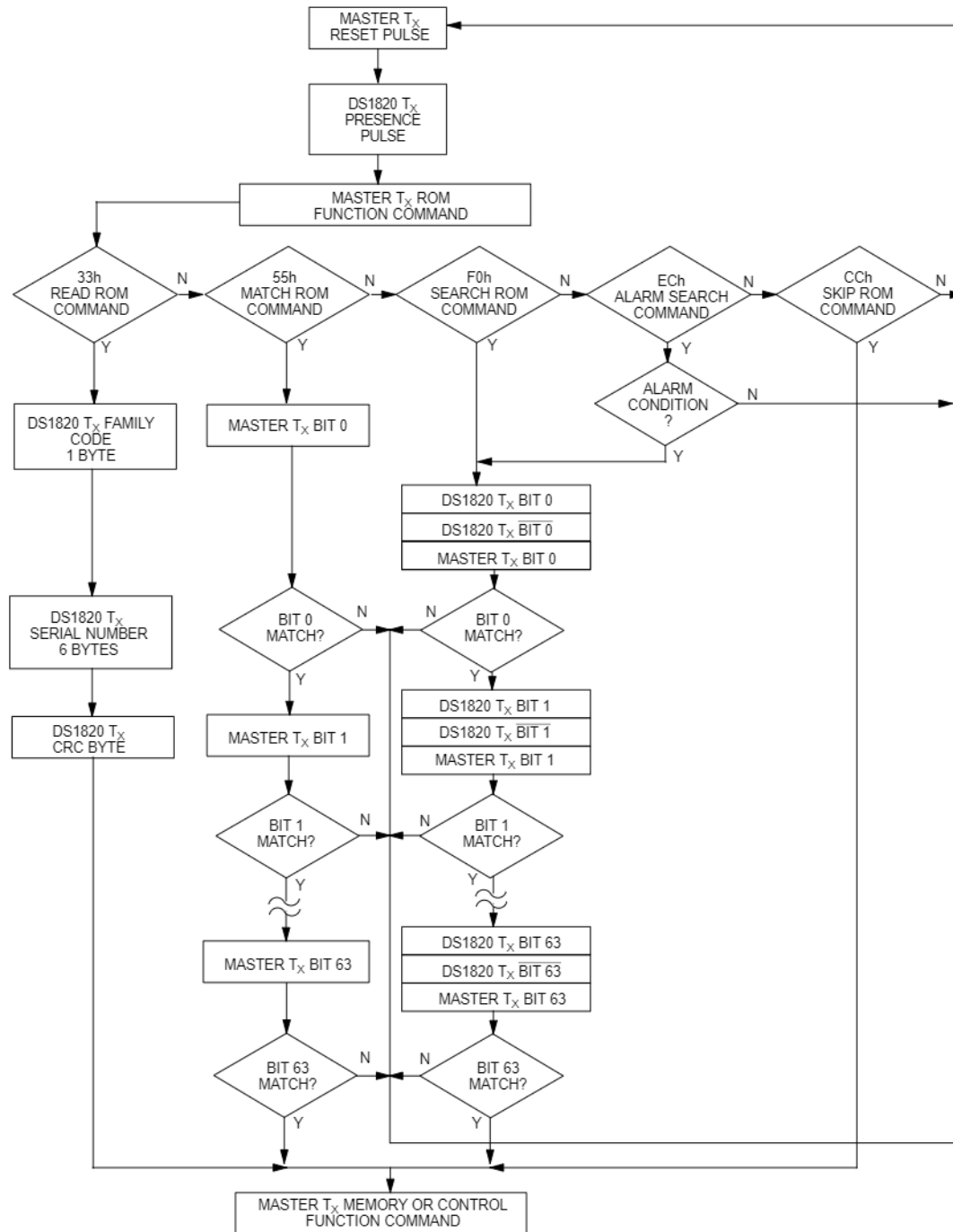
The shift register bits are initialized to zero. Then starting with the least significant bit of the family code, one bit at a time is shifted in. After the 8th bit of the family code has been entered, then the serial number is entered. After the 48th bit of the serial number has been entered, the shift register contains the CRC value. Shifting in the eight bits of CRC should return the shift register to all zeros.

The DS1820 also generates an 8-bit CRC value using the same polynomial function shown above and provides this value to the bus master to validate the transfer of data bytes. In each case where a CRC is used for data transfer validation, the bus master must calculate a CRC value using the polynomial function given above and compare the calculated value to either the 8-bit CRC value stored in the 64-bit ROM portion of the DS1820 (for ROM reads) or the 8-bit CRC value computed within the DS1820 (which is read as a ninth byte when the scratchpad is read). The comparison of CRC values and decision to continue with an operation are determined entirely by the bus master. There is no circuitry inside the DS1820 that prevents a command sequence from proceeding if the CRC stored in or calculated by the DS1820 does not match the value generated by the bus master.

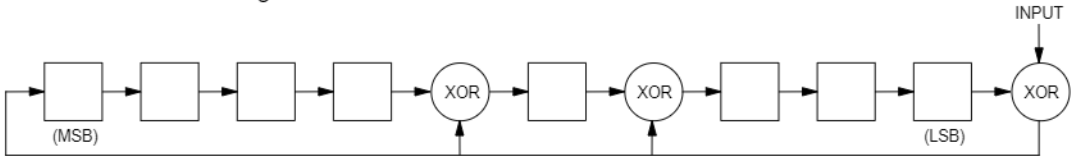
64-BIT LASERED ROM Figure 5



ROM FUNCTIONS FLOW CHART Figure 6



1-WIRE CRC CODE Figure 7



MEMORY

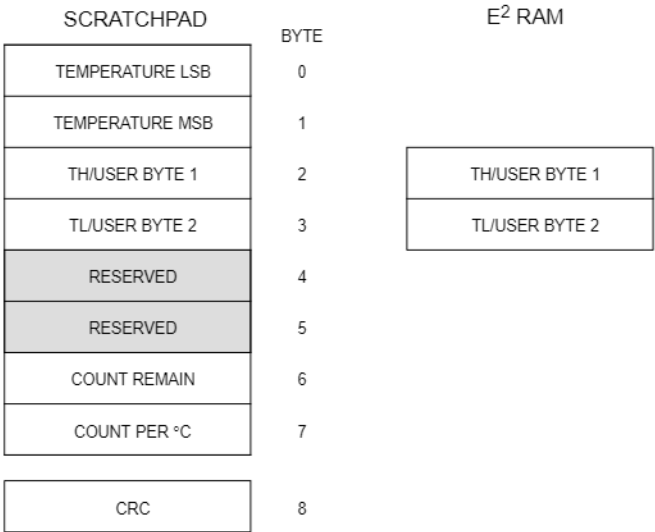
The DS1820's memory is organized as shown in Figure 8. The memory consists of a scratchpad RAM and a nonvolatile, electrically erasable (E²) RAM, which stores the high and low temperature triggers TH and TL. The scratchpad helps insure data integrity when communicating over the 1-Wire bus. Data is first written to the scratchpad where it can be read back. After the data has been verified, a copy scratchpad command will transfer the data to the nonvolatile (E²) RAM. This process insures data integrity when modifying the memory.

The scratchpad is organized as eight bytes of memory. The first two bytes contain the measured temperature

information. The third and fourth bytes are volatile copies of TH and TL and are refreshed with every power-on reset. The next two bytes are not used; upon reading back, however, they will appear as all logic 1's. The seventh and eighth bytes are count registers, which may be used in obtaining higher temperature resolution (see "Operation-measuring Temperature" section).

There is a ninth byte which may be read with a Read Scratchpad command. This byte contains a cyclic redundancy check (CRC) byte which is the CRC over all of the eight previous bytes. This CRC is implemented in the fashion described in the section titled "CRC Generation".

DS1820 MEMORY MAP Figure 8



1-WIRE BUS SYSTEM

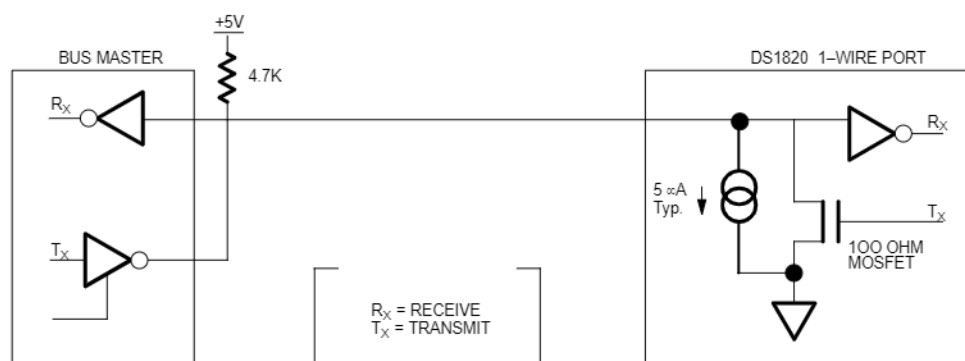
The 1-Wire bus is a system which has a single bus master and one or more slaves. The DS1820 behaves as a slave. The discussion of this bus system is broken down into three topics: hardware configuration, transaction sequence, and 1-Wire signaling (signal types and timing).

HARDWARE CONFIGURATION

The 1-Wire bus has only a single line by definition; it is important that each device on the bus be able to drive it

at the appropriate time. To facilitate this, each device attached to the 1-Wire bus must have open drain or 3-state outputs. The 1-Wire port of the DS1820 (I/O pin) is open drain with an internal circuit equivalent to that shown in Figure 9. A multidrop bus consists of a 1-Wire bus with multiple slaves attached. The 1-Wire bus requires a pullup resistor of approximately 5K Ω .

HARDWARE CONFIGURATION Figure 9



The idle state for the 1-Wire bus is high. If for any reason a transaction needs to be suspended, the bus MUST be left in the idle state if the transaction is to resume. Infinite recovery time can occur between bits so long as the 1-Wire bus is in the inactive (high) state during the recovery period. If this does not occur and the bus is left low for more than 480 μ s, all components on the bus will be reset.

TRANSACTION SEQUENCE

The protocol for accessing the DS1820 via the 1-Wire port is as follows:

- Initialization
- ROM Function Command
- Memory Function Command
- Transaction/Data

INITIALIZATION

All transactions on the 1-Wire bus begin with an initialization sequence. The initialization sequence consists of a reset pulse transmitted by the bus master followed by presence pulse(s) transmitted by the slave(s).

The presence pulse lets the bus master know that the DS1820 is on the bus and is ready to operate. For more details, see the "1-Wire Signaling" section.

ROM FUNCTION COMMANDS

Once the bus master has detected a presence, it can issue one of the five ROM function commands. All ROM function commands are 8-bits long. A list of these commands follows (refer to flowchart in Figure 6):

Read ROM [33h]

This command allows the bus master to read the DS1820's 8-bit family code, unique 48-bit serial number, and 8-bit CRC. This command can only be used if there is a single DS1820 on the bus. If more than one slave is present on the bus, a data collision will occur when all slaves try to transmit at the same time (open drain will produce a wired AND result).

Match ROM [55h]

The match ROM command, followed by a 64-bit ROM sequence, allows the bus master to address a specific DS1820 on a multidrop bus. Only the DS1820 that exactly matches the 64-bit ROM sequence will respond to the following memory function command. All slaves that do not match the 64-bit ROM sequence will wait for a reset pulse. This command can be used with a single or multiple devices on the bus.

Skip ROM [CCh]

This command can save time in a single drop bus system by allowing the bus master to access the memory functions without providing the 64-bit ROM code. If more than one slave is present on the bus and a read command is issued following the Skip ROM command, data collision will occur on the bus as multiple slaves transmit simultaneously (open drain pullups will produce a wired AND result).

Search ROM [F0h]

When a system is initially brought up, the bus master might not know the number of devices on the 1-Wire bus or their 64-bit ROM codes. The search ROM command allows the bus master to use a process of elimination to identify the 64-bit ROM codes of all slave devices on the bus.

Alarm Search [ECh]

The flowchart of this command is identical to the Search ROM command. However, the DS1820 will respond to this command only if an alarm condition has been encountered at the last temperature measurement. An alarm condition is defined as a temperature higher than TH or lower than TL. The alarm condition remains set as long as the DS1820 is powered up, or until another temperature measurement reveals a non-alarming value. For alarming, the trigger values stored in EEPROM are taken into account. If an alarm condition exists and the TH or TL settings are changed, another temperature

conversion should be done to validate any alarm conditions.

Example of a ROM Search

The ROM search process is the repetition of a simple 3-step routine: read a bit, read the complement of the bit, then write the desired value of that bit. The bus master performs this simple, 3-step routine on each bit of the ROM. After one complete pass, the bus master knows the contents of the ROM in one device. The remaining number of devices and their ROM codes may be identified by additional passes.

The following example of the ROM search process assumes four different devices are connected to the same 1-Wire bus. The ROM data of the four devices is as shown:

ROM1	00110101...
ROM2	10101010...
ROM3	11110101...
ROM4	00010001...

The search process is as follows:

1. The bus master begins the initialization sequence by issuing a reset pulse. The slave devices respond by issuing simultaneous presence pulses.
2. The bus master will then issue the Search ROM command on the 1-Wire bus.
3. The bus master reads a bit from the 1-Wire bus. Each device will respond by placing the value of the first bit of their respective ROM data onto the 1-Wire bus. ROM1 and ROM4 will place a 0 onto the 1-Wire bus, i.e., pull it low. ROM2 and ROM3 will place a 1 onto the 1-Wire bus by allowing the line to stay high. The result is the logical AND of all devices on the line, therefore the bus master sees a 0. The bus master reads another bit. Since the Search ROM data command is being executed, all of the devices on the 1-Wire bus respond to this second read by placing the complement of the first bit of their respective ROM data onto the 1-Wire bus. ROM1 and ROM4 will place a 1 onto the 1-Wire, allowing the line to stay high. ROM2 and ROM3 will place a 0 onto the 1-Wire, thus it will be pulled low. The bus master again observes a 0 for the complement of the first ROM data bit. The bus master has determined that there are some devices on the 1-Wire bus that have a 0 in the first position and others that have a 1.

The data obtained from the two reads of the 3-step routine have the following interpretations:

- 00 There are still devices attached which have conflicting bits in this position.
 - 01 All devices still coupled have a 0-bit in this bit position.
 - 10 All devices still coupled have a 1-bit in this bit position.
 - 11 There are no devices attached to the 1-Wire bus.
4. The bus master writes a 0. This deselects ROM2 and ROM3 for the remainder of this search pass, leaving only ROM1 and ROM4 connected to the 1-Wire bus.
 5. The bus master performs two more reads and receives a 0-bit followed by a 1-bit. This indicates that all devices still coupled to the bus have 0's as their second ROM data bit.
 6. The bus master then writes a 0 to keep both ROM1 and ROM4 coupled.
 7. The bus master executes two reads and receives two 0-bits. This indicates that both 1-bits and 0-bits exist as the third bit of the ROM data of the attached devices.
 8. The bus master writes a 0-bit. This deselects ROM1 leaving ROM4 as the only device still connected.
 9. The bus master reads the remainder of the ROM bits for ROM4 and continues to access the part if desired. This completes the first pass and uniquely identifies one part on the 1-Wire bus.
 10. The bus master starts a new ROM search sequence by repeating steps 1 through 7.
 11. The bus master writes a 1-bit. This decouples ROM4, leaving only ROM1 still coupled.
 12. The bus master reads the remainder of the ROM bits for ROM1 and communicates to the underlying logic if desired. This completes the second ROM search pass, in which another of the ROMs was found.
 13. The bus master starts a new ROM search by repeating steps 1 through 3.
 14. The bus master writes a 1-bit. This deselects ROM1 and ROM4 for the remainder of this search pass, leaving only ROM2 and ROM3 coupled to the system.

15. The bus master executes two read time slots and receives two zeros.

16. The bus master writes a 0-bit. This decouples ROM3, and leaving only ROM2.
17. The bus master reads the remainder of the ROM bits for ROM2 and communicates to the underlying logic if desired. This completes the third ROM search pass, in which another of the ROMs was found.
18. The bus master starts a new ROM search by repeating steps 13 through 15.
19. The bus master writes a 1-bit. This decouples ROM2, leaving only ROM3.
20. The bus master reads the remainder of the ROM bits for ROM3 and communicates to the underlying logic if desired. This completes the fourth ROM search pass, in which another of the ROMs was found.

Note the following:

The bus master learns the unique ID number (ROM data pattern) of one 1-Wire device on each ROM Search operation. The time required to derive the part's unique ROM code is:

$$960 \mu\text{s} + (8 + 3 \times 64) 61 \mu\text{s} = 13.16 \text{ ms}$$

The bus master is therefore capable of identifying 75 different 1-Wire devices per second.

I/O SIGNALING

The DS1820 requires strict protocols to insure data integrity. The protocol consists of several types of signaling on one line: reset pulse, presence pulse, write 0, write 1, read 0, and read 1. All of these signals, with the exception of the presence pulse, are initiated by the bus master.

The initialization sequence required to begin any communication with the DS1820 is shown in Figure 11. A reset pulse followed by a presence pulse indicates the DS1820 is ready to send or receive data given the correct ROM command and memory function command.

The bus master transmits (TX) a reset pulse (a low signal for a minimum of 480 μs). The bus master then releases the line and goes into a receive mode (RX). The 1-Wire bus is pulled to a high state via the 5K pull-up resistor. After detecting the rising edge on the

I/O pin, the DS1820 waits 15–60 μ s and then transmits the presence pulse (a low signal for 60–240 μ s).

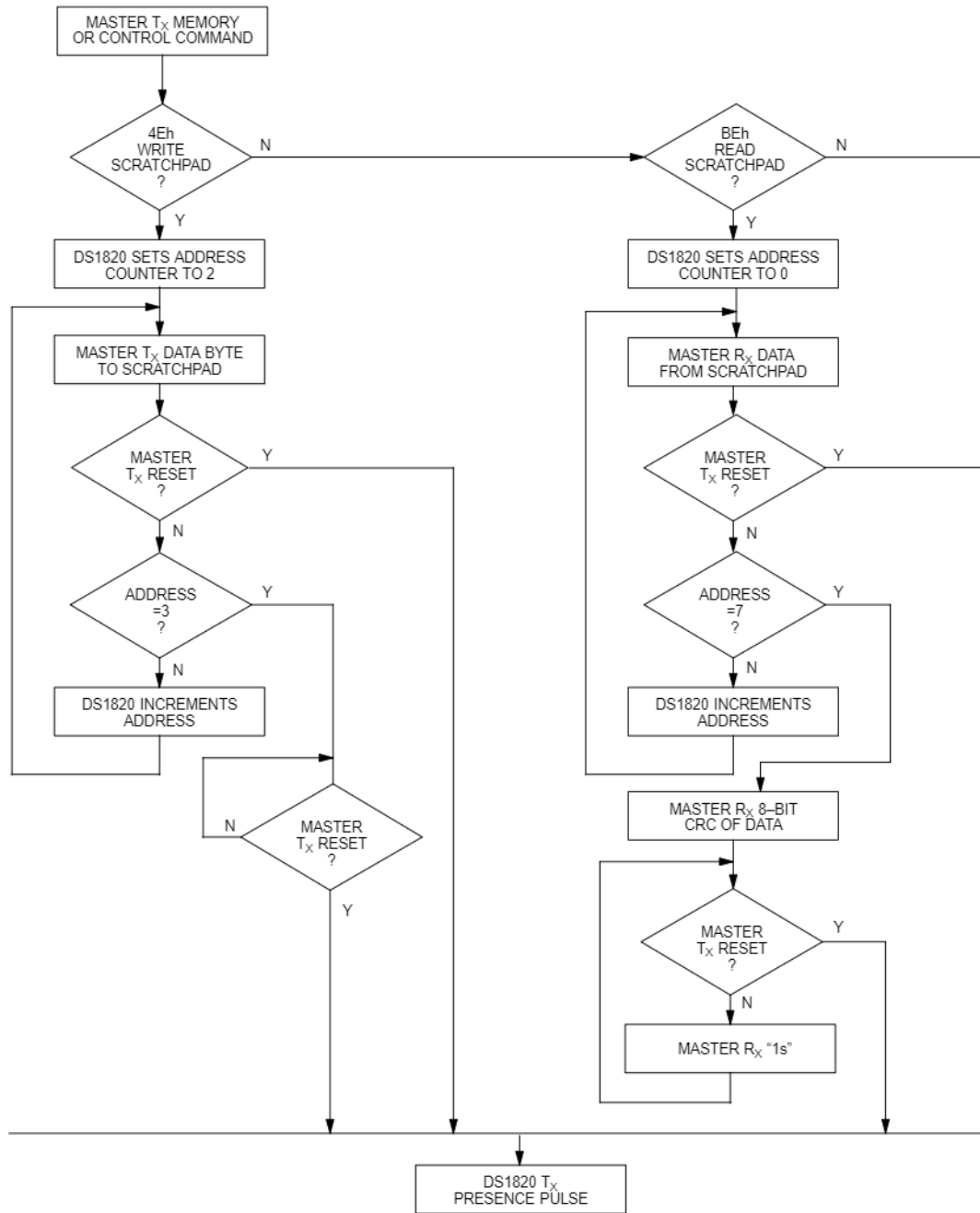
MEMORY COMMAND FUNCTIONS

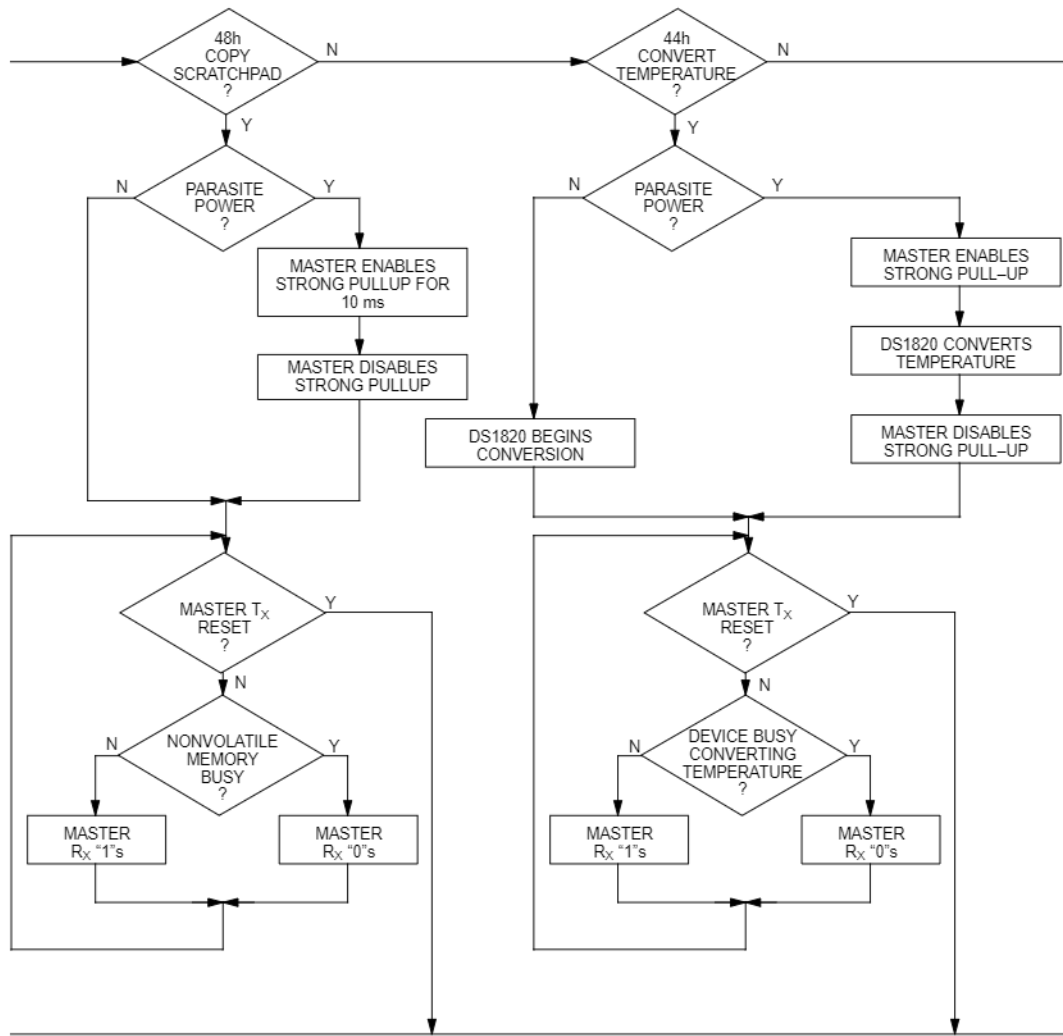
The following command protocols are summarized in Table 2, and by the flowchart of Figure 10.

Write Scratchpad [4Eh]

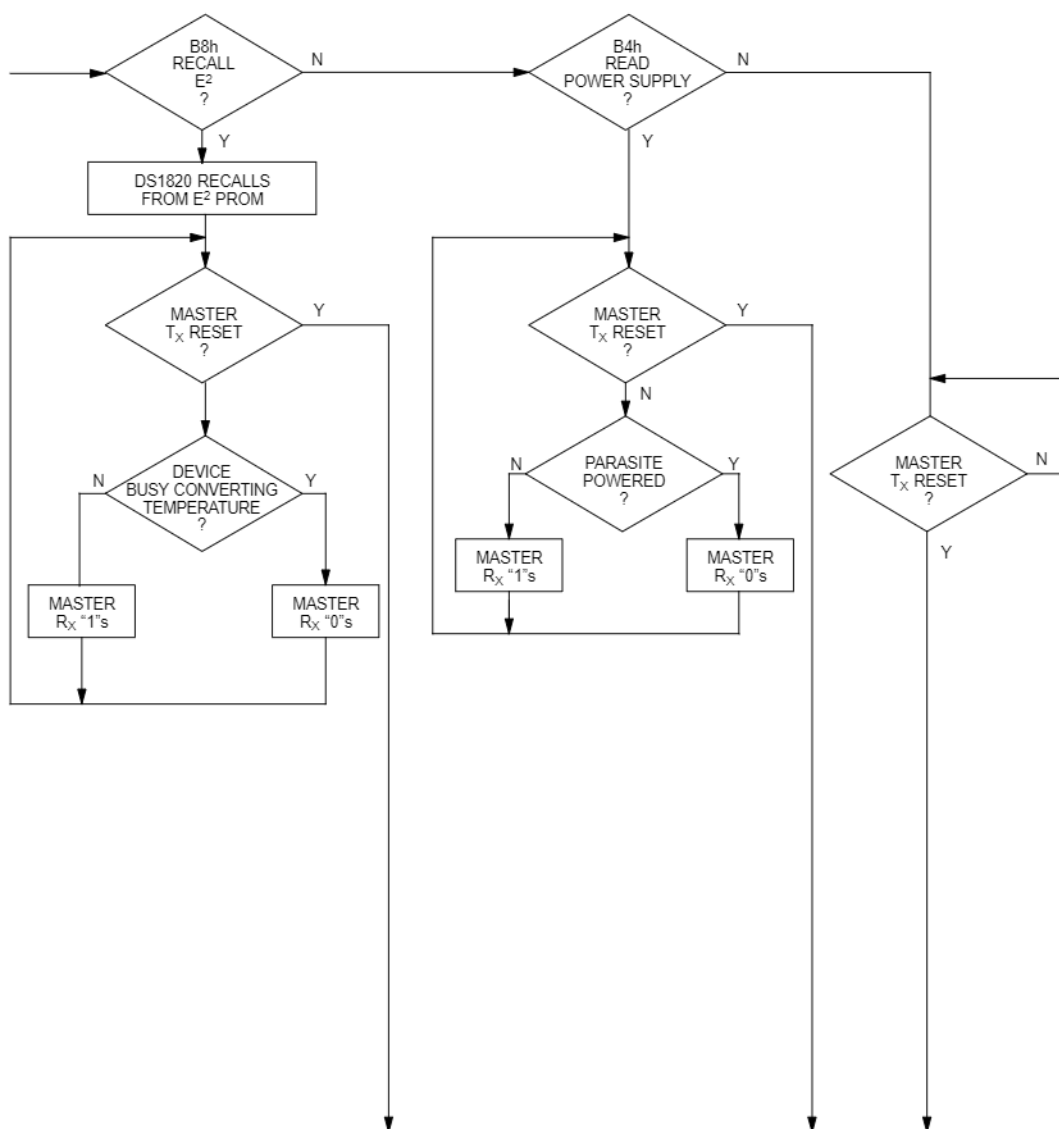
This command writes to the scratchpad of the DS1820, starting at address 2. The next two bytes written will be saved in scratchpad memory, at address locations 2 and 3. Writing may be terminated at any point by issuing a reset.

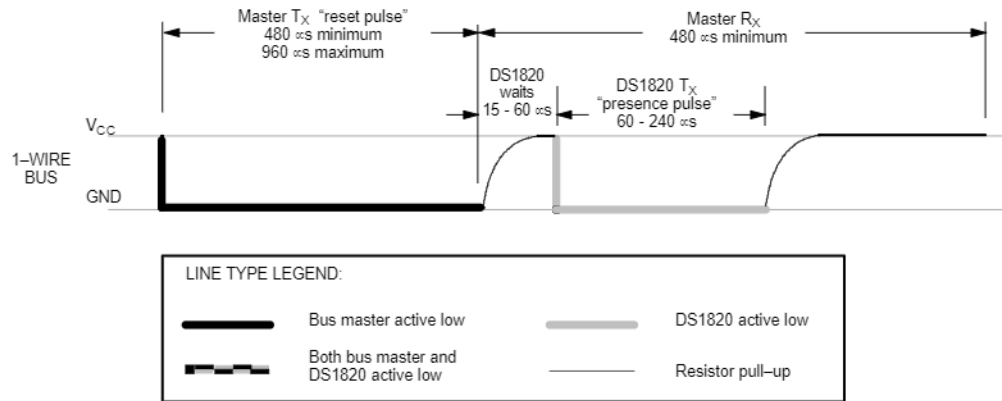


MEMORY FUNCTIONS FLOW CHART Figure 10

MEMORY FUNCTIONS FLOW CHART Figure 10 (cont'd)

MEMORY FUNCTIONS FLOW CHART Figure 10 (cont'd)



INITIALIZATION PROCEDURE “RESET AND PRESENCE PULSES” Figure 11**DS1820 COMMAND SET** Table 2

INSTRUCTION	DESCRIPTION	PROTOCOL	1-WIRE BUS AFTER ISSUING PROTOCOL	NOTES
TEMPERATURE CONVERSION COMMANDS				
Convert T	Initiates temperature conversion.	44h	<read temperature busy status>	1
MEMORY COMMANDS				
Read Scratchpad	Reads bytes from scratchpad and reads CRC byte.	BEh	<read data up to 9 bytes>	
Write Scratchpad	Writes bytes into scratchpad at addresses 2 and 3 (TH and TL temperature triggers).	4Eh	<write data into 2 bytes at addr. 2 and addr. 3>	
Copy Scratchpad	Copies scratchpad into nonvolatile memory (addresses 2 and 3 only).	48h	<read copy status>	2
Recall E ²	Recalls values stored in nonvolatile memory into scratchpad (temperature triggers).	B8h	<read temperature busy status>	
Read Power Supply	Signals the mode of DS1820 power supply to the master.	B4h	<read supply status>	

NOTES:

- Temperature conversion takes up to 500 ms. After receiving the Convert T protocol, if the part does not receive power from the V_{DD} pin, the I/O line for the DS1820 must be held high for at least 500 ms to provide power during the conversion process. As such, no other activity may take place on the 1-Wire bus for at least this period after a Convert T command has been issued.
- After receiving the Copy Scratchpad protocol, if the part does not receive power from the V_{DD} pin, the I/O line for the DS1820 must be held high for at least 10 ms to provide power during the copy process. As such, no other activity may take place on the 1-Wire bus for at least this period after a Copy Scratchpad command has been issued.

Read Scratchpad [BEh]

This command reads the contents of the scratchpad. Reading will commence at byte 0, and will continue through the scratchpad until the 9th (byte=8, CRC) byte is read. If not all locations are to be read, the master may issue a reset to terminate reading at any time.

Copy Scratchpad [48h]

This command copies the scratchpad into the E² memory of the DS1820, storing the temperature trigger bytes in nonvolatile memory. If the bus master issues read time slots following this command, the DS1820 will output "0" on the bus as long as it is busy copying the scratchpad to E²; it will return a "1" when the copy process is complete. If parasite powered, the bus master has to enable a strong pull-up for at least 10 ms immediately after issuing this command.

Convert T [44h]

This command begins a temperature conversion. No further data is required. The temperature conversion will be performed and then the DS1820 will remain idle. If the bus master issues read time slots following this command, the DS1820 will output "0" on the bus as long as it is busy making a temperature conversion; it will return a "1" when the temperature conversion is complete. If parasite powered, the bus master has to enable a strong pullup for 500 ms immediately after issuing this command.

Recall E2 [B8h]

This command recalls the temperature trigger values stored in E² to the scratchpad. This recall operation happens automatically upon power-up to the DS1820 as well, so valid data is available in the scratchpad as soon as the device has power applied. With every read data time slot issued after this command has been sent, the device will output its temperature converter busy flag "0"=busy, "1"=ready.

Read Power Supply [B4h]

With every read data time slot issued after this command has been sent to the DS1820, the device will signal its power mode: "0"=parasite power, "1"=external power supply provided.

READ/WRITE TIME SLOTS

DS1820 data is read and written through the use of time slots to manipulate bits and a command word to specify the transaction.

Write Time Slots

A write time slot is initiated when the host pulls the data line from a high logic level to a low logic level. There are two types of write time slots: Write One time slots and Write Zero time slots. All write time slots must be a minimum of 60 μ s in duration with a minimum of a one μ s recovery time between individual write cycles.

The DS1820 samples the I/O line in a window of 15 μ s to 60 μ s after the I/O line falls. If the line is high, a Write One occurs. If the line is low, a Write Zero occurs (see Figure 12).

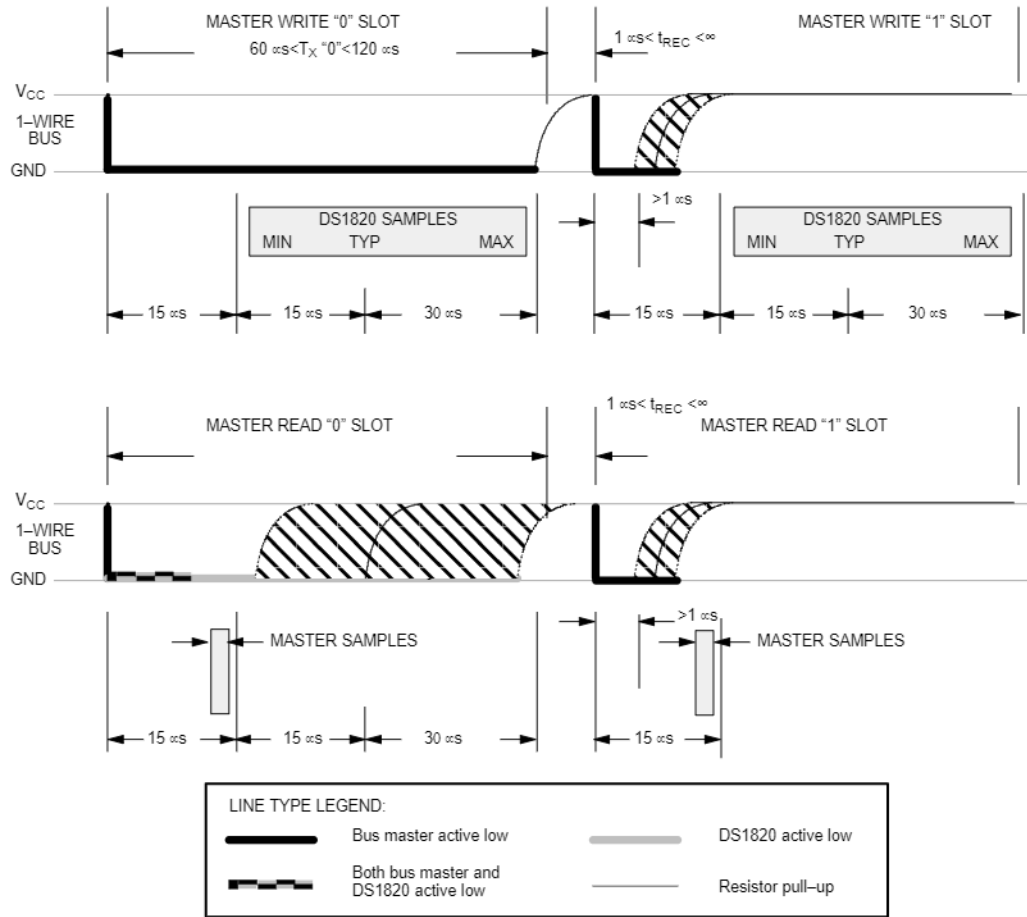
For the host to generate a Write One time slot, the data line must be pulled to a logic low level and then released, allowing the data line to pull up to a high level within 15 μ s after the start of the write time slot.

For the host to generate a Write Zero time slot, the data line must be pulled to a logic low level and remain low for 60 μ s.

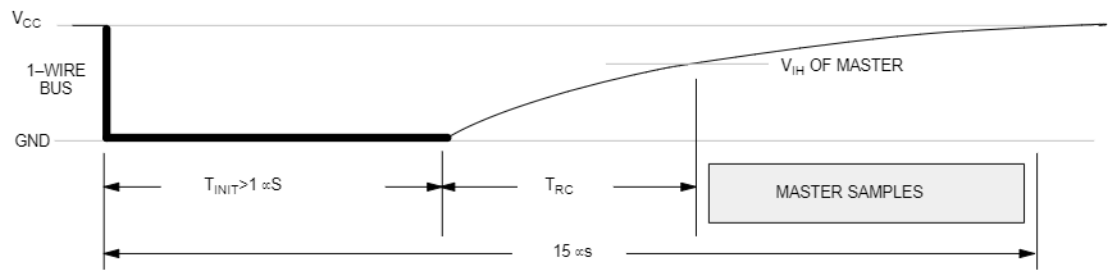
Read Time Slots

The host generates read time slots when data is to be read from the DS1820. A read time slot is initiated when the host pulls the data line from a logic high level to logic low level. The data line must remain at a low logic level for a minimum of one μ s; output data from the DS1820 is valid for 15 μ s after the falling edge of the read time slot. The host therefore must stop driving the I/O pin low in order to read its state 15 μ s from the start of the read slot (see Figure 12). By the end of the read time slot, the I/O pin will pull back high via the external pull-up resistor. All read time slots must be a minimum of 60 μ s in duration with a minimum of a one μ s recovery time between individual read slots.

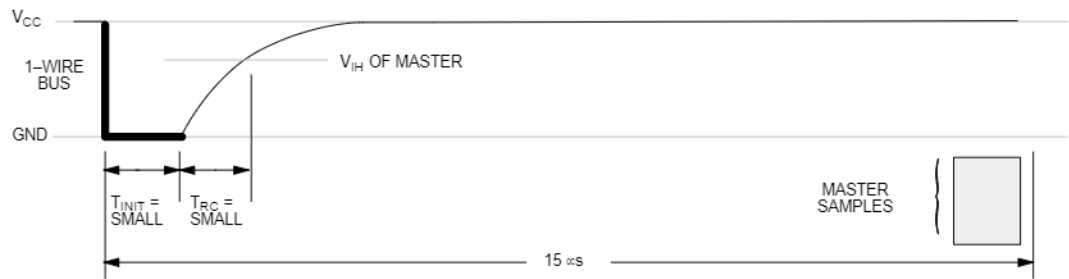
Figure 13 shows that the sum of T_{INIT}, T_{RC}, and T_{SAMPLE} must be less than 15 μ s. Figure 14 shows that system timing margin is maximized by keeping T_{INIT} and T_{RC} as small as possible and by locating the master sample time towards the end of the 15 μ s period.

READ/WRITE TIMING DIAGRAM Figure 12

DETAILED MASTER READ “1” TIMING Figure 13



RECOMMENDED MASTER READ “1” TIMING Figure 14



LINE TYPE LEGEND:			
	Bus master active low		DS1820 active low
	Both bus master and DS1820 active low		Resistor pull-up

Related Application Notes

The following Application Notes can be applied to the DS1820. These notes can be obtained from the Dallas

Semiconductor "Application Note Book", via our website at <http://www.dalsemi.com/>, or through our faxback service at (214) 450-0441.

Application Note 27: "Understanding and Using Cyclic Redundancy Checks with Dallas Semiconductor Touch Memory Product"

Application Note 55: "Extending the Contact Range of Touch Memories"

Application Note 74: "Reading and Writing Touch Memories via Serial Interfaces"

Application Note 104: "Minimalist Temperature Control Demo"

Application Note 105: "High Resolution Temperature Measurement with Dallas Direct-to-Direct Temperature Sensors"

Application Note 106: "Complex MicroLANs"

Application Note 108: "MicroLAN – In the Long Run"

Sample 1-Wire subroutines that can be used in conjunction with AN74 can be downloaded from the website or our Anonymous FTP Site.

MEMORY FUNCTION EXAMPLE Table 3

Example: Bus Master initiates temperature conversion, then reads temperature (parasite power assumed).

MASTER MODE	DATA (LSB FIRST)	COMMENTS
TX	Reset	Reset pulse (480–960 μ s).
RX	Presence	Presence pulse.
TX	55h	Issue "Match ROM" command.
TX	<64-bit ROM code>	Issue address for DS1820.
TX	44h	Issue "Convert T" command.
TX	<I/O LINE HIGH>	I/O line is held high for at least 500 ms by bus master to allow conversion to complete.
TX	Reset	Reset pulse.
RX	Presence	Presence pulse.
TX	55h	Issue "Match ROM" command.
TX	<64-bit ROM code>	Issue address for DS1820.
TX	BEh	Issue "Read Scratchpad" command.
RX	<9 data bytes>	Read entire scratchpad plus CRC; the master now recalculates the CRC of the eight data bytes received from the scratchpad, compares the CRC calculated and the CRC read. If they match, the master continues; if not, this read operation is repeated.
TX	Reset	Reset Pulse.
RX	Presence	Presence pulse, done.

MEMORY FUNCTION EXAMPLE Table 4

Example: Bus Master writes memory (parasite power and only one DS1820 assumed).

MASTER MODE	DATA (LSB FIRST)	COMMENTS
TX	Reset	Reset pulse.
RX	Presence	Presence pulse.
TX	CCh	Skip ROM command.
TX	4Eh	Write Scratchpad command.
TX	<2 data bytes>	Writes two bytes to scratchpad (TH and TL).
TX	Reset	Reset pulse.
RX	Presence	Presence pulse.
TX	CCh	Skip ROM command.
TX	BEh	Read Scratchpad command.
RX	<9 data bytes>	Read entire scratchpad plus CRC. The master now recalculates the CRC of the eight data bytes received from the scratchpad, compares the CRC and the two other bytes read back from the scratchpad. If data match, the master continues; if not, repeat the sequence.
TX	Reset	Reset pulse.
RX	Presence	Presence pulse.
TX	CCh	Skip ROM command.
TX	48h	Copy Scratchpad command; after issuing this command, the master must wait 6 ms for copy operation to complete.
TX	Reset	Reset pulse.
RX	Presence	Presence pulse, done.

MEMORY FUNCTION EXAMPLE Table 5

Example: Temperature conversion and interpolation (external power supply and only one DS1820 assumed).

MASTER MODE	DATA (LSB FIRST)	COMMENTS
TX	Reset	Reset pulse.
TR	Presence	Presence pulse.
TX	CCh	Skip ROM command.
TX	44h	Convert T command.
RX	<1 data byte>	Read busy flag eight times. The master continues reading one byte (or bit) after another until the data is FFh (all bits 1).
TX	Reset	Reset pulse.
RX	Presence	Presence pulse.
TX	CCh	Skip ROM command.
TX	BEh	Read Scratchpad command.
RX	<9 data bytes>	Read entire scratchpad plus CRC. The master now recalculates the CRC of the eight data bytes received from the scratchpad and compares both CRCs. If the CRCs match, the data is valid. The master saves the temperature value and stores the contents of the count register and count per °C register as COUNT_REMAIN and COUNT_PER_C, respectively.
TX	Reset	Reset pulse.
RX	Presence	Presence pulse, done.
–	–	CPU calculates temperature as described in the data sheet for higher resolution.

ABSOLUTE MAXIMUM RATINGS*

Voltage on Any Pin Relative to Ground	−0.5V to +7.0V
Operating Temperature	−55°C to +125°C
Storage Temperature	−55°C to +125°C
Soldering Temperature	260°C for 10 seconds

* This is a stress rating only and functional operation of the device at these or any other conditions above those indicated in the operation sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods of time may affect reliability.

RECOMMENDED DC OPERATING CONDITIONS

PARAMETER	SYMBOL	CONDITION	MIN	TYP	MAX	UNITS	NOTES
Supply Voltage	V_{DD}	I/O Functions	2.8	5.0	5.5	V	1, 2
		$\pm 1/2^\circ\text{C}$ Accurate Temperature Conversions	4.3		5.5		
Data Pin	I/O		−0.5		+5.5	V	2
Logic 1	V_{IH}		2.0		$V_{CC}+0.3$	V	2, 3
Logic 0	V_{IL}		−0.3		+0.8	V	2, 4

DC ELECTRICAL CHARACTERISTICS

(−55°C to +125°C; $V_{DD}=3.6\text{V}$ to 5.5V)

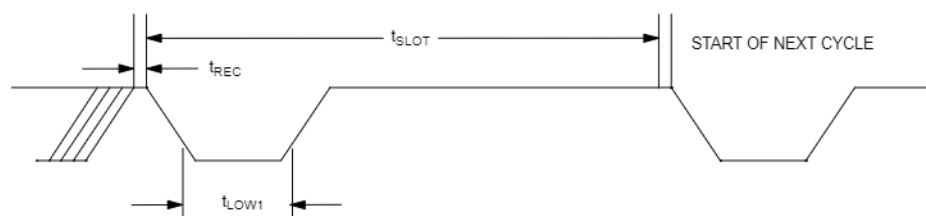
PARAMETER	SYMBOL	CONDITION	MIN	TYP	MAX	UNITS	NOTES
Thermometer Error	t_{ERR}	−0°C to +70°C			$\pm 1/2$	$^\circ\text{C}$	1, 9, 10
		−55°C to 0°C and +70°C to +125°C			See Typical Curve		
Input Logic High	V_{IH}		2.2		5.5	V	2, 3
Input Logic Low	V_{IL}		−0.3		+0.8	V	2, 4
Sink Current	I_L	$V_{I/O}=0.4\text{V}$	−4.0			mA	2
Standby Current	I_Q			200	350	nA	8
Active Current	I_{DD}			1	1.5	mA	5, 6
Input Load Current	I_L			5		μA	7

AC ELECTRICAL CHARACTERISTICS:(-55°C to +125°C; V_{DD} =3.6V to 5.5V)

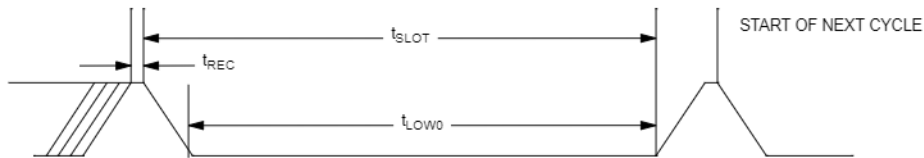
PARAMETER	SYMBOL	MIN	TYP	MAX	UNITS	NOTES
Temperature Conversion Time	t_{CONV}		200	500	ms	
Time Slot	t_{SLOT}	60		120	μ s	
Recovery Time	t_{REC}	1			μ s	
Write 0 Low Time	t_{LOW0}	60		120	μ s	
Write 1 Low Time	t_{LOW1}	1		15	μ s	
Read Data Valid	t_{RDV}			15	μ s	
Reset Time High	t_{RSTH}	480			μ s	
Reset Time Low	t_{RSTL}	480		4800	μ s	
Presence Detect High	t_{PDHIGH}	15		60	μ s	
Presence Detect Low	t_{PDLOW}	60		240	μ s	
Capacitance	$C_{IN/OUT}$			25	pF	

NOTES:

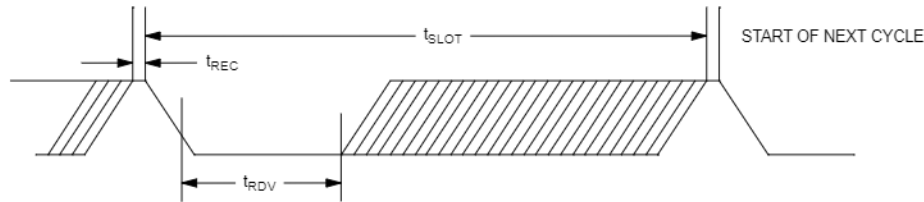
1. Temperature conversion will work with $\pm 2^\circ\text{C}$ accuracy down to $V_{DD} = 3.4$ volts.
2. All voltages are referenced to ground.
3. Logic one voltages are specified at a source current of 1 mA.
4. Logic zero voltages are specified at a sink current of 4 mA.
5. I_{DD} specified with V_{CC} at 5.0 volts.
6. Active current refers to either temperature conversion or writing to the E^2 memory. Writing to E^2 memory consumes approximately 200 μ A for up to 10 ms.
7. Input load is to ground.
8. Standby current specified up to 70°C. Standby current typically is 5 μ A at 125°C.
9. See Typical Curve for specification limits outside the 0°C to 70°C range. Thermometer error reflects sensor accuracy as tested during calibration.
10. Typical accuracy curve valid for $4.3\text{V} \leq V_{DD} \leq 5.5\text{V}$.

1-WIRE WRITE ONE TIME SLOT

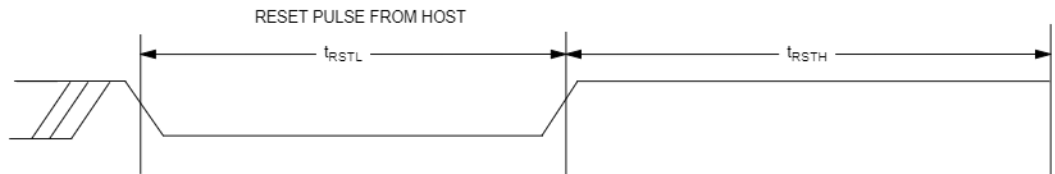
1-WIRE WRITE ZERO TIME SLOT



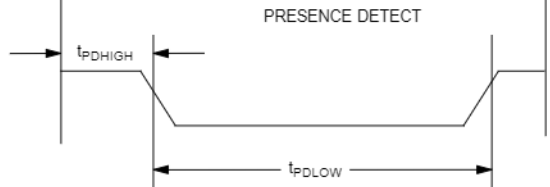
1-WIRE READ ZERO TIME SLOT



1-WIRE RESET PULSE

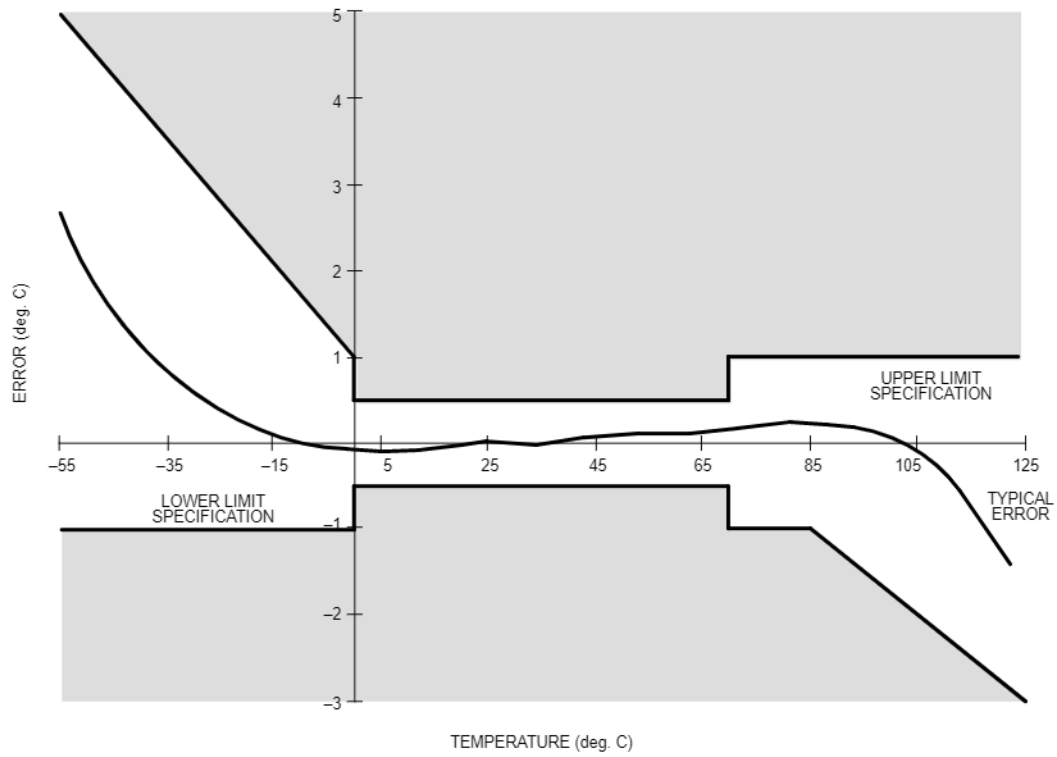


1-WIRE PRESENCE DETECT



TYPICAL PERFORMANCE CURVE

DS1820 DIGITAL THERMOMETER AND THERMOSTAT
TEMPERATURE READING ERROR



Atmel ATmega328PB

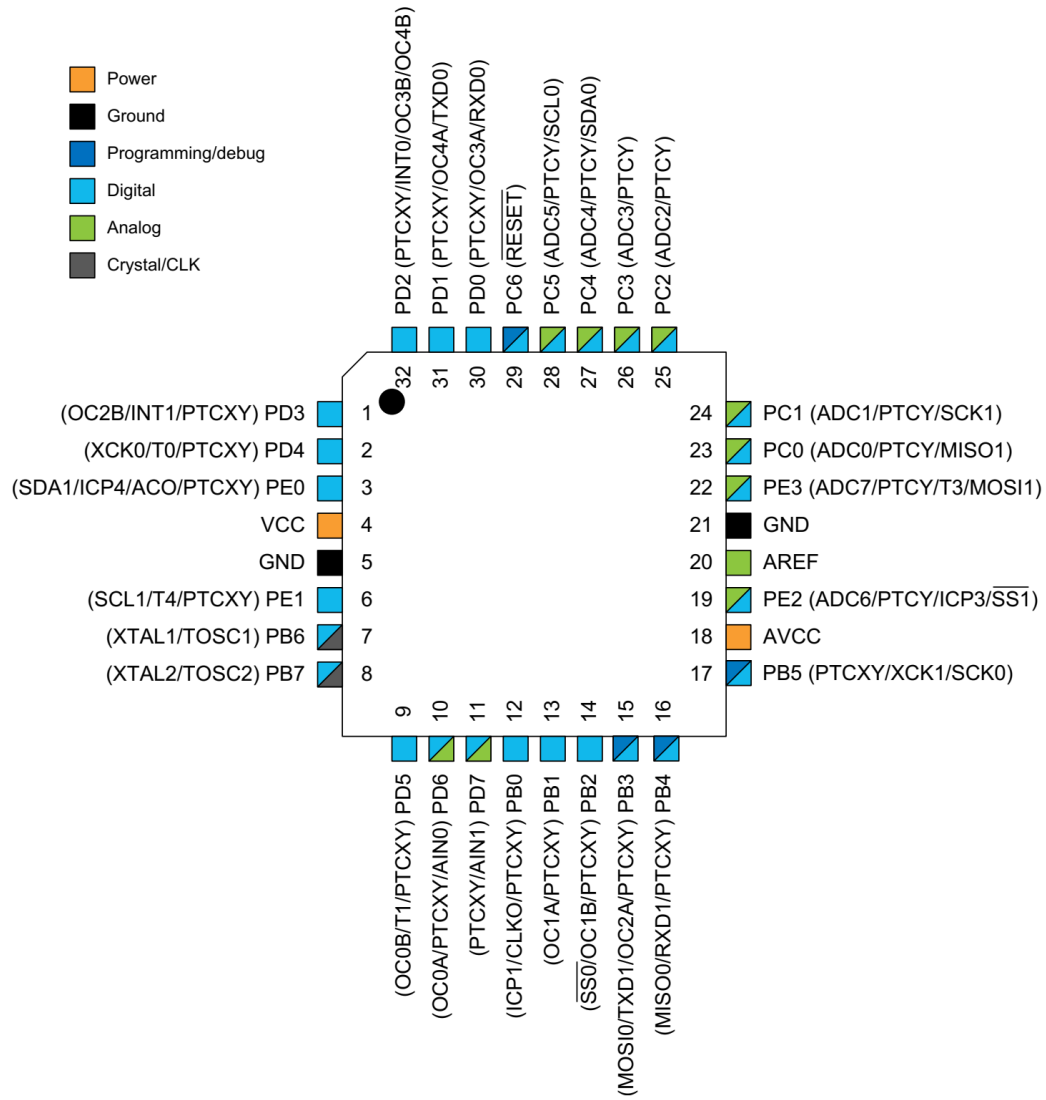
The picoPower® ATmega328PB is a low-power CMOS 8-bit microcontroller based on the AVR® enhancedRISC architecture. By executing powerful instructions in a single clock cycle, the ATmega328PB achieves throughputs close to 1MIPS per MHz. This empowers system designers to optimize the device for power consumption versus processing speed.

Features

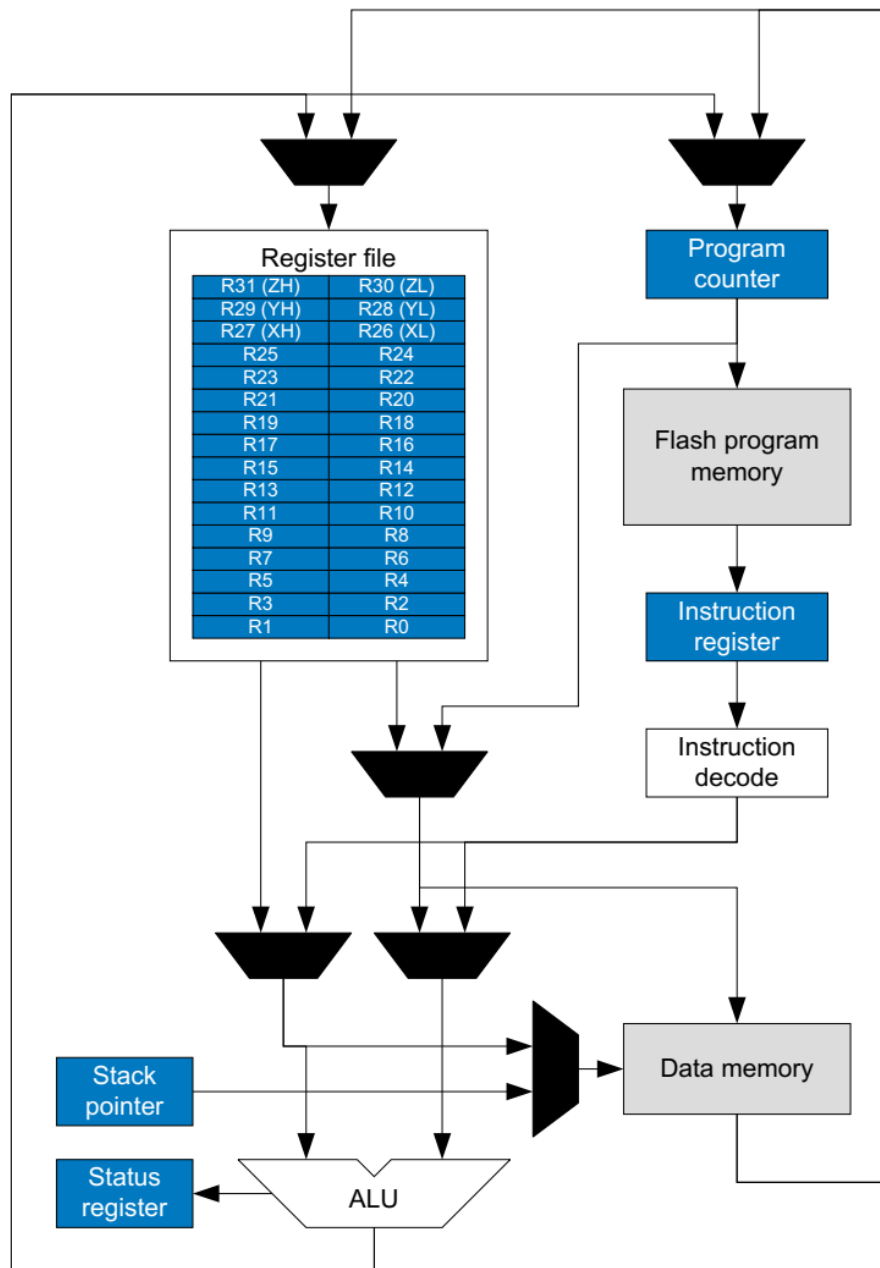
High Performance, Low Power AVR® 8-Bit Microcontroller Family

- 131 Powerful Instructions
- Most Single Clock Cycle Execution
- 32 x 8 General Purpose Working Registers
- Fully Static Operation
- Up to 20 MIPS Throughput at 20MHz
- On-Chip 2-Cycle Multiplier
- 32KBytes of In-System Self-Programmable Flash program memory
- 1KBytes EEPROM
- 2KBytes Internal SRAM
- Programming Lock for Software Security
- Two 8-bit Timer/Counters with Separate Prescaler and Compare Mode
- Three 16-bit Timer/Counters with Separate Prescaler, Compare Mode, and Capture Mode
- Real Time Counter with Separate Oscillator
- Ten PWM Channels
- 8-channel 10-bit ADC in TQFP and QFN/MLF package
- Two Programmable Serial USARTs
- Two Master/Slave SPI Serial Interfaces
- Two Byte-Oriented 2-Wire Serial Interfaces (Philips I2C Compatible)
- Programmable Watchdog Timer with Separate On-chip Oscillator
- On-Chip Analog Comparator
- Interrupt and Wake-Up on Pin Change
- Internal 8 MHz Calibrated Oscillator
- External and Internal Interrupt Sources
- Six Sleep Modes: Idle, ADC Noise Reduction, Power-save, Power-down, Standby, and
- 27 Programmable I/O Lines
- Operating Voltage: 1.8 - 5.5V
- Temperature Range: -40°C to 105°C
- Speed Grade:
 - 0 - 4MHz @ 1.8 - 5.5V
 - 0 - 10MHz @ 2.7 - 5.5.V
 - 0 - 20MHz @ 4.5 - 5.5V
- Power Consumption at 1MHz, 1.8V, 25°C
 - Active Mode: 0.24mA
 - Power-Down Mode: 0.2µA
 - Power-Save Mode: 1.3µA (Including 32kHz RTC)

Pin Configuration



Block Diagram of the AVR Architecture



I/O Registers

When addressing I/O Registers as data space using LD and ST instructions, the provided offset must be used. When using the I/O specific commands IN and OUT, the offset is reduced by 0x20, resulting in an I/O address offset within 0x00 - 0x3F.

The device is a complex microcontroller with more peripheral units than can be supported within the 64 locations reserved in Opcode for the IN and OUT instructions. For the Extended I/O space from 0x60 in SRAM, only the ST/STS/STD and LD/LDS/LDD instructions can be used.

Status Register(SREG)

The Status Register contains information about the result of the most recently executed arithmetic instruction. This information can be used for altering program flow in order to perform conditional operations. The Status Register is updated after all ALU operations, as specified in the Instruction Set Reference. This will in many cases remove the need for using the dedicated compare instructions, resulting in faster and more compact code.

The Status Register is not automatically stored when entering an interrupt routine and restored when returning from an interrupt. This must be handled by software.

SREG (Offset: 0x5F)

Bit	7	6	5	4	3	2	1	0
	I	T	H	S	V	N	Z	C
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit 7 – I: Global Interrupt Enable

The Global Interrupt Enable bit must be set for the interrupts to be enabled. The individual interrupt enable control is then performed in separate control registers. If the Global Interrupt Enable Register is cleared, none of the interrupts are enabled independent of the individual interrupt enable settings. The Ibit is cleared by hardware after an interrupt has occurred, and is set by the RETI instruction to enable subsequent interrupts. The I-bit can also be set and cleared by the application with the SEI and CLI instructions, as described in the instruction set reference.

Bit 6 – T: Copy Storage

The Bit Copy instructions BLD (Bit Load) and BST (Bit Store) use the T-bit as source or destination for the operated bit. A bit from a register in the Register File can be copied into T by the BST instruction, and a bit in T can be copied into a bit in a register in the Register File by the BLD instruction.

Bit 5 – H: Half Carry Flag

The Half Carry Flag H indicates a Half Carry in some arithmetic operations. Half Carry Flag is useful in BCD arithmetic.

Bit 4 – S: Sign Flag, $S = N \oplus V$

The S-bit is always an exclusive or between the Negative Flag N and the Two's Complement Overflow Flag V.

Bit 3 – V: Two's Complement Overflow Flag

The Two's Complement Overflow Flag V supports two's complement arithmetic.

Bit 2 – N: Negative Flag

The Negative Flag N indicates a negative result in an arithmetic or logic operation.

Bit 1 – Z: Zero Flag

The Zero Flag Z indicates a zero result in an arithmetic or logic operation.

Bit 0 – C: Carry Flag

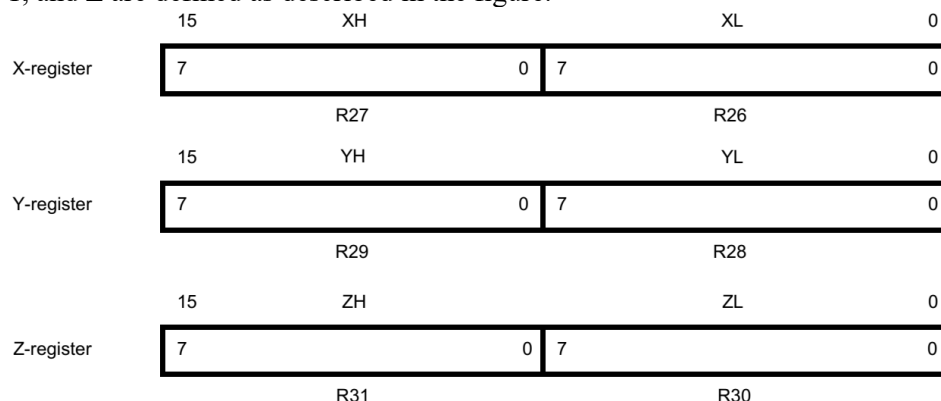
The Carry Flag C indicates a carry in an arithmetic or logic operation.

AVR CPU General Purpose Working Registers

Most of the instructions operating on the Register File have direct access to all registers, and most of them are single cycle instructions. As shown in the figure, each register is also assigned a data memory address, mapping them directly into the first 32 locations of the user Data Space.

The X-register, Y-register, and Z-register

The registers R26...R31 have some added functions to their general purpose usage. These registers are 16-bit address pointers for indirect addressing of the data space. The three indirect address registers X, Y, and Z are defined as described in the figure:



Stack Pointer(SPH:SPL)

The Stack is mainly used for storing temporary data, for storing local variables and for storing return addresses after interrupts and subroutine calls. The Stack is implemented as growing from higher to lower memory locations. The Stack Pointer Register always points to the top of the Stack. The Stack Pointer points to the data SRAM Stack area where the Subroutine and Interrupt Stacks are located. A Stack PUSH command will decrease the Stack Pointer. The Stack in the data SRAM must be defined by the program before any subroutine calls are executed or interrupts are enabled. Initial Stack Pointer value equals the last address of the internal SRAM and the Stack Pointer must be set to point above start of the SRAM.

The AVR Stack Pointer is implemented as two 8-bit registers in the I/O space. The number of bits actually used is implementation dependent.

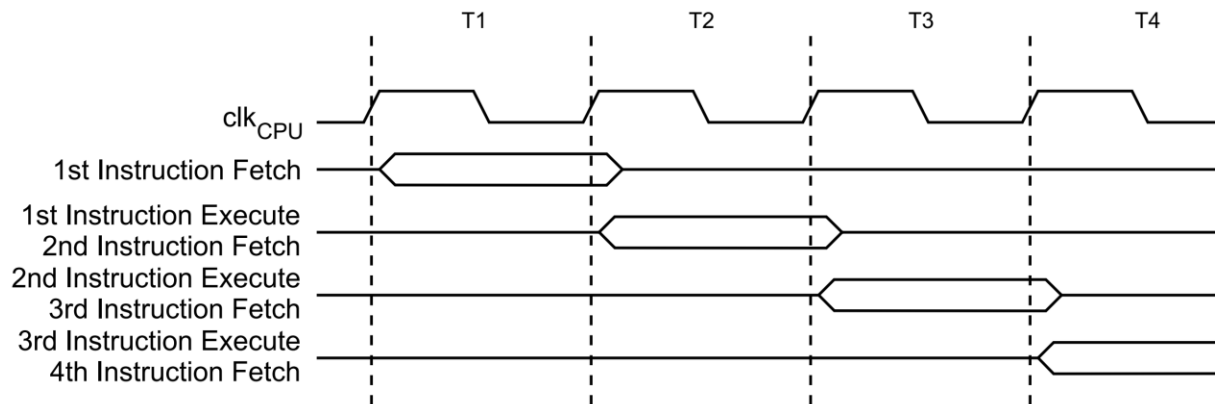
SPH-SPL (Offset: 0x5E-0x5D)

Bit	15	14	13	12	11	10	9	8
					SP11	SP10	SP9	SP8
Access	R	R	R	R	RW	RW	RW	RW
Reset	0	0	0	0	1	0	0	0

Bit	7	6	5	4	3	2	1	0
	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0
Access	RW	RW	RW	RW	RW	RW	RW	RW
Reset	1	1	1	1	1	1	1	1

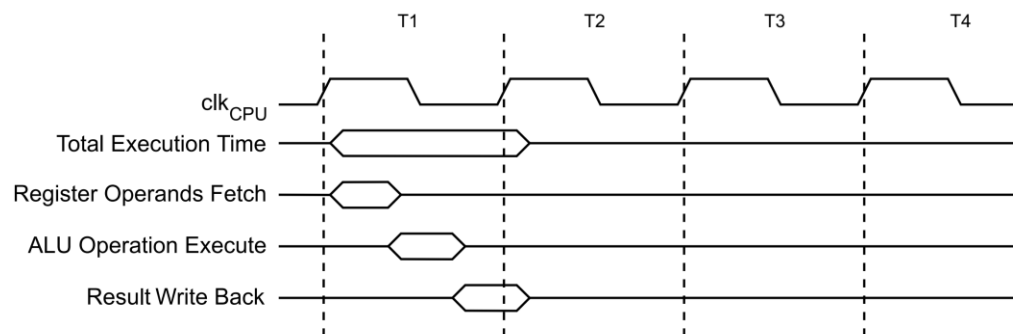
Instruction Execution Timing

The AVR CPU is driven by the CPU clock clk_{CPU} , directly generated from the selected clock source for the chip. No internal clock division is used. The Figure below shows the parallel instruction fetches and instruction executions enabled by the Harvard architecture and the fast-access Register File concept. This is the basic pipelining concept to obtain up to 1 MIPS per MHz with the corresponding unique results for functions per cost, functions per clocks, and functions per power-unit.



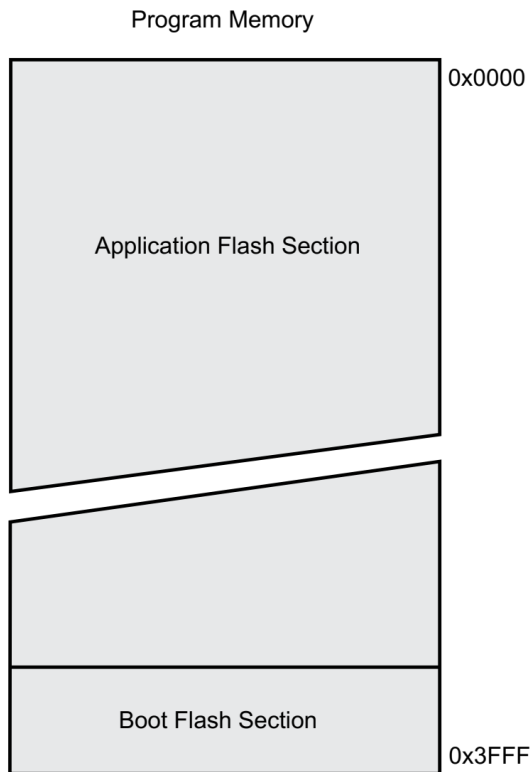
Single Cycle ALU Operation

In a single clock cycle an ALU operation using two register operands is executed, and the result is stored back to the destination register.



In-System Reprogrammable Flash Program Memory

The ATmega328PB contains 32Kbytes on-chip in-system reprogrammable Flash memory for program storage. Since all AVR instructions are 16 or 32 bits wide, the Flash is organized as 16K x 16. The ATmega328PB Program Counter (PC) is 14 bits wide, thus addressing the 16K program memory locations. Constant tables can be allocated within the entire program memory address space, using the Load Program Memory (LPM) instruction.



SRAM Data Memory

IN/OUT		Load/Store
0x0000 – 0x001F	32 registers	0x0000 – 0x001F
	64 I/O registers	0x0020 – 0x005F
	160 Ext I/O registers	0x0060 – 0x00FF
	Internal SRAM (2048x8)	0x0100
		0x08FF

The data memory locations address both the Register File, the I/O memory, Extended I/O memory, and the internal data SRAM.

The first 32 locations address the Register File, the next 64 location the standard I/O memory, then 160 locations of Extended I/O memory, and the next 2K locations address the internal data SRAM.

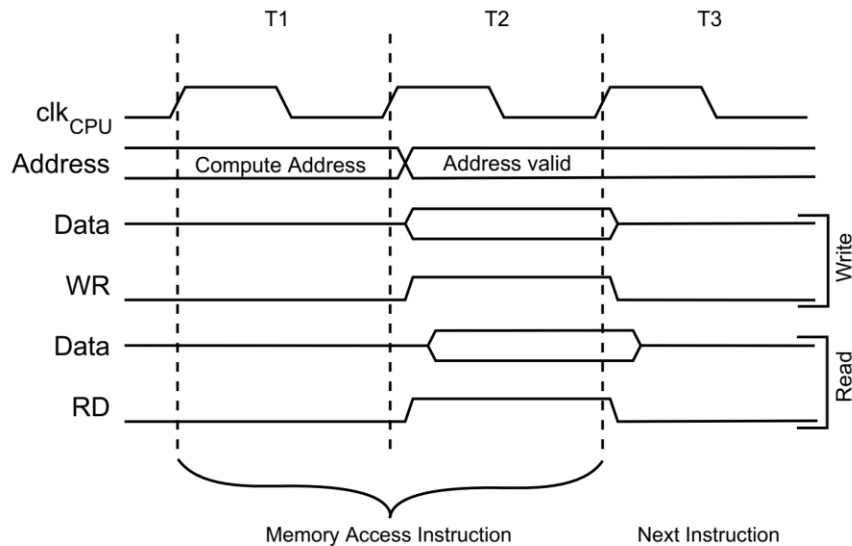
The five different addressing modes for the data memory cover:

- Direct – The direct addressing reaches the entire data space.

- Indirect with Displacement – The Indirect with Displacement mode reaches 63 address locations from the base address given by the Y- or Z-register.
- Indirect – In the Register File, registers R26 to R31 feature the indirect addressing pointer registers.
- Indirect with Pre-decrement – The address registers X, Y, and Z are decremented.
- Indirect with Post-increment – The address registers X, Y, and Z are incremented.

The 32 general purpose working registers, 64 I/O Registers, 160 Extended I/O Registers, and the 2K bytes of internal data SRAM in the device are all accessible through all these addressing modes.

The internal data SRAM access is performed in two clk_{CPU} cycles as described in the following Figure:



Reset and Interrupt Handling

During reset, all I/O Registers are set to their initial values, and the program starts execution from the Reset Vector. The instruction placed at the Reset Vector must be a Relative Jump instruction (RJMP) to the reset handling routine.

The ATmega328PB provides several different interrupt sources. Activating an interrupt source forces the microcontroller to immediately stop the current task and execute the code contained in predetermined address, called the interrupt vector. A routine is usually attached at this point interrupt service (different for each application). After the end of the interrupt service routine, the microcontroller resumes the work it interrupted, returning to the exact point where it was interrupted.

All interrupts are assigned individual enable bits which must be written logic one together with the Global Interrupt Enable bit in the Status Register in order to enable the interrupt.

The lowest addresses in the program memory space are by default defined as the Reset and Interrupt Vectors. They have determined priority levels: The lower the address the higher is the priority level. RESET has the highest priority, and next is INT0 – the External Interrupt Request 0. If the program never enables an interrupt source, the Interrupt Vectors are not used, and regular program code can be placed at these locations.

When an interrupt occurs, the Global Interrupt Enable I-bit is cleared and all interrupts are disabled. The user software can write logic one to the I-bit to enable nested interrupts. All enabled interrupts can then interrupt the current interrupt routine. The I-bit is automatically set when a Return from Interrupt instruction – RETI – is executed.

There are basically two types of interrupts:

- The first type is triggered by an event that sets the Interrupt Flag. For these interrupts, the Program Counter is vectored to the actual Interrupt Vector in order to execute the interrupt handling routine, and hardware clears the corresponding Interrupt Flag. Interrupt Flags can also be cleared by writing a logic one to the flag bit position(s) to be cleared. If an interrupt condition occurs while the corresponding interrupt enable bit is cleared, the Interrupt Flag will be set and remembered until the interrupt is enabled, or the flag is cleared by software. Similarly, if one or more interrupt conditions occur while the Global Interrupt Enable bit is cleared, the corresponding Interrupt Flag(s) will be set and remembered until the Global Interrupt Enable bit is set, and will then be executed by order of priority.
- The second type of interrupts will trigger as long as the interrupt condition is present. These interrupts do not necessarily have Interrupt Flags. If the interrupt condition disappears before the interrupt is enabled, the interrupt will not be triggered. When the AVR exits from an interrupt, it will always return to the main program and execute one more instruction before any pending interrupt is served.

The interrupt execution response for all the enabled AVR interrupts is four clock cycles minimum. After four clock cycles the program vector address for the actual interrupt handling routine is executed. During this four-clock cycle period, the Program Counter is pushed onto the Stack. The vector is normally a jump to the interrupt routine, and this jump takes three clock cycles. If an interrupt occurs during execution of a multi-cycle instruction, this instruction is completed before the interrupt is served. If an interrupt occurs when the MCU is in sleep mode, the interrupt execution response time is increased by four clock cycles. This increase comes in addition to the start-up time from the selected sleep mode.

A return from an interrupt handling routine takes four clock cycles. During these four clock cycles, the Program Counter (two bytes) is popped back from the Stack, the Stack Pointer is incremented by two, and the I-bit in SREG is set

The Status Register is not automatically stored when entering an interrupt routine, nor restored when returning from an interrupt routine. This must be handled by software.

When using the CLI instruction to disable interrupts, the interrupts will be immediately disabled. No interrupt will be executed after the CLI instruction, even if it occurs simultaneously with the CLI instruction.

Reset and Interrupt Vectors in ATmega328PB

Vector No	Program Address	Source	Interrupts definition
1	0x0000	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 0
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2_COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2_COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2_OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1_CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1_COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1_COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1_OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0_COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0_COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0_OVF	Timer/Counter0 Overflow
18	0x0022	SPI0_STC	SPI1 Serial Transfer Complete
19	0x0024	USART0_RX	USART0 Rx Complete
20	0x0026	USART0_UDRE	USART0, Data Register Empty
21	0x0028	USART0_TX	USART0, Tx Complete
22	0x002A	ADC	ADC Conversion Complete
23	0x002C	EE READY	EEPROM Ready
24	0x002E	ANALOG COMP	Analog Comparator
25	0x0030	TWI	2-wire Serial Interface (I ² C
26	0x0032	SPM READY	Store Program Memory Ready
27	0x0034	USART0_START	USART0 Start frame detection
28	0x0036	PCINT3	Pin Change Interrupt Request 3
29	0x0038	USART1_RX	USART0 Rx Complete
30	0x003A	USART1_UDRE	USART0, Data Register Empty
31	0x003C	USART1_TX	USART0, Tx Complete
32	0x003E	USART1_START	USART1 Start frame detection
33	0x0040	TIMER3_CAPT	Timer/Counter3 Capture Event
34	0x0042	TIMER3_COMPA	Timer/Counter3 Compare Match A
35	0x0044	TIMER3_COMPB	Timer/Counter3 Compare Match B
36	0x0046	TIMER3_OVF	Timer/Counter3 Overflow
37	0x0048	CFD	Clock failure detection interrupt
38	0x004A	PTC_EOC	PTC End of Conversion
39	0x004C	PTC_WCOMP	PTC Window comparator mode
40	0x004E	SPI1_STC	SPI1 Serial Transfer Complete
41	0x0050	TWI1	TWI1 Transfer complete
42	0x0052	TIMER4_CAPT	Timer/Counter3 Capture Event
43	0x0054	TIMER4_COMPA	Timer/Counter3 Compare Match A
44	0x0056	TIMER4_COMPB	Timer/Counter3 Compare Match B
45	0x0058	TIMER4_OVF	Timer/Counter3 Overflow

External interrupts on ATmega328PB

The ATmega328PB is equipped with interrupt inputs for immediate response to external conditions. For the operation of the interrupt system of each Microcontroller, it is first necessary to activate its flags and options, which determine the exact mode of operation.

In the AVR ATmega328PB Microcontroller, the selection of the activation level of the external interrupts is done via the EICRA register (offset 0x69):

Bit	7	6	5	4	3	2	1	0
					ISC1n	ISC1n	ISC0n	ISC0n
Access					R/W	R/W	R/W	R/W
Reset					0	0	0	0

ISC11	ISC10	Description
0	0	The low level of INT1 generates an interrupt request.
0	1	Any logical change on INT1 generates an interrupt request.
1	0	The falling edge of INT1 generates an interrupt request.
1	1	The rising edge of INT1 generates an interrupt request.

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

External Interrupt Mask Register (EIMSK, offset = 0x3D)

When the INTx bit of EIMSK is set and the I-bit in the Status Register (SREG) is set, the External Interrupt x is enabled.

Bit	7	6	5	4	3	2	1	0
							INT1	INT0
Access							R/W	R/W
Reset							0	0

External Interrupt Flag Register (EIFR, offset = 0x3C)

When an edge or logic change on the INTx pin triggers an interrupt request, INTFx will be set. If the I-bit in SREG and the INTx bit in EIMSK are set, the MCU will jump to the corresponding Interrupt Vector. Any of these flags can be cleared by writing a logical one to it.

Bit	7	6	5	4	3	2	1	0
							INTF1	INTF0
Access							R/W	R/W
Reset							0	0

External interrupt inputs

In the AVR ATmega328P Microcontroller, the **PD2** pin, in addition to being a general-purpose I/O pin, also works alternatively as an external input of the INT0 interrupt. Similarly, the **PD3** pin also works alternatively as an external input of the INT1 interrupt.

External interrupt vectors

The INT0 interrupt vector is **0x002**.

INT1 interrupt vector is **0x004**.

Save / Restore the SREG to the stack

The Status Register (SREG) is not automatically stored when entering an interrupt routine, nor restored when returning from an interrupt routine. This must be handled by software:

a) Store the SREG

```
push r0;      // Push the register r0 to make space for SREG
in r0, SREG;   // Read the value of SREG into a register r0
push r0;      // Push the value of SREG (r0) onto the stack
```

b) Restore the SREG

```
pop r0;       // Pop the value of SREG (r0) from the stack into the register r0
out SREG, r0; // Write the value of the register r0 back to SREG
```

Any of the 32 General Purpose Working Registers (r0 – r31) can be used in place of r0.

I/O-Ports

The port pins are tri-stated when reset condition becomes active, even if no clocks are running. In normal operation the Port pins are controlled through the three registers DDRx, PORTx, and PINx. All registers and bit references in this section are written in general form. A lower case “x” represents the numbering letter for the port, and a lower case “n” represents the bit number. However, when using the register or bit defines in a program, the precise form must be used.

Port direction

All AVR ports can be configured as input ports or output ports and have true Read-Modify-Write functionality. This means that the direction of one port pin can be changed without unintentionally changing the direction of any other pin with the SBI and CBI instructions. The DDRxn bit in the DDRx register selects the direction of this pin. If DDRxn is written to '1', the pin is configured as output. If DDRxn is written to '0', the pin is configured as input.

Pull-up resistors

All port pins have individually selectable internal pull-up resistors with a supply-voltage invariant resistance. If PORTxn bit of PORTx register is written to '1' when the pin is configured as an input pin, the pull-up resistor is activated. To switch the pull-up resistor off, PORTxn has to be written to '0' or the pin has to be configured as output. The Pull-up Disable – PUD bit in MCUCR register disables the pull-up function for all pins in all ports when set.

Output pins

If PORTxn bit of PORTx register is written to '1' when the pin is configured as an output pin, the port pin is driven high. If PORTxn is written logic zero the port pin is driven low. The output port pin has a buffer with symmetrical drive characteristics with both high sink and source capability. The pin driver is strong enough to drive LED displays directly.

Toggling a Pin

Writing a '1' to PINxn bit of PINx register toggles the value of PORTxn bit of PORTx register, independent on the value of DDRxn bit of the DDRx register. The SBI instruction can be used to toggle one single bit in a port.

Reading the Pin Value

Independent of the setting of Data Direction bit DDRxn of DDRx register, the port pin can be read through the PINxn bit of PINx register.

The following table summarizes the control signals for the pin value.

DDRxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

Register Summary (Page 1)

Offset	Name	Bit Pos.								
0x23	PINB	7:0	PINBn	PINBn	PINBn	PINBn	PINBn	PINBn	PINBn	PINBn
0x24	DDRB	7:0	DDRB7	DDRB6	DDRB5	DDRB4	DDRB3	DDRB2	DDRB1	DDRB0
0x25	PORTB	7:0	PORTBn	PORTBn	PORTBn	PORTBn	PORTBn	PORTBn	PORTBn	PORTBn
0x26	PINC	7:0		PINCn	PINCn	PINCn	PINCn	PINCn	PINCn	PINCn
0x27	DDRC	7:0		DDRC6	DDRC5	DDRC4	DDRC3	DDRC2	DDRC1	DDRC0
0x28	PORTC	7:0		PORTCn	PORTCn	PORTCn	PORTCn	PORTCn	PORTCn	PORTCn
0x29	PIND	7:0	PINDn	PINDn	PINDn	PINDn	PINDn	PINDn	PINDn	PINDn
0x2A	DDRD	7:0	DDRD7	DDRD6	DDRD5	DDRD4	DDRD3	DDRD2	DDRD1	DDRD0
0x2B	PORTD	7:0	PORTDn	PORTDn	PORTDn	PORTDn	PORTDn	PORTDn	PORTDn	PORTDn
0x2C	PINE	7:0					PINE3	PINE2	PINE1	PINE0
0x2D	DDRE	7:0					DDRE3	DDRE2	DDRE1	DDRE0
0x2E	PORTE	7:0					PORTE3	PORTE2	PORTE1	PORTE0
0x2F ... 0x34	Reserved									
0x35	TIFR0	7:0						OCF0B	OCF0A	TOV0
0x36	TIFR1	7:0			ICF1			OCF1B	OCF1A	TOV1
0x37	TIFR2	7:0						OCF2B	OCF2A	TOV2
0x38	TIFR3	7:0			ICF3			OCF3B	OCF3A	TOV3
0x39	TIFR4	7:0			ICF4			OCF4B	OCF4A	TOV4
0x3A	Reserved									
0x3B	PCIFR	7:0						PCIF2	PCIF1	PCIF0
0x3C	EIFR	7:0							INTF1	INTF0
0x3D	EIMSK	7:0							INT1	INT0
0x3E	GPOR0	7:0	GPOR0[7:0]							
0x3F	EECR	7:0			EEPmn	EEPmn	EERIE	EEMPE	EEPE	EERE
0x40	EEDR	7:0	EEDR[7:0]							
0x41	EEAR	7:0	EEAR7	EEAR6	EEAR5	EEAR4	EEAR3	EEAR2	EEAR1	EEAR0
0x42		15:8							EEAR9	EEAR8
0x43	GTCCR	7:0	TSM						PSRASY	PSRSYNC
0x44	TCCR0A	7:0	COM0An	COM0An	COM0Bn	COM0Bn			WGM0n	WGM0n
0x45	TCCR0B	7:0	FOC0A	FOC0B			WGM02	CS0[2:0]		
0x46	TCNT0	7:0	TCNT0[7:0]							
0x47	OCR0A	7:0	OCR0A[7:0]							
0x48	OCR0B	7:0	OCR0B[7:0]							
0x49	Reserved									
0x4A	GPOR1	7:0	GPOR1[7:0]							
0x4B	GPOR2	7:0	GPOR2[7:0]							
0x4C	SPCR0	7:0	SPIE0	SPE0	DORD0	MSTR0	CPOL0	CPHA0	SPR0n	SPR0n
0x4D	SPSR0	7:0	SPIF0	WCOL0						SPI2X0
0x4E	SPDR0	7:0	SPID[7:0]							
0x4F	Reserved									
0x50	ACSR	7:0	ACD	ACBG	ACO	ACI	ACIE	ACIC	ACISn	ACISn
0x51	DWDR	7:0	DWDR[7:0]							

Register Summary (Page 3)

Offset	Name	Bit Pos.								
0x84	TCNT1L and TCNT1H	7:0	TCNT1[7:0]							
0x85		15:8	TCNT1[15:8]							
0x86	ICR1L and ICR1H	7:0	ICR1[7:0]							
0x87		15:8	ICR1[15:8]							
0x88	OCR1AL and OCR1AH	7:0	OCR1A[7:0]							
0x89		15:8	OCR1A[15:8]							
0x8A	OCR1BL and OCR1BH	7:0	OCR1B[7:0]							
0x8B		15:8	OCR1B[15:8]							
0x8C ... 0x8F	Reserved									
0x90	TCCR3A	7:0	COM3A1	COM3A0	COM3B1	COM3B0			WGM31	WGM30
0x91	TCCR3B	7:0	ICNC3	ICES3		WGM33	WGM32	CS3[2:0]		
0x92	TCCR3C	7:0								
0x93	Reserved									
0x94	TCNT3L and TCNT3H	7:0	TCNT3[7:0]							
0x95		15:8	TCNT3[15:8]							
0x96	ICR3L and ICR3H	7:0	ICR3[7:0]							
0x97		15:8	ICR3[15:8]							
0x98	OCR3AL and OCR3AH	7:0	OCR3A[7:0]							
0x99		15:8	OCR3A[15:8]							
0x9A	OCR3BL and OCR3BH	7:0	OCR3B[7:0]							
0x9B		15:8	OCR3B[15:8]							
0x9C ... 0x9F	Reserved									
0xA0	TCCR4A	7:0	COM4A1	COM4A0	COM4B1	COM4B0			WGM43	WGM42
0xA1	TCCR4B	7:0	ICNC4	ICES4		WGM43	WGM42	CS4[2:0]		
0xA2	TCCR4C	7:0	FOC4A	FOC4B						
0xA3	Reserved									
0xA4	TCNT4L and TCNT4H	7:0	TCNT4[7:0]							
0xA5		15:8	TCNT4[15:8]							
0xA6	ICR4L and ICR4H	7:0	ICR4[7:0]							
0xA7		15:8	ICR4[15:8]							
0xA8 ... 0xA9	Reserved									
0xAA	OCR4BL and OCR4BH	7:0	OCR4B[7:0]							
0xAB		15:8	OCR4B[15:8]							
0xAC	SPCR1	7:0	SPIE1	SPE1	DORD1	MSTR1	CPOL1	CPHA1	SPR1n	SPR1n
0xAD	SPSR1	7:0	SPIF1	WCOL1						SPI2X1
0xAE	SPDR1	7:0	SPID1[7:0]							
0xAF	Reserved									
0xB0	TCCR2A	7:0	COM2An	COM2An	COM2Bn	COM2Bn			WGM2n	WGM2n
0xB1	TCCR2B	7:0	FOC2A	FOC2B			WGM22	CS2[2:0]		
0xB2	TCNT2	7:0	TCNT2[7:0]							
0xB3	OCR2A	7:0	OCR2A[7:0]							

Register Summary (Page 4)

Offset	Name	Bit Pos.									
0xB4	OCR2B	7:0	OCR2B[7:0]								
0xB5	Reserved										
0xB6	ASSR	7:0		EXCLK	AS2	TCN2UB	OCR2AUB	OCR2BUB	TCR2AUB	TCR2BUB	
0xB7	Reserved										
0xB8	TWBR0	7:0	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	
0xB9	TWSR0	7:0	TWS7	TWS6	TWS5	TWS4	TWS3		TWPS[1:0]		
0xBA	TWAR0	7:0	TWA[6:0]							TWGCE	
0xBB	TWDR0	7:0	TWD[7:0]								
0xBC	TWCR0	7:0	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN		TWIE	
0xBD	TWAMR0	7:0	TWAM[6:0]								
0xBE ... 0xBF	Reserved										
0xC0	UCSR0A	7:0	RXC	TXC	UDRE	FE	DOR	UPE	U2X	MPCM	
0xC1	UCSR0B	7:0	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	
0xC2	UCSR0C	7:0	UMSEL[1:0]		UPM[1:0]		USBS	UCSZ1 / UDORD	UCSZ0 / UCPHA	UCPOL	
0xC3	Reserved										
0xC4	UBRR0L and UBRR0H	7:0	UBRR[7:0]								
0xC5		15:8					UBRR[11:8]				
0xC6	UDR0	7:0	TXB / RXB[7:0]								
0xC7	Reserved										
0xC8	UCSR1A	7:0	RXC	TXC	UDRE	FE	DOR	UPE	U2X	MPCM	
0xC9	UCSR1B	7:0	RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8	
0xCA	UCSR1C	7:0	UMSEL[1:0]		UPM[1:0]		USBS	UCSZ1 / UDORD	UCSZ0 / UCPHA	UCPOL	
0xCB	Reserved										
0xCC	UBRR1L and UBRR1H	7:0	UBRR[7:0]								
0xCD		15:8					UBRR[11:8]				
0xCE	UDR1	7:0	TXB / RXB[7:0]								
0xCF ... 0xD7	Reserved										
0xD8	TWBR1	7:0	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	TWBRn	
0xD9	TWSR1	7:0	TWS7	TWS6	TWS5	TWS4	TWS3		TWPS[1:0]		
0xDA	TWAR1	7:0	TWA[6:0]							TWGCE	
0xDB	TWDR1	7:0	TWD[7:0]								
0xDC	TWCR1	7:0	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN		TWIE	
0xDD	TWAMR1	7:0	TWAM[6:0]								
0xDE ... 0x09A7	Reserved										
0x09A8	OCR4AL and	7:0	OCR4A[7:0]								
0x09A9	OCR4AH	15:8	OCR4A[15:8]								

Instruction Set Summary

ARITHMETIC AND LOGIC INSTRUCTIONS					
Mnemonics	Operands	Description	Operation	Flags	#Clocks
ADD	Rd, Rr	Add two Registers without Carry	$Rd \leftarrow Rd + Rr$	Z,C,N,V,H	1
ADC	Rd, Rr	Add two Registers with Carry	$Rd \leftarrow Rd + Rr + C$	Z,C,N,V,H	1
ADIW	RdI, K	Add Immediate to Word	$Rdh:Rdl \leftarrow Rdh:Rdl + K$	Z,C,N,V,S	2
SUB	Rd, Rr	Subtract two Registers	$Rd \leftarrow Rd - Rr$	Z,C,N,V,H	1
SUBI	Rd, K	Subtract Constant from Register	$Rd \leftarrow Rd - K$	Z,C,N,V,H	1
SBC	Rd, Rr	Subtract two Registers with Carry	$Rd \leftarrow Rd - Rr - C$	Z,C,N,V,H	1
SBCI	Rd, K	Subtract Constant from Reg with Carry.	$Rd \leftarrow Rd - K - C$	Z,C,N,V,H	1
SBIW	RdI, K	Subtract Immediate from Word	$Rdh:Rdl \leftarrow Rdh:Rdl - K$	Z,C,N,V,S	2
AND	Rd, Rr	Logical AND Registers	$Rd \leftarrow Rd \cdot Rr$	Z,N,V	1
ANDI	Rd, K	Logical AND Register and Constant	$Rd \leftarrow Rd \cdot K$	Z,N,V	1
OR	Rd, Rr	Logical OR Registers	$Rd \leftarrow Rd \vee Rr$	Z,N,V	1
ORI	Rd, K	Logical OR Register and Constant	$Rd \leftarrow Rd \vee K$	Z,N,V	1
EOR	Rd, Rr	Exclusive OR Registers	$Rd \leftarrow Rd \oplus Rr$	Z,N,V	1
COM	Rd	One's Complement	$Rd \leftarrow 0xFF - Rd$	Z,C,N,V	1
NEG	Rd	Two's Complement	$Rd \leftarrow 0x00 - Rd$	Z,C,N,V,H	1
SBR	Rd, K	Set Bit(s) in Register	$Rd \leftarrow Rd \vee K$	Z,N,V	1
CBR	Rd, K	Clear Bit(s) in Register	$Rd \leftarrow Rd \cdot (0xFF - K)$	Z,N,V	1
INC	Rd	Increment	$Rd \leftarrow Rd + 1$	Z,N,V	1
DEC	Rd	Decrement	$Rd \leftarrow Rd - 1$	Z,N,V	1
TST	Rd	Test for Zero or Minus	$Rd \leftarrow Rd \cdot Rd$	Z,N,V	1
CLR	Rd	Clear Register	$Rd \leftarrow Rd \oplus Rd$	Z,N,V	1
SER	Rd	Set Register	$Rd \leftarrow 0xFF$	None	1
MUL	Rd, Rr	Multiply Unsigned	$R1:R0 \leftarrow Rd \times Rr$	Z,C	2
MULS	Rd, Rr	Multiply Signed	$R1:R0 \leftarrow Rd \times Rr$	Z,C	2
MULSU	Rd, Rr	Multiply Signed with Unsigned	$R1:R0 \leftarrow Rd \times Rr$	Z,C	2
FMUL	Rd, Rr	Fractional Multiply Unsigned	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$	Z,C	2
FMULS	Rd, Rr	Fractional Multiply Signed	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$	Z,C	2
FMULSU	Rd, Rr	Fractional Multiply Signed with Unsigned	$R1:R0 \leftarrow (Rd \times Rr) \ll 1$	Z,C	2

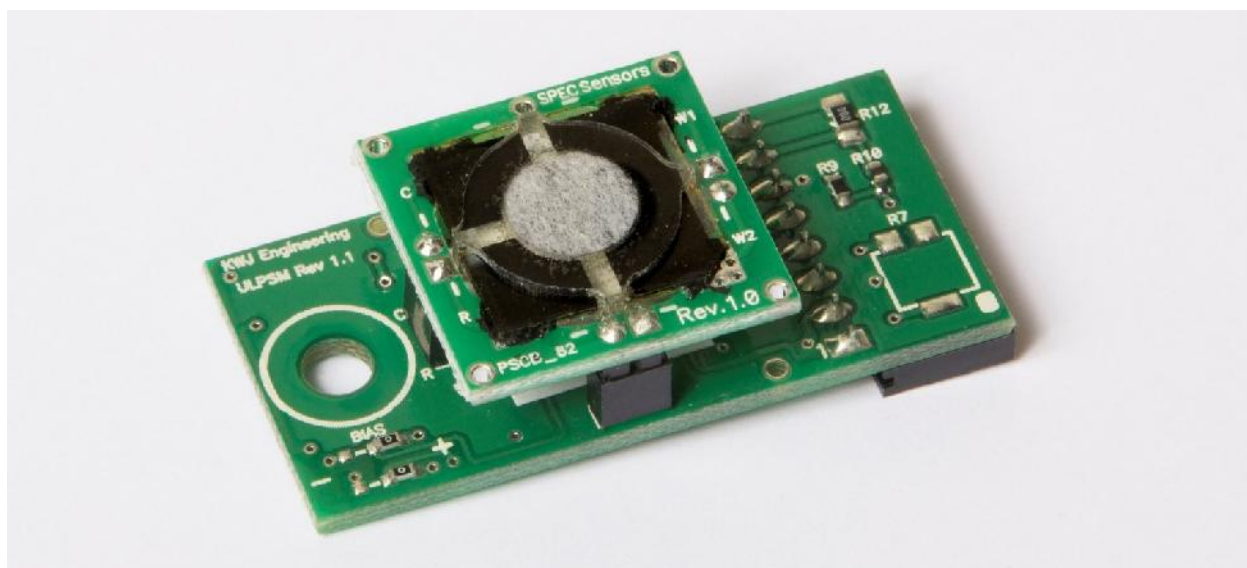
BRANCH INSTRUCTIONS					
Mnemonics	Operands	Description	Operation	Flags	#Clocks
RJMP	k	Relative Jump	$PC \leftarrow PC + k + 1$	None	2
IJMP		Indirect Jump to (Z)	$PC \leftarrow Z$	None	2
JMP(1)	k	Direct Jump	$PC \leftarrow k$	None	3
RCALL	k	Relative Subroutine Call	$PC \leftarrow PC + k + 1$	None	3
ICALL		Indirect Call to (Z)	$PC \leftarrow Z$	None	3
CALL(1)	k	Direct Subroutine Call	$PC \leftarrow k$	None	4
RET		Subroutine Return	$PC \leftarrow \text{STACK}$	None	4
RETI		Interrupt Return	$PC \leftarrow \text{STACK}$	I	4
CPSE	Rd,Rr	Compare, Skip if Equal	if (Rd = Rr) $PC \leftarrow PC + 2$ or 3	None	1/2/3
CP	Rd,Rr	Compare	Rd - Rr	Z, N,V,C,H	1
CPC	Rd,Rr	Compare with Carry	Rd - Rr - C	Z, N,V,C,H	1
CPI	Rd,K	Compare Register with Immediate	Rd - K	Z, N,V,C,H	1
SBRC	Rr, b	Skip if Bit in Register Cleared	if (Rr(b)=0) $PC \leftarrow PC + 2$ or 3	None	1/2/3
SBRS	Rr, b	Skip if Bit in Register is Set	if (Rr(b)=1) $PC \leftarrow PC + 2$ or 3	None	1/2/3
SBIC	A, b	Skip if Bit in I/O Register Cleared	if (I/O(A,b)=1) $PC \leftarrow PC + 2$ or 3	None	1/2/3
SBIS	A, b	Skip if Bit in I/O Register is Set	if (I/O(A,b)=1) $PC \leftarrow PC + 2$ or 3	None	1/2/3
BRBS	s, k	Branch if Status Flag Set	if (SREG(s) = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRBC	s, k	Branch if Status Flag Cleared	if (SREG(s) = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BREQ	k	Branch if Equal	if (Z = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRNE	k	Branch if Not Equal	if (Z = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRCS	k	Branch if Carry Set	if (C = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRCC	k	Branch if Carry Cleared	if (C = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRSH	k	Branch if Same or Higher	if (C = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRLO	k	Branch if Lower	if (C = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRMI	k	Branch if Minus	if (N = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRPL	k	Branch if Plus	if (N = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRGE	k	Branch if Greater or Equal, Signed	if (N $\oplus$ V = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRLT	k	Branch if Less Than Zero, Signed	if (N $\oplus$ V = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRHS	k	Branch if Half Carry Flag Set	if (H = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRHC	k	Branch if Half Carry Flag Cleared	if (H = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRTS	k	Branch if T Flag Set	if (T = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRTC	k	Branch if T Flag Cleared	if (T = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRVS	k	Branch if Overflow Flag is Set	if (V = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRVC	k	Branch if Overflow Flag is Cleared	if (V = 0) then $PC \leftarrow PC + k + 1$	None	1/2
BRIE	k	Branch if Interrupt Enabled	if (I = 1) then $PC \leftarrow PC + k + 1$	None	1/2
BRID	k	Branch if Interrupt Disabled	if (I = 0) then $PC \leftarrow PC + k + 1$	None	1/2

BIT AND BIT-TEST INSTRUCTIONS					
Mnemonics	Operands	Description	Operation	Flags	#Clocks
SBI	P,b	Set Bit in I/O Register	$I/O(P,b) \leftarrow 1$	None	2
CBI	P,b	Clear Bit in I/O Register	$I/O(P,b) \leftarrow 0$	None	2
LSL	Rd	Logical Shift Left	$Rd(n+1) \leftarrow Rd(n), Rd(0) \leftarrow 0$	Z,C,N,V	1
LSR	Rd	Logical Shift Right	$Rd(n) \leftarrow Rd(n+1), Rd(7) \leftarrow 0$	Z,C,N,V	1
ROL	Rd	Rotate Left Through Carry	$Rd(0) \leftarrow C, Rd(n+1) \leftarrow Rd(n), C \leftarrow Rd(7)$	Z,C,N,V	1
ROR	Rd	Rotate Right Through Carry	$Rd(7) \leftarrow C, Rd(n) \leftarrow Rd(n+1), C \leftarrow Rd(0)$	Z,C,N,V	1
ASR	Rd	Arithmetic Shift Right	$Rd(n) \leftarrow Rd(n+1), n=0...6$	Z,C,N,V	1
SWAP	Rd	Swap Nibbles	$Rd(3...0) \leftarrow Rd(7...4), Rd(7...4) \leftarrow Rd(3...0)$	None	1
BSET	s	Flag Set	$SREG(s) \leftarrow 1$	SREG(s)	1
BCLR	s	Flag Clear	$SREG(s) \leftarrow 0$	SREG(s)	1
BST	Rr, b	Bit Store from Register to T	$T \leftarrow Rr(b)$	T	1
BLD	Rd, b	Bit load from T to Register	$Rd(b) \leftarrow T$	None	1
SEC		Set Carry	$C \leftarrow 1$	C	1
CLC		Clear Carry	$C \leftarrow 0$	C	1
SEN		Set Negative Flag	$N \leftarrow 1$	N	1
CLN		Clear Negative Flag	$N \leftarrow 0$	N	1
SEZ		Set Zero Flag	$Z \leftarrow 1$	Z	1
CLZ		Clear Zero Flag	$Z \leftarrow 0$	Z	1
SEI		Global Interrupt Enable	$I \leftarrow 1$	I	1
CLI		Global Interrupt Disable	$I \leftarrow 0$	I	1
SES		Set Signed Test Flag	$S \leftarrow 1$	S	1
CLS		Clear Signed Test Flag	$S \leftarrow 0$	S	1
SEV		Set Two's Complement Overflow.	$V \leftarrow 1$	V	1
CLV		Clear Two's Complement Overflow	$V \leftarrow 0$	V	1
SET		Set T in SREG	$T \leftarrow 1$	T	1
CLT		Clear T in SREG	$T \leftarrow 0$	T	1
SEH		Set Half Carry Flag in SREG	$H \leftarrow 1$	H	1
CLH		Clear Half Carry Flag in SREG	$H \leftarrow 0$	H	1

MCU CONTROL INSTRUCTIONS					
Mnemonics	Operands	Description	Operation	Flags	#Clocks
NOP		No Operation	No Operation	None	1
SLEEP		Sleep	(see specific descr. for Sleep function)	None	1
WDR		Watchdog Reset	(see specific descr. for WDR/timer)	None	1
BREAK		Break	For On-chip Debug Only	None	N/A

DATA TRANSFER INSTRUCTIONS					
Mnemonics	Operands	Description	Operation	Flags	#Clocks
MOV	Rd, Rr	Move Between Registers	$Rd \leftarrow Rr$	None	1
MOVW	Rd, Rr	Copy Register Word	$Rd+1:Rd \leftarrow Rr+1:Rr$	None	1
LDI	Rd, K	Load Immediate	$Rd \leftarrow K$	None	1
LD	Rd, X	Load Indirect	$Rd \leftarrow (X)$	None	2
LD	Rd, X+	Load Indirect and Post-Increment	$Rd \leftarrow (X), X \leftarrow X + 1$	None	2
LD	Rd, -X	Load Indirect and Pre-Decrement	$X \leftarrow X - 1, Rd \leftarrow (X)$	None	2
LD	Rd, Y	Load Indirect	$Rd \leftarrow (Y)$	None	2
LD	Rd, Y+	Load Indirect and Post-Increment	$Rd \leftarrow (Y), Y \leftarrow Y + 1$	None	2
LD	Rd, -Y	Load Indirect and Pre-Decrement	$Y \leftarrow Y - 1, Rd \leftarrow (Y)$	None	2
LDD	Rd, Y+q	Load Indirect with Displacement	$Rd \leftarrow (Y + q)$	None	2
LD	Rd, Z	Load Indirect	$Rd \leftarrow (Z)$	None	2
LD	Rd, Z+	Load Indirect and Post-Increment	$Rd \leftarrow (Z), Z \leftarrow Z + 1$	None	2
LD	Rd, -Z	Load Indirect and Pre-Decrement	$Z \leftarrow Z - 1, Rd \leftarrow (Z)$	None	2
LDD	Rd, Z+q	Load Indirect with Displacement	$Rd \leftarrow (Z + q)$	None	2
LDS	Rd, k	Load Direct from SRAM	$Rd \leftarrow (k)$	None	2
ST	X, Rr	Store Indirect	$(X) \leftarrow Rr$	None	2
ST	X+, Rr	Store Indirect and Post-Increment	$(X) \leftarrow Rr, X \leftarrow X + 1$	None	2
ST	-X, Rr	Store Indirect and Pre-Decrement	$X \leftarrow X - 1, (X) \leftarrow Rr$	None	2
ST	Y, Rr	Store Indirect	$(Y) \leftarrow Rr$	None	2
ST	Y+, Rr	Store Indirect and Post-Increment	$(Y) \leftarrow Rr, Y \leftarrow Y + 1$	None	2
ST	-Y, Rr	Store Indirect and Pre-Decrement	$Y \leftarrow Y - 1, (Y) \leftarrow Rr$	None	2
STD	Y+q, Rr	Store Indirect with Displacement	$(Y + q) \leftarrow Rr$	None	2
ST	Z, Rr	Store Indirect	$(Z) \leftarrow Rr$	None	2
ST	Z+, Rr	Store Indirect and Post-Increment	$(Z) \leftarrow Rr, Z \leftarrow Z + 1$	None	2
ST	-Z, Rr	Store Indirect and Pre-Decrement	$Z \leftarrow Z - 1, (Z) \leftarrow Rr$	None	2
STD	Z+q, Rr	Store Indirect with Displacement	$(Z + q) \leftarrow Rr$	None	2
STS	k, Rr	Store Direct to SRAM	$(k) \leftarrow Rr$	None	2
LPM		Load Program Memory	$R0 \leftarrow (Z)$	None	3
LPM	Rd, Z	Load Program Memory	$Rd \leftarrow (Z)$	None	3
LPM	Rd, Z+	Load Program Memory and Post-Inc	$Rd \leftarrow (Z), Z \leftarrow Z + 1$	None	3
SPM		Store Program Memory	$(Z) \leftarrow R1:R0$	None	-
IN	Rd, A	In from I/O Location	$Rd \leftarrow I/O(A)$	None	1
OUT	A, Rr	Out to I/O Location	$I/O(A) \leftarrow Rr$	None	1
PUSH	Rr	Push Register on Stack	$STACK \leftarrow Rr$	None	2
POP	Rd	Pop Register from Stack	$Rd \leftarrow STACK$	None	2

Ultra-Low Power Analog Sensor Module for Carbon Monoxide



BENEFITS

- 0 to 3 V Analog Signal Output
- Low Power Consumption < 45 μ W
- Fast Response (< 30 seconds)
- On-board Temperature Sensor
- Easy Sensor Replacement
- Standard 8-pin connector

APPLICATIONS

- Residential and Commercial CO Monitoring
- Industrial CO Monitors
- Ventilation Control
- RV and Marine CO Monitoring

DESCRIPTION

Quickly integrate Carbon Monoxide Sensing into your system with very low power consumption and a simple analog sensor signal output.

The ULPSM converts the Carbon Monoxide sensor's linear current signal output to a linear voltage signal, while maintaining the sensor at its ideal biased operation settings.

**Incorporates SPEC Sensors' 100-102
ETL and UL Recognized Component**



Conforms to UL STDS 2034 & 2075

Certified to CSA STD 6.19-01



File E471165

MEASUREMENT PERFORMANCE CHARACTERISTICS

Measurement Range	0 to 500 ppm
Lower Detection Limit	2 ppm
Resolution	1 ppm
Accuracy	< 3 % of reading
Response Time T90	< 30 seconds
Power-On Stabilization Time	15 minutes recommended

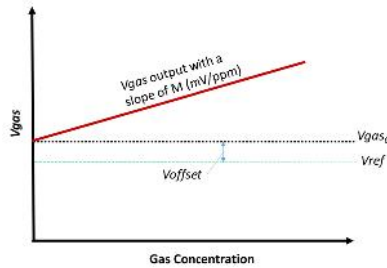
ABSOLUTE MAXIMUM RATINGS

Parameter	Conditions	Min.	Rec.	Max.	Units
Supply Voltage		2.7	3	3.3	V
Storage Temperature	Vapor Sealed @ 50% RH	5	20	30	°C
Storage Humidity	Non-condensing, Vapor Sealed	20	50	80	% RH
Storage Pressure	Vapor Sealed	0.8	1	1.2	atm.
Storage Time	Vapor Sealed	-	12	-	Months
Operating Temperature	< 10 hours	-40	-	50	°C
Operating Humidity	< 10 hours, Non-Condensing	0	-	100	% RH
Operating Temperature	Continuous	-20	25	40	°C
Operating Humidity	Continuous, Non-Condensing	15	50	95	% RH
Operating Pressure	Continuous	0.8	1	1.2	Atm.

ELECTRICAL CHARACTERISTICS

Parameter	Conditions	Min.	Typ.	Max.	Units
Supply Current	V+ = 3.0 V	5	10	15	μA
Power Consumption	V+ = 3.0 V	15	30	45	μW
Vref			V+/2		Volts
Vgas Zero		V+/2 – 0.005	V+/2	V+/2 + 0.005	Volts
Vgas Span (M)	Room temperature	0.200	0.475	0.750	mV/ppm

CALCULATING GAS CONCENTRATION



The target gas concentration is calculated by the following method:

$$Cx = \frac{1}{M} \cdot (V_{gas} - V_{gas0}),$$

where Cx is the gas concentration (ppm), V_{gas} is the voltage output gas signal (V), V_{gas0} is the voltage output gas signal in a clean-air environment (free of analyte gas) and M is the sensor calibration factor (V/ppm). The value, M , is calculated by the following method:

$$M (V/ppm) = Sensitivity\ Code (nA/ppm) \times TIA\ Gain (kV/A) \times 10^{-9} (A/nA) \times 10^3 (V/kV),$$

where the *Sensitivity Code* is provided on the sensor label and the *TIA Gain* is the gain of the trans-impedance amplifier (TIA) stage of the ULPSM circuit. Standard gain configurations are listed in the table below.

The value V_{gas0} can also be represented by:

$$V_{gas0} = V_{ref} + V_{offset},$$

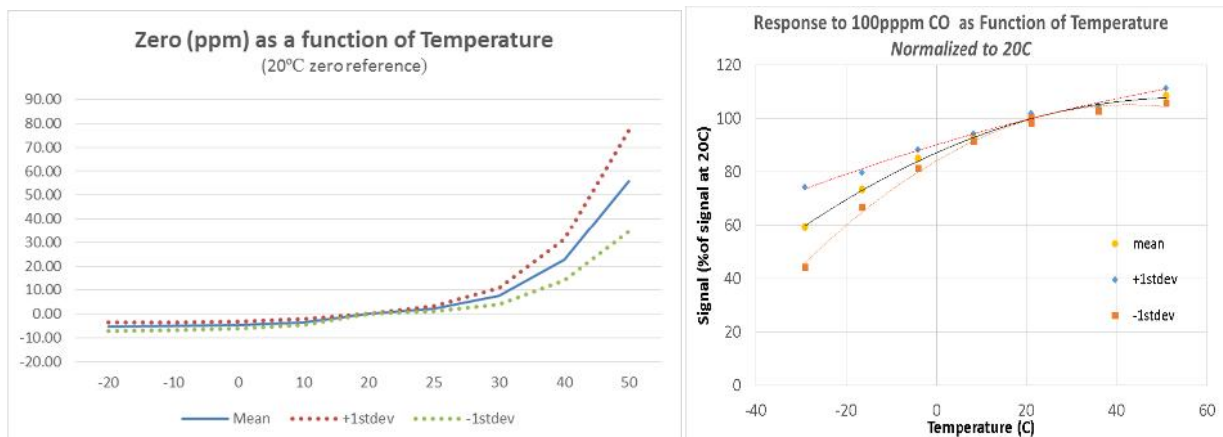
where, V_{ref} is the voltage output reference signal (V) and V_{offset} is a voltage offset factor. The V_{ref} output acts as the reference voltage for zero concentration even as the battery voltage decreases. Measuring V_{ref} in-situ compensates for variations in battery or supply voltage, minimizing these effects on Cx . A difference amplifier or instrumentation amplifier can be used to subtract V_{ref} from V_{gas} . Alternatively, when measuring V_{ref} directly, always use a unity gain buffer.

V_{offset} , accounts for a small voltage offset that is caused by a normal sensor background current and circuit background voltage. To start, $V_{offset} = 0$ is an adequate approximation. To achieve higher-precision measurements, V_{offset} must be quantified. Once the sensor has been powered-on and allowed to stabilize in a clean-air environment (free of the analyte gas) and is providing a stable output within your application's measurement goals, the value of V_{gas} may be stored as V_{gas0} and used in subsequent calculations of gas concentration, Cx .

Target Gas	TIA Gain (kV/A)
Carbon Monoxide	100
Hydrogen Sulfide	49.9
Nitrogen Dioxide	499
Sulfur Dioxide	100
Ozone	499
Ethanol	249
Indoor Air Quality	100
Respiratory Irritants	499

TEMPERATURE COMPENSATION

Temperature fluctuations have a predictable, easily compensated effect on the sensor signal. The figures below show the typical Temperature dependency of the output and baseline of 3SP_CO_1000 sensors under constant humidity of 40-50% RH. This is a very uniform and repeatable effect, easily compensated for in hardware or software.



From the graphs above:

- The temperature effect of zero shift is expressed as ppm change.
- The temperature effect of span (sensitivity) is expressed with respect to sensitivity at the calibration temperature of 20 °C.

When implementing temperature compensation, first correct the temperature effect on the zero (offset) and then correct the temperature effect on the span (sensitivity) of the sensor.

These corrections can be done in software by implementing one of the following:

- Curve fit
- Look up table
- A set of linear approximations, as outline in the following table.

Temperature Coefficient of Span (%/°C) (Typical)	-20 °C to 20 °C	-0.8%/°C
	20 °C to 40 °C	0.25%/°C
Temperature Coefficient of Zero Shift (ppm/°C) (Typical)	-20 °C to 0 °C	0.06 ppm/°C
	0 °C to 25 °C	0.3 ppm/°C
	25 °C to 40 °C	1.4 ppm/°C

CROSS SENSITIVITY

Most chemical sensors exhibit some cross-sensitivity to other gases. The following table lists the relative response of common potential interfering gases, and the concentration at which the data was gathered.

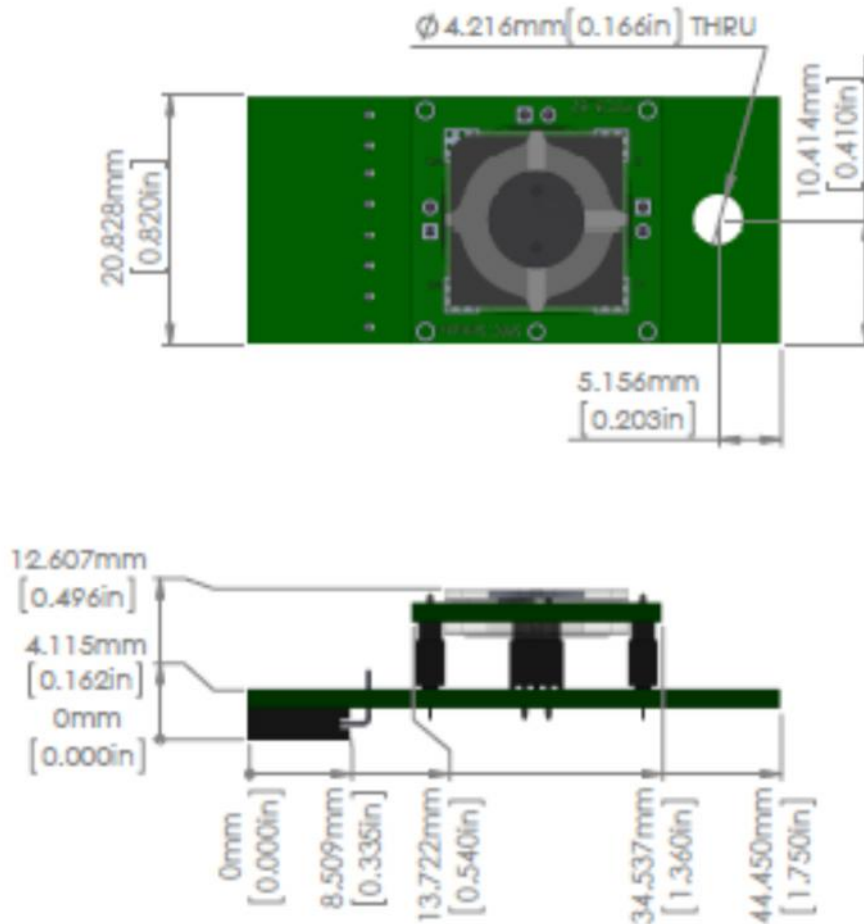
Gas/Vapor	Applied Concentration (PPM)	Typical Response (PPM CO)
Carbon Dioxide	5,000	< 1
Hydrogen	100	17
Methane	3,000	< 1
Ammonia	100	< 1
Nitrogen Dioxide	10	< 1
Hydrogen Sulfide	25	< 1
Carbon Monoxide	400	400
Ozone	5	< 1
Sulfur Dioxide	20	< 1
Chlorine	10	< 1
n-Heptane	500	< 1
Toluene	200	< 1
Isopropyl Alcohol	200	1.3
Acetone	200	< 1

MARKING INFORMATION

All gas sensors are tested and marked at the SPEC Sensors factory. Sensors include a label with an alpha-numeric code and a two-dimensional bar code. The codes include the information indicated in the table below.

	Unique Serial Number	Sensor Part Number	Target Gas	Date Code (YYMM)	Sensitivity Code (nA/ppm)
Alph-Numerica Code:		100102	CO	1510	4.94
2D Code:	101915010906	100102	CO	1510	4.94

PACKAGE OUTLINE DRAWING & DIMENSIONS



PINOUT

Electrical connections to the ULPSM are made via a rectangular female socket connector (Sullins Connector Solutions P/N: PPPC041LGBN-RC; recommended mate for host board: P/N: PBC08SBAN). This connector also provides mechanical rigidity on one end of the board. A through-hole is located on the opposite end of the board to provide additional mechanical connection.



Pin #	ULPSM Function	Notes
1	Vgas	Voltage Output. Vgas is proportional to the target gas concentration.
2	Vref	Voltage Output. Vref is approximately half the supply voltage. Useful as a fixed reference; equivalent to zero for Vgas. NOTE: High impedance output requires a buffer to connect to any measurement device.
3	Vtemp	Voltage Output. Vtemp is proportional to temperature. NOTE: High impedance output requires a buffer to connect to any measurement device.
4	N/C	
5	N/C	
6	GND	Universal ground for power and signal
7	V+	Voltage Supply Input: 2.7 to 3.3 V
8	V+	Voltage Supply Input: 2.7 to 3.3 V